5004

Shri Vile Parle Kelavani Mandal's
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Autonomous College Affiliated to the University of Mumbai)
NAAC Accredited with "A" Grade (CGPA : 3.18)

Continuous Assessment for Laboratory / Assignment sessions

Academic Year 2026-27

Name: Aaryan Nandkishor Raorane SAP ID: 60018250070

Course: Python for Data Science Laboratory Course Code: DJS23SLPC303 Year: SE

Sem: III Batch: A1

Department: Artificial Intelligence (AI) & Data Science

Performance Indicators(Any no. of Indicators) (Maximum 3 marks per indicator)	1	2	3	4	5	6	7	8	9	10	11	12	13	14	Q1	Q2	Σ	Avg
				✗		✗												
Course Outcome	1	1	1	4	1	1	2	2	3	3	4	4	4	4	1,2	3,4		
1. Knowledge (Factual/Conceptual/Procedural/ Metacognitive)	3	3	3	3	3	3	3	3	3	3	3	3	3	3				
2. Describe (Factual/Conceptual/Procedural/ Metacognitive)	3	2	3	2	3	3	3	3	3	2	3	3	2	3				
3. Demonstration (Factual/Conceptual/Procedural/ Metacognitive)	3	3	3	2	2	2	2	2	2	3	3	3	3	3				
4. Strategy (Analyse & / or Evaluate) (Factual/Conceptual/ Procedural/Metacognitive)	3	3	3	2	3	2	3	3	3	3	2	3	3	3				
5. Interpret/ Develop (Factual/Conceptual/ Procedural/Metacognitive)																		
6. Attitude towards learning (receiving, attending, responding, valuing, organizing, characterization by value)	3	3	✓	3	3	3	2	3	3	3	3	3	3	3				
Total	15	14	14	12	14	13	13	14	14	14	14	15	14	15	13	24		
Signature of the faculty member																		

Outstanding (3), Good (2), Fair (1)

Laboratory marks ∑Avg. =	Quiz marks ∑ Avg. =	Total Term-work (25) = 23
Laboratory Scaled to (15) = 14	Quiz Scaled to (10) = 9	Sign of the Student: Aaryan

Signature of the Faculty member:
Name of the Faculty member: Shruti Dodani

Signature of Head of the Department
Date: 29/10/26

PLOT NO. U-15, JVPD SCHEME, BHAKTIVEDANTA SWAMI MARG, VILE PARLE (WEST), MUMBAI - 400056.
Tel.: 42335000/42335001 Email: info@djsce.ac.in / admin@djsce.ac.in Website : www.djsce.ac.in

Python

1. int(integer):
The int type stores

First Lecture - Python Basics (assigning values to variables, different operator,New Identity and Membership operators)

In [86]:
```
#hello World
"""
multiline
commenting
"""
print("Hello World!")
```

```
Hello World!
```

In [87]:
```
#addition python code
a=10
b=10
c=a+b
print("Sum is ",c)
```

```
Sum is  20
```

In [88]:
```
#if else python code
if 5>2:
    print("5 > 2")
else :
    print("5 < 2")
```

```
5 > 2
```

In [89]:
```
#normal vraiable printing
n=4
print(n)
```

```
4
```

In [90]:
```
subject = "Klien Moretti"
classification ="Lord Of The Mysteries"
Danger ="Fighting Cosmic Corruption and Old Half Lord Of the Mysteries"
Age = 24.25
Artifacts = 8
Sefirot ='Sefirah_Castle'
is_real=0+0j
print(f"Subject : {subject}")
print(f"classification : {classification}")
print(f"Danger : {Danger}")
print(f"Age : {Age}")
print(f"Artifacts : {Artifacts}")
print(f"is_real : {is_real}")
print(type(Age))
print(type(Artifacts))
print(type(Sefirot))
print(type(classification))
print(type(is_real))
```

```
Subject : Klien Moretti
classification : Lord Of The Mysteries
Danger : Fighting Cosmic Corruption and Old Half Lord Of the Mysteries
Age : 24.25
Artifacts : 8
is_real : 0j
<class 'float'>
<class 'int'>
<class 'str'>
<class 'str'>
<class 'complex'>
```

In [91]:
```
x=-36.978
y=12E78
print(type(x))
print(type(y))
```

```
<class 'float'>
<class 'float'>
```

In [92]:
```
x=True
print(x)
print(type(x))
```

```
True
<class 'bool'>
```

In [93]:
```
x=1
y=2.8
z=1j
a=float(x)
b=int(y)
c=complex(z)
print(a)
print(b)
print(c)
print(type(a))
print(type(b))
print(type(c))
print(f"a : {type(a)} : {a}")
print(f"b : {type(b)} : {b}")
print(f"c : {type(c)} : {c}")
```

```
1.0
2
1j
<class 'float'>
<class 'int'>
<class 'complex'>
a : <class 'float'> : 1.0
b : <class 'int'> : 2
c : <class 'complex'> : 1j
```

In [94]:
```
#arithmetic operators
a=5
b=3
print(a+b)#addition
print(a-b)#substraction
print(a*b)#multiplication
print(a/b)#division
print(a%b)#modulus
```

```
print(a//b)#floor division
print(a**b)#exponent
```

```
8
2
15
1.6666666666666667
2
1
125
```

In [95]:
```
#conditional operators
a=5
b=3
print(a>b)#greater then
print(a<b)#less then
print(a==b)#equal to
print(a!=b)#not equal to
print(a>=b)#greater then or equal to
print(a<=b)#less then or equal to
```

```
True
False
False
True
True
False
```

In [96]:
```
#bitwise operators
a=60
b=13
print(a & b)
print(a | b)
print(a ^ b)
print(~ a)
print(~ b)
print(bin(a))
print(bin(b))
```

```
12
61
49
-61
-14
0b111100
0b1101
```

In [97]:
```
a+=5
print(a)
```

```
65
```

In [98]:
```
#Membership operators
x="Hello World!"
print('H' in x)
print('Hello' not in x)
print('hello' in x)
```

```
True
False
False
```

In [3]:
```
#identity operator
x=["cat","mat"]
y=["cat","mat"]
z=x
print(x is z)
print(x is y)
print(x==y)
```

```
True
False
True
```

In [ ]:

Input and Output in Python

In [1]:
```
#by default String
num = input("Enter a number : ")
print("You entered : ", num)
print("Data type of num : ", type(num))
```

```
Enter a number : 1
You entered :  1
Data type of num :  <class 'str'>
```

In [2]:
```
#typecasting input
a=int(input("Enter first no : "))
b=int(input("Enter second no : "))
sum = a+b
print("Sum is",sum)
```

```
Enter first no : 1
Enter second no : 2
Sum is 3
```

In [4]:
```
# print object,seperator and end
print("GFG")
print("G","F","G",sep="#",end="@!")
print("H&M")
print("H&MMM")
```

```
GFG
G#F#G@!H&M
H&MMM
```

In [5]:
```
print('New Year',2026,'Never see you again',sep='.')
```

```
New Year.2026.Never see you again
```

In [7]:
```
print('Science '+'is '+'Elegant')
```

```
Science is Elegant
```

In [8]:
```
print("Hello \t World")
print("Hello \\n World")
print(3*"Hello World")
```

```
Hello    World
Hello \n World
Hello WorldHello WorldHello World
```

In [10]:
```
a,b=1,2
print(a,b)
```

```
1 2
```

In [11]:
```
lst = [10,"A","hai"]
print(lst)
```

```
[10, 'A', 'hai']
```

In [12]:
```
d = {10:"Ram",20:"Amar"}
print(d)
```

```
{10: 'Ram', 20: 'Amar'}
```

In [16]:
```
print(f"The value of a is {a} and b is {b}")
print("The value of a is {} and b is {}".format(a,b))
```

```
The value of a is 1 and b is 2
The value of a is 1 and b is 2
```

ClassWork 1

In [17]:
```
# WAP to display your name and college name
name = "Aaryan"
c_name = "DJSCE"
print(name,c_name)
```

```
Aaryan DJSCE
```

In [19]:
```
# display designation
dsg = "Web Dev Intern"
name = "Aaryan"
c_name = "DJSCE"
print(dsg,name,"from",c_name)
```

```
Web Dev Intern Aaryan from DJSCE
```

In [20]:
```
pi = float(3.14)
r = int(2)
area = pi * (r ** 2)
print(area)
```

```
12.56
```

In [24]:
```
pi = float(3.14)
r = int(2)
area = pi * (r ** 2)
print(int(area))
```

```
12
```

In [23]:
```
pi = float(3.14)
r = int(input("Enter Radius : "))
area = pi * (r ** 2)
print(int(area))
```

```
Enter Radius : 3
28
```

In [ ]:
```

```

Department of AI_DS
Academic Year: 2026-27 (Odd Sem)
Course: Python for Data Science Lab (DJS23SLPC303)
Second Year B. Tech Semester: III

PRACTICAL 1

Part A (To be referred by students)

Data Types, Input / Output Statements, Operators (Arithmetic, Relational, Logical, Bitwise) and Expressions

Aim: To implement program to demonstrate data types operator and input output in python.

Problem Statements:

1. Create a program that takes user input and determines its data type.
2. Write a program to enter a number and print its square, cube and square root.
3. Write a program to perform arithmetic operations on two numbers.
4. Create a program that uses all comparison operators and prints the results.
5. Write a program to calculate the area of a right-angle triangle.
6. Calculate average of 3 numbers.
7. Calculate simple interest.
8. Convert Celsius to Fahrenheit.
9. Find Euclidean distance between two points.
10. Swap two variables using: Temp variable, Comma operator, Bitwise XOR
11. Find roots of a quadratic equation $ax^2 + bx + c = 0$.

Learning Objective: The Learner will be able to

1. Analyze the scenario to write a script declaring variables, printing output and using expressions.
2. Infer the features of various operators for different operations as per variable data type.
3. Evaluate the working of the algorithm using variables, printing and inputting user values and expressions.

Theory:

- **Data types:**
  Number: int, float, complex
  String

```
a = 11.80
x  = 100
y = "Hello"
z = 3+5.j

print(a)
print(x)
print(y)
print(z)
```

Output:

```
11.8
100
Hello
(3+5j)
```

- **Accepting Input from the Console:**

```
# if input() used it accepts input as string
num=input('Enter a number: ')
print(num)

# to accept numbers
num=int(input('Enter a number: '))
print(num)

num=float(input('Enter a number: '))
print(num)
```

Output:
Enter a number: 10
'10'
Enter a number: 45
45
Enter a number: 78
78.0

- **Output using print() with formatting:**

```
x=6
y=89
print('The value of x is {1} and y is {0}'.format(x,y))
x = 12.3456789
print('The value of x is %3.2f' %x)

print('The value of x is %3.4f' %x)

print ('The %.2f and %.1f ' %(x, y))
```

Output:
The value of x is 89 and y is 6
The value of x is 12.35
The value of x is 12.3457
The 12.35 and 456.8

```
name = "Alice"
age = 25
print(f"{name} is {age} years old.")
```

Output:
Alice is 25 years old.

- **Operators:**

Operators are used to perform operations on variables and values.
Python divides the operators in the following groups:

- **Arithmetic operators**
- Assignment operators
- **Relational/Comparison operators**
- **Logical operators**
- Identity operators
- Membership operators
- **Bitwise operators**

- **Python Arithmetic Operators**

Arithmetic operators are used with numeric values to perform common mathematical operations:

Operator	Example
+	**x + y**
-	**x - y**
*	**x * y**
/	**x / y**
%	**x % y**
**	**x ** y**
//	x // y

- **Python Assignment Operators**

Assignment operators are used to assign values to variables:

Operator	Example

=	x = 5
+=	x += 3
-=	x -= 3
*=	x *= 3
/=	x /= 3
%=	x %= 3
//=	x //= 3
**=	x **= 3
&=	x &= 3
|=	x |= 3
^=	x ^= 3
>>=	x >>= 3
<<=	x <<= 3

- **Python Relational/Comparison Operators**

Comparison operators are used to compare two values:

Operator	Example
==	x == y
!=	x != y
>	x > y
<	x < y
>=	x >= y
<=	x <= y

- **Python Logical Operators**

Logical operators are used to combine conditional statements:

Operator	Description	Example
and	Returns True if both statements are true	x < 5 and x < 10
or	Returns True if one of the statements is true	x < 5 or x < 4
not	Reverse the result, returns False if the result is true	not(x < 5 and x < 10)

- **Python Bitwise Operators**

Bitwise operators are used to compare (binary) numbers:

Operator	Name	Description
&	AND	Sets each bit to 1 if both bits are 1
|	OR	Sets each bit to 1 if one of two bits is 1
^	XOR	Sets each bit to 1 if only one of two bits is 1
~	NOT	Inverts all the bits
<<	Zero fill left shift	Shift left by pushing zeros in from the right and let the leftmost bits fall off
>>	Signed right shift	Shift right by pushing copies of the leftmost bit in from the left, and let the rightmost bits fall off

PRACTICAL 1

Part B (to be completed by students)

Data Types, Input / Output Statements, Operators (Arithmetic, Relational, Logical, Bitwise) and Expressions

1. All the students are required to perform the given tasks in Jupyter Notebook
2. Create a new notebook for each experiment. The filename should be RollNo_Name_Exp1)
3. In the first cell, the student must write his/her Name, roll no and class in the form of comments

4. Every program should be written in separate cells and in the given sequence
5. After completing the experiment, download the notebook in pdf format. The filename should be RollNo_Name_Exp1.pdf
6. Upload the pdf on the web portal (Final Submission)
7. Submit the handwritten experiment in the next lab session

In [1]:
```
#Determine user input
something = input("Enter something : ")
print("You entered : ", something)
print("Data type of num : ", type(something))
```

```
Enter something : fhsjigvhuhb
You entered :  fhsjigvhuhb
Data type of num :  <class 'str'>
```

In [2]:
```
num = int(input("Enter a number : "))
print("Square : ",(num ** 2))
print("Cube : ",(num ** 3))
print("Square root : ",(num ** 0.5))
```

```
Enter a number : 4
Square :  16
Cube :  64
Square root :  2.0
```

In [5]:
```
#arithmentic operation
a=int(input("Enter first no : "))
b=int(input("Enter second no : "))
print("addition is : ",(a+b))
print("substraction is : ",(a-b))
print("multiplication is : ",(a*b))
print("division is :",(a/b))
print("modulus is : ",(a%b))
print("floor division is : ",(a//b))
print("exponent is : ",(a**b))
```

```
Enter first no : 1
Enter second no : 2
addition is :  3
substraction is :  -1
multiplication is :  2
division is : 0.5
modulus is :  1
floor division is :  0
exponent is :  1
```

In [8]:
```
#comparison operator
a=int(input("Enter first no : "))
b=int(input("Enter second no : "))
print((a>b))#greater then
print((a<b))#less then
print((a==b))#equal to
print((a!=b))#not equal to
print((a>=b))#greater then or equal to
print((a<=b))#less then or equal to
```

```
Enter first no : 2
Enter second no : 3
False
True
False
True
False
True
```

In [10]:
```
#area of right angled triangle
l=float(input("Enter length : "))
h=float(input("Enter breadth : "))
print("Area of the right angled triange : ",(0.5*l*b))
```

```
Enter length : 2
Enter breadth : 3
Area of the right angled triange :  3.0
```

In [11]:
```
# average
a=float(input("Enter first no : "))
b=float(input("Enter second no : "))
c=float(input("Enter third no : "))
avg = (a+b+c)/3
print("Average : ",avg)
```

```
Enter first no : 1
Enter second no : 2
Enter third no : 3
Average :  2.0
```

In [12]:
```
#simple interest
p=float(input("Enter principal value : "))
r=float(input("Enter interest rate : "))
n=float(input("Enter no of year: "))
si = (p*r*n)/100
print("Simple Interest : ",si)
```

```
Enter principal value : 1000
Enter interest rate : 5
Enter no of year: 3
Simple Interest :  150.0
```

In [13]:
```
# celsius to fahrenheit
c = float(input("Enter the temperature in celsius : "))
f = ((9/5)*c)+32
print("Temperature in Fahrenheit : ",f)
```

```
Enter the temperature in celsius : 29
Temperature in Fahrenheit :  84.2
```

In [14]:
```
# euclidean distance
x1 = float(input("Enter first x co-ordinate : "))
y1 = float(input("Enter first y co-ordinate : "))
x2 = float(input("Enter second x co-ordinate : "))
y2 = float(input("Enter second y co-ordinate : "))
dist = ((((x2-x1)**2)+((y2-y1)**2)) ** 0.5)
print("Euclidean distance : ",dist)
```

```
Enter first x co-ordinate : 1
Enter first y co-ordinate : 2
Enter second x co-ordinate : 3
Enter second y co-ordinate : 4
Euclidean distance :  2.8284271247461903
```

In [18]:
```
# swap 2 variables using comma,temp,bitwiseXOR
a=3
b=4
a,b=b,a
print(a,b)
```

```
4 3
```

In [19]:
```
a=3
b=4
t=a
a=b
b=t
print(a,b)
```

```
4 3
```

In [29]:
```
a=9
b=4
a= a^b
b = a^b
a = a^b
print(a,b)
```

```
4 9
```

In [30]:
```
# solution to quadratic equation
a=float(input("Enter first co-effiecient : "))
b=float(input("Enter second co-effiecient : "))
c=float(input("Enter third co-effiecient : "))
D = ((b**2)-(4*a*c))
if D > 0:
    x1=(((b*(-1))+(D ** 0.5))/(2*a))
    x2=(((b*(-1))-(D ** 0.5))/(2*a))
    print("Roots of the equation are : ",x1,"and",x2)
elif D==0:
    x=((b*(-1))/(2*a))
    print("Root of the equation is : ",x)
else:
    print("Real solution doesn't exist")
    x1=(((b*(-1))+(D ** 0.5))/(2*a))
    x2=(((b*(-1))-(D ** 0.5))/(2*a))
    print("Complex Roots of the equation are : ",x1,"and",x2)
```

```
Enter first co-effiecient : 1
Enter second co-effiecient : 1
Enter third co-effiecient : 1
Real solution doesn't exist
Complex Roots of the equation are :  (-0.49999999999999994+0.8660254037844386j) a
nd (-0.5-0.8660254037844386j)
```

In [ ]:
```
```

Department of Artificial Intelligence (AI) & Data Science
AY: 2026-27

Course: Data Science Laboratory (DJS23DSPL303)

Class: SE/A1 Sem: III Roll No.: SAPID:

Experiment no 1: Data Types and Input-Output in Python

Aim: To implement program to demonstrate data types operator and input output in python.

Theory:

Basic Data Types in python with Examples

Basic Operators in Python with Examples

Input and Output in Python

Conclusion:

Name :- Aaryan N. Rodrane Roll no :- 5004
branch :- AI-DS batch : A1 SAPID :- 60018250070

Experiment No 1

Title :- Data-Types, operators and input-output in Python

Aim :- To implement a program to demonstrate data types operator and input output in Python

Theory :-

Basic Data types in Python with Examples

Python is dynamically typed, interpreted language. Unlike statically typed languages such as C or Java, Python determines the data type of a variable at runtime based on the value assigned to it. Every value in Python is an object, and each object belongs to a specific class.

Python provides the following built-in data types:

1. **Int (Integer) :**

The int type stores whole numbers without a decimal component. Python integers have arbitary precision, meaning they are not constrained by a fixed bit width like 32-bit or 64-bit integers in other languages.

FOR EDUCATIONAL USE

code:
```
x = 10
y = -25
z = 0
print (type(x)) # <class 'int'>
print (x + y) # [output: -15]
```

2. float (floating-point Number)

The float type represents real numbers with a decimal point. Internally, Python floats follow the IEEE 754 double-precision standard, which provides approximately 15 to 17 significant decimal digits of precision.

code:
```
pi = 3.14159
temperature = -7.5
print (type(pi)) # output: <class 'float'>
print (pi * 2) # output: 6.28318
```

3. complex

The complex type stores numbers of the form $a+bj$, where a is the real part and b is the imaginary part. Python uses j to denote the imaginary unit.

code:
```
c = 3 + 4j
print (c.real) # output: 3.0
print (c.imag) # output: 4.0
print (type(c)) # output: <class 'complex'>
```

Sundaram
FOR EDUCATIONAL USE

4. str (string)

A string is an ordered, immutable sequence of Unicode characters. Strings can be enclosed in single quotes, double quotes or triple quotes (for multiline strings).

code :
```
name = "Python"
message = 'Hello, World!'
multiline = """This is a
multiline string."""
print (type(name)) # output : <class 'str'>
print (name[0]) # output : P
print (len(name)) # output : 6
```

5. Bool (Boolean)

The Bool type holds one of two values : True or False. Booleans are technically a subclass of int in Python, where True evaluates to 1 and False evaluates to 0

code :
```
is_active = True
is_empty = False
print (type(is_active)) # output : <class 'bool'>
print (is_active + 6) # output : 7
print (int(is_empty)) # output : 0
```

Sundaram
FOR EDUCATIONAL USE

*) Basic Operators in Python with Examples

Operators are tokens that instruct the interpreter to perform a specific computation on one or more operands. Python supports several categories of operators.

1. Arithmetic Operator

These operators perform standard mathematical computations. The floor division operator (//) returns the quotient rounded down to the nearest integer, and the modulus operator (%) returns the remainder.

code:-

```
a = 10
b = 3

print(a+b) # Addition output: 13
print(a-b) # Substraction output: 7
print(a*b) # Multiplication output: 30
print(a/b) # Division output: 3.3333333....
print(a//b) # floor Division output: 3
print(a%b) # Modulus Output: 1
print(a**b) # Exponent output: 1000
```

Sundaram
FOR EDUCATIONAL USE

2. Comparison (Relational) operators

Comparison operators evaluate the relationship between two operands and return a Boolean value (True or False). These are widely used in conditional statements and loop control.

Code:-

```
x = 5
y = 10

print(x == y) # equal to output : False
print(x != y) # Not equal to output : True
print(x > y) # greater than output : False
print(x < y) # Less than output : True
print(x >= y) # greater or equal to output : False
print(x <= y) # Less or equal to output : True
```

3. Assignment operators

Assignment operators binds value to a variable. Compound assignment operators (such as += and *=) combine an arithmetic operation with assignment into a single statement.

Code:-

```
a = 10
a += 5 # Equivalent to a = a+5 → a = 15
a -= 3 # Equivalent to a = a-3 → a = 12
a *= 2 # Equivalent to a = a*2 → a = 24
a //= 4 # Equivalent to a = a//4 → a = 6
print(a) # output : 6
```

Sundaram
FOR EDUCATIONAL USE

4. Logical operators

Logical operators combine or negate Boolean expressions. Python evaluates and and or using short-circuit evaluation: it stops evaluating as soon as the result is determined.

code:
```
p = True
q = False

print(p and q) # both must be True output: False
print(p or q) # at least one true output: True
print(not p) # Negation output: False
print(not q) # Negation output: True
```

5. Bitwise operators

Bitwise operators operate directly on the Binary representation of integer operands. They are used in low-level programming, cryptography and performance-sensitive applications.

code:
```
a = 5    # binary : 0101
b = 3    # binary : 0011

print(a & b) # AND output: 1 (0001)
print(a | b) # OR output: 7 (0111)
print(a ^ b) # XOR output: 6 (0110)
print(~a) # NOT output: -6 (1010)
print(a << 1) # left shift output: 10 (1010)
print(a >> 1) # Right shift output: 2 (0010)
```

Sundaram
FOR EDUCATIONAL USE

5. Membership Operators

Membership operators test whether a value is present in a sequence such as a list, tuple, string or set. They return a Boolean result.

Code:
```
fruits=["apple","Banana","mango"]

print("apple" in fruits) # output: True
print("grape" in fruits) # output: False
print("grape" not in fruits) # output: True

text = "Python Programming"
print("Python" in text) # output: True
```

6. Identity Operators

Identity operators check whether two variables refer to the same object in memory. This is different from equality: two objects can hold the same value but occupy different memory locations.

Code:
```
a=[1,2,3]
b=a # b refers to the same object as a
c=[1,2,3] # c is a new object with the same value
print(a is b) # output: True (same object in memory)
print(a is c) # output: False (different objects)
print(a==c) # output: True (same values)
print(a is not c) # output: True
```

Sundaram
FOR EDUCATIONAL USE

3. Input and Output in Python

Python provides built in functions for reading data from the user and displaying results to the console.

* Output using print()

The print() function writes data to the standard output stream. It accepts multiple positional arguments and two keywords: sep (separator between arguments, default is a space) and end (character printed after the last argument, default is a newline)

Syntax: print (object(s), sep=' ', end='\n')

code:
```
print ("Hello, World!") # output: Hello, World!
print (10, 20, 30)  # output: 10 20 30
print (10, 20, 30, sep=":", ) #output: 10, 20, 30
print ("First", end = " ")
print ("second") # output: First second
print ("Name:", "Carlson", "Age:", 20)
    # output: Name: Carlson Age: 20
```

* formatted output

Python supports three methods for formatted string output:

Sundaram
FOR EDUCATIONAL USE

Method 1 :- % formatting (old style)

code :-
```
name = "Aaryan"
marks = 95.5
print ("student : %s, Marks : %.2f" %
       (name, marks))
#output : student : Aaryan , Marks : 95.50
```

Method 2 :- str.format() method

code :-
```
print ("Student : {} , Marks : {:.2f}".format(name,
       marks))
#output : student : Aaryan, Marks : 95.50
```

Method 3 :- f-strings (Python 3.6 and above)

code :-
```
print (f "Student : {name}, Marks : {marks:.2f}")
#output : Student : Aaryan , Marks : 95.50
```

f-strings are the most readable and efficient of the three methods. they allow expressions to be embedded directly inside the string literal

* Input using input()

The input() function pauses program execution and reads a line of text from the standard input stream. It always return the value as a string (str), regardless of what the user types.

Sundaram
FOR EDUCATIONAL USE

Syntax: input ([prompt])

code:
```
name = input("Enter your name: ")
print("Hello,", name)
# input : Karl
# output : Hello, Aaryan.
```

* Type Conversion (casting)

Since input() returns a string, numeric input must be explicitly converted using type-casting functions.

code:
```
age = int(input("Enter your age: "))
cgpa = float(input("Enter your CGPA: "))
print(f"Age: {age}, CGPA: {cgpa:.2f}")
```

Python's built-in type conversion functions:

1. int(x) : converts x to an integer
2. float(x): converts x to a floating-point number
3. str(x) : converts x to a string.
4. bool(x): converts x to a Boolean
5. complex(x): converts x to a complex number.

code:
```
x = "42"
print(int(x))  # output: 42
print(float(x)) # output: 42.0
print(bool(0)) # output: False
print(bool(99)) # output: True
print(str(3.14)) # output: '3.14'
```

Sundaram
FOR EDUCATIONAL USE

* Python input - output code :-

```
length = float(input("Enter length : "))
breadth = float(input("Enter breadth : "))
area = length * breadth
perimeter = 2 * (length + breadth)

print(f"\n Rectangle Details")
print(f"length : {length}")
print(f" breadth : {breadth}")
print(f"Area : {area :.2f} sq. units")
print(f"Perimeter : {perimeter :.2f} units")
```

output: Enter length : 5.0
Enter breadth : 3.0

Rectangle Details
length : 5.0
breadth : 3.0
Area : 15.00 sq. units
Perimeter : 16.00 units

* Conclusion :-

Through this experiment, the Basic data types in Python such as int, float, complex, str and bool were studied along with their usage in programs. The various operators like arithmetic, logical, relational and bitwise were understood and implemented with examples. The input() and print() functions were used to handle user input and display formatted output on the console.

Sundaram
FOR EDUCATIONAL USE

Control Statement (If else , switch ,for , while, break,etc.)

In [1]:
```
# if else with indentation error
a=23
b=302
if a>b:
    print("b is greater then a")
else:
print("a is greater then b")
```

```
  File "<ipython-input-1-990ba1a9f7f2>", line 7
    print("a is greater then b")
        ^
IndentationError: expected an indented block
```

In [2]:
```
# if else
a=23
b=302
if a>b:
    print("b is greater then a")
else:
    print("a is greater then b")
```

```
a is greater then b
```

In [3]:
```
msg = input("Are willing to learn Python 'yes' or 'no' : \n")
if msg=="yes":
    print("Welcome to Python Programming")
```

```
Are willing to learn Python 'yes' or 'no' :
yes
Welcome to Python Programming
```

In [4]:
```
#using only if
a=int(input("Enter no. 1 : "))
b=int(input("Enter no. 2 : "))
if a>b:
    print("no. 1 is greater then no. 2")
if a==b:
    print("no. 1 is equal to no. 2")
if b>a:
    print("no. 2 is greater then no. 1")
```

```
Enter no. 1 : 10
Enter no. 2 : 56
no. 2 is greater then no. 1
```

In [5]:
```
#using only if and else
a=int(input("Enter no. 1 : "))
b=int(input("Enter no. 2 : "))
if a>b:
    print("no. 1 is greater then no. 2")
if a==b:
    print("no. 1 is equal to no. 2")
else:
    print("no. 2 is greater then no. 1")
```

```
Enter no. 1 : 10
Enter no. 2 : 10
no. 1 is equal to no. 2
```

In [6]:
```python
#using only if and else
a=int(input("Enter no : "))
if a>0:
    print("Positive")
if a==0:
    print("Zero")
else:
    print("Negative")
```

```
Enter no : -88
Negative
```

In [7]:
```python
#using only if and else
a=int(input("Enter no : "))
if a%2==0 :
    print("Even")
else:
    print("Odd")
```

```
Enter no : 45646478313
Odd
```

In [8]:
```python
#using only if , elif and else
a=int(input("Enter age : "))
if a>=18:
    print("Eligible to Vote")
elif a>0:
    print("Not eligible to Vote")
else:
    print("U are not a human !! Go away Demon !! (Splashing Holy water on you)")
```

```
Enter age : -9999
U are not a human !! Go away Demon !! (Splashing Holy water on you)
```

In [9]:
```python
a=int(input("Enter no. 1 : "))
b=int(input("Enter no. 2 : "))
c=int(input("Enter no. 3 : "))
if a>b and a>c:
    print(f"{a} is greatest")
elif a>c and b>a:
    print(f"{b} is greatest")
else:
    print(f"{c} is greatest")
```

```
Enter no. 1 : 10
Enter no. 2 : 50
Enter no. 3 : 13
13 is greatest
```

In [11]:
```python
a=int(input("Enter no : "))
if a>0:
    print("Positive")
elif a==0:
    print("Zero")
else:
    print("Negative")
```

```
Enter no : -9
Negative
```

In [13]:
```
# nested if-else
a = int(input("Enter no. : "))
if a> 10:
    print("Above 10")
    if a>20:
        print("And also above 20")
    else :
        print("but not above 20")
```

```
Enter no. : 41
Above 10
And also above 20
```

In [19]:
```
# one line if-else
a=23
b=24
print("A") if a>b else print("=") if a==b else print("B")
```

```
B
```

Strings and string operations

In [20]:
```
a = "This is name1"
b = 'This is name2'
print(a)
print(b)
```

```
This is name1
This is name2
```

In [24]:
```
a = "This is 'name1'"
print(a)
b = 'This is "name2"'
print(b)
```

```
This is 'name1'
This is "name2"
```

In [26]:
```
# backslash character
a = "\"hello\" This isn\'t name1"
print(a)
```

```
"hello" This isn't name1
```

In [30]:
```
# accessing string character
print(a[0])
print(a[5])
print(a[-3])
```

```
"
o
m
```

In [34]:
```
# slicing
print(a[4:-11])
print(a[4:])
```

```
print(a[:-11])
print(a[-11:-4])
```

```
lo" This
lo" This isn't name1
"hello" This
isn't n
```

In [38]:
```
#concatenation
x = "Helllo"
y = "World"
z = "I"
a = "am"
b = "???"
print(a + b + y + z + x)
# string repition
print(3*(a + b + y + z + x))
```

```
am???WorldIHelllo
am???WorldIHellloam???WorldIHellloam???WorldIHelllo
```

In [40]:
```
# == operator
print("Hello" == "Hello")
print("Hello" == "hello")
```

```
True
False
```

In [41]:
```
# != operator
print("Hello" != "Hello")
print("Hello" != "hello")
```

```
False
True
```

In [42]:
```
# > operator
print("Hello" > "Hello")
print("Hello" > "hello")
```

```
False
False
```

In [43]:
```
# < operator
print("Hello" < "Hello")
print("Hello" < "hello")
```

```
False
True
```

In [44]:
```
print('jas' in 'jaspreet')
```

```
True
```

In [45]:
```
print('jas' not in 'jaspreet')
```

```
False
```

In [48]:
```
age = int(31)
print("age is %d"%(age))
```

```
age is 31
```

In [50]:
```
s='I am Jaspreet'
```

```
print(s[0:8])
```

```
I am Jas
```

In [20]:
```
s='abc'*3
print(s)
print(s[0:6:2])
print(s[0:6:2])
print(s[2:10:2])
print(s[::3])
print(s[1::3])
print(s[:8:3])

#3rd parameter as negative index(1st parameter then must be greater then 2nd par

print(s[8:1:-2])
print(s[8:1:-1])
print(s[8:-1:-1])
print(s[::-1])
```

```
abcabcabc
acb
acb
cbac
aaa
bbb
aaa
cabc
cbacbac

cbacbacba
abcabcabc
```

Strip Method

In [4]:
```
s="          abcabcabc      "
print(s.strip())
s="aaaaaaaabcabcabcaaaa"
print(s.strip('a'))
s="Hello world"
print(s.strip('ldoH'))
print(s.strip('ldoh'))
```

```
abcabcabc
bcabcabc
ello wor
Hello wor
```

Lstrip and Rstrip Method

Removes only leading and trailing whitespaces by default respectively

In [7]:
```
s="         Hello world      "
print(s.lstrip())
print(s.rstrip())
```

```
Hello world
         Hello world
```

In [9]:
```
"  solving any problem is an art.  ".strip('. ')
```

Out[9]:
```
'solving any problem is an art'
```

In [11]:
```
"  solving any problem is an art.  ".strip(' ')
```

Out[11]:
```
'solving any problem is an art.'
```

In [18]:
```
str="I am Jaspreet      ."
str = str.rstrip('I am.')
str
```

Out[18]:
```
'I am Jaspreet'
```

In [20]:
```
str="I am Jaspreet      ."
str = str.lstrip('I am.')
str
```

Out[20]:
```
'Jaspreet      .'
```

In [23]:
```
sentence ="My name is Jaspreet  ".rstrip("my ")
sentence
```

Out[23]:
```
'My name is Jaspreet'
```

Split method

syntax : string.split(separator,maxsplit)

In [25]:
```
"Hell$I$am$Jaspreet".split('$',maxsplit=2)
```

Out[25]:
```
['Hell', 'I', 'am$Jaspreet']
```

In [27]:
```
"Hell$I$am$Jaspreet".rsplit('$',maxsplit=2)
```

Out[27]:
```
['Hell$I', 'am', 'Jaspreet']
```

In [40]:
```
"Hello world! I am tired saying this sentence again and again".split()
```

Out[40]:
```
['Hello',
 'world!',
 'I',
 'am',
 'tired',
 'saying',
 'this',
 'sentence',
 'again',
 'and',
 'again']
```

Join method

Syntax : separator.join(iterator)

In [42]:
```
s = "Hello world! I am tired saying this sentence again and again".rsplit()
print(s)
s='    '.join(s)
print(s)
```

```
['Hello', 'world!', 'I', 'am', 'tired', 'saying', 'this', 'sentence', 'again', 'a
nd', 'again']
Hello    world!    I    am    tired    saying    this    sentence    again    and
again
```

In [43]:
```
s=s.split()
print(s)
s='_@_'.join(s)
print(s)
```

```
['Hello', 'world!', 'I', 'am', 'tired', 'saying', 'this', 'sentence', 'again', 'a
nd', 'again']
Hello_@_world!_@_I_@_am_@_tired_@_saying_@_this_@_sentence_@_again_@_and_@_again
```

In [44]:
```
s='aBc'
print(s.upper())
print(s.lower())
print(s.capitalize())
```

```
ABC
abc
Abc
```

In [51]:
```python
s='Python is Fun'
print(s.replace('Py','Kr'))
print(s.count('n'))
print(len(s))
print(s.find('is'))
```

```
Krthon is Fun
2
13
7
```

Department of AI_DS
Academic Year: 2026-27 (Odd Sem)
Course: Python for Data Science Lab
Second Year B. Tech Semester: III

PRACTICAL 2

Part A (To be referred by students)

Conditional Statements & Strings

SAVE THE FILE AND UPLOAD AS (RollNo_Name_Exp2)

Aim: To Implement python programs to demonstrate different decision-making and Control Statements and string handling.

Problem Statement: Write Python program to

1. Accept three sides of a triangle and print if the triangle is equilateral, isosceles or scalene.
2. A function f is defined as follows :

   $f(x) = ax^3 - bx^2 + cx - d$, if $x > k$
   $= 0$, if $x = k$
   $= -ax^3 + bx^2 - cx + d$, if $x < k$

   Write a program that reads a, b, c, d, k and x and prints the value of f(x).
3. Implement Body Mass Index calculator. Define BMI function to accept weight and height as parameter and displays appropriate BMI status.
   BMI = weight(kg)/Height^2

BM	Status
<= 18.4	Underweight
18.5 – 24.9	Normal
25.0 – 39.9	Overweight
>=40	Obese

4. Implement distance convertor to convert distances by reading choice from user
   1. inches to feet
   2. cm to meter
5. Write a program to read and display a string
6. Create a string variable s with the value "Hello World" and print its length.
7. Convert the string "Python Programming" to lowercase and print it.
8. Concatenate the strings "Data" and "Science" with a space in between and print the result.
9. Remove all leading and trailing whitespace from the string " Data Analysis " and print it.
10. Define one string and display all elements at odd indexes using slicing operator
11. Write a program to check entered string is palindrome or not.
12. Reverse the string 'Hello' only using string slicing
13. WAP to split the string into a list of words and then join them back into a single string with a user-defined delimiter.
14. Write a python program to take a list of words, reverse the order of the words, and then join them into a single string with a hyphen - as the separator.

Department of AI_DS
Academic Year: 2026-27 (Odd Sem)
Course: Python for Data Science Lab
Second Year B. Tech Semester: III

Topic covered: Conditional Statements (if, if…else, nested if, elif, pass and Strings,),

Learning Objective: Learner would be able to

1. Analyze the scenario to write script applying conditional statements.
2. Infer the features of conditions for true and false decision-making.
3. Solve the problems using decision-making statements
4. Process string and perform various slicing operations on string

Theory:

- **The if statement:**

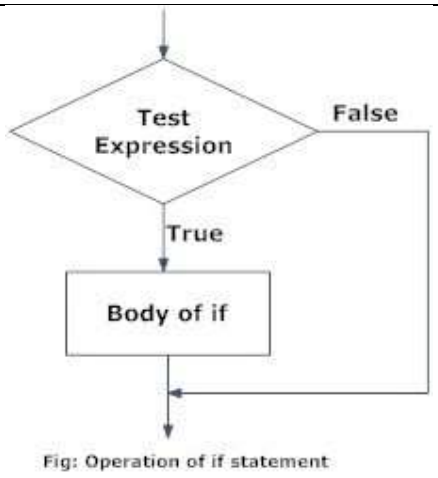
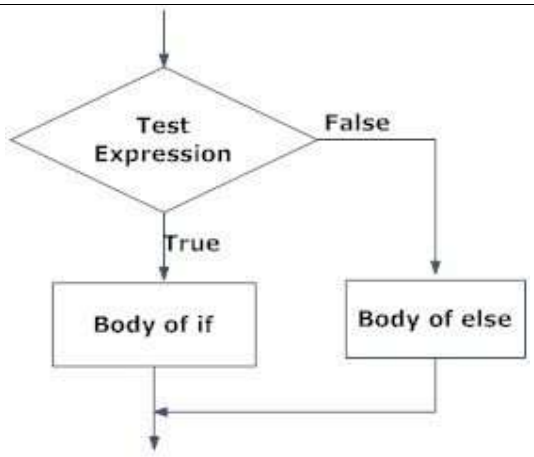
Syntax of if statement	
if test_expression: **body of if**	
The program evaluates the test expression and will execute statement(s) only if the test expression is True. If the test expression is False, the statement(s) is not executed. In Python, the body of the if statement is indicated by the indentation. The body starts with an indentation and the first unindented line marks the end. Python interprets non-zero values as True. None and 0 are interpreted as False.	

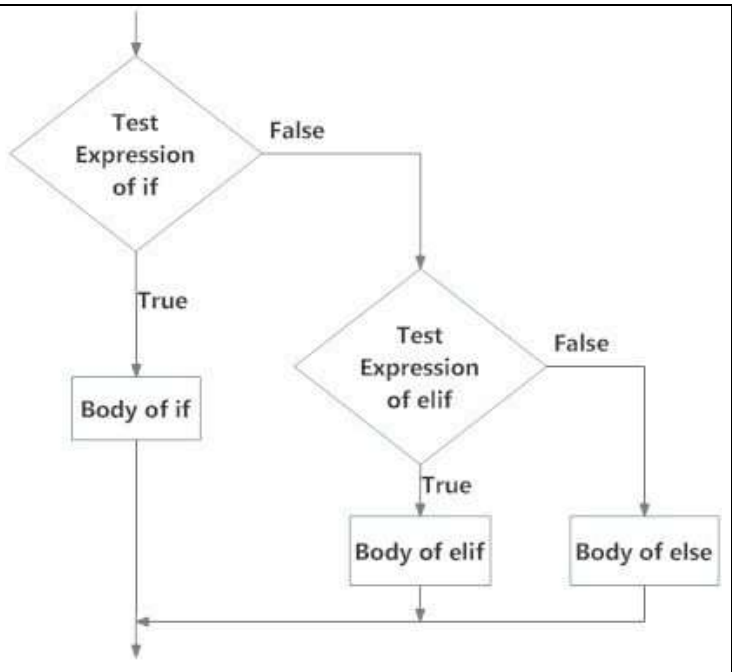
Fig: Operation of if statement

- **The if…else statement:**

Syntax of if…else statement	
if test_expression: **body of if**	
The if..else statement evaluates test expression and will execute the body of if only when the test condition is True. If the condition is False, the body of else is executed. Indentation is used to separate the blocks.	Fig: Operation of if...else statement

- **The if…elif…else statement:**

  The if…elif…else statement is used for decision making.

Syntax of for Loop	
if test_expression: **body of if** **elif test_expression:** **body of elif** **else:** **body of else**	
The elif is short for else if. It allows us to check for multiple expressions. If the condition for if is False, it checks the condition of the next elif block and so on.	Fig: Operation of if...elif...else statement

If all the conditions are False, the body of else is executed. Only one block among the several if...elif...else block s is executed according to the condition. The if block can have only one else block. But it can have multiple elif blocks.	

```
# Check if number is positive or negative or zero:
num = int(input("Enter a number "))
if num > 0:
   print("Positive number")
elif num == 0:
   print("Zero")
else:
   print("Negative number")
```

Output:

Enter a number 2

Positive number

- **The Nested if statement:**

  We can have a if...elif...else statement inside another if...elif...else statement. This is called nesting in computer programming.

  Any number of these statements can be nested inside one another. Indentation is the only way to figure out the level of nesting.

```
# Check if number is positive or negative or zero:
num = int(input("Enter a number: "))
if num >= 0:
   if num == 0:
      print("Zero")
   else:
      print("Positive number")
else:
   print("Negative number")
```

Output:
Enter a number: 0
Zero

String

- String literals in python are surrounded by either single quotation marks, or double quotation marks.
- 'hello' is the same as "hello".
- You can display a string literal with the print( ) function:
- Example

```
print("Hello")
print('Hello')
```

- Assigning a string to a variable is done with the variable name followed by an equal sign and the string:
- Example

```
a = "Hello"
print(a)
```

Str --- String

```
name=u'\u092a\u0948\u0925\u0964\u0928'
print(name)
```

 # Slicing operations on string [start:stop:stepsize], if start and stop is not specified it takes 0 and n-1 and if step size in not specified it is 1.

 # slicing the string stringname[start:stop:stepsize]

```
str='core python'
print(str[0:9:1])
```

```
#str[0:9:2]  --> stepsize 2
#str[2:4:1]  -->  2 to 3 in size 1
#str[ : :2]  -->  access entire string in steps of 2.
#str[ :4: ]  -->  access string from 0 to 3 and stepsize is 1.
#str[-4:-1: ]-->  access string from -4 to -2 from left to right
#str[-1:-4:-1]->  -1 to -3 from right to left...
#str[-1::-1] -->  retrieves from last to first element [reverse]
```

#Repeating String...

```
print(4*"Python")
```

Concatenation (Joining)

```
s1="Core"
s2="Python"
s=s1+s2
print(s)
```

#Checking Membership

Write a program to find sub string in main string...

```
#operator:- in -- if found returns true  ---And--- not in-- returns false if not found
str=input('Enter String')
sub=input('Enter sub string')
if sub in str:
   print("Found")
else:
   print("Not Found")
```

 # Comparing String...

```
s1='Box'
s2="box"
if(s1==s2):
   print("Same")
else:
   print("Not Same")
```

Department of AI_DS
Academic Year: 2026-27 (Odd Sem)
Course: Python for Data Science Lab
Second Year B. Tech Semester: III

PRACTICAL 2

Part B (to be completed by students)

Conditional Statements and Strings

1. All the students are required to perform the given tasks in Jupyter Notebook
2. Create a new notebook for each experiment. The filename should be RollNo_Name_Exp1)
3. In the first cell, the student must write his/her Name, roll no and class in the form of comments
4. Every program should be written in separate cells and in the given sequence
5. After completing the experiment, download the notebook in pdf format. The filename should be RollNo_Name_Exp1.pdf).
6. Upload the pdf on the web portal

In [5]:
```
# accept 3 sides of triangle and tell its type
a = float(input("Enter 1st Side of the triangle : "))
b = float(input("Enter 2nd Side of the triangle : "))
c = float(input("Enter 3rd Side of the triangle : "))

if(a==b and b==c):
    print("Equilateral Triangle")
elif(a==b or b==c or a==c):
    print("Isoceles Triangle")
else:
    print("Scalene Triangle")
```

```
Enter 1st Side of the triangle : 1
Enter 2nd Side of the triangle : 1
Enter 3rd Side of the triangle : 1
Equilateral Triangle
```

In [6]:
```
#function solver

print("f(x) = ax^3 - bx^2 + cx -d            if x > k")
print("f(x) = 0                             if x = k")
print("f(x) = -ax^3 + bx^2 - cx +d          if x < k")

a=float(input("Enter value of a : "))
b=float(input("Enter value of b : "))
c=float(input("Enter value of c : "))
d=float(input("Enter value of d : "))
k=float(input("Enter value of k : "))
x=float(input("Enter value of x : "))

if(x>k):
    f = (a*(x**3))-(b*(x**2))+(c*x)-d
    print(f"Solution : {f}")
elif(x==k):
    print("Solution : 0")
else:
    f = (a*(x**3))-(b*(x**2))+(c*x)-d
    print(f"Solution : {f}")
```

```
f(x) = ax^3 - bx^2 + cx -d            if x > k
f(x) = 0                              if x = k
f(x) = -ax^3 + bx^2 - cx +d           if x < k
Enter value of a : 1
Enter value of b : 2
Enter value of c : 3
Enter value of d : 4
Enter value of k : 5
Enter value of x : 6
Solution : 158.0
```

In [14]:
```
# BMI calculator
w = float(input("Enter weight in kg : "))
h = float(input("Enter height in m : "))

bmi = w/(h**2)

if(bmi <= 18.4):
    print(f"BMI : {bmi:.2f} Status : Underweight")
elif(bmi <= 24.9):
```

```
    print(f"BMI : {bmi:.2f} Status : Normal")
elif(bmi <= 39.9):
    print(f"BMI : {bmi:.2f} Status : Overweight")
else:
    print(f"BMI : {bmi:.2f} Status : Obese")
```

```
Enter weight in kg : 105
Enter height in m : 1.83
BMI : 31.35 Status : Overweight
```

In [1]:
```
#DISTANCE CONVERTER

print("Enter the value of the convereter u want to use")
print("1. Inches to Feet ")
print("2. Centimeter to Meter ")

choice = int(input("Enter the value : "))

if (choice == 1):
    inch = float(input("Enter distance in inches : "))
    feet = inch/12
    print(f"feet : {feet:.2f}")
elif(choice == 2):
    cm = float(input("Enter distance in centimeter : "))
    m = cm/100
    print(f"meter : {m:.2f}m")
else:
    print("Invalid Choice")
```

```
Enter the value of the convereter u want to use
1. Inches to Feet
2. Centimeter to Meter
Enter the value : 2
Enter distance in centimeter : 183
meter : 1.83m
```

In [3]:
```
#read and display string
s = input("Enter String : ")
print(s)
```

```
Enter String : Hello World1
Hello World1
```

In [8]:
```
# length of string
s = "Hello World"
print(len(s))
```

```
11
```

In [7]:
```
# lower case
s = "Python Programming"
s=s.lower()
print(s)
```

```
python programming
```

In [10]:
```
# concat
s1="Data"
s2="Science"
s = s1 + " " + s2
print(s)
```

```
Data Science
```

In [12]:
```
#remove trailing spaces
s = "    Data Analysis    "
print(s)
s = s.strip()
print(s)
```

```
   data analysis
data analysis
```

In [20]:
```
#odd string elements
s ="Data Science"
print(s[1::2])
```

```
aaSine
```

In [3]:
```
#check if palindrom or not
s = input("Enter String : ")
rs = s[::-1]
if (s == rs ):
    print("Palindrome")
else :
    print("Not Palindrome")
```

```
Enter String : amma
Palindrome
```

In [4]:
```
#string reverse
s="Hello"
print(s[::-1])
```

```
olleH
```

In [5]:
```
# the seperator and joiner
s = "Hello world! I am Tired saying this same word"
a = s.split()
sym = input("enter joiner : ")
print(s)
print(a)
print(sym.join(a))
```

```
enter joiner : -
Hello world! I am Tired saying this same word
['Hello', 'world!', 'I', 'am', 'Tired', 'saying', 'this', 'same', 'word']
Hello-world!-I-am-Tired-saying-this-same-word
```

In [7]:
```
# input words then reverse words order then join by hypen
s = input("Enter the words : ")
a = s.split()
a = a[::-1]
a = '-'.join(a)
print(a)
```

```
Enter the words : Hello world! I am Tired saying this same word
word-same-this-saying-Tired-am-I-world!-Hello
```

In [ ]:
```

```

Department of Artificial Intelligence (AI) & Data Science

AY: 2026-27

Course: Python for Data Science Laboratory (DJS23SLPC303)
Class: SE/A11/A12/A21/A22 Sem: III Roll No.: SAPID:

Experiment No 1: Decision-making and Control Statements.

Aim: To Implement python programs to demonstrate different decision-making and Control Statements.

Theory

1. What is the difference between if, if…else, and if…elif…else in Python? Give examples.
2. Explain nested if statements with a real-life example.
3. How does Python determine equality between two strings?
4. Why is indentation important in Python conditional statements? What happens if it is incorrect?
5. Differentiate between string **concatenation** and **repetition** in Python with examples.

Conclusion

Name :- Aaryan N. Raorane Roll no :- 5004 branch :- AI & DS
batch :- A1 SAPID :- 60018250070

Experiment No 2

Title :- Decision - making and control statements

Aim :- To Implement python programs to demonstrate different decision making and control statements.

Theory :-

1. What is the difference between if, if.... else, and if... elf in Python? Give Examples.

⇒ **if statement :-**

The if statement is the simplest form of decision making in python. The block of code inside it executes only when the given condition evaluates to True. If the condition is False, the block is skipped Entirely and the program moves on.

code :-
```
marks = 75
if marks >= 40:
    print("student has passed")

# output : student has passed.
```

if.... else statement :-

The if.... else construct provides exactly two execution paths. If the condition is True, the if block runs. It is

Sundaram
FOR EDUCATIONAL USE

false, the else block runs. One of the two block is guaranteed to execute.

code:
```
marks = 30

if marks >= 40:
    print("Student has passed")
else:
    print("student has failed")

# output: student has failed.
```

if.... elif..... else statement

when there are more than two possible condition to check, the elif (short for 'else if') clause is used. Python evaluates each condition from top to bottom and executes the first block whose condition is True all remaining conditions are skipped. The else block at the end acts as a default case when no condition above it is satisfied.

code:
```
marks = 82
if marks >= 90:
    print("grade: O")
elif marks >= 75:
    print("grade: A")
elif marks >= 60:
    print("grade: B")
```

Sundaram
FOR EDUCATIONAL USE

```
elif marks >= 40:
    print("Grade: C")
else:
    print("Grade: F")

# output: Grade: A
```

The Key difference among the three is the number of conditions handled. if handles one condition, if...else handles two mutually exclusive outcomes, and if.... elif.... else handles multiple conditions in a single, clean structure.

2. Explain nested if statements with a real-life example.

=> A nested if statement is an if block placed inside another if block. Python first evaluates the outer condition. only if the outer condition is true does it enter that block and then evaluate the inner condition. This allows multiple levels of decision-making to be chained together.

Syntax:
```
if condition1:
    if condition2:
        # executes only when both condition1 and 2 are
          True
    else:
        # executes when condition1 is true but condition2
          is False
else:
    # Executes when condition1 is false
```

Sundaram
FOR EDUCATIONAL USE

Example code:

"""A bank approves a loan only if the applicant is above 18 years of age AND has a credit score above 700. If the applicant is underage, the loan is rejected outright without even checking the credit score"""

```
age = 22
credit_score = 750

if age >= 18:
    if credit_score >= 700:
        print("Loan approved.")
    else:
        print("Loan rejected: Low credit score.")
else:
    print("Loan rejected: Applicant is underage")
```

If age = 15, the output would be "Loan rejected: Applicant is underage" without the inner condition ever being checked. This is exactly how nested conditionals behave; the inner block is entirely unreachable when the outer condition fails.

Sundaram®
FOR EDUCATIONAL USE

3. How does Python determine equality between two strings?

→ Python compares two strings using the == operator, which performs a character-by-character comparison based on Unicode code points. Two strings are considered equal if and only if they have the same length and every character at each corresponding index is identical. The comparison is case-sensitive, meaning "Python" and "python" are not equal.

Code:
```
s1 = "Python"
s2 = "Python"
s3 = "python"

print(s1==s2)  #output: True
print(s1==s3)  #output: False.
```

The == operator checks value equality (same content), while is checks identity (whether both variables point to the same object in memory). Two string variables can hold identical content but still fail an is comparison if they are stored at different memory locations.

Code:
```
a = "Hello"
b = a
c = "Hello"
print(a==b)  # True
print(a is b)  # True
print(a is c)  # False
```

Sundaram® FOR EDUCATIONAL USE

4. Why is Indentation important in Python conditional statements? What happens if it is incorrect?

→ Unlike languages such as C, C++, and Java which use curly braces {} to define code blocks, Python uses indentation (whitespaces at the beginning of a line) to indicate which statements belong to a particular block. The Python interpreter treats indentation as part of the language syntax, not just a formatting convention.

All statements inside the same block must be indented by the same ~~block must~~ number of spaces. The standard convention is 4 spaces per indentation level.

Code:

```
x = 10
if x > 5:
    print("x is greater than 5")
    print("This also belongs to the if block")
print("This is outside the if block")

# output:
# x is greater than 5
# This also belongs to the if block
# This is outside the if block
```

If Indentation is missing or inconsistent, Python raises an IndentationError at parse time and the program does not run at all

Sundaram
FOR EDUCATIONAL USE

code: if a>5:
print("missing indentation")
# IndentationError: expected an indented block

d More dangerous situation is logically incorrect indentation, where the code runs without error but produces wrong output. This is harder to catch

code: a = 3

if a>5:
print("a is greater than 5")
print("this should be inside but is not")

The second print runs regardless of the condition.
output: This should be inside but is not.

In this case, the second print was intended to be part of the if block but was placed at the outer level by mistake, causing it to execute unconditionally. No error is raised, making such bugs difficult to detect.

Sundaram
FOR EDUCATIONAL USE

5. Differentiate between string concatenation and repetition in Python with examples.

→ String concatenation :-

Concatenation is the process of joining two or more strings end to end into a single string. In Python, this is done using the + operator. Both operands must be of type str; concatenating a string with an integer directly raises a TypeError.

code :-
```
first = "Data"
second = "Science"

result = first + " " + second
print (result) # output: Data Science
print(type(result)) # output: <class 'str'>

Date = "30th" + " " + "July"
print (Date) # output: 30th July.
```

If one operand is not a string, it must be explicitly converted using str() before concatenation

code :-
```
course = "Semester :"
num = 3
print(course + str(num)) # output: Semester: 3
```

Sundaram®
FOR EDUCATIONAL USE

String Repetition :-

Repetition produces a new string by repeating the original string a specified number of times. In Python, this is done using the * operator between a string and an integer

code :-
```
word = "Ha"
print(word * 3)  # output: HaHaHa

separator = "=" * 20
print(separator) #output: ====================

line="Python!"
print(line * 2)  #output: Python! Python!
```

Key difference :-

Concatenation (+) joins distinct strings together, while repetition (*) multiplies a single string. Concatenation requires atleast two string operands, whereas repetition requires one string and one integer operand. Mixing up the two operations or their types will result in a TypeError at runtime.

Sundaram®
FOR EDUCATIONAL USE

Conclusion :-

Through this experiment, the decision-making constructs in Python namely if, if---else, if---elif--else and nested if statements were studied and implemented with examples. The role of indentation as a syntactic rule in Python was understood along with the errors caused by incorrect indentation. String equality using the == operator and the difference between string concatenation and repetition were also demonstrated practically.

Sundaram
FOR EDUCATIONAL USE

Strip Method

In [4]:
```
s="          abcabcabc      "
print(s.strip())
s="aaaaaaaabcabcabcaaaa"
print(s.strip('a'))
s="Hello world"
print(s.strip('ldoH'))
print(s.strip('ldoh'))
```

```
abcabcabc
bcabcabc
ello wor
Hello wor
```

Lstrip and Rstrip Method

Removes only leading and trailing whitespaces by default respectively

In [7]:
```
s="         Hello world      "
print(s.lstrip())
print(s.rstrip())
```

```
Hello world
         Hello world
```

In [9]:
```
"  solving any problem is an art.  ".strip('. ')
```

Out[9]:
```
'solving any problem is an art'
```

In [11]:
```
"  solving any problem is an art.  ".strip(' ')
```

Out[11]:
```
'solving any problem is an art.'
```

In [18]:
```
str="I am Jaspreet      ."
str = str.rstrip('I am.')
str
```

Out[18]:
```
'I am Jaspreet'
```

In [20]:
```
str="I am Jaspreet      ."
str = str.lstrip('I am.')
str
```

Out[20]:
```
'Jaspreet      .'
```

In [23]:
```
sentence ="My name is Jaspreet  ".rstrip("my ")
sentence
```

Out[23]:
```
'My name is Jaspreet'
```

Split method

syntax : string.split(separator,maxsplit)

In [25]:
```
"Hell$I$am$Jaspreet".split('$',maxsplit=2)
```

Out[25]:
```
['Hell', 'I', 'am$Jaspreet']
```

In [27]:
```
"Hell$I$am$Jaspreet".rsplit('$',maxsplit=2)
```

Out[27]:
```
['Hell$I', 'am', 'Jaspreet']
```

In [40]:
```
"Hello world! I am tired saying this sentence again and again".split()
```

Out[40]:
```
['Hello',
 'world!',
 'I',
 'am',
 'tired',
 'saying',
 'this',
 'sentence',
 'again',
 'and',
 'again']
```

Join method

Syntax : separator.join(iterator)

In [42]:
```
s = "Hello world! I am tired saying this sentence again and again".rsplit()
print(s)
s='    '.join(s)
print(s)
```

```
['Hello', 'world!', 'I', 'am', 'tired', 'saying', 'this', 'sentence', 'again', 'a
nd', 'again']
Hello    world!    I    am    tired    saying    this    sentence    again    and
again
```

In [43]:
```
s=s.split()
print(s)
s='_@_'.join(s)
print(s)
```

```
['Hello', 'world!', 'I', 'am', 'tired', 'saying', 'this', 'sentence', 'again', 'a
nd', 'again']
Hello_@_world!_@_I_@_am_@_tired_@_saying_@_this_@_sentence_@_again_@_and_@_again
```

In [44]:
```
s='aBc'
print(s.upper())
print(s.lower())
print(s.capitalize())
```

```
ABC
abc
Abc
```

In [51]:
```python
s='Python is Fun'
print(s.replace('Py','Kr'))
print(s.count('n'))
print(len(s))
print(s.find('is'))
```

```
Krthon is Fun
2
13
7
```

Control Statements Loop

In [1]:
```
#range
range(0,10)
```

Out[1]:
```
range(0, 10)
```

In [3]:
```
a=list(range(10))
print(a)
```

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

In [4]:
```
a=list(range(0,10))
print(a)
a=list(range(1,10,2))
print(a)
```

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
[1, 3, 5, 7, 9]
```

In [5]:
```
print(range(0,20))
print(list(range(0,20)))
print(list(range(0,20,3)))
```

```
range(0, 20)
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19]
[0, 3, 6, 9, 12, 15, 18]
```

for loop

In [7]:
```
#syntax : for Val in sequence :
numbers = [6,5,3,8,4,2,5,4,11]
sum=0
for val in numbers:
    sum = sum + val
print("The sum is ",sum)
```

```
The sum is  48
```

for loop with else

In [12]:
```
list_of_digit = [0,1,2,3,4,5,6]
input_digit = int(input("Enter a digit : "))
for i in list_of_digit :
    if (i == input_digit ):
        print("Digit is in the list")
        break
else :
        print("Digit not found")
```

```
Enter a digit : 55
Digit not found
```

In [13]:
```
list_of_digit = [0,1,2,3,4,5,6]
input_digit = int(input("Enter a digit : "))
for i in list_of_digit :
    if (i == input_digit ):
        print("Digit is in the list")
else :
        print("Digit not found")
```

```
Enter a digit : 2
Digit is in the list
Digit not found
```

In [14]:
```
sum = 0
for i in range(11):
    sum += i
print(sum)
```

```
55
```

In [17]:
```
n1 = 5
n2 = 5
for i in range(n1):
    for j in range(n2):
        print("*",end=" ")
    print()
```

```
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
```

In [18]:
```
n1 = 5
n2 = 5
for i in range(n1):
    for j in range(n2):
        print("* ",end=" ")
    print()
```

```
*  *  *  *  *
*  *  *  *  *
*  *  *  *  *
*  *  *  *  *
*  *  *  *  *
```

In [20]:
```
n1 = 5
n2 = 5
for i in range(n1):
    for j in range(i,n1):
        print("*",end=" ")
    print()
```

```
* * * * *
* * * *
* * *
* *
*
```

In [25]:
```
n1 = 5
n2 = 5
for i in range(n1):
```

```
    for j in range(i):
        print("*",end=" ")
    print()
```

```
*
* *
* * *
* * * *
```

In [26]:
```
n1 = 5
n2 = 5
for i in range(n1):
    for j in range(i+1):
        print("*",end=" ")
    print()
```

```
*
* *
* * *
* * * *
* * * * *
```

In [29]:
```
n1 = 5
n2 = 5
for i in range(n1):
    for j in range(i+1):
        print("*",end="")
    print()
```

```
*
**
***
****
*****
```

In [30]:
```
n1 = 5
n2 = 5
for i in range(n1):
    for j in range(i,n1):
        print(" ",end="")
    for k in range(i+1):
        print("*",end="")
    print()
```

```
    *
   **
  ***
 ****
*****
```

In [32]:
```
n1 = 5
n2 = 5
for i in range(n1):
    for j in range(i+1):
        print("*",end="")
    print()

for i in range(n1):
    for j in range(i,n1):
        print("*",end="")
    print()
for i in range(n1):
```

```
    for j in range(i,n1):
        print(" ",end="")
    for k in range(i+1):
        print("*",end="")
    print()
```

```
*
**
***
****
*****
*****
****
***
**
*
     *
    **
   ***
  ****
 *****
```

While Loop

In [33]:
```
n = int(input("Enter n : "))
i=1
sum = 0
while i<=n:
    sum += i
    i +=1
print(sum)
```

```
Enter n : 11
66
```

In [34]:
```
counter = 0
while counter < 3:
    print("Inside Loop")
    counter += 1
else :
    print("Inside Else")
```

```
Inside Loop
Inside Loop
Inside Loop
Inside Else
```

In [1]:
```
n = int(input("Enter a number : "))
i = 2
count = 0

while (i < n):
    if (n % i == 0):
        count += 1
        break  # Move break inside the if-condition to stop immediately if a div
    i += 1     # Increment i at the while-loop level, not inside the if-statemen

if (count == 0 and n > 1):  # A prime number has 0 divisors between 2 and n-1, a
    print("Prime")
```

```
else:
    print("Not prime")
```

```
Enter a number : 7
Prime
```

In [3]:
```
for i in range (1,51):
    if(i%2!=0):
        continue
    if(i%2==0):
        print(i,end=" ")
```

```
2 4 6 8 10 12 14 16 18 20 22 24 26 28 30 32 34 36 38 40 42 44 46 48 50
```

In [2]:
```
c=0
while c<10:
    if c==3:
        print("using continue stateemnt : skipping iteration when count is : ",c
        c=c+1
        continue
    if c==6:
        print("using pass statement : count is : ",c)
        c=c+1
        pass
    if c==8:
        print("using break statement : count is : ",c)
        c=c+1
        break
    print("Current count is : ",c)
    c=c+1
```

```
Current count is :  0
Current count is :  1
Current count is :  2
using continue stateemnt : skipping iteration when count is :  3
Current count is :  4
Current count is :  5
using pass statement : count is :  6
Current count is :  7
using break statement : count is :  8
```

In [4]:
```
for i in range(1,21,2):
    print(i)
```

```
1
3
5
7
9
11
13
15
17
19
```

In [6]:
```
for i in range(6):
    print(i)
```

```
0
1
2
3
4
5
```

In [9]:
```python
# 1 to 10
for i in range(1,11):
    print(i,end=" ")
```

```
1 2 3 4 5 6 7 8 9 10
```

In [14]:
```python
# 10 to 1
for i in range(10,0,-1):
    print(i,end=" ")
```

```
10 9 8 7 6 5 4 3 2 1
```

In [17]:
```python
# table of
n = int(input("Enter n: "))
for i in range(1,11):
    print(f"{n} X {i} = {n*i}")
```

```
Enter n: 5
5 X 1 = 5
5 X 2 = 10
5 X 3 = 15
5 X 4 = 20
5 X 5 = 25
5 X 6 = 30
5 X 7 = 35
5 X 8 = 40
5 X 9 = 45
5 X 10 = 50
```

In [20]:
```python
# factorial of a num
n = int(input("Enter n : "))
fact = 1
for i in range(1,n+1):
    fact = fact * i
print(fact)
```

```
Enter n : 5
120
```

In [31]:
```python
# sum of digits
n = int(input("Enter n : "))
sum =0
while(n!=0) :
    sum = sum + int(n%10)
    n=int(n/10)
print(sum)
```

```
Enter n : 123
6
```

In [1]:
```python
# Reverse of a number
n = int(input("Enter n : "))
rn = 0

while n != 0:
```

```
    rn = (rn * 10) + (n % 10)
    n = n // 10

print(rn)
```

```
Enter n : 445
544
```

In [4]:
```
#palindrome
# Reverse of a number
n = int(input("Enter n : "))
n1 = n
rn = 0

while n != 0:
    rn = (rn * 10) + (n % 10)
    n = n // 10  # Integer division to reduce n

if (rn == n1):
    print("palindrome")
else :
    print("not palindrome")
```

```
Enter n : 252
palindrome
```

In [9]:
```
# fibonacci
a = 0
b = 1
n = int(input("enter no of elements in series u want : "))
for i in range (n):
    print(f"{a}",end=" ")
    a,b = b,a+b
```

```
enter no of elements in series u want : 6
0 1 1 2 3 5
```

In [17]:
```
#traverse string

s = input("Enter a string")

for i in range(len(s)):
    print(f"{i} -> {s[i]}")
```

```
Enter a stringraphael
0 -> r
1 -> a
2 -> p
3 -> h
4 -> a
5 -> e
6 -> l
```

In [20]:
```
#pattern 1
for i in range(1,6):
    for k in range(6-i):
        print(" ",end=" ")
    for j in range(i):
        print("*",end=" ")
    print()
```

```
    *
   * *
  * * *
 * * * *
* * * * *
```

In [25]:
```
#pattern 2
for i in range(5):
    for j in range(1,6):
        print(j,end="")
    print()
```

```
12345
12345
12345
12345
12345
```

In [29]:
```
#pattern 3
for i in range(1,6):
    for j in range(1,i+1):
        print(j,end="")
    print()
```

```
1
12
123
1234
12345
```

In [33]:
```
#patttern 4
k=65
for i in range(1,6):
    for j in range(1,i+1):
        print(chr(k),end="")
        k+=1
    print()
```

```
A
BC
DEF
GHIJ
KLMNO
```

In [34]:
```
#patttern 5
k=65
for i in range(1,6):
    for j in range(1,i+1):
        print(chr(k),end="")
    k+=1
    print()
```

```
A
BB
CCC
DDDD
EEEEE
```

Department of AI_DS
Academic Year: 2026-27 (Odd Sem)
Course: Python for Data Science Lab
Second Year B. Tech Semester: III

PRACTICAL 3

Part A (To be referred by students)

Looping & Strings

SAVE THE FILE AND UPLOAD AS (RollNo_Name_Exp3)

Aim: Write a program to demonstrate Looping Statements (while, for, nested loops, break, continue, else with loop), Pass and string functions.

Problem Statement: Write Python program to

1. WAP to print the sum of the first n natural numbers using a for loop
2. Count the number of even and odd numbers from a series of numbers P to Q using a for loop.
3. Write a program to check whether the entered number is prime or not (make use of break)
4. Display the Fibonacci sequence up to the nth term, where n is provided by the user (use a while loop).
5. Print the following pattern using a nested loop.

```
  *
 ***
*****
```

6. WAP to print the following pattern

```
*****        1          *
****         12         **
***          123        ***
**           1234       **
*            12345      *
```

7. Write a Python program that iterates the integers from 1 to 50. For multiples of three print "Fizz" instead of the number and for multiples of five print "Buzz". For numbers that are multiples of three and five, print "FizzBuzz".
   Sample Output :
   fizzbuzz
   1
   2
   fizz
   4
   buzz
8. Demonstrate 5 string operations using string function
9. Demonstrate using the continue statement in a loop
10. Demonstrate use of Pass in loop and else with loop

Department of AI_DS
Academic Year: 2026-27 (Odd Sem)
Course: Python for Data Science Lab
Second Year B. Tech Semester: III

Topic covered: Loops (for, while) else with loop, pass & string functions

Learning Objective: Learner would be able to

1. Analyze the scenario to write script applying looping.
2. Solve the problems using looping statements
3. Understand and use pass, else, continue & break
4. Process string and perform various operations on string

Theory:

- **The for loop:**

  The for loop in Python is used to iterate over a sequence (list, tuple, string) or other iterable objects. With for loop we can execute a set of statements, once for each item in a list, tuple, set etc.

Syntax of for Loop

```
for val in sequence:
    loop body
```

Here, val is the variable that takes the value of the item inside the sequence on each iteration.

Loop continues until we reach the last item in the sequence. The body of for loop is separated from the rest of the code using indentation.

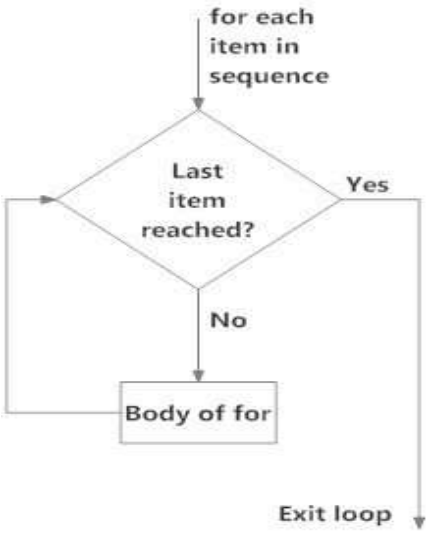


Fig: operation of for loop

The for loop does not require an indexing variable to set beforehand, as the for command itself allows for this.

```
# Print each fruit in a fruit list:
fruits = ["apple", "banana", "cherry"]
for x in fruits:
```

```
print(x)
```

Output:

apple

banana

cherry

- **The while loop:**

  The for loop in Python is used to iterate over a sequence (list, tuple, string) or other iterable objects. With for loop we can execute a set of statements, once for each item in a list, tuple, set etc.

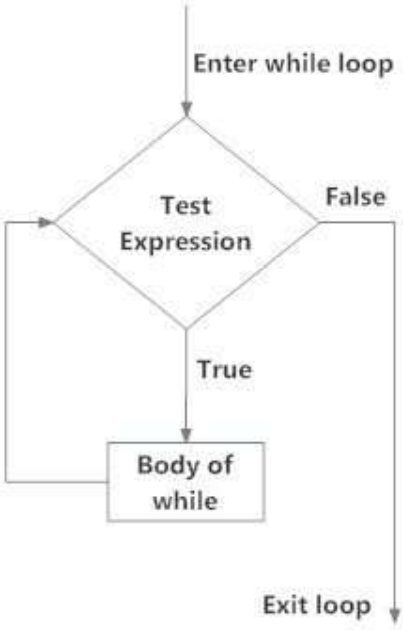
Syntax of while Loop	
while test_expression: **loop body**	
In the while loop, test expression is checked first. The body of the loop is entered only if the test_expression evaluates to True. The body starts with indentation and the first unindented line marks the end. Python interprets any non-zero value as True. None and 0 are interpreted as False.	

Fig: operation of while loop

```
# Program to add natural numbers up to sum = 1+2+3+...+n
n = 10
# initialize sum and counter
sum = 0
i = 1
while i <= n:
    sum = sum + i
    i = i+1    # update counter
# print the sum
print("The sum is", sum)
```

Output:

```
The sum is 55
```

- **The range() function:**

  To loop through a set of code a specified number of times, we can use the range() function, The range() function returns a sequence of numbers, starting from 0 by default, and increments by 1 (by default), and ends at a specified number.

  We can also define the start, stop and step size as range(start, stop,step_size). step_size defaults to 1 if not provided.

```
# range(n) prints numbers 0 to n-1
# Outputs 0 to 5
for x in range(6):
    print(x)
# Outputs 2 to 5
for x in range(2, 6):
    print(x)
# Outputs 2, 5, 8, 11, 14, 17
for x in range(2, 20, 3):
    print(x)
```

- **The break Statement:**

  The break statement terminates the loop containing it. Control of the program flows to the statement immediately after the body of the loop.

  If the break statement is inside a nested loop (loop inside another loop), the break statement will terminate the innermost loop.

```
# Exit the loop when x is banana
fruits = ["apple", "banana", "cherry"]
for x in fruits:
    if x == "banana":
        break
    print(x)
```

Output:

apple

- **The continue Statement:**

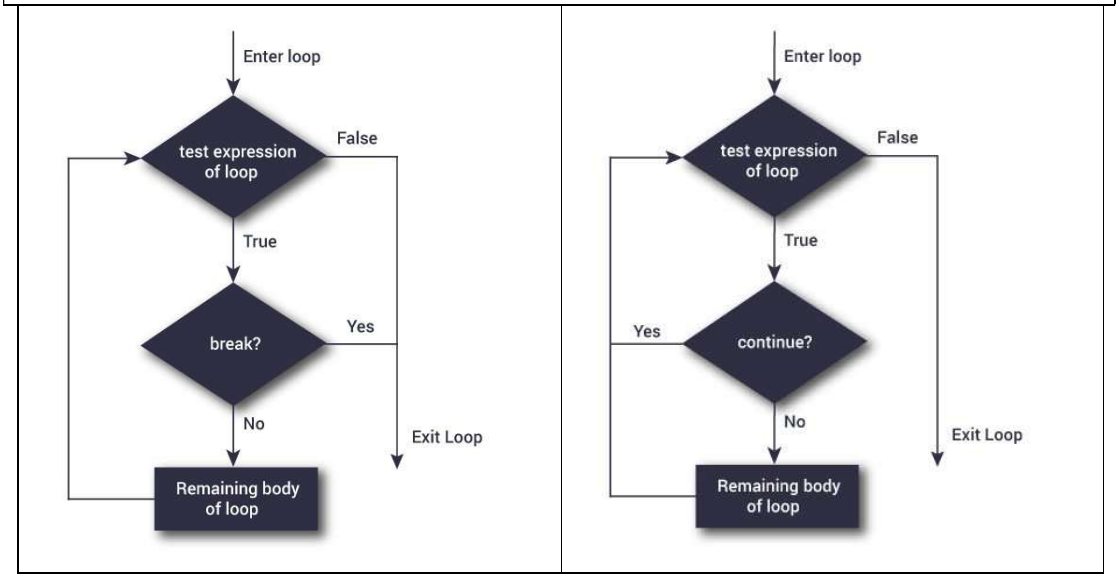
  The continue statement is used to skip the rest of the code inside a loop for the current iteration only. Loop does not terminate but continues on with the next iteration.

```
# Exit the loop when x is banana
fruits = ["apple", "banana", "cherry"]
for x in fruits:
    if x == "banana":
        continue
    print(x)
```

Output:

apple

cherry



- **Loop with else (use with for and while loop):**

A for loop can have an optional else block as well. The else part is executed if the items in the sequence used in for loop exhausts.

The break keyword can be used to stop a for loop. In such cases, the else part is ignored. Hence, a for loop's else part runs if no break occurs.

```
digits = [0, 1, 5]
for i in digits:
    print(i)
else:
    print("No items left.")
```

Output:

0

1

5

No items left.

Strings….

- **Python Strings are immutable**

  In Python, strings are immutable. That means the characters of a string cannot be changed.

```
message = 'Hola Amigos'
message[0] = 'H'
print(message)
```

Output

TypeError: 'str' object does not support item assignment

However, we can assign the variable name to a new string.

```
message = 'Hola Amigos'
# assign new string to message variable
message = 'Hello Friends'
prints(message); # prints "Hello Friends"
```

- **Python Multiline String**

  We can also create a multiline string in Python. For this, we use triple double quotes """ or triple single quotes '''.

```
# multiline string
message = """
Never gonna give you up
Never gonna let you down
"""
print(message)
```

Output

```
Never gonna give you up
Never gonna let you down
```

- **String Operations:**

  There are many operations that can be performed with strings which makes it one of the most used data types in Python.

  1. Compare Two Strings

  We use the == operator to compare two strings. If two strings are equal, the operator returns True. Otherwise, it returns False.

```
str1 = "Hello, world!"
```

```
str2 = "I love Python."
str3 = "Hello, world!"
# compare str1 and str2
print(str1 == str2)
# compare str1 and str3
print(str1 == str3)
```

Output

```
False
True
```

In the above example,

str1 and str2 are not equal. Hence, the result is False.

str1 and str3 are equal. Hence, the result is True.

2. Join Two or More Strings

In Python, we can join (concatenate) two or more strings using the + operator.

```
greet = "Hello, "
name = "Jack"

# using + operator
result = greet + name
print(result)
```

Output

```
Hello, Jack
```

Run Code

3. Iterate Through a Python String

We can iterate through a string using a for loop.

```
greet = 'Hello'
# iterating through greet string
for letter in greet:
    print(letter)
```

Output

```
H
e
l
l
o
```

5. Python String Length

In Python, we use the len() method to find the length of a string.

```
greet = 'Hello'
# count length of greet string
print(len(greet))
```

Output

```
5
```

6. String Membership Test

We can test if a substring exists within a string or not, using the keyword in.

```
print('a' in 'program') # True
print('at' not in 'battle') False
```

7. Methods of Python String

Besides those mentioned above, there are various string methods present in Python. Here are some of those methods:

Methods	Description
upper()	converts the string to uppercase
lower()	converts the string to lowercase
partition()	returns a tuple
replace()	replaces substring inside
find()	returns the index of first occurrence of substring
rstrip()	removes trailing characters
split()	splits string from left
startswith()	checks if string starts with the specified string
isnumeric()	checks numeric characters
index()	returns index of substring

Department of AI_DS
Academic Year: 2026-27 (Odd Sem)
Course: Python for Data Science Lab
Second Year B. Tech Semester: III

PRACTICAL 3

Part B (to be completed by students)

Looping and Strigs

1. All the students are required to perform the given tasks in Jupyter Notebook
2. Create a new notebook for each experiment. The filename should be RollNo_Name_Exp3)
3. In the first cell, the student must write his/her Name, roll no and class in the form of comments
4. Every program should be written in separate cells and in the given sequence
5. After completing the experiment, download the notebook in pdf format. The filename should be RollNo_Name_Exp3.pdf).
6. Upload the pdf on the web portal

In [3]:
```
#sum of 1st n natural numbers
sum = 0
n=int(input("Enter no of elements : "))
for i in range(1,n+1):
    sum+=i
print(sum)
```

```
Enter no of elements : 10
55
```

In [2]:
```
#count number of even and odd numbers in the given series
p=int(input("enter p :"))
q=int(input("enter q :"))
even = 0
odd = 0
for i in range(p,q+1):
    if(i%2 == 0):
        even+=1
    else:
        odd+=1

print(f"Total Even numbers : {even}")
print(f"Total Odd numbers : {odd}")
```

```
enter p :51
enter q :60
Total Even numbers : 5
Total Odd numbers : 5
```

In [3]:
```
#prime checker
n = int(input("Enter a number : "))
i = 2
count = 0

while (i < n):
    if (n % i == 0):
        count += 1
        break
    i += 1

if (count == 0 and n > 1):
    print("Prime")
else:
    print("Not prime")
```

```
Enter no : 4
```

In [7]:
```
#fibonnaci
n = int(input("Enter the last term u want in fibonacci series : "))
a=0
b=1
while(a <= n):
    print(a,end=" ")
    a,b=b,a+b
```

```
Enter the last term u want in fibonacci series : 13
0 1 1 2 3 5 8 13
```

In [27]:
```
#nested pattern loop
for i in range(1,6):
```

```
    for j in range(6-i):
        print(" ",end="")
    for k in range(2*i-1):
        print("*",end="")
    print()
```

```
     *
    ***
   *****
  *******
 *********
```

In [30]:
```
#pattern 1
for i in range(1,6):
    for j in range (6-i):
        print("*",end=" ")
    print()
```

```
* * * * *
* * * *
* * *
* *
*
```

In [32]:
```
#pattern 2
for i in range(1,6):
    for j in range(1,i+1):
        print(j,end="")
    print()
```

```
1
12
123
1234
12345
```

In [38]:
```
#pattern 3
n1 = 3
for i in range(n1):
    for j in range(i+1):
        print("* ",end="")
    print()

for i in range(n1):
    for j in range(i,n1-1):
        print("* ",end="")
    print()
```

```
*
* *
* * *
* *
*
```

In [42]:
```
#fizzbuzz
for i in range(1,51):
    if(i%3==0 and i%5!=0):
        print("Fizz")
    elif(i%3!=0 and i%5==0):
        print("Buzz")
    elif(i%3==0 and i%5==0):
```

```
        print("FizzBuzz")
    else:
        print(i)
```

```
1
2
Fizz
4
Buzz
Fizz
7
8
Fizz
Buzz
11
Fizz
13
14
FizzBuzz
16
17
Fizz
19
Buzz
Fizz
22
23
Fizz
Buzz
26
Fizz
28
29
FizzBuzz
31
32
Fizz
34
Buzz
Fizz
37
38
Fizz
Buzz
41
Fizz
43
44
FizzBuzz
46
47
Fizz
49
Buzz
```

In [43]:
```
#string operations
s = input("enter string sentence: ")
print(s.upper())
print(s.lower())
print(s.capitalize())
```

```
s=s.split()
print(s)
s='_@_'.join(s)
print(s)
```

```
enter string sentence: Hello world i am python not language snake
HELLO WORLD I AM PYTHON NOT LANGUAGE SNAKE
hello world i am python not language snake
Hello world i am python not language snake
['Hello', 'world', 'i', 'am', 'python', 'not', 'language', 'snake']
Hello_@_world_@_i_@_am_@_python_@_not_@_language_@_snake
```

In [46]:
```
#continue
for i in range(10):
    if(i%2==0):
        continue
    print(i)
```

```
1
3
5
7
9
```

In [48]:
```
#pass and else
for i in range(11):
    if(i==3):
        pass
    print(i)
else:
    print("Loop complete")
```

```
0
1
2
3
4
5
6
7
8
9
10
Loop complete
```

Department of Artificial Intelligence (AI) & Data Science

AY: 2026-27

Course: Python for Data Science Laboratory (DJS23SLPC303)
Class: SE/A11/A12/A21/A22 Sem: III Roll No.: SAPID:
Experiment No 3: Looping and Strings

Aim: Write a program to demonstrate Looping Statements (while, for, nested loops, break, continue, else with loop), Pass and string functions.

Theory

1. Differentiate between for loop and while loop with examples.
2. Explain the role of the range() function in loops. Give two examples.
3. What is the purpose of the else block in a loop? Write one example using for loop with else.
4. Differentiate between break, continue, and pass with suitable examples.
5. Why are strings called immutable in Python? Explain with an example.

Conclusion

Name:- Aaryan NRaorane Roll No:- 5004 Batch:- A1
branch:- AI&DS SAP ID:- 60018250070

Experiment No 3

Title: Looping and strings

Aim: Write a program to demonstrate looping statements (while, for, nested loops, break, continue, else with loop), lists and string functions

Theory:

1) Differentiate between for loop and while loop with examples.

⇒ The for loop is used when the number of iterations is known in advance. It iterates over a sequence such as a list, string, or range object, and the loop variable takes each value from that sequence automatically. The while loop is used when the number of iterations depends on a condition. The loop body executes as long as the condition remains true, and the programmer must manually update the condition variable to avoid infinite execution.

Parameter	for loop	while loop
• Iteration control	over a sequence	Based on a condition
• Number of iterations	Known in advance	May be unknown
• counter update	Automatic	Manual
• Risk of infinite loop	Lower	Higher if condition not updated.

Sundaram
FOR EDUCATIONAL USE

Example of for loop:

```
code: for i in range(1, 6):
          print("Iteration:", i)
```

output:
```
Iteration : 1
Iteration : 2
Iteration : 3
Iteration : 4
Iteration : 5
```

Example of while loop:

```
code: i = 1
      while i <= 5
          print("Iteration:", i)
          i += 1
```

output:
```
Iteration : 1
Iteration : 2
Iteration : 3
Iteration : 4
Iteration : 5
```

Sundaram
FOR EDUCATIONAL USE

2) Explain the role of the range() function in loops. Give two examples.

=> The range() function generates a sequence of integers and is commonly used with for loops to control the number of iterations. It accepts up to three arguments: start, stop, and step. The start value defaults to 0 and the step defaults to 1. The stop value is exclusive, meaning the sequence ends one integer before it.

Syntax:

```
range(stop)
range(start, stop)
range(start, stop, step)
```

Example 1:

~~for code:~~

```
for i in range(1,6)
    print("Iteration:", i)
```

output:

```
Iteration : 1
Iteration : 2
Iteration : 3
Iteration : 4
Iteration : 5
```

Sundaram®
FOR EDUCATIONAL USE

Example 2: ~~Print~~

code:- for i in range(1,10,2)
print(i)

output:- 1 3 5 7 9

3) what is the purpose of the else block in a loop? Write one example using for loop with else.

=> The else block in a loop executes after the loop completes all its iterations normally. If the loop is terminated early by a break statement, the else block is skipped entirely. This makes it useful for detecting whether a loop ran to completion without interruption, such as in search operations where a break is used when the target is found.

code:- for i in range(1,6):
print(i)
else:
print("Loop completed without break")

output:- 1
2
3
4
5
Loop completed without break.

since no break was encountered, the else block is executed.

Sundaram FOR EDUCATIONAL USE

4) Differentiate between break, continue, and pass with suitable examples.

=>

Keyword	Effect on loop	common use case
break	Exits the loop actually	stop execution when condition is met
continue	skips the current iteration	Bypass specific value in sequence
pass	no effect; loop proceeds normally	Placeholder where syntax requires a statement

Example of break:

```
for i in range(1, 10):
    if i == 5:
        break
    print(i, end=" ")
```

output: 1 2 3 4

The loop exits as soon as i reaches 5

Example of continue:

```
for i in range(1, 6):
    if i == 3:
        continue
    print(i, end=" ")
```

output: 1 2 4 5

The value 3 is skipped, all other iterations proceed normally.

Sundaram
FOR EDUCATIONAL USE

Example of pass:

```
for j in range(1,6):
    if j == 3:
        pass
    print(j, end=" ")
```

output: 1 2 3 4 5

5) Why are strings called immutable in Python? Explain with an example.

=> Strings in Python are called immutable because once a string object is created in memory, its individual characters cannot be modified in place. Any operation that appears to change a string doesn't alter the original object. Instead, Python creates a new string object with the modified content and assigns it to the variable. The original object remains unchanged.

This can be verified using the id() function, which returns the memory address of an object. After reassignment, the address changes, confirming that a new object was created.

code:

```
s = "hello"
print("Before:", id(s))

s = s + "world"
print("After:", id(s))
```

Sundaram
FOR EDUCATIONAL USE

output: Before: 140245678401234
After: 140245 67890 1235

The two different id values confirm that ~~assignment~~ reassignment created a new object rather than modifying the original string.

conclusion:-

This Experiment covered the core looping constructs in Python including for, while, nested loops, and loop control statement such as break, continue and pass. The behaviour of the else block with loops and the use of the range() function were also observed through code execution. String immutability was verified by attempting direct character modification and comparing object identifies using id().

Sundaram®
FOR EDUCATIONAL USE

List,Tuple,Dictionary

1. List

In [1]:
```
thisList = ["apple","banana","Cherry","dragonfruit"]
print(thisList)
```

In [2]:
```
thisList = list(("apple","banana","Cherry","dragonfruit"))
print(thisList)
```

```
['apple', 'banana', 'Cherry', 'dragonfruit']
```

List allows duplicate,it is changeable and ordered(like array)

In [3]:
```
print(type(thisList))
```

```
<class 'list'>
```

1. Len

In [4]:
```
print(len(thisList))
```

```
4
```

In [5]:
```
list1 = ["abc",34,True,40,'z']
print(list1)
print(type(list1))
```

```
['abc', 34, True, 40, 'z']
<class 'list'>
```

The slicing and element callling in list is similar to that of string

In [6]:
```
print("list1[2] = ",list1[2])
```

```
list1[2] =  True
```

In [7]:
```
print("list1[:2] = ",list1[:2])
```

```
list1[:2] =  ['abc', 34]
```

In [8]:
```
print("list1[-1] = ",list1[-1])
```

```
list1[-1] =  z
```

In [9]:
```
fruit = ["apple","banana","cherry","orange","Kiwi","Melon","Mango"]
```

In [10]:
```
print(fruit)
print(fruit[2:5])
print(fruit[:4])
```

```
['apple', 'banana', 'cherry', 'orange', 'Kiwi', 'Melon', 'Mango']
['cherry', 'orange', 'Kiwi']
['apple', 'banana', 'cherry', 'orange']
```

In [11]:
```
print(fruit[-4:-1])
```

```
['orange', 'Kiwi', 'Melon']
```

In [12]:
```
fruit[1]="blackcurrent"
```

In [13]:
```
print(fruit)
```

```
['apple', 'blackcurrent', 'cherry', 'orange', 'Kiwi', 'Melon', 'Mango']
```

In [14]:
```
fruit[2:4]=["chickoo","dragonfruit"]
```

In [15]:
```
print(fruit)
```

```
['apple', 'blackcurrent', 'chickoo', 'dragonfruit', 'Kiwi', 'Melon', 'Mango']
```

In [16]:
```
fruit.append("pineapple")
```

In [17]:
```
print(fruit)
```

```
['apple', 'blackcurrent', 'chickoo', 'dragonfruit', 'Kiwi', 'Melon', 'Mango', 'pi
neapple']
```

In [18]:
```
fruit.insert(3,"watermelon")
```

In [19]:
```
print(fruit)
```

```
['apple', 'blackcurrent', 'chickoo', 'watermelon', 'dragonfruit', 'Kiwi', 'Melo
n', 'Mango', 'pineapple']
```

In [20]:
```
tropical=["papaya","jackfruit","gauva"]
fruit.extend(tropical)
```

In [21]:
```
print(fruit)
```

```
['apple', 'blackcurrent', 'chickoo', 'watermelon', 'dragonfruit', 'Kiwi', 'Melo
n', 'Mango', 'pineapple', 'papaya', 'jackfruit', 'gauva']
```

In [22]:
```
fruittuple=("lichi","grapes")
```

In [23]:
```
fruit.extend(fruittuple)
```

In [24]:
```
print(fruit)
```

```
['apple', 'blackcurrent', 'chickoo', 'watermelon', 'dragonfruit', 'Kiwi', 'Melo
n', 'Mango', 'pineapple', 'papaya', 'jackfruit', 'gauva', 'lichi', 'grapes']
```

In [25]:
```
fruit.remove("papaya")
print(fruit)
```

```
['apple', 'blackcurrent', 'chickoo', 'watermelon', 'dragonfruit', 'Kiwi', 'Melo
n', 'Mango', 'pineapple', 'jackfruit', 'gauva', 'lichi', 'grapes']
```

In [26]:
```
fruit.pop(2)
print(fruit)
```

```
['apple', 'blackcurrent', 'watermelon', 'dragonfruit', 'Kiwi', 'Melon', 'Mango',
'pineapple', 'jackfruit', 'gauva', 'lichi', 'grapes']
```

In [27]:
```
del fruit[5]
print(fruit)
```

```
['apple', 'blackcurrent', 'watermelon', 'dragonfruit', 'Kiwi', 'Mango', 'pineappl
e', 'jackfruit', 'gauva', 'lichi', 'grapes']
```

In [28]:
```
#fruit.clear()
#print(fruit)
```

In [29]:
```
# del fruit
# print(fruit)
```

In [30]:
```
for x in fruit:
    print(x)
```

```
apple
blackcurrent
watermelon
dragonfruit
Kiwi
Mango
pineapple
jackfruit
gauva
lichi
grapes
```

In [31]:
```
print("apple" in fruit)
```

```
True
```

In [32]:
```
fruit.sort()
print(fruit)
```

```
['Kiwi', 'Mango', 'apple', 'blackcurrent', 'dragonfruit', 'gauva', 'grapes', 'jac
kfruit', 'lichi', 'pineapple', 'watermelon']
```

In [33]:
```
fruit.sort(reverse=True)
print(fruit)
```

```
['watermelon', 'pineapple', 'lichi', 'jackfruit', 'grapes', 'gauva', 'dragonfrui
t', 'blackcurrent', 'apple', 'Mango', 'Kiwi']
```

In [34]:
```
!jt -t onedork -T -N
```

In [35]:
```
fruit2 = fruit#updates both list even after altering
print(fruit2)
fruit.append("berry")
print(fruit)
print(fruit2)
```

```
['watermelon', 'pineapple', 'lichi', 'jackfruit', 'grapes', 'gauva', 'dragonfrui
t', 'blackcurrent', 'apple', 'Mango', 'Kiwi']
['watermelon', 'pineapple', 'lichi', 'jackfruit', 'grapes', 'gauva', 'dragonfrui
t', 'blackcurrent', 'apple', 'Mango', 'Kiwi', 'berry']
['watermelon', 'pineapple', 'lichi', 'jackfruit', 'grapes', 'gauva', 'dragonfrui
t', 'blackcurrent', 'apple', 'Mango', 'Kiwi', 'berry']
```

In [36]:
```
fruit2 = fruit.copy()
print(fruit2)#doesnt updates
fruit.append("blueberry")
print(fruit)
print(fruit2)
```

```
['watermelon', 'pineapple', 'lichi', 'jackfruit', 'grapes', 'gauva', 'dragonfrui
t', 'blackcurrent', 'apple', 'Mango', 'Kiwi', 'berry']
['watermelon', 'pineapple', 'lichi', 'jackfruit', 'grapes', 'gauva', 'dragonfrui
t', 'blackcurrent', 'apple', 'Mango', 'Kiwi', 'berry', 'blueberry']
['watermelon', 'pineapple', 'lichi', 'jackfruit', 'grapes', 'gauva', 'dragonfrui
t', 'blackcurrent', 'apple', 'Mango', 'Kiwi', 'berry']
```

In [37]:
```
fruit3 = fruit + fruit2
fruit3.sort()
print(fruit3)
```

```
['Kiwi', 'Kiwi', 'Mango', 'Mango', 'apple', 'apple', 'berry', 'berry', 'blackcurr
ent', 'blackcurrent', 'blueberry', 'dragonfruit', 'dragonfruit', 'gauva', 'gauv
a', 'grapes', 'grapes', 'jackfruit', 'jackfruit', 'lichi', 'lichi', 'pineapple',
'pineapple', 'watermelon', 'watermelon']
```

In [38]:
```
for x in fruit2:
    fruit.append(x)
fruit.sort()
print(fruit)
```

```
['Kiwi', 'Kiwi', 'Mango', 'Mango', 'apple', 'apple', 'berry', 'berry', 'blackcurr
ent', 'blackcurrent', 'blueberry', 'dragonfruit', 'dragonfruit', 'gauva', 'gauv
a', 'grapes', 'grapes', 'jackfruit', 'jackfruit', 'lichi', 'lichi', 'pineapple',
'pineapple', 'watermelon', 'watermelon']
```

In [39]:
```
berries = ["blueberry","strawberry","mulberry"]
berries*3
print(berries)
```

```
['blueberry', 'strawberry', 'mulberry']
```

In [40]:
```
sq = [x**2 for x in range(1,6)]
print(sq)
```

```
[1, 4, 9, 16, 25]
```

In [41]:
```
even = [x for x in range(10) if x%2==0]
print(even)
```

```
[0, 2, 4, 6, 8]
```

In [42]:
```
upper_fruit=[w.upper() for w in fruit]
print(upper_fruit)
```

```
['KIWI', 'KIWI', 'MANGO', 'MANGO', 'APPLE', 'APPLE', 'BERRY', 'BERRY', 'BLACKCURR
ENT', 'BLACKCURRENT', 'BLUEBERRY', 'DRAGONFRUIT', 'DRAGONFRUIT', 'GAUVA', 'GAUV
A', 'GRAPES', 'GRAPES', 'JACKFRUIT', 'JACKFRUIT', 'LICHI', 'LICHI', 'PINEAPPLE',
'PINEAPPLE', 'WATERMELON', 'WATERMELON']
```

nested list

In [43]:
```
a = [[11,12,13],[21,22,23],[31,32,33]]
print(a)
```

```
[[11, 12, 13], [21, 22, 23], [31, 32, 33]]
```

Tuple

tuple is immutable

In [44]:
```
tfruit=("apple","banana","chickoo","dragonfruit","grape","gauva","jackfruit","le
print(tfruit)
print(type(tfruit))
```

```
('apple', 'banana', 'chickoo', 'dragonfruit', 'grape', 'gauva', 'jackfruit', 'lem
on')
<class 'tuple'>
```

indexing in tuple is similar to list

In [45]:
```
print(type(tfruit[0]))
print(tfruit[0])
```

```
<class 'str'>
apple
```

In [46]:
```
print(len(tfruit))
```

```
8
```

In [47]:
```
print(tfruit[1:2])
print(tfruit[:-1])
print(tfruit[-2:])
```

```
('banana',)
('apple', 'banana', 'chickoo', 'dragonfruit', 'grape', 'gauva', 'jackfruit')
('jackfruit', 'lemon')
```

we can update a element in tuple by first convertin it into a list and then do changes and convert back to a tuple

In [48]:
```
x=tfruit
y=list(x)
y[1]="kiwi"
x=tuple(y)
print(x)
```

```
('apple', 'kiwi', 'chickoo', 'dragonfruit', 'grape', 'gauva', 'jackfruit', 'lemo
n')
```

In [49]:
```
t=('JP morgan','20','education')
print(t)
```

```
('JP morgan', '20', 'education')
```

In [50]:
```
(company,empid,profile)=t
print(company)
print(empid)
print(profile)
```

```
JP morgan
20
education
```

In [51]:
```
t1=(1,2,3,4)
a,b,c,d=t1
print(a,b,c,d)
```

```
1 2 3 4
```

In [52]:
```
a,b,c=t1
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
<ipython-input-52-408f20357701> in <module>
----> 1 a,b,c=t1

ValueError: too many values to unpack (expected 3)
```

In [53]:
```
(a,b,c)=t1[0:3]
```

In [54]:
```
print(a,b,c)
```

```
1 2 3
```

In [ ]:

In [9]:
```
# Tuples are Comparable

print((0,1,2) < (5,1,2)) # 0 is less than 5 - True

print((0,1,200) < (0,3,4)) # 0 is equal to 0 --> 1 is less than 3 - True

print(('Jones', 'Sally') < ('Jones', 'Sam')) # Jones is equal to Jones --> Sally

print(('Jones', 'Sally') < ('Adams', 'Sam')) # Jones is not equal to Adams but i
```

```
True
True
True
False
```

In [10]:
```
# Concatenation, Repetition, Index

t1=(1,2,3)
t2=(3,'Priya',4)
t3=t1+t2
print(t3)
print()
print(t1*3)
print()
print(t2.index('Priya'))
```

```
(1, 2, 3, 3, 'Priya', 4)

(1, 2, 3, 1, 2, 3, 1, 2, 3)

1
```

In [11]:
```
l=(1,2,3,1,2,3,1,2,3)
print(2 in l)
```

```
True
```

In [12]:
```
# Iterating through a tuple

for name in ('John', 'Kate'):
    print("Hello", name)
```

```
Hello John
Hello Kate
```

In [13]:
```
fruits=('apple', 'banana', 'mango', 'grapes', 'pineapple')
for x in fruits:
    print(x)
```

```
apple
banana
mango
grapes
pineapple
```

Tuples do not support these methods: append remove insert reverse sort

Set

In [14]:
```
fruits={'apple', 'banana', 'mango', 'grapes', 'pineapple'}
print(fruits)

a={1,3,5,2,4}
print(a)
print(type(a))

# RANDOM ORDER
```

```
{'grapes', 'mango', 'banana', 'pineapple', 'apple'}
{1, 2, 3, 4, 5}
<class 'set'>
```

In [15]:
```
thisSet=set(('apple', 'banana', 'mango', 'grapes', 'pineapple'))
print(thisSet)
print(type(thisSet))
```

```
{'grapes', 'mango', 'banana', 'pineapple', 'apple'}
<class 'set'>
```

In [16]:
```
a={1,2,2,3,3,3,4,4,4,4}
print(a)
```

```
{1, 2, 3, 4}
```

In [17]:
```
a={1,2,3}
print(a[1]) # 'set' object does not support indexing
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
<ipython-input-17-61a87b252975> in <module>
      1 a={1,2,3}
----> 2 print(a[1]) # 'set' object does not support indexing

TypeError: 'set' object does not support indexing
```

In [18]:
```
fruits={'apple', 'banana', 'mango', 'grapes', 'pineapple'}
for x in fruits:
    print(x)
print()
print(len(fruits))

# Once a set is created, you cannot change its item but you can add new items
```

```
grapes
mango
banana
pineapple
apple

5
```

In [19]:
```
fruits={'apple', 'banana', 'mango', 'grapes', 'pineapple'}
fruits.add('orange')
print(fruits)

fruits.update(['cherry', 'blackberry', 'dragonfruit'])
print(fruits)

fruits.discard('strawberry') # if the item does not exist, discard() will not ra
print(fruits)
```

```
fruits.remove('strawberry') # if the item does not exist, remove() will raise an
print(fruits)
```

```
{'orange', 'grapes', 'mango', 'banana', 'pineapple', 'apple'}
{'orange', 'cherry', 'grapes', 'mango', 'banana', 'dragonfruit', 'blackberry', 'p
ineapple', 'apple'}
{'orange', 'cherry', 'grapes', 'mango', 'banana', 'dragonfruit', 'blackberry', 'p
ineapple', 'apple'}
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
<ipython-input-19-93efbfaa3e7d> in <module>
      9 print(fruits)
     10
---> 11 fruits.remove('strawberry') # if the item does not exist, remove() will r
aise an error
     12 print(fruits)

KeyError: 'strawberry'
```

In [20]:
```
fruits={"apple", "banana", "mango", "grapes", "pineapple", "papaya", "guava", "l

print()
fruits.clear()
print(fruits)

del fruits
print(fruits)
```

```
set()
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
<ipython-input-20-2b95bb775634> in <module>
      6
      7 del fruits
----> 8 print(fruits)

NameError: name 'fruits' is not defined
```

In [21]:
```
fruits={"apple", "banana", "mango", "grapes", "pineapple", "papaya", "guava", "l

x=fruits.pop()
print(x)
print(fruits)
```

```
grapes
{'lemon', 'guava', 'mango', 'papaya', 'banana', 'pineapple', 'apple'}
```

In [22]:
```
set1={1,2,3,'b'}
set2={'a', 'b', 'c', 1, 3}
set3=set1.union(set2)
print(set3)

print()

set1.update(set2)
print(set1)

# both union and update will exclude any duplicate values
```

```
{1, 2, 3, 'b', 'c', 'a'}

{1, 2, 3, 'b', 'c', 'a'}
```

In [23]:
```
null_set=set()
print(null_set)

a={1, 2.5, ("pineapple", "pumpkin"), "Jack"}
print(a)

b=set("abcd")
print(b) # {'b', 'a', 'd', 'c'}

c={"abcd"}
print(c) # {'abcd'}

print(len(a))

print(("pineapple", "pumpkin") in a) # True
print(("pineapple") in a) # False

print()

for x in a:
    print(x)
```

```
set()
{1, 2.5, ('pineapple', 'pumpkin'), 'Jack'}
{'d', 'b', 'c', 'a'}
{'abcd'}
4
True
False

1
2.5
('pineapple', 'pumpkin')
Jack
```

In [24]:
```
a_set=set("abcd")
b_set=set("cdef")
print(a_set & b_set) # OR print(a_set.intersection(b_set))

print()

print(a_set - b_set) # OR print(a_set.difference(b_set)) # {'b', 'a'}
print(b_set - a_set) # OR print(b_set.difference(a_set)) # {'e', 'f'}

print()
print(a_set ^ b_set) #{'f', 'e', 'a', 'b'}
# OR b_set.symmetric_difference(a_set) # {'a', 'b', 'e', 'f'}
```

```
{'d', 'c'}

{'b', 'a'}
{'e', 'f'}

{'f', 'b', 'e', 'a'}
```

In [25]:
```
my_set={'a', 'b', 'c'}
print(my_set)
```

```
print()

my_copy=my_set.copy()
print(my_copy)

print()

my_ref_copy=my_set
print(my_ref_copy)

print()

my_set.remove('b')
print(my_set) # {'a', 'c'}
print(my_copy) # {'b', 'a', 'c'}
print(my_ref_copy) # {'a', 'c'}
```

```
{'b', 'c', 'a'}

{'b', 'c', 'a'}

{'b', 'c', 'a'}

{'c', 'a'}
{'b', 'c', 'a'}
{'c', 'a'}
```

In [26]:
```
# frozenset() Function - making set immutable
a=(1,2,3,4,5,6,7,8,9,10)
#converting tuple to frozenset
fnum=frozenset(a)
print("Frozenset Object is: ", fnum)
```

```
Frozenset Object is:  frozenset({1, 2, 3, 4, 5, 6, 7, 8, 9, 10})
```

Dictionary

In [27]:
```
thisDict={
    "Brand": "Mahindra",
    "Model": "BE 6e Batman Edition",
    "Year": 2025
}

print(thisDict)
```

```
{'Brand': 'Mahindra', 'Model': 'BE 6e Batman Edition', 'Year': 2025}
```

In [28]:
```
thisDict={
    "Brand": "Mahindra",
    "Model": "BE 6e Batman Edition",
    "Year": 2025,
    "Year": 2026
}

print(thisDict)
```

```
{'Brand': 'Mahindra', 'Model': 'BE 6e Batman Edition', 'Year': 2026}
```

In [29]:
```
x=thisDict["Model"]
print(x)

y=thisDict.get("Model")
print(y)

print(len(thisDict))

z=thisDict["Year"]
print(z)
```

```
BE 6e Batman Edition
BE 6e Batman Edition
3
2026
```

In [30]:
```
x={
    'a':'b',
    'b':2,
    '2':6
}

print(x['2']) # 6
print(x[2]) # KeyError: 2
```

```
6
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
<ipython-input-30-da7ca0d41bc8> in <module>
      6
      7 print(x['2']) # 6
----> 8 print(x[2]) # KeyError: 2

KeyError: 2
```

In [31]:
```
thisDict={
    "Brand": "Mahindra",
    "Model": "BE 6e Batman Edition",
    "Year": 2025
}

thisDict["Year"] = 2026
print(thisDict)
```

```
{'Brand': 'Mahindra', 'Model': 'BE 6e Batman Edition', 'Year': 2026}
```

In [32]:
```
for x in thisDict:
    print(x)
```

```
Brand
Model
Year
```

In [33]:
```
for x in thisDict:
    print(thisDict[x])
```

```
Mahindra
BE 6e Batman Edition
2026
```

In [34]:
```
for x in thisDict.values():
    print(x)
```

```
Mahindra
BE 6e Batman Edition
2026
```

In [35]:
```
for x in thisDict.keys():
    print(x)
```

```
Brand
Model
Year
```

In [36]:
```
for x, y in thisDict.items():
    print(x, ":", y)
```

```
Brand : Mahindra
Model : BE 6e Batman Edition
Year : 2026
```

In [37]:
```
thisDict={
    "Brand": "Mahindra",
    "Model": "BE 6e Batman Edition",
    "Year": 2026
}

print(thisDict)
```

```
{'Brand': 'Mahindra', 'Model': 'BE 6e Batman Edition', 'Year': 2026}
```

In [38]:
```
thisDict["Color"]="Matt Black"
print(thisDict)
```

```
{'Brand': 'Mahindra', 'Model': 'BE 6e Batman Edition', 'Year': 2026, 'Color': 'Ma
tt Black'}
```

In [39]:
```
thisDict.popitem()
print(thisDict)
```

```
{'Brand': 'Mahindra', 'Model': 'BE 6e Batman Edition', 'Year': 2026}
```

In [40]:
```
thisDict.pop("Brand")
print(thisDict)
```

```
{'Model': 'BE 6e Batman Edition', 'Year': 2026}
```

In [41]:
```
thisDict={
    "Brand": "Mahindra",
    "Model": "BE 6e Batman Edition",
    "Year": 2026
}
```

In [42]:
```
del thisDict["Model"]
print(thisDict)
```

```
{'Brand': 'Mahindra', 'Year': 2026}
```

In [43]:
```
thisDict={
    "Brand": "Mahindra",
    "Model": "BE 6e Batman Edition",
```

```
    "Year": 2026
}
```

In [44]:
```
del thisDict
print(thisDict)
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
<ipython-input-44-af22f29e0d2e> in <module>
      1 del thisDict
----> 2 print(thisDict)

NameError: name 'thisDict' is not defined
```

In [45]:
```
thisDict={
    "Brand": "Mahindra",
    "Model": "BE 6e Batman Edition",
    "Year": 2026
}
```

In [46]:
```
thisDict.clear()
print(thisDict)
```

```
{}
```

In [ ]:

Department of AI_DS
Academic Year: 2026-27 (Odd Sem)
Course: Python for Data Science Lab
Second Year B. Tech Semester: III

PRACTICAL 4

Part A (To be referred by students)

Sequences and Dictionary

SAVE THE FILE AND UPLOAD AS (RollNo_Name_Exp4)

Aim: To understand and utilize Python collections (lists, tuples, sets, and dictionaries) and explore their applications in data science and statistics.

Problem Statement:

Solve the following computational problems by **recognizing** core Python data structures and **apply** them to implement programs

1. Create a list of the first 10 positive integers and print the sum of the even numbers in the list
2. Demonstrate the following operations on a number list using list functions
    a. insert 13 at position 4
    b. Sort the list in ascending order
    c. delete the last element
    d. remove 31
    e. Reverse the list
    f. Append one number to list
    g. extend the list with [20,30,40]
    h. print the number of elements in the list using while
3. Given the tuple (10, 20, 30, 40, 50), use slicing to print a new tuple containing only the last three elements.
4. Create a set of the numbers {1, 2, 3, 4, 5} and compute the intersection with {4, 5, 6, 7}; print the result.
5. Define a dictionary with the keys 'a', 'b', and 'c' and their respective values 1, 2, and 3, then update the value for key 'b' to 10 and print the updated dictionary.
6. Using a list comprehension, create a list of squares for the numbers from 1 to 5 and print the resulting list.
7. Create a tuple with elements 'cat', 'dog', and 'bird', then check and print whether 'dog' is in the tuple.
8. Given the list [1, 1, 2, 3, 5, 8, 13], convert it to a set and print the set's length.
9. Create two dictionaries {'A': [1, 2], 'B': [3, 4]} and {'B': [5, 6], 'C': [7, 8]}, then merge them into one dictionary and print the result.

Department of AI_DS
Academic Year: 2026-27 (Odd Sem)
Course: Python for Data Science Lab
Second Year B. Tech Semester: III

10. For the list [3, 5, 7, 3, 5, 9, 3], create a dictionary to count the occurrences of each element and print the dictionary.
11. Create a set, add member(s) in a set and perform the following operations: intersection of sets, union of sets, set difference, symmetric difference, find length, maximum, minimum value in a set, and clear a set.
12. Create a dictionary with the day no as the key and the day as the value & display it. (ex. 1 – Monday, …)
13. Write a Python program to find the sum of all items in the dictionary.
14. Create a dictionary to keep students' marks, use students' sapid as the key. Perform the task mentioned below on this dictionary.
    a. Display all the keys
    b. Display all the values
    c. Take the sapid as the input and modify the grade given by the user
    d. Take the sapid from the user to remove that user from the dictionary
    e. Give 5 marks as a bonus to all the students and display the new marks
    f. Find the length of the dictionary using the len function
    g. Create a new copy of the dictionary using the copy method
15. Calculate total sales from a product-sales dictionary.
16. Find highest and lowest temperatures in [22, 25, 19, 30, 28, 32, 26].
17. Print the name of the student with the highest score from a list of tuples [(90, 'Alice'), (85, 'Bob'), (88, 'Charlie')].
18. Compute and print the cumulative sum of [2, 4, 6, 8, 10].
19. Create a list where numbers divisible by 3 become "Fizz", divisible by 5 become "Buzz", both become "FizzBuzz", for numbers 1 to 15.
20. Create a list of numbers from 1 to 100 divisible by both 2 and 5.
21. Write a program that takes a dictionary where each value is a list and converts it into a dictionary where each value is a set containing the unique elements from the original list.
22. Write a program that takes two dictionaries and returns a set of keys that are present in the first dictionary but not in the second.
23. Write a function that takes two sets of tuples and returns a tuple containing three sets:
    i. Elements that are in both sets.
    ii. Elements that are only in the first set.
    iii. Elements that are only in the second set.
24. Write a program that takes a list of sets and returns a single set that is the union of all the sets in the list.
25. Write a program that takes a list of tuples and rotates each tuple by a given number of positions. For example, rotating `(1, 2, 3, 4)` by 2 positions would result in `(3, 4, 1,2)`.
26. Write a program that takes a list of tuples where each tuple contains integers and returns the tuple with the highest sum of its elements.

Topic covered: List, Tuple, Set & Dictionary

Department of AI_DS
Academic Year: 2026-27 (Odd Sem)
Course: Python for Data Science Lab
Second Year B. Tech Semester: III

Learning Objective: Learner would be able to

1. Recognize core python data structures & apply them to solve problems
2. Solve the problems using sequences like list, tuple, set
3. Understand dictionary and solving problems using dictionary
4. Perform various operations (using functions) on sequences and dictionary

Theory:

Lists:

Lists are used to store multiple items in a single variable. Lists are created using square brackets. List items are ordered, changeable, and allow duplicate values. List can contain strings, integers, as well as objects. It can be used to implement stacks and queues.
List items are indexed, the first item has index [0], the second item has index [1] etc.

- **Create List:**

A tuple a list is created by placing elements inside square brackets [], separated by commas. A list can have any number of items and they may be of different types (integer, float, string, etc.). A list can also have another list as an item. This is called a nested list.

```
# list of integers
my_list = [1, 2, 3]
print(my_list)

# empty list
my_list = []
print(my_list)
# list with mixed data types
my_list = [1, "Hello", 3.4]
print(my_list)

# nested list
```

```
my_list = ["mouse", [8, 4, 6], ['a']]
print(my_list)
```

Output:

[1, 2, 3]
[]
[1, 'Hello', 3.4]
['mouse', [8, 4, 6], ['a']]

- **Access Items:**

  List Index:

  We can use the index operator [] to access an item in a list. In Python, indices start at 0. So, a list having 5 elements will have an index from 0 to. Python allows negative indexing for its sequences. The index of -1 refers to the last item, -2 to the second last item and so on. We can access a range of items in a list by using the slicing operator : .

```
my_list = ['p', 'r', 'o', 'b', 'e']

# first item
print(my_list[0])  # p

# third item
print(my_list[2])  # o

# fifth item
print(my_list[4])  # e

# Nested List
n_list = ["Happy", [2, 0, 1, 5]]
```

```
# Nested indexing
print(n_list[0][1])

print(n_list[1][3])

# Error! Only integer can be used for indexing
print(my_list[4.0])

# Negative indexing in lists
my_list = ['p','r','o','b','e']

# last item
print(my_list[-1])

# fifth last item
print(my_list[-5])

# List slicing in Python

my_list = ['p','r','o','g','r','a','m','i','z']

# elements from index 2 to index 4
print(my_list[2:5])
# elements from index 5 to end
print(my_list[5:])

# elements beginning to end
print(my_list[:])
```

Output:

```
p
o
e
a
5
Traceback (most recent call last):
File "<string>", line 21, in <module>
TypeError: list indices must be integers or slices, not float
e
p
['o', 'g', 'r']
['a', 'm', 'i', 'z']
['p', 'r', 'o', 'g', 'r', 'a', 'm', 'i', 'z']
```

- **Update Items:**

  We can use the assignment operator = to change an item or a range of items.

  We can add one item to a list using the append() method or add several items using the extend() method.

  We can also use + operator to combine two lists. This is also called concatenation.

  The * operator repeats a list for the given number of times.

  We can insert one item at a desired location by using the method insert() or insert multiple items by squeezing it into an empty slice of a list.

```
# Correcting mistake values in a list
odd = [2, 4, 6, 8]


# change the 1st item
odd[0] = 1
```

```
print(odd)

# change 2nd to 4th items
odd[1:4] = [3, 5, 7]
print(odd)

# Appending and Extending lists in Python
odd = [1, 3, 5]
odd.append(7)
print(odd)

odd.extend([9, 11, 13])
print(odd)

# Concatenating and repeating lists
odd = [1, 3, 5]
print(odd + [9, 7, 5])

print(["re"] * 3)
# Demonstration of list insert() method
odd = [1, 9]
odd.insert(1,3)
print(odd)

odd[2:2] = [5, 7]
print(odd)
```

Output:

[1, 4, 6, 8]

```
[1, 3, 5, 7]
[1, 3, 5, 7]
[1, 3, 5, 7, 9, 11, 13]
[1, 3, 5, 9, 7, 5]
['re', 're', 're']
[1, 3, 9]
[1, 3, 5, 7, 9]
```

- **Remove Item:**

  We can delete one or more items from a list using the Python del statement. It can even delete the list entirely.

```
# Deleting list items
my_list = ['p', 'r', 'o', 'b', 'l', 'e', 'm']

# delete one item
del my_list[2]

print(my_list)

# delete multiple items
del my_list[1:5]

print(my_list)

# delete the entire list
del my_list

# Error: List not defined
print(my_list)
```

Output:

['p', 'r', 'b', 'l', 'e', 'm']

['p', 'm']

Traceback (most recent call last):

File "<string>", line 21, in <module>

NameError: name 'my_list' is not defined

We can use remove() to remove the given item or pop() to remove an item at the given index.

The pop() method removes and returns the last item if the index is not provided. This helps us implement lists as stacks (first in, last out data structure).

And, if we have to empty the whole list, we can use the clear() method.

```
my_list = ['p','r','o','b','l','e','m']
my_list.remove('p')

# Output: ['r', 'o', 'b', 'l', 'e', 'm']
print(my_list)

# Output: 'o'
print(my_list.pop(1))

# Output: ['r', 'b', 'l', 'e', 'm']
print(my_list)

# Output: 'm'
print(my_list.pop())

# Output: ['r', 'b', 'l', 'e']
```

```
print(my_list)


my_list.clear()

# Output: []
print(my_list)
```

Output:

['r', 'o', 'b', 'l', 'e', 'm']
o
['r', 'b', 'l', 'e', 'm']
m
['r', 'b', 'l', 'e']
[]

We can also delete items in a list by assigning an empty list to a slice of elements. The sliced elements are removed from the original list.

```
my_list = ['p','r','o','b','l','e','m']
print(my_list[2:3])
my_list[2:3] = []
print(my_list)

print(my_list[2:5])
my_list[2:5] = []
print(my_list)
```

Output:

['o']
['p', 'r', 'b', 'l', 'e', 'm']

```
['b', 'l', 'e']
['p', 'r', 'm']
```

- **List Operations:**

1. List Membership Test:

We can test if an item exists in a list or not, using the keyword in.

```
my_list = ['p', 'r', 'o', 'b', 'l', 'e', 'm']

# Output: True
print('p' in my_list)

# Output: False
print('a' in my_list)

# Output: True
print('c' not in my_list)
```

Output:

True

False

True

2. Iterating through a List:

We can use a for loop to iterate through each item in a list.

```
for fruit in ['apple','banana','mango']:
    print("I like",fruit)
```

Output:

I like apple

```
I like banana
I like mango
```

- **List Comprehension:**

  List comprehension is an elegant and concise way to create a new list from an existing list in Python.

  A list comprehension consists of an expression followed by **for statement** inside square brackets.

```
pow2 = [2 ** x for x in range(10)]
print(pow2)

Output:
[1, 2, 4, 8, 16, 32, 64, 128, 256, 512]
```

  A list comprehension can optionally contain more for or **if statements**.

```
pow2 = [2 ** x for x in range(10) if x > 5]
print(pow2)
odd = [x for x in range(20) if x % 2 == 1]
print(odd)
z=[x+y for x in ['Python ','C '] for y in ['Language','Programming']]
print(z)

Output:
[64, 128, 256, 512]
[1, 3, 5, 7, 9, 11, 13, 15, 17, 19]
['Python Language', 'Python Programming', 'C Language', 'C Programming']
```

Tuple:

A tuple is a sequence of **immutable** (cannot be changed) Python objects. Tuples use parentheses. Tuple are used to store sequence of python objects which are static in nature. Just like a list, a tuple contains a sequence of objects **in order** but once created a tuple, it cannot be changed anything about it. Tuple indices start from 0. Since tuples are indexed, they can have items with the same value.

- **Create a Tuple:**

  A tuple can have any number of items and they may be of different types (integer, float, list, string, etc.).

- **Create Tuple with One Item:**

  To create a tuple with only one item, you have to add a comma after the item, otherwise Python will not recognize it as a tuple.

```
tup1 = ('physics', 'chemistry', 1997, 2000);
tup2 = (1, 2, 3, 4, 5);
tup3 = "a", "b", "c", "d";
print(tup1)
print(tup2)
print(tup3)
tup4 = ()
print(tup4)
tup4 = (50, );
print(tup4)
```

Output: Tuples are ordered and items cannot be changed once added.

('physics', 'chemistry', 1997, 2000)

(1, 2, 3, 4, 5)

('a', 'b', 'c', 'd')

()

(50,)

- **Access Items:**

  To access values in tuple, use the square brackets for slicing along with the index or indices to obtain value available at that index.

```
tup1 = ('physics', 'chemistry', 1997, 2000)
tup2 = (1, 2, 3, 4, 5, 6, 7)
print ("tup1[0]: ", tup1[0])
print ("tup2[1:5]: ", tup2[1:5])
print(tup2[-1])
print(tup2[-6])
```

Output:

tup1[0]: physics

tup2[1:5]: (2, 3, 4, 5)

7

2

- **Update Items:**

  Tuples are immutable which means you cannot update or change the values of tuple elements. You are able to take portions of existing tuples to create new tuples.

```
tup1 = (12, 34.56)
tup2 = ('abc', 'xyz')
# Following action is not valid for tuples
# tup1[0] = 100
# So let's create a new tuple as follows
tup3 = tup1 + tup2
print (tup3)
```

Output:

(12, 34.56, 'abc', 'xyz')

- **Remove Item:**

  Removing individual tuple elements is not possible. There is, of course, nothing wrong with putting together another tuple with the undesired elements discarded. To explicitly remove an entire tuple, just use the del statement.

```
tup = ('physics', 'chemistry', 1997, 2000)
print (tup)
del (tup)
print (tup)
```

Output:

('physics', 'chemistry', 1997, 2000)

Traceback (most recent call last):

File "<string>", line 4, in <module>

NameError: name 'tup' is not defined

- **Tuple Operations:**

  3. Tuple Membership Test:

  We can test if an item exists in a tuple or not, using the keyword in.

```
# Membership test in tuple
my_tuple = ('a', 'p', 'p', 'l', 'e',)

# In operation
print('a' in my_tuple)
print('b' in my_tuple)

# Not in operation
print('g' not in my_tuple)
```

Output:

True

False

True

4. Iterating through a Tuple:

We can use a for loop to iterate through each item in a tuple.

```
# Using a for loop to iterate through a tuple
for name in ('John', 'Kate'):
    print("Hello", name)
```

Output:

Hello John

Hello Kate

5. Repetition:

Repeat the elements in a tuple for a given number of times using the * operator.

6. Concatenation:

Use + operator to combine two tuples. This is called concatenation.

7. Length:

Count the number of elements in the tuple.

```
# Concatenation
t = (1, 2, 3)
p =(4, 5, 6)
print(t + p)
```

```
# length
print(len(t))


# Repeat
# Output: ('Repeat', 'Repeat', 'Repeat')
print(("Repeat",) * 3)
```

Output:

(1, 2, 3, 4, 5, 6)

3

('Repeat', 'Repeat', 'Repeat')

Set:

A set is a collection which is **unordered** and **unindexed**. In Python sets are written with **curly brackets**. Every element is **unique (no duplicates)** and must be immutable (which cannot be changed). It can have any number of items and they may be of **different types (integer, float, tuple, string etc.)**. The sets in Python are typically used for mathematical operations like union, intersection, difference and complement etc.

- **Create a Set:**

```
thisset = {"apple", "banana", "cherry"}
print(thisset)
```

Output: Sets are unordered, so the items will appear in a random order.

{'banana', 'cherry', 'apple'}

- **Access Items:**

You cannot access items in a set by referring to an index, since sets are unordered the items has no index. But you can loop through the set items using a for loop, or ask if a specified value is present in a set, by using thein keyword.

```
thisset = {"apple", "banana", "cherry"}
print(thisset)

for x in thisset: print(x)

print("banana" in thisset)
```

Output:

{'cherry', 'banana', 'apple'}
cherry
banana
apple
True

- **Update Items:**

  Once a set is created, you cannot change its items, but you can add new items.

- **Add Items:**

  To add one item to a set use the add() method.

  To add more than one item to a set use the update() method.

```
thisset = {"apple", "banana", "cherry"}
thisset.add("orange")
print(thisset)
thisset.add("apple")
print(thisset)
```

Output:

{'apple', 'cherry', 'orange', 'banana'}

{'apple', 'cherry', 'orange', 'banana'}

Add multiple items to a set, using the update() method:

```
thisset = {"apple", "banana", "cherry"}
thisset.update(["orange", "mango", "grapes"])
print(thisset)
```

Output:

{'apple', 'cherry', 'orange', 'banana'}

- **Length of a Set:**

  To determine how many items a set has, use the len() method.

```
thisset = {"apple", "banana", "cherry"}
print(len(thisset))
```

Output:

3

- **Remove Item:**

  To remove an item in a set, use the remove(), or the discard() method.

```
# remove()
thisset = {"apple", "banana", "cherry"}
thisset.remove("banana")
print(thisset)

thisset.remove("banana")
print(thisset)
```

Output:

{'cherry', 'apple'}

```
Traceback (most recent call last):
  File "<string>", line 6, in <module>
KeyError: 'banana'
```

Note: If the item to remove does not exist, remove() will raise an error.

Note: If the item to remove does not exist, discard() will NOT raise an error.

```
# discard()
thisset = {"apple", "banana", "cherry"}
thisset.remove("banana")
print(thisset)
thisset.remove("banana")
print(thisset)

Output:
{'apple', 'cherry'}
{'apple', 'cherry'}
```

Note: Sets are unordered, so when using the gets removed. pop() method, you will not know which item that gets removed.

You can also use the pop(), method to remove an item, but this method will remove last item. Since set is unordered, so you will not know what item that gets. The return value of the pop() method is the removed item.

```
thisset = {"apple", "banana", "cherry"}
x = thisset.pop()
print(x)
print(thisset)

Output:
banana
```

```
{'cherry', 'apple'}
```

The clear() method empties the set:

```
thisset = {"apple", "banana", "cherry"}
thisset.clear()
print(thisset)
thisset.add("organge")
print(thisset)

Output:
set()
{'organge'}
```

The del() method will delete the set comple:

```
thisset = {"apple", "banana", "cherry"}
del thisset
print(thisset)

Output:
Traceback (most recent call last):
  File "<string>", line 5, in <module>
NameError: name 'thisset' is not defined
```

Dictionary:

A dictionary is a collection which is unordered, changeable and indexed. In Python dictionaries are written with curly brackets, and they have keys and values. It is a collection which is ordered (Python version 3.7 and above) and changeable (meaning that we can change, add or remove items after the dictionary has been created). No duplicate members.

- **Create a Dictionary:**

```
thisdict = {
"brand": "Ford",
"model": "Mustang",
"year": 1964
}
print(thisdict)
```

Output: Dictionary are ordered.

{'brand': 'Ford', 'model': 'Mustang', 'year': 1964}

- **Access Items:**

  You can access the items of a dictionary by referring to its key name, inside square brackets.

```
# Get the value of the "model" key:
thisdict = {
"brand": "Ford",
"model": "Mustang",
"year": 1964
}
x = thisdict["model"]
print(x)
```

Output:

Mustang

get() : There is also a method called get() that will give you the same result:

```
# Get the value of the "model" key:
thisdict = {
"brand": "Ford",
```

```
"model": "Mustang",
"year": 1964
}
x = thisdict.get("model")
print(x)
```

Output:

Mustang

- **Update and Add Items:**

  Dictionaries are mutable. We can add new items or change the value of existing items using an assignment operator.

  If the key is already present, then the existing value gets updated. In case the key is not present, a new (key: value) pair is added to the dictionary.

```
# Changing and Adding Dictionary elements:
thisdict = {
"brand": "Ford",
"model": "Mustang",
"year": 1964
}
print(thisdict)
# update value
thisdict['year'] = 1966
print(thisdict)
# add item
thisdict["color"] = 'White'
print(thisdict)
```

Output:

{'brand': 'Ford', 'model': 'Mustang', 'year': 1964}

{'brand': 'Ford', 'model': 'Mustang', 'year': 1966}

{'brand': 'Ford', 'model': 'Mustang', 'year': 1966, 'color': 'White'}

- **Built-in functions of Dictionary:**

  all()

  Return True if all keys of the dictionary are True (or if the dictionary is empty).

  any()

  Return True if any key of the dictionary is true. If the dictionary is empty, return False.

  len()

  Return the length (the number of items) in the dictionary.

  cmp()

  Compares items of two dictionaries. (Not available in Python 3)

  sorted()

  Return a new sorted list of keys in the dictionary.

```
# Dictionary Built-in Functions
squares = {0: 0, 1: 1, 3: 9, 5: 25, 7: 49, 9: 81}

# Output: False
print(all(squares))

# Output: True
print(any(squares))

# Output: 6
print(len(squares))

# Output: [0, 1, 3, 5, 7, 9]
```

```
print(sorted(squares))
```

Output:

False

True

6

[0, 1, 3, 5, 7, 9]

- **Remove Item:**

  We can remove a particular item in a dictionary by using the **pop()** method. This method removes an item with the provided key and returns the value.

  The **popitem()** method can be used to remove and return an arbitrary (key, value) item pair from the dictionary.

  All the items can be removed at once, using the **clear()** method.

  We can also use the **del** keyword to remove individual items or the entire dictionary itself.

```
# Removing elements from a dictionary

# create a dictionary
squares = {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

# remove a particular item, returns its value
# Output: 16
print(squares.pop(4))

# Output: {1: 1, 2: 4, 3: 9, 5: 25}
print(squares)

# remove an arbitrary item, return (key,value)
# Output: (5, 25)
```

```
print(squares.popitem())

# Output: {1: 1, 2: 4, 3: 9}
print(squares)

# remove all items
squares.clear()
# Output: {}
print(squares)

# delete the dictionary itself
del squares

# Throws Error
print(squares)
```

Output:

```
16
{1: 1, 2: 4, 3: 9, 5: 25}
(5, 25)
{1: 1, 2: 4, 3: 9}
{}
Traceback (most recent call last):
  File "<string>", line 30, in <module>
NameError: name 'squares' is not defined
```

- **Looping through Dictionary:**

  A dictionary can be iterated using the for loop. If you want to get both keys and the values in the output. You just have to add the keys and values as the argument of the print

statement in comma separation. After each iteration of the for loop, you will get both the keys its relevant values in the output.

```
# The output shows the given key is related to the given value in the output.
dict={'Name':'Zara','Age':7,'Class':'First'}
for key, value in dict.items():
    print(key, '-',value)
```

Output:

Name - Zara

Age - 7

Class - First

- **Python Dictionary Comprehension:**

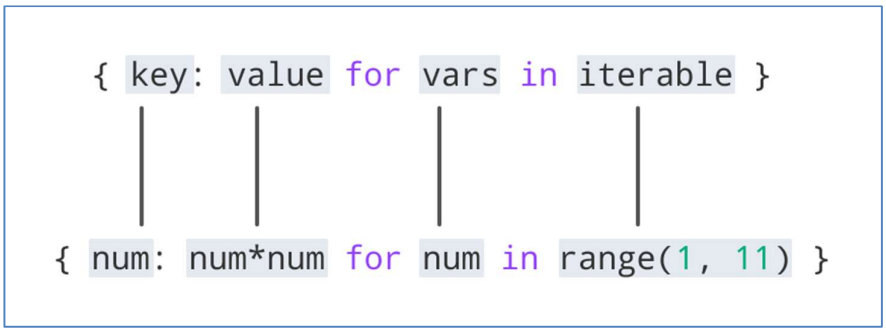
  Python Dictionary Comprehension helps to create a new dictionary from an iterable in Python.

  Dictionary comprehension consists of an expression pair (**key: value**) followed by a for statement inside curly braces {}.

  The minimal syntax for dictionary comprehension is:

```
dictionary = {key: value for vars in iterable}
```

Let's compare this syntax with dictionary comprehension from the above example.



```
# Dictionary Comprehension to create square of numbers from 0 to 5
squares = {x: x*x for x in range(6)}
print(squares)
```

Output:

{0: 0, 1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

Conditionals in Dictionary Comprehension:

We can further customize dictionary comprehension by adding conditions to it.

The syntax for dictionary comprehension with conditionals is:

```
dictionary = {key: true_value for vars in iterable   if  condition }
```

```
dictionary = {key: ( true_value if condition else false_value ) for vars in iterable}
```

```
# Apply dictionary comprehension to create squares of only odd numbers from 1 to 11

odd_squares = {x: x*x for x in range(11) if x % 2 == 1}
print(odd_squares)
```

Output: {1: 1, 3: 9, 5: 25, 7: 49, 9: 81}

```
# apply dictionary comprehension to pick up only items having even age values
original_dict = {'jack': 38, 'michael': 48, 'guido': 57, 'john': 33}

even_dict = {k: v for (k, v) in original_dict.items() if v % 2 == 0}
print(even_dict)
```

Output: {'jack': 38, 'michael': 48}

Department of AI_DS
Academic Year: 2026-27 (Odd Sem)
Course: Python for Data Science Lab
Second Year B. Tech Semester: III

PRACTICAL 4

Part B (to be completed by students)

Sequences and Dictionary

1. All the students are required to perform the given tasks in Jupyter Notebook
2. Create a new notebook for each experiment. The filename should be RollNo_Name_Exp4)
3. In the first cell, the student must write his/her Name, roll no and class in the form of comments
4. Every program should be written in separate cells and in the given sequence
5. After completing the experiment, download the notebook in pdf format. The filename should be RollNo_Name_Exp4.pdf).
   Note: - To download as a PDF, go to File and choose Print Preview. This will open a new tab. Now, press Ctrl + P and opt to save as PDF.
6. Upload the pdf on the web portal

Create a list of the first 10 positive integers and print the sum of the even numbers in the list

In [1]:
```
num=list(range(1,11))
```

Out[1]: list

In [2]:
```
sum=0
for i in num:
    if(i%2==0):
        sum+=i
print(sum)
```

```
30
```

Demonstrate the following operations on a number list using list functions

a.insert 13 at position 4

b.Sort the list in ascending order

c.delete the last element

d.remove 31

e.Reverse the list

f.Append one number to list

g.extend the list with [20,30,40]

h.print the number of elements in the list using while

In [21]:
```
num=list(range(1,11))
print(num)
```

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

In [22]:
```
num.insert(4,13)
print(num)
```

```
[1, 2, 3, 4, 13, 5, 6, 7, 8, 9, 10]
```

In [23]:
```
num.sort()
print(num)
```

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 13]
```

In [24]:
```
num.pop()
print(num)
```

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

In [25]:
```
num.remove(31)
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
<ipython-input-25-04da6c47085e> in <module>
----> 1 num.remove(31)

ValueError: list.remove(x): x not in list
```

In [26]:
```
num.sort(reverse=True)
print(num)
```

```
[10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
```

In [27]:
```
num.append(45)
print(num)
```

```
[10, 9, 8, 7, 6, 5, 4, 3, 2, 1, 45]
```

In [28]:
```
l=[20,30,40]
num.extend(l)
print(num)
```

```
[10, 9, 8, 7, 6, 5, 4, 3, 2, 1, 45, 20, 30, 40]
```

In [29]:
```
i=0
while i < len(num):
    print(num[i])
    i+=1
```

```
10
9
8
7
6
5
4
3
2
1
45
20
30
40
```

Given the tuple (10, 20, 30, 40, 50), use slicing to print a new tuple containing only the last three elements.

In [30]:
```
num=(10,20,30,40,50)
print(num[2:])
```

```
(30, 40, 50)
```

Create a set of the numbers {1, 2, 3, 4, 5} and compute the intersection with {4, 5, 6, 7}; print the result.

In [32]:
```
set1={1,2,3,4,5}
set2={4,5,6,7}

print(set1&set2)
```

```
{4, 5}
```

Define a dictionary with the keys 'a', 'b', and 'c' and their respective values 1, 2, and 3, then update the value for key 'b' to 10 and print the updated dictionary.

In [39]:
```
thisdict ={
    'a' : 1,
    'b' : 2,
    'c' : 3
}
print(thisdict)
```

```
dict['b']=10
print(thisdict)
```

```
{'a': 1, 'b': 2, 'c': 3}
{'a': 1, 'b': 2, 'c': 3}
```

Using a list comprehension, create a list of squares for the numbers from 1 to 5 and print the resulting list.

In [42]:
```
sq = [x**2 for x in range(1,6)]
print(sq)
```

```
[1, 4, 9, 16, 25]
```

Create a tuple with elements 'cat', 'dog', and 'bird', then check and print whether 'dog' is in the tuple.

In [43]:
```
animal=("cat","dog","bird")
print("dog" in animal)
```

```
True
```

Given the list [1,1,2,3,5,8,13], convert it to a set and print the set's length.

In [46]:
```
ls = [1,1,2,3,5,8,13]
print(ls)
s = set(ls)
print(s)
print(len(s))
```

```
[1, 1, 2, 3, 5, 8, 13]
{1, 2, 3, 5, 8, 13}
6
```

Create two dictionaries {'A': [1, 2], 'B': [3, 4]} and {'B': [5, 6], 'C': [7, 8]}, then merge them into one dictionary and print the result.

In [54]:
```
dict1 = {'A':[1,2], 'B':[3,4]}
dict2 = {'B':[5,6], 'C':[7,8]}
merged_dict = {**dict1, **dict2}
print(merged_dict)
```

```
{'A': [1, 2], 'B': [5, 6], 'C': [7, 8]}
```

For the list [3, 5, 7, 3, 5, 9, 3], create a dictionary to count the occurrences of each element and print the dictionary.

In [55]:
```
elements = [3, 5, 7, 3, 5, 9, 3]
counts = {}

for num in elements:
    if num in counts:
        counts[num] += 1
    else:
        counts[num]=1
print(counts)
```

```
{3: 3, 5: 2, 7: 1, 9: 1}
```

In [ ]:

Department of Artificial Intelligence (AI) & Data Science

AY: 2026-27
Course: Python for Data Science Laboratory (DJS23SLPC303)
Class: SE/A11/A12/A21/A22 Sem: III Roll No.: SAPID:

Experiment No. 4: Sequences and Dictionary.

Aim: To understand and utilize Python collections (lists, tuples, sets, and dictionaries) and explore their applications in data science and statistics.

Theory

1. What is a list in Python? How is it different from an array in other programming languages?
2. Explain the concept of list mutability with an example.
3. Differentiate between append() and extend() methods.
4. What is the difference between list.sort() and sorted(list)?
5. What is a tuple in Python? How is it different from a list?
6. Why are tuples called immutable? What are the advantages of immutability?
7. Can a tuple contain mutable objects like lists? Explain with an example.
8. What is a set in Python and what are its main properties?
9. How are sets different from lists and tuples?
10. Why are duplicates not allowed in a set?
11. What is the difference between remove() and discard() methods in sets?
12. What is a dictionary in Python? How is it different from a list?
13. Explain the concept of key-value pairs in dictionaries.
14. Why must dictionary keys be immutable (like strings, numbers, tuples)?
15. Differentiate between get() and direct key access (e.g., d['key']).

Conclusions:

Name:- Aaryan N. Rodrane Roll No:- 5004 Div:- 5
batch:- A1 SAP ID:- 60018250070 branch:- A I & DS

Experiment No 4

* Title:- Sequences and dictionary

* Aim:- To understand and utilize Python collections (lists, tuples, sets and dictionaries) and explore their applications in data science and statistics.

* Theory:-

1) What is a list in Python? How is it different from an array in other programming languages?

⇒ A list is an ordered, mutable collection in Python that can store elements of different data types. Arrays in languages like C or Java require all elements to be of the same type and have a fixed size, whereas Python lists resize dynamically and accept mixed types.

Feature	Python list	Array (C / Java)
data types	Multiple types allowed	single type only
size	Dynamic	Fixed at declaration
Memory layout	Non contiguous	contiguous

code: data = [85, 90.5, "A", True]
print (data)

output: [85, 90.5, 'A', True]

2) Explain the concept of list Mutability with an example.

—> A list is mutable meaning its contents can be modified after creation. Elements can be updated, added or removed without replacing the entire list object.

code: marks = [70, 80, 90]
marks[0] = 95
print (marks)

output: [95, 80, 90]

3) Differentiate between append() and extend() methods.

=>

Method	Behaviour	Argument type
append()	adds a single element to the end	any object
extend()	unpacks and adds each element of an iterable	Iterable (list, tuple, etc.

Sundaram
FOR EDUCATIONAL USE

code:
```
a = [1, 2, 3]
a.append([4, 5])
print(a)

b = [1, 2, 3]
b.extend([4, 5])
print(b)
```

output:
```
[1, 2, 3, [4, 5]]
[1, 2, 3, 4, 5]
```

4) what is the difference between list.sort() and sorted(list)?

=>

feature	list.sort()	sorted(list)
Modifies original list	Yes (in place)	No
Return value	None	A new sorted list
works on	Lists only	Any iterable

code:
```
a = [3, 1, 2]
b = sorted(a)
print("sorted():", a, b)
a.sort()
print("list.sort():", a)
```

output:
```
sorted(): [3, 1, 2] [1, 2, 3]
list.sort(): [1, 2, 3]
```

Sundaram
FOR EDUCATIONAL USE

5) What is a tuple in Python? How is it different from list?

=> A tuple is an ordered, immutable collection defined using parentheses. Its elements cannot be changed once assigned.

Feature	Tuple	List
Mutability	immutable	Mutable
Syntax	()	[]
Performance	Faster	Slower
Use case	fixed/constant data	Dynamic data

Code :-
```
cords = (10.5, 20.3)
print(cords[0])
```

Output :- 10.5

6) Why are tuples called immutable? What are the advantages of immutability?

=> Tuples are immutable because their elements cannot be modified, added or deleted after creation. Any attempt to do so raises a TypeError.

The advantage of tuple is that they are safe for storing data that should not change, they can be used as dictionary keys (since they are hashable), and they iterate faster than

Sundaram
FOR EDUCATIONAL USE

lists due to lower memory overhead

code :-
```
t = (1, 2, 3)
t[0] = 99
```

output :- TypeError: 'tuple' object does not support item assignment

7) Can a tuple contain mutable objects like lists? Explain with an example.

=> Yes, the tuple itself remains immutable (its references cannot be reassigned), but a mutable object stored inside it can still be modified through its own methods

code :-
```
t = (1, [2, 3], 4)
t[1].append(99)
print(t)
```

output :- (1, [2, 3, 99], 4)

8) What is a set in Python and what are its main properties?

=> A set is an unordered collection of unique, hashable elements. It is defined using curly braces or the set() constructor. Sets support mathematical operations such as union, intersection and difference.

Sundaram
FOR EDUCATIONAL USE

Main properties :-

- unordered (no indexing)
- duplicates not allowed
- elements must be immutable

Code :-
```
s = {10, 20, 30, 20, 10}
print(s)
print(type(s))
```

Output :-
```
{10, 20, 30}
<class 'set'>
```

Q) How are sets different from lists and tuples?

⇒

Feature	set	List	Tuple
order	unordered	ordered	ordered
duplicates	not allowed	allowed	allowed
Mutability	Mutable	Mutable	Immutable
Indexing	Not supported	supported	supported

Code :-
```
lst = [1, 2, 2, 3]
tpl = (1, 2, 2, 3)
st = {1, 2, 2, 3}
print(lst, tpl, st)
```

output :- [1, 2, 2, 3] (1, 2, 2, 3) {1, 2, 3}

Sundaram
FOR EDUCATIONAL USE

10) Why are duplicates not allowed in a set?

⇒ sets use a hash table internally. Each element is stored at a position determined by its hash value. When a element with an identical hash (ie, or equal value) is inserted, it replaces the existing entry rather than creating a new one. This mechanism enforces uniqueness automatically.

code :-
```
s = set([4,4,5,5,6])
print(s)
```

output : {4,5,6}

11) What is the difference between remove() and discard() methods in sets?

⇒

method	when element is not found
remove()	Raises KeyError
discard()	Does nothing, no error raised

code :-
```
s = {1,2,3}
s.discard(99)
print(s)
s.remove(99)
```

output :- {1,2,3}
KeyError: 99

Sundaram FOR EDUCATIONAL USE

(2) what is a dictionary in Python? How is it different from a list?

⇒ A dictionary is an unordered collection of key value pairs. Elements are accessed using keys rather than numeric indices.

Feature	Dictionary	List
Access Method	By key	By index
Element structure	Key value pairs	single values
Key requirement	must be immutable	Not applicable

code :- student = {"name": "Raj", "Marks": 88}
print (student ["name"])

output :- Raj

(3) Explain the concept of key-value pairs in dictionaries.

⇒ Every entry in a dictionary has two parts: a key and a value. The key acts as a unique identifier and the value holds the associated data. Keys must be unique within a dictionary, but values can repeat.

Sundaram
FOR EDUCATIONAL USE

code :- course = {
"subject" : "Python",
"code" : "DJS23SLPC303",
"credits" : 2
}
print(course["subject"], "-->", course["code"])

output :- Python --> DJS23SLPC303

(a) Why must dictionary keys be immutable (like strings, numbers, tuples)?

=> Python uses a hash table to store dictionary keys. Hashing requires the value of an object to remain constant. Mutable objects like lists have no stable hash, so attempting to use them as keys raises a TypeError.

code :- d = {}
d[(1,2)] = "valid"
print(d)
d[[1,2]] = "invalid"

output :- {(1,2) : 'valid'}
TypeError : unhashable type : 'list'

Sundaram
FOR EDUCATIONAL USE

(5) Differentiate between get() and direct key access (eg. d['key']).

⟹

Feature	get()	d['key']
Key not found	Returns none or a specified default	Raises KeyError
safety	Safe for unknown Keys	Risky without prior check
Default value	can be specified	Not applicable

code:-
```
d={"a":100}
print(d.get("b",0))
print(d["b"])
```

output:-
```
0
KeyError: 'b'
```

*Conclusion:-

This experiment covers the four built in collection types in Python, namely lists, tuples, sets, and dictionaries, each differing in mutability, ordering, and element constraints. Lists and tuples handle ordered sequence while sets enforce uniqueness through hashing and dictionaries provide fast key based data access. Selecting the appropriate collection type based on these properties is essential for writing correct and efficient data science programs.

Sundaram
FOR EDUCATIONAL USE

Numpy (Numerical Python)

In [4]:
```
import numpy as np
```

Creating Numpy Array and Studying its types

In [3]:
```
# normal array
list1=[2,4,6,8]
array = np.array(list1)
print(array)
```

```
[2 4 6 8]
```

In [4]:
```
# array of 0 and 1
array0 = np.zeros(4)
array1 = np.ones(4)
print(array0,array1)
```

```
[0. 0. 0. 0.] [1. 1. 1. 1.]
```

In [6]:
```
# creating array with range
arr1 = np.arange(5)
arr2 = np.arange(1,6)
arr3 = np.arange(1,11,2)
print(arr1,arr2,arr3)
```

```
[0 1 2 3 4] [1 2 3 4 5] [1 3 5 7 9]
```

In [21]:
```
# random array generator
arr = np.random.rand(5)
print(arr)
```

```
[0.83177336 0.10457029 0.03207396 0.96526122 0.61636389]
```

In [22]:
```
# random integer array generator
arr = np.random.randint(1,101,size=5)
print(arr)
```

```
[39 55 62 68 42]
```

In [26]:
```
# random integer array generator with seed
np.random.seed(0)
arr = np.random.randint(1,101,size=5)
print(arr)
```

```
[45 48 65 68 68]
```

In [27]:
```
# generate 5 random number uniformly distributed distributed between 0.0 and 1.0
arr = np.random.uniform(0.0,1.0,size=5)
print(arr)
```

```
[0.84725174 0.6235637  0.38438171 0.29753461 0.05671298]
```

In [33]:
```
arr = np.empty(5)
print(arr)
```

```
[0.84725174 0.6235637  0.38438171 0.29753461 0.05671298]
```

In [34]:
```
# 1d array
arr1d = np.zeros(5)
print(arr1d)
print(arr1d.shape)
```

```
[0. 0. 0. 0. 0.]
(5,)
```

In [36]:
```
# 2d array
arr2d = np.zeros((2,3))
print(arr2d)
print(arr2d.shape)
```

```
[[0. 0. 0.]
 [0. 0. 0.]]
(2, 3)
```

In [37]:
```
# 3d array
arr3d = np.zeros((2,3,4))
print(arr3d)
print(arr3d.shape)
```

```
[[[0. 0. 0. 0.]
  [0. 0. 0. 0.]
  [0. 0. 0. 0.]]

 [[0. 0. 0. 0.]
  [0. 0. 0. 0.]
  [0. 0. 0. 0.]]]
(2, 3, 4)
```

In [51]:
```
# 4d array
arr4d = np.zeros((2,3,4,5))
print(arr4d)
print(arr4d.shape)
# (no of 3d arrays,no of 2d arrays,rows,columns)
```

```
[[[[0. 0. 0. 0. 0.]
   [0. 0. 0. 0. 0.]
   [0. 0. 0. 0. 0.]
   [0. 0. 0. 0. 0.]]

  [[0. 0. 0. 0. 0.]
   [0. 0. 0. 0. 0.]
   [0. 0. 0. 0. 0.]
   [0. 0. 0. 0. 0.]]

  [[0. 0. 0. 0. 0.]
   [0. 0. 0. 0. 0.]
   [0. 0. 0. 0. 0.]
   [0. 0. 0. 0. 0.]]]


 [[[0. 0. 0. 0. 0.]
   [0. 0. 0. 0. 0.]
   [0. 0. 0. 0. 0.]
   [0. 0. 0. 0. 0.]]

  [[0. 0. 0. 0. 0.]
   [0. 0. 0. 0. 0.]
   [0. 0. 0. 0. 0.]
   [0. 0. 0. 0. 0.]]

  [[0. 0. 0. 0. 0.]
   [0. 0. 0. 0. 0.]
   [0. 0. 0. 0. 0.]
   [0. 0. 0. 0. 0.]]]]
(2, 3, 4, 5)
```

In [52]:
```
arr = np.array([[1,2,3],[4,5,6]])
print(arr.ndim)
print(len(arr))
print(arr.shape)
print(arr.dtype.name)
print(arr.astype(int))
print(arr.dtype)
```

```
2
2
(2, 3)
int32
[[1 2 3]
 [4 5 6]]
int32
```

In [53]:
```
arr = np.array([[1,2,3],[4,5,6],[7,8,9]])
print("Number of Dimension : ",arr.ndim)
print("Size of array : ",arr.size)
print("Shape of array : ",arr.shape)
print("Data type of elements : ",arr.dtype)
print("Size of each elements in bytes : ",arr.itemsize)
```

```
Number of Dimension :  2
Size of array :  9
Shape of array :  (3, 3)
Data type of elements :  int32
Size of each elements in bytes :  4
```

In [54]:
```
arr2 = np.array([[1,2,3],[4,5,6],[7,8,9]],dtype=np.int64)
print(arr2.itemsize)
print(arr2.data)
```

```
8
<memory at 0x0000024E90E92EA0>
```

In [55]:
```
arr2d = np.array([[1,2,3,4],[5,6,7,8]])
print(arr2d)
```

```
[[1 2 3 4]
 [5 6 7 8]]
```

In [57]:
```
arr4d = np.array([[[1,2,3,4],[5,6,7,8],[9,10,11,12]],[[1,2,3,4],[5,6,7,8],[9,10,
print(arr4d)
```

```
[[[ 1  2  3  4]
  [ 5  6  7  8]
  [ 9 10 11 12]]

 [[ 1  2  3  4]
  [ 5  6  7  8]
  [ 9 10 11 12]]]
```

In [59]:
```
arr = np.zeros((2,3))
print("2d array")
print(arr)
```

```
2d array
[[0. 0. 0.]
 [0. 0. 0.]]
```

In [60]:
```
arr = np.zeros((2,3,4))
print("3d array")
print(arr)
```

```
3d array
[[[0. 0. 0. 0.]
  [0. 0. 0. 0.]
  [0. 0. 0. 0.]]

 [[0. 0. 0. 0.]
  [0. 0. 0. 0.]
  [0. 0. 0. 0.]]]
```

In [61]:
```
arr = np.ones((2,3,4))
print("3d array")
print(arr)
```

```
3d array
[[[1. 1. 1. 1.]
  [1. 1. 1. 1.]
  [1. 1. 1. 1.]]

 [[1. 1. 1. 1.]
  [1. 1. 1. 1.]
  [1. 1. 1. 1.]]]
```

In [63]:
```
numpy_array = np.full((2,2),5)
print(numpy_array)
```

```
[[5 5]
 [5 5]]
```

In [64]:
```
numpy_array_3d = np.random.rand(2,2,2)
print(numpy_array_3d)
```

```
[[[0.27265629 0.47766512]
  [0.81216873 0.47997717]]

 [[0.3927848  0.83607876]
  [0.33739616 0.64817187]]]
```

In [67]:
```
numpy_array_3d = np.empty((2,2,2))
print(numpy_array_3d)
```

```
[[[0.27265629 0.47766512]
  [0.81216873 0.47997717]]

 [[0.3927848  0.83607876]
  [0.33739616 0.64817187]]]
```

In [70]:
```
carr1 = np.array([1+2j,2+3j,3+8j])
print(carr1)
print(carr1.dtype)
print(carr1.itemsize)
```

```
[1.+2.j 2.+3.j 3.+8.j]
complex128
16
```

In [71]:
```
farr = carr1.astype('float')
print(farr)
```

```
[1. 2. 3.]
C:\Users\djsce.student\AppData\Local\Continuum\anaconda3\lib\site-packages\ipykernel_launcher.py:1: ComplexWarning: Casting complex values to real discards the imaginary part
  """Entry point for launching an IPython kernel.
```

Numpy array Indexing

starts with 0

In [74]:
```
arr1 = np.array([1,3,5,9])
print(arr1[0])
print(arr1[2])
print(arr1[-1])
```

```
1
5
9
```

In [75]:
```
arr1[1]=11
arr1[3]=13
print(arr1)
```

```
[ 1 11  5 13]
```

In [76]:
```
print(arr1[-2])
```

```
5
```

20th August

In [6]:
```
arr1 = np.array([[1, 3, 5], [7, 9, 2], [4, 6, 8]])
print("output : ",arr1[2, 1])
```

```
6
```

In [9]:
```
print("2nd Row: ",arr1[1,:])
```

```
2nd Row:  [7 9 2]
```

In [10]:
```
print("3rd column: ",arr1[:,2])
```

```
3rd column:  [5 2 8]
```

In [13]:
```
arr2 = np.array([[[1,2,3,4],[5,6,7,8],[9,10,11,12]],[[13,14,15,16],[17,18,19,20]
print("element : ",arr2[1,2,1])
print("row : ",arr2[1,2,:])
print("column : ",arr2[1,:,1])
print("what to call it ? : ",arr2[:,2,1])
```

```
element :  22
row :  [21 22 23 24]
column :  [14 18 22]
what to call it ? :  [10 22]
```

In [14]:
```
arr = np.array([1,3,5,7,9,2,4,6,8])
print(arr[2:6])
print(arr[0:8:2])
print(arr[3:])
```

```
[5 7 9 2]
[1 5 9 4]
[7 9 2 4 6 8]
```

In [21]:
```
arr = np.array([10,20,30,40,50])
print(arr[:2])
print(arr[2:])
print(arr[::-1])
```

```
[10 20]
[30 40 50]
[50 40 30 20 10]
```

In [22]:
```
# changing by indexing
arr[3:]=100
print(arr)
```

```
[ 10  20  30 100 100]
```

In [23]:
```
arr[:3]=20
print(arr)
```

```
[ 20  20  20 100 100]
```

In [24]:
```
arr[::2]=0
print(arr)
```

```
[  0  20   0 100   0]
```

In [25]:
```
arr = np.array([10,20,30,40,50])
print(arr[-3:])
print(arr[-5:-2])
print(arr[-1::-2])
```

```
[30 40 50]
[10 20 30]
[50 30 10]
```

In [35]:
```
arr = np.array([[1,3,5,7],[9,11,13,15]])
print(arr[:2,:2])
```

```
[[ 1  3]
 [ 9 11]]
```

In [37]:
```
print(arr[:2,2:])
```

```
[[ 5  7]
 [13 15]]
```

In [39]:
```
arr = np.array([1,3,5,7,2,4,6,8])
result = np.reshape(arr,(2,4))
print(result)
```

```
[[1 3 5 7]
 [2 4 6 8]]
```

In [41]:
```
result = np.reshape(result,(2,2,2))
print(result)
```

```
[[[1 3]
  [5 7]]

 [[2 4]
  [6 8]]]
```

In [42]:
```
print(np.reshape(arr,(2,2,2)))
```

```
[[[1 3]
  [5 7]]

 [[2 4]
  [6 8]]]
```

In [43]:
```
# flattening or array
result = np.reshape(result,-1)
print(result)
```

```
[1 3 5 7 2 4 6 8]
```

In [47]:
```
# adding 2 arrays
arr1 = np.array([1,3,5,7])
arr2 = np.array([8,6,4,2])
print(arr1+arr2)
```

```
[9 9 9 9]
```

In [48]:
```
# mean,median,mode
marks = np.array([76,78,81,66,85])
print("mean : ",np.mean(marks))
print("median : ",np.median(marks))
```

```
print("minimum : ",np.min(marks))
print("maximum : ",np.max(marks))
```

```
mean :  77.2
median :  78.0
minimum :  66
maximum :  85
```

In [53]:
```
# less then , greater then , equal to
print("arr1 < arr2 : ",arr1 < arr2)
print("arr1 > arr2 : ",arr1 > arr2)
print("arr1 == arr2 : ",arr1 == arr2)
print("arr1 <= arr2 : ",arr1 <= arr2)
print("arr1 <= arr2 : ",np.less_equal(arr1,arr2))
print("arr1 >= arr2 : ",arr1 >= arr2)
```

```
arr1 < arr2 :  [ True  True False False]
arr1 > arr2 :  [False False  True  True]
arr1 == arr2 :  [False False False False]
arr1 <= arr2 :  [ True  True False False]
arr1 <= arr2 :  [ True  True False False]
arr1 >= arr2 :  [False False  True  True]
```

In [56]:
```
# column-wise axis=0
# row-wise axis=1 similar for mean,median,comparison
arr = np.array([[1,2,3],[4,5,6],[7,8,9]])
print(np.sum(arr,axis=0))
print(np.sum(arr,axis=1))
```

```
[12 15 18]
[ 6 15 24]
```

In [57]:
```
#slicing
arr=np.array([10,20,30,40,50,60,70,80])
print(arr[1:6])
```

```
[20 30 40 50 60]
```

In [60]:
```
#negative slicing
numbers=np.array([5,10,15,20,25,30,35,40])
print(numbers[-5:-1])
```

```
[20 25 30 35]
```

In [61]:
```
#slicing extract 2nd and 3d row
arr = np.array([[1,2,3,4],[5,6,7,8],[9,10,11,12],[13,14,15,16]])
print(arr[1:3,:])
```

```
[[ 5  6  7  8]
 [ 9 10 11 12]]
```

In [64]:
```
#slicing extract 1sr and 3d column
arr = np.array([[1,2,3,4],[5,6,7,8],[9,10,11,12],[13,14,15,16]])
print(arr[:,0:3:2])
```

```
[[ 1  3]
 [ 5  7]
 [ 9 11]
 [13 15]]
```

In [65]:
```
#numpy array 1 to 12 matrix 3x4
arr = np.arange(1,13)
```

```
arr = np.reshape(arr,(3,4))
print(arr)
```

```
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
```

In [66]:
```
#numpy array 1 to 24 in 3d shape
arr = np.arange(1,25)
arr = np.reshape(arr,(2,3,4))
print(arr)
```

```
[[[ 1  2  3  4]
  [ 5  6  7  8]
  [ 9 10 11 12]]

 [[13 14 15 16]
  [17 18 19 20]
  [21 22 23 24]]]
```

In [67]:
```
#numpy array 1 to 20 in 4x5 and 5x4 shape
arr = np.arange(1,21)
arr = np.reshape(arr,(4,5))
print(arr)
arr = np.reshape(arr,(5,4))
print(arr)
```

```
[[ 1  2  3  4  5]
 [ 6  7  8  9 10]
 [11 12 13 14 15]
 [16 17 18 19 20]]
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]
 [13 14 15 16]
 [17 18 19 20]]
```

In [ ]:

Department of AI_DS
Academic Year: 2026-27 (Odd Sem)
Course: Python for Data Science Lab
Second Year B. Tech Semester: III

PRACTICAL 5

Part A (To be referred by students)

NumPy

SAVE THE FILE AND UPLOAD AS (RollNo_Name_Exp5)

Aim: To understand and apply basic and advanced NumPy operations and built-in matrixfunctions, focusing on their application in data science and machine learning tasks.

Problem Statement:

Solve the following computational problems by **recognizing** python data structures of NumPy and **apply** them to implement programs

1. Create a 3x3 matrix of random integers between 1 and 10 and find the total sum of its elements.
2. Generate a 4x4 matrix of random numbers and compute the average of all its elements.
3. Create a 2x3 matrix and find the sum of all elements in the second row.
4. Generate a 5x5 matrix of random integers and determine the highest value in each column.
5. Create a 4x4 matrix and calculate the sum of its main diagonal elements.
6. Generate a 3x3 matrix and find the smallest value among all its elements.
7. Create a 4x4 matrix of random integers between 1 and 20 and compute the sum of all elements that are greater than 10.
8. Generate a 5x5 matrix of random numbers and find the mean of the maximum values of each row.
9. Create a 3x3 matrix and calculate the difference between the sum of the elements in the first row and the sum of the elements in the last row.
10. Generate a 6x6 matrix of random integers and replace all values below the median with zero.
11. Create a 4x4 matrix and compute the sum of elements in the upper triangular part of the matrix.
12. Generate a 3x4 matrix of random values and find the standard deviation of the elements in the third column.

Topic covered: Numpy

Learning Objective: Learner would be able to

1. Learn how to import packages in python
2. Features and available functions of Numpy

3. Create multi-dimensional data structures and manipulate them

Theory:

- NumPy is 'Numerical Python package consisting of multidimensional array (ndarray) objects and a collection of routines for processing of array.

#Numpy with one dimension import numpy as np a = np.array([1,2,3]) print(a) O/P - [1, 2, 3]	# more than one dimensions import numpy as np a = np.array([[1, 2], [3, 4]]) print(a) O/P - [[1, 2] [3, 4]]

NumPy's array class is called ndarray. The important attributes of an ndarray object are:
ndarray.ndim - the number of axes (dimensions) of the array.
ndarray.shape - the dimensions of the array. This is a tuple of integers indicating the size of the array in each dimension. For a matrix with n rows and m columns, shape will be (n,m). The length of the shape tuple is therefore the number of axes, ndim.
ndarray.size -the total number of elements of the array. This is equal to the product of the elements of shape.
ndarray.dtype -an object describing the type of the elements in the array. One can create or specify dtype's using standard Python types. Additionally, NumPy provides types of its own. numpy.int32, numpy.int16, and numpy.float64 are some examples.
ndarray.itemsize - the size in bytes of each element of the array. For example, an array of elements of type float64 has itemsize 8 (=64/8), while one of type complex32 has itemsize 4 (=32/8). It is equivalent to ndarray.dtype. itemsize.
ndarray.data - the buffer containing the actual elements of the array. Normally, we won't need to use this attribute because we will access the elements in an array using indexing facilities.
Example –

import numpy as np a = np.arange(15).reshape(3, 5) print(a) array([[0, 1, 2, 3, 4], [5, 6, 7, 8, 9], [10, 11, 12, 13, 14]]) print(a.shape) (3, 5) print(a.ndim) 2 print(a.dtype.name) 'int64'	print(a.itemsize) 8 print(a.size) 15 print(type(a)) <class 'numpy.ndarray'> print(b = np.array([6, 7, 8]) print(b) array([6, 7, 8]) print(type(b)) <class 'numpy.ndarray'>

Department of AI_DS
Academic Year: 2026-27 (Odd Sem)
Course: Python for Data Science Lab
Second Year B. Tech Semester: III

Array Creation There are several ways to create arrays. For example, you can create an array from a regular Python list or tuple using the array function

import numpy as np a = np.array([2, 3, 4]) print(a) array([2, 3, 4]) print(a.dtype) dtype('int64') print(b = np.array([1.2, 3.5, 5.1]) print(b.dtype) dtype('float64')	print(a = np.array(1, 2, 3, 4) # WRONG Traceback (most recent call last): ... TypeError: array() takes from 1 to 2 positional arguments but 4 were given print(a = np.array([1, 2, 3, 4]) # RIGHT	b = np.array([(1.5, 2, 3), (4, 5, 6)]) print(b) array([[1.5, 2. , 3.], [4, 5 , 6]]) **The type of the array can also be explicitly specified at creation time:** print(c = np.array([[1, 2], [3, 4]], dtype=**complex**) print(c) array([[1.+0.j, 2.+0.j], [3.+0.j, 4.+0.j]])

To create sequences of numbers, NumPy provides the arange function which is analogous to the Python built-in range, but returns an array.
print(np.arange(10, 30, 5))
array([10, 15, 20, 25])
print(np.arange(0, 2, 0.3)) # it accepts float arguments
array([0. , 0.3, 0.6, 0.9, 1.2, 1.5, 1.8])

Universal Functions
NumPy provides familiar mathematical functions such as sin, cos, and exp

B = np.arange(3) print(B) array([0, 1, 2]) print(np.exp(B)) array([1. , 2.71828183, 7.3890561])	print(np.sqrt(B)) array([0. , 1. , 1.41421356]) print(C = np.array([2., -1., 4.]) print(np.add(B, C)) array([2., 0., 6.])

Changing the **shape** of an array

a = np.floor(10 * rg.random((3, 4)))
print(a)
array([[3., 7., 3., 4.],
[1., 4., 2., 2.],
[7., 2., 4., 9.]])
print(a.shape)
(3, 4)

The shape of an array can be changed with various commands. Note that the following three commands all return a modified array, but do not change the original array:

```
print( a.ravel()) # returns the array, flattened
array([3., 7., 3., 4., 1., 4., 2., 2., 7., 2., 4., 9.])
print( a.reshape(6, 2)) # returns the array with a modified shape
array([[3., 7.],
[3., 4.],
[1., 4.],
[2., 2.],
[7., 2.],
[4., 9.]])
print( a.T) # returns the array, transposed
array([[3., 1., 7.],
[7., 4., 2.],
[3., 2., 4.],
[4., 2., 9.]])
print( a.T.shape)
(4, 3)
print( a.shape)
(3, 4)
```

Department of AI_DS
Academic Year: 2026-27 (Odd Sem)
Course: Python for Data Science Lab
Second Year B. Tech Semester: III

PRACTICAL 5

Part B (to be completed by students)

Numpy

1. All the students are required to perform the given tasks in Jupyter Notebook
2. Create a new notebook for each experiment. The filename should be RollNo_Name_Exp5)
3. In the first cell, the student must write his/her Name, roll no and class in the form of comments
4. Every program should be written in separate cells and in the given sequence
5. After completing the experiment, download the notebook in pdf format. The filename should be RollNo_Name_Exp5.pdf).
   Note: - To download as a PDF, go to File and choose Print Preview. This will open a new tab. Now, press Ctrl + P and opt to save as PDF.
6. Upload the pdf on the web portal

In [1]:
```
import numpy as np
```

In [6]:
```
#sum of element of 3 x 3
arr3d = np.random.randint(1,11,size=(3,3))
print(arr3d)
Sum = np.sum(arr3d)
print("Sum of all Elements : ",Sum)
```

```
[[ 2  5  2]
 [ 4  9  4]
 [ 2  6 10]]
Sum of all Elements :  44
```

In [7]:
```
#avg of element of 4 x 4
arr4d = np.random.rand(4,4)
print(arr4d)
avg = np.mean(arr4d)
print("Average of all Elements : ",avg)
```

```
[[0.31008133 0.30666187 0.03657576 0.03351542]
 [0.40332796 0.04365721 0.43461072 0.13389531]
 [0.86378715 0.21645602 0.27535305 0.07617275]
 [0.8475716  0.15397093 0.68304676 0.56588243]]
Average of all Elements :  0.33653539215167105
```

In [9]:
```
#sum of element of 2 x 3
arr2x3 = np.random.rand(2,3)
print(arr2x3)
Sum = np.sum(arr2x3[:,1])
print("Sum of all Elements of 2nd row : ",Sum)
```

```
[[0.62572517 0.34119676 0.20156804]
 [0.80421055 0.28441033 0.87016062]]
Sum of all Elements of 2nd row :  0.6256070889323414
```

In [10]:
```
# find highest element in 5x5
arr5x5 = np.random.randint(1,10001,size=(5,5))
print(arr5x5)
Max = np.max(arr5x5)
print("Max among all Elements : ",Max)
```

```
[[1508 4004 1511 6881 7283]
 [ 530 7680  213 7470 5627]
 [1296 2579 2639 3159 1268]
 [1358 4418 7124  343 2146]
 [2876 8985  344  278 2659]]
Max among all Elements :  8985
```

In [13]:
```
# find trace in 4x4
arr4x4 = np.random.rand(4,4)
print(arr4x4)
tr = np.trace(arr4x4)
print("sum of all diagonal elements : ",tr)
```

```
[[0.38969828 0.32538589 0.29082692 0.38099127]
 [0.87866076 0.99828621 0.93642776 0.59968742]
 [0.3412576  0.03916221 0.23970246 0.04576408]
 [0.48057372 0.27434913 0.94587712 0.26478667]]
sum of all diagonal elements :  1.8924736319021438
```

In [15]:
```
# find highest element in 3x3
arr3x3 = np.random.rand(3,3)
print(arr3x3)
Min = np.min(arr3x3)
print("Minimum among all Elements : ",Min)
```

```
[[0.38374901 0.23625982 0.61691999]
 [0.14352691 0.03455422 0.05900485]
 [0.64946168 0.99616589 0.63250666]]
Minimum among all Elements :  0.03455422138235509
```

In [16]:
```
# sum all elements > 10 in 3X3
arr3x3 = np.random.randint(1,21,size=(3,3))
print(arr3x3)
Sum = np.sum(arr3x3[arr3x3 > 10])
print("Sum of all Elements of 2nd row : ",Sum)
```

```
[[17  5  1]
 [13  1  4]
 [10  6 20]]
Sum of all Elements of 2nd row :  50
```

In [18]:
```
# Generate a 5x5 matrix of random numbers and find the mean of the maximum value
arr5x5 = np.random.randint(1, 101, size=(5, 5))
print(arr5x5)
row_maxima = np.max(arr5x5, axis=1)
print("Row Max :", row_maxima)
mean_of_maxima = np.mean(row_maxima)
print("Mean of Max :", mean_of_maxima)
```

```
[[37 73 95 60 10]
 [21 89 84 70 62]
 [73 34 42 12 89]
 [24 71  6 20 25]
 [ 2 63 97 58 16]]
Row Maxima: [95 89 89 71 97]
Mean of Maxima: 88.2
```

In [20]:
```
# Create a 3x3 matrix and calculate the difference between the sum of the elemen
# the sum of the elements in the last row.
arr3x3 = np.random.randint(1, 21, size=(3, 3))
print(arr3x3)
sum_first_row = np.sum(arr3x3[0, :])
sum_last_row = np.sum(arr3x3[-1, :])
difference = sum_first_row - sum_last_row
print("Difference between sum of first and last row : ", difference)
```

```
[[ 9 19 15]
 [18  5  5]
 [14  9 16]]
Difference between sum of first and last row :  4
```

In [22]:
```
# Generate a 6x6 matrix of random integers and replace all values below the medi
arr6x6 = np.random.randint(1, 101, size=(6, 6))
print(arr6x6)
median_val = np.median(arr6x6)
print("Median value : ",median_val)
arr6x6[arr6x6 < median_val] = 0
print("Matrix after replacing values below median with zero:\n", arr6x6)
```

```
[[18 53  2 18 50 81]
 [12 16 15 92 29 77]
 [58 48 29 36 57 80]
 [33 69 16 18 48  9]
 [16 51 16 96 70 96]
 [56 25 44 28  7 47]]
Median value :  40.0
Matrix after replacing values below median with zero:
 [[ 0 53  0  0 50 81]
 [ 0  0  0 92  0 77]
 [58 48  0  0 57 80]
 [ 0 69  0  0 48  0]
 [ 0 51  0 96 70 96]
 [56  0 44  0  0 47]]
```

In [23]:
```python
# Create a 4x4 matrix and compute the sum of elements in the upper triangular pa
arr4x4 = np.random.randint(1, 101, size=(4, 4))
print(arr4x4)
upper_tri_elements = arr4x4[np.triu_indices(4)]
sum_upper_tri = np.sum(upper_tri_elements)
print("Sum of elements in the upper triangular part : ", sum_upper_tri)
```

```
[[90 84 62 47]
 [ 8 97 73 54]
 [97 85 61 23]
 [33 95 90 53]]
Sum of elements in the upper triangular part :  644
```

In [24]:
```python
# Generate a 3x4 matrix of random values and find the standard deviation of the
arr3x4 = np.random.randint(1, 101, size=(3, 4))
print(arr3x4)
third_column_std = np.std(arr3x4[:, 2])
print("Standard deviation : ", third_column_std)
```

```
[[19 65 16 88]
 [12 74 19 34]
 [69 18 89 82]]
Standard deviation :  33.7276675083759
```

In [ ]:
```python

```

Department of Artificial Intelligence (AI) & Data Science

AY: 2026-27
Course: Python for Data Science Laboratory (DJS23SLPC303)
Class: SE/ Sem: III Roll No.: SAPID:

Experiment No. 5: NumPy

Aim: To understand and apply basic and advanced NumPy operations and built-in matrix functions, focusing on their application in data science and machine learning tasks.

Theory

1. What is array slicing? Provide an example where you extract a sub-array from a 2D NumPy array.

2. Describe element-wise operations in NumPy. Why are they useful in data science applications?

3. What is the purpose of using functions like np.zeros(), np.ones(), and np.empty()? When would you use each?

4. Explain the difference between np.arange() and np.linspace(). Provide a use-case for each function.

5. How does reshaping an array work in NumPy? What will happen if the total number of elements does not match the new shape?

6. What is matrix multiplication in NumPy, and how is it different from element-wise multiplication?

7. Explain what broadcasting is in NumPy. Provide an example where broadcasting simplifies operations on arrays of different shapes.

8. Describe how you would normalize a dataset using NumPy. Why is normalization important in machine learning tasks?

9. What is a covariance matrix, and how does NumPy help in computing it? Explain its importance in data science.

Conclusions:

Name :- Aaryan N. Raorane Roll No :- 5004

Branch :- AI-DS batch : A1 SAP ID : 60018250070

Experiment No 5 : Numpy

Aim : To understand and apply basic and advanced Numpy operations and built in matrix functions, focusing on their application in data science and machine learning tasks.

Theory :

1. What is array slicing ? Provide an example where you extract a sub array from a 2D Numpy array.

=> Array slicing is a technique in Numpy that allows extraction of a contiguous portion of an array by specifying index ranges along each dimension. For a 2D array, the syntax is

```
array[row_start : row_end, col_start : col_end]
```

The indices follow Python conventions, where the start index is inclusive and the end index is exclusive. Slicing returns a view, not copy, meaning modifications to the slice affect the original array

Example :

```
import numpy as np
arr = np.array([[10,20,30,40],
                [50,60,70,80],
                [90,100,110,120]])
sub = arr[0:2, 1:3]
print("original array:")
print(arr)
```

Sundaram FOR EDUCATIONAL USE

```
print ("Extracted subarray:")
print (sub)
```

output:

```
Original Array:
[[10 20 30 40]
 [50 60 70 80]
 [90 100 110 120]]
Extracted Subarray:
[[20 30]
 [60 70]]
```

2. Describe element-wise operations in Numpy. Why are they useful in data science applications?

=> Elementwise operations in Numpy are arithmetic or mathematical operations applied independently to each corresponding pair of elements across two arrays of the same shape. Numpy vectorizes these computations internally using optimized C routines, which eliminates the need for explicit Python loops and result in significantly faster execution. In data science, elementwise operations are widely used for feature scaling, computing residuals between predicted and actual value, applying transformations such as squaring or taking square roots across an entire dataset, and performing column-wise arithmetic on numerical data.

Sundaram®
FOR EDUCATIONAL USE

Example :-

```
import numpy as np
a= np.array([2,4,6,8])
b= np.array([1,3,5,7])

print("a+b:", a+b)
print("a*b:", a*b)
print("a**2:", a**2)
print("np.sqrt(a):", np.sqrt(a))
```

output :-

```
a+b: [3 7 11 15]
a*b: [2 12 30 56]
a**2: [4 16 36 64]
np.sqrt(a): [1.41421356 2.
             2.44948974 2.82842712]
```

3. What is the purpose of using ~~like np~~ functions like np.zeros(), np.ones(), and np.empty()? When would you use each?

⇒ These three functions are used to create Numpy arrays of a specified shape with requiring explicit data input. Each serves a distinct initialization purpose.

i) np.zeros(shape) creates an array where every element is initialized to 0.0 It is used when a zero ~~beta~~ baseline is required, such as initializing weight matrices before training a neural network or creating placeholder arrays for accumulating sums.

Sundaram
FOR EDUCATIONAL USE

ii) np.ones(shape) creates an array where every element to 1.0. It is commonly used to construct bias vectors, mask arrays, or initial arrays where a multiplicative identity is needed.

iii) np.empty(shape) creates an array without initializing its values. The memory is allocated but the contents are arbitary (garbage values from previously stored data). It is the fastest of the three fn and is appropriate only when the array will be completely overwritten before use, making it useful in performance critical numerical routines.

Example: import numpy as np

```
z = np.zeros((2,3))
o = np.ones((2,3))
e = np.empty((2,3))
print("np.zeros((2,3)):\n", z)
print("np.ones((2,3)):\n", o)
print("np.empty((2,3)):\n", e)
```

Output:
```
np.zeros((2,3)):
[[0. 0. 0.]
 [0. 0. 0.]]
np.ones((2,3)):
[[1. 1. 1.]
 [1. 1. 1.]]
```

Sundaram FOR EDUCATIONAL USE

np.empty((2,3)):

[[1.0094556 6e-315 0.00000000e+000
0.00000000e+000]
[0.00000000e+000 0.00000000e+000
0.00000000e+000]]

4. Explain the difference between np.arange() and np.linspace(). Provide a use-case for each function.

=>

Feature	np.range()	np.linspace()
control parameter	step size (interval between values)	Number of evenly spaced points.
Endpoint included	No (stop is exclusive)	Yes (stop is included by default)
output data type	Inferred from arguments (int or float)	Always float 64
Primary use case	Iterating over integer index ranges	Generating uniform sample points for plots or interpolation.
Predictability of Endpoint	Endpoint may be omitted due to floating point rounding	Endpoint is always guaranteed.

Sundaram®
FOR EDUCATIONAL USE

Use case for np.arange(): Generating indices for iteration, such as creating a sequence of pixel values from 0 to 255 in step of 1.

Use case for np.linspace(): Generating uniformly distributed x values for plotting a mathematical function or creating equally spaced sample points for signal processing.

Example: import numpy as np

```
r = np.arange(0,10,2)
print("np.arange(0,10,2):", r)

l = np.linspace(0,1,5)
print("np.linspace(0,1,5):", l)
```

Output: np.arange(0,10,2): [0 2 4 6 8]
np.linspace(0,1,5): [0 0.25 0.5 0.75 1.]

5. How does reshaping an array work in Numpy? What will happen if the total number of elements does not match the new shape?

=> Reshaping in Numpy refers to changing the dimensional structure of an array without altering any of its underlying data values. The total number of elements must remain identical before and after the reshape operation. Numpy simply reinterprets the flat memory

Sundaram®
FOR EDUCATIONAL USE

layout of the array under new shape specification. If the product of the new shape dimension does not equal the total number of Element in the orignal array, Numpy raises a ValueError at runtime. For example, an array of 12 elements can be reshaped to (3,4), (2, 6), (4,3), or (12,), but attempting to reshape it to (3, 5) which requires 15 elements will cause an Error

Example: import numpy as np

```
a = np.arange(1,13)
print("orignal:", a)

b = a.reshape(3, 4)
print("Reshaped to (3,4):\n", b)

try:
    c = a.reshape(3,5)
except ValueError as e:
    print("Error:", e)
```

output: orignal: [1 2 3 4 5 6 7 8 9 10 11 12]
Reshaped to (3, 4):
[[1 2 3 4]
[5 6 7 8]
[9 10 11 12]]
error: cannot reshape array of size 12 into shape (3,5)

Sundaram
FOR EDUCATIONAL USE

6. What is matrix multiplication in Numpy, and how is it different from element wise multiplication?

=>

Aspect	Elementwise Multiplication	Matrix multiplication
operator / function	* operator	@ operator or np.matmul()
operation performed	Multiplies each element at position (i,j) in A with the element at the same position (i,j) in B	computes the dot product of rows of A with columns of B
Shape requirement	Both arrays must have identical shapes	Number of columns in A must equal number of rows in B.
Result shape	Same as input arrays	depends on outer dimensions of operands
Mathematical meaning	No standard linear algebra equivalent	standard matrix product from linear algebra

Sundaram
FOR EDUCATIONAL USE

Typical use case	masking, scaling, Hadamard product in ML	Linear transformation, neural network forward pass, solving systems of equations.

Example: import numpy as np

```
A = np.array([[1,2], [3,4]])
B = np.array([[5,6], [7,8]])

print("Elementwise (A * B):\n", A * B)
print("Matrix Multiplication (A @ B):\n", A @ B)
```

Output:
```
Elementwise (A * B):
[[5 12]
 [21 32]]
Matrix Multiplication (A @ B):
[[19 22]
 [43 50]]
```

7. Explain what broadcasting is in Numpy. Provide an example where broadcasting simplifies operations on arrays of different shapes.

⟹ Broadcasting is the mechanism in Numpy that allows arithmetic operations to be performed between arrays of different shapes without explicitly copying the data; Numpy automatically stretches the smaller

Sundaram
FOR EDUCATIONAL USE

array along dimensions where it has size 1, so that it matches the shape of the larger array during computation. The rules governing broadcasting are applied by comparing dimension from the trailing end. Two dimensions are compatible if they are equal or if one of them is 1. If neither condition holds, Numpy raises a ValueError. Broadcasting avoids the memory overhead of creating expanded copies and allows concise, readable code.

Example: adding a row vector of shape (3,) to a matrix of shape (3,3). The row vector is broadcast across all three rows of the matrix

```
import numpy as np

matrix = np.array([ [1,2,3],
                    [4,5,6],
                    [7,8,9] ])
row = np.array([10, 20, 30])
result = matrix + row
print("Matrix + Row (Broadcasting):\n", result)
```

Output:
```
Matrix + Row (Broadcasting):
[[11 22 33]
 [14 25 36]
 [17 28 39]]
```

Sundaram
FOR EDUCATIONAL USE

8. Describe how you would normalize a dataset using Numpy. Why is normalization important in machine learning tasks?

=> Normalization scales all values in a dataset to fixed range, typically [0, 1], using the min-max formula:

$$normalized = (x - x_{min}) / (x_{max} - x_{min})$$

This transformation ensures that smallest value in the dataset maps to 0 and the largest maps to 1, with all other values scaled proportionally in between. Normalization is critical in machine learning for several reasons. Algorithms based on distance computation such as K-Nearest neighbours and support vector machines are heavily biased toward features with larger numerical scales if normalization is not applied. Gradient descent based algorithm converge significantly faster when all input features are on a comparable scale, avoiding situations where the loss surface is elongated along certain dimensions.

Example:

```
import numpy as np
data = np.array([10.0, 20.0, 30.0, 40.0, 50.0])
normalized = (data - data.min()) / (data.max() - data.min())
print("Original data:", data)
print("Normalized data:", normalized)
```

Sundaram
FOR EDUCATIONAL USE

output:

Original data: [10. 20. 30. 40. 50.]
Normalized data: [0. 0.25 0.5 0.75 1.]

9. What is a covariance Matrix, and how does numpy help in computing it? Explain its importance in data science.

⟹ A covariance Matrix is a square symmetric matrix where the entry at position (i, j) represents the covariance between the ith and jth features of a dataset. The diagonal entries represent the variance of each individual feature, while the off-diagonal entries indicate how two features vary together. A positive covariance between features i and j means they tend to increase and decrease together, while a negative covariance means they vary in opposite directions. Numpy computes the covariance matrix using np.cov(), where each row of the input matrix is treated as one variable (feature) and each column as one observation. The function applies the sample covariance formula, dividing by N minus 1 rather than N to produce an unbiased estimate. The covariance matrix is central to several data science techniques. In principal component analysis (PCA), the eigenvectors of the covariance

Sundaram
FOR EDUCATIONAL USE

matrix define the principal components, enabling dimensionality reduction in multivariate statistics, it is used for linear discriminant analysis, Mahalanobis distance computation, and gaussian distribution modelling

Example:

```
import numpy as np

data = np.array([[2,3,5],[7,4,6],[1,8,9]])
cov = np.cov(data)
print("Dataset:\n", data)
print("Covariance:\n", cov)
```

Output:

```
Dataset:
[[2 3 5]
 [7 4 6]
 [1 8 9]]
Covariance Matrix:
[[2.33333333 -0.33333333  5.5       ]
 [-0.33333333 2.33333333 -4.5       ]
 [5.5        -4.5         19.       ]]
```

Sundaram®
FOR EDUCATIONAL USE

conclusions:

This experiment showed that Numly offers an efficient framework for numerical computing, covering array creation, slicing, reshaping, and vectorized mathematical operations that eliminate explicit loops and boost performance. Matrix operations like multiplication and covariance computation confirmed Numly's accurate implementation of linear algebra fundamentals underlying ML algorithms such as PCA and linear regression. Techniques like min-max normalization and covariance computation, applied to real datasets, established clear link between Numly's API and core data science workflows.

Sundaram®
FOR EDUCATIONAL USE

Pandas

In [2]:
```
import pandas as pd
```

```
C:\Users\djsce.student\AppData\Local\Continuum\anaconda3\lib\site-packages\pandas
\compat\_optional.py:138: UserWarning: Pandas requires version '2.7.0' or newer o
f 'numexpr' (version '2.6.8' currently installed).
  warnings.warn(msg, UserWarning)
```

In [3]:
```
data = [10,20,30,'Jhon',50]
my_series = pd.Series(data)
print(my_series)  #default labelels 0,1,2
```

```
0      10
1      20
2      30
3    Jhon
4      50
dtype: object
```

In [4]:
```
print(my_series[2])
```

```
30
```

In [5]:
```
a = [1,3,5]
my_series = pd.Series(a,index = ["X","Y","Z"]);
print(my_series)
```

```
X    1
Y    3
Z    5
dtype: int64
```

In [6]:
```
print(my_series["X"])
```

```
1
```

In [7]:
```
grades = {"sem1":3.25,"sem2":3.28,"sem3":3.75}
my_grades = pd.Series(grades)  #We can use dictionary to give labels
print(my_grades)
```

```
sem1    3.25
sem2    3.28
sem3    3.75
dtype: float64
```

In [8]:
```
my_series = pd.Series(grades,index = ["sem1","sem2","sem4"])  #values not existi
print(my_series)
```

```
sem1    3.25
sem2    3.28
sem4     NaN
dtype: float64
```

DataFrames

In [9]:
```
data = {"Name":["John","Alice","Bob"],
        "Age":[25,30,35],
        "City":["New York","London","Paris"]}
df = pd.DataFrame(data,index = [1,2,3])
print(df)
```

```
    Name  Age      City
1   John   25  New York
2  Alice   30    London
3    Bob   35     Paris
```

In [10]:
```
data = [["Alice",25,"New York"],
        ["John",30,"London"],
        ["Bob",35,"Paris"]]
df = pd.DataFrame(data,columns = ["Name","Age","City"])
print(df)  #data shall be entered properly else any value is entered anywhere
```

```
    Name  Age      City
0  Alice   25  New York
1   John   30    London
2    Bob   35     Paris
```

In [11]:
```
import numpy as np
data_array=  np.array([["ProductA",15.50,100],
                       ["ProductB",22.00,45],
                       ["ProductC",9.99,250]])
df = pd.DataFrame(data_array,columns = ["item","Price","stcok"])
print(df)
```

```
       item Price stcok
0  ProductA  15.5   100
1  ProductB  22.0    45
2  ProductC  9.99   250
```

In [12]:
```
df = pd.DataFrame()
print(df)
```

```
Empty DataFrame
Columns: []
Index: []
```

In [13]:
```
import requests

url = 'https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic
response = requests.get(url)

with open('titanic.csv', 'wb') as f:
    f.write(response.content)

print('titanic.csv downloaded successfully.')
```

```
titanic.csv downloaded successfully.
```

In [14]:
```
df = pd.read_csv('titanic.csv')
display(df.head())
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fa
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.250
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.283
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.925
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.100
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.050

In [15]:
```
print(df.shape)
```

```
(891, 12)
```

In [16]:
```
data = {"Name":["John","Alice","Bob"],
        "Age":[25,30,35],
        "City":["New York","London","Paris"]}
df = pd.DataFrame(data)
df.set_index("Name",inplace = True)
print(df) #treated as index not as column
```

```
       Age      City
Name
John    25  New York
Alice   30    London
Bob     35     Paris
```

In [17]:
```
data = {"Name":["John","Alice","Bob"],
        "Age":[25,30,35],
        "City":["New York","London","Paris"]}
df = pd.DataFrame(data,index = pd.RangeIndex(5,8,name = "index"))  # index cha n
print(df)
```

```
        Name  Age      City
index
5       John   25  New York
6      Alice   30    London
7        Bob   35     Paris
```

In [18]:
```
print(df.iloc[2])
```

```
Name      Bob
Age        35
City    Paris
Name: 7, dtype: object
```

In [19]:
```
data = {"Name":["John","Alice","Bob"],
        "Age":[25,30,35],
        "City":["New York","London","Paris"]}
df = pd.DataFrame(data)
print("Original df")
print(df)
#rename index
df.rename(index = {0:'A',1:'B',2:'C'},inplace = True)  #inplace = same data fram
print("Modified")
print(df)

#reset index
df.reset_index(inplace = True)
print("Reset")
print(df)
```

```
Original df
    Name  Age      City
0   John   25  New York
1  Alice   30    London
2    Bob   35     Paris
Modified
    Name  Age      City
A   John   25  New York
B  Alice   30    London
C    Bob   35     Paris
Reset
  index   Name  Age      City
0     A   John   25  New York
1     B  Alice   30    London
2     C    Bob   35     Paris
```

In [20]:
```
print(df.index)
print(df.index.values)
```

```
RangeIndex(start=0, stop=3, step=1)
[0 1 2]
```

In [21]:
```
arr=pd.array([1,2,3],dtype="Int64")
print(arr)
```

```
<IntegerArray>
[1, 2, 3]
Length: 3, dtype: Int64
```

In [22]:
```
s = pd.Series([1,2,3],index=["a","b","c"],dtype="Int64")
print(s)
```

```
a    1
b    2
c    3
dtype: Int64
```

In [23]:
```
pip install --upgrade pandas
```

```
The following command must be run outside of the IPython shell:

    $ pip install --upgrade pandas

The Python package manager (pip) can only be used from outside of IPython.
Please reissue the `pip` command in a separate terminal or command prompt.

See the Python documentation for more information on how to install packages:

    https://docs.python.org/3/installing/
```

In [24]:
```python
string_array = pd.array(['apple','banana','cherry','date'],dtype='str')
print(string_array)
print()
```

```
<PandasArray>
['apple', 'banana', 'cherry', 'date']
Length: 4, dtype: str192
```

In [25]:
```python
import pandas as pd
print("New Version:", pd.__version__)

arr = pd.array([1, 2, 3], dtype="Int64")
print(arr)
```

```
New Version: 1.3.5
<IntegerArray>
[1, 2, 3]
Length: 3, dtype: Int64
```

In [26]:
```python
data = {
    'Name':['John','Alice','Bob','Emma','Mike','Sarah','David','Linda','Tom','Em
    'Age':[25,30,35,28,32,27,40,33,29,31],
    'City':['New York','Paris','London','Sydney','Tokyo','Berlin','Rome','Madrid
}
df=pd.DataFrame(data)
print(df)
print()
print(df.head(3))
print()
print(df.tail(3))
```

```
    Name  Age      City
0   John   25  New York
1  Alice   30     Paris
2    Bob   35    London
3   Emma   28    Sydney
4   Mike   32     Tokyo
5  Sarah   27    Berlin
6  David   40      Rome
7  Linda   33    Madrid
8    Tom   29   Toronto
9  Emily   31    Moscow

    Name  Age      City
0   John   25  New York
1  Alice   30     Paris
2    Bob   35    London

    Name  Age     City
7  Linda   33   Madrid
8    Tom   29  Toronto
9  Emily   31   Moscow
```

In [27]:
```
df.info()
df.describe()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype 
---  ------  --------------  ----- 
 0   Name    10 non-null     object
 1   Age     10 non-null     int64 
 2   City    10 non-null     object
dtypes: int64(1), object(2)
memory usage: 368.0+ bytes
```

Out[27]:

	Age
count	10.000000
mean	31.000000
std	4.320494
min	25.000000
25%	28.250000
50%	30.500000
75%	32.750000
max	40.000000

In [28]:
```
data = {
    'Name':['John','Emma','Michael','Sophia'],
    'Height':[5.5,6.0,5.8,5.3],
    'Qualification':['BSc','BBA','MBA','BSc']
}
df = pd.DataFrame(data)
print(df)
print()
```

```
address = ['New York','London','Sydney','Toronto']
df['Address']=address
print(df)
```

```
      Name  Height Qualification
0     John     5.5           BSc
1     Emma     6.0           BBA
2  Michael     5.8           MBA
3   Sophia     5.3           BSc

      Name  Height Qualification   Address
0     John     5.5           BSc  New York
1     Emma     6.0           BBA    London
2  Michael     5.8           MBA    Sydney
3   Sophia     5.3           BSc   Toronto
```

In [29]:
```
df.loc[len(df.index)]=['Amy',5.2,'BIT',None]
print(df)
```

```
      Name  Height Qualification   Address
0     John     5.5           BSc  New York
1     Emma     6.0           BBA    London
2  Michael     5.8           MBA    Sydney
3   Sophia     5.3           BSc   Toronto
4      Amy     5.2           BIT      None
```

In [30]:
```
df.drop(4,axis=0,inplace=True)
print(df)
```

```
      Name  Height Qualification   Address
0     John     5.5           BSc  New York
1     Emma     6.0           BBA    London
2  Michael     5.8           MBA    Sydney
3   Sophia     5.3           BSc   Toronto
```

In [31]:
```
df.drop([1,3],axis=0,inplace=True)
print(df)
```

```
      Name  Height Qualification   Address
0     John     5.5           BSc  New York
2  Michael     5.8           MBA    Sydney
```

In [32]:
```
data={
    'Name':['Alice','Bob','Charlie','David'],
    'Age':[25,30,35,40],
    'City':['New York','London','Paris','Tokyo'],
    'Height':['165','178','185','171'],
    'Profession':['Engineer','Entrepreneur','Unemployed','Actor'],
    'Marital Status':['Single','Marries','Divorced','Engaged']
}
df = pd.DataFrame(data)
print(df)
```

```
      Name  Age      City Height    Profession Marital Status
0    Alice   25  New York    165      Engineer         Single
1      Bob   30    London    178  Entrepreneur        Marries
2  Charlie   35     Paris    185    Unemployed       Divorced
3    David   40     Tokyo    171         Actor        Engaged
```

In [33]:
```
df.drop('Age',axis=1,inplace=True)
print(df)
```

```
      Name      City Height    Profession Marital Status
0    Alice  New York    165      Engineer         Single
1      Bob    London    178  Entrepreneur        Marries
2  Charlie     Paris    185    Unemployed       Divorced
3    David     Tokyo    171         Actor        Engaged
```

In [34]:
```
df.drop(columns='Marital Status',inplace=True)
print(df)
```

```
      Name      City Height    Profession
0    Alice  New York    165      Engineer
1      Bob    London    178  Entrepreneur
2  Charlie     Paris    185    Unemployed
3    David     Tokyo    171         Actor
```

In [35]:
```
df.drop(['Height','Profession'],axis=1,inplace=True)
print(df)
```

```
      Name      City
0    Alice  New York
1      Bob    London
2  Charlie     Paris
3    David     Tokyo
```

In [36]:
```
data={
    'Name':['Alice','Bob','Charlie','David'],
    'Age':[25,30,35,40],
    'City':['New York','London','Paris','Tokyo']
}
df = pd.DataFrame(data)
print(df)
```

```
      Name  Age      City
0    Alice   25  New York
1      Bob   30    London
2  Charlie   35     Paris
3    David   40     Tokyo
```

In [37]:
```
df.rename(columns={'Name':'First_Name'},inplace=True)
print(df)
```

```
  First_Name  Age      City
0      Alice   25  New York
1        Bob   30    London
2    Charlie   35     Paris
3      David   40     Tokyo
```

In [38]:
```
df.rename(columns={'Age':'Number','City':'Address'},inplace=True)
print(df)
```

```
  First_Name  Number   Address
0      Alice      25  New York
1        Bob      30    London
2    Charlie      35     Paris
3      David      40     Tokyo
```

In [39]:
```
df.rename(index={0:7},inplace=True)
print(df)
```

```
  First_Name  Number   Address
7      Alice      25  New York
1        Bob      30    London
2    Charlie      35     Paris
3      David      40     Tokyo
```

In [40]:
```
data={
    'Name':['Alice','Bob','Charlie','David','Eve'],
    'Age':[25,30,35,40,18],
    'City':['New York','London','Paris','Tokyo','Sydney'],
    'Height':['165','178','185','171','179'],
    'Profession':['Engineer','Entrepreneur','Unemployed','Actor','Professor'],
    'Marital Status':['Single','Married','Divorced','Engaged','Single']
}
df = pd.DataFrame(data)
print(df)
```

```
      Name  Age      City Height    Profession Marital Status
0    Alice   25  New York    165      Engineer         Single
1      Bob   30    London    178  Entrepreneur        Married
2  Charlie   35     Paris    185    Unemployed       Divorced
3    David   40     Tokyo    171         Actor        Engaged
4      Eve   18    Sydney    179     Professor         Single
```

In [41]:
```
boolean_index=df.loc[df['Age']>30]
print("filtered Dataframe: ")
print(boolean_index)
```

```
filtered Dataframe:
      Name  Age   City Height  Profession Marital Status
2  Charlie   35  Paris    185  Unemployed       Divorced
3    David   40  Tokyo    171       Actor        Engaged
```

In [42]:
```
data = {'Name' : ['John','Emma','Michael','Sophia','Aadi'],
        'Age' : [25,30,35,28,69],
        'City' : ['New York','Paris','London','Sydney','Kalyan']}
df = pd.DataFrame(data)
single_row = df.loc[2] #access a single row
print("single row:")
print(single_row)
print()
row_list = df.loc[[0,3,4]] #access rows 0, 3 and 4
print("list of rows:")
print(row_list)
print()
column_list = df.loc[:,['Name', 'Age']]
print("list of cols:")
print(column_list)
print()
specific_value = df.loc[1, 'Name']
print("specific value:")
print(specific_value)
print()
```

```
single row:
Name    Michael
Age          35
City     London
Name: 2, dtype: object

list of rows:
     Name  Age      City
0    John   25  New York
3  Sophia   28    Sydney
4    Aadi   69    Kalyan

list of cols:
      Name  Age
0     John   25
1     Emma   30
2  Michael   35
3   Sophia   28
4     Aadi   69

specific value:
Emma
```

In [43]:
```
data = {'Name' : ['John','Emma','Michael','Sophia','Aadi'],
        'Age' : [25,30,35,28,69],
        'City' : ['New York','Paris','London','Sydney','Kalyan']}
df = pd.DataFrame(data)
slice_rows = df.loc[1:3] #slice row from index 1 to 3
print("slice rows:")
print(slice_rows)
print()
slice_cols = df.loc[:, ['Name', 'Age']]
print("sliced columns:")
print(slice_cols)
print()
```

```
slice rows:
      Name  Age    City
1     Emma   30   Paris
2  Michael   35  London
3   Sophia   28  Sydney

sliced columns:
      Name  Age
0     John   25
1     Emma   30
2  Michael   35
3   Sophia   28
4     Aadi   69
```

In [44]:
```
df = pd.read_csv('titanic.csv')
```

In [45]:
```
print(df.head())
print()
print(df.tail())
print()
print(df.info())
```

```
print()
print(df.describe())
```

```
   PassengerId  Survived  Pclass  \
0            1         0       3
1            2         1       1
2            3         1       3
3            4         1       1
4            5         0       3

                                                Name     Sex   Age  SibSp  \
0                            Braund, Mr. Owen Harris    male  22.0      1
1  Cumings, Mrs. John Bradley (Florence Briggs Th...  female  38.0      1
2                             Heikkinen, Miss. Laina  female  26.0      0
3       Futrelle, Mrs. Jacques Heath (Lily May Peel)  female  35.0      1
4                           Allen, Mr. William Henry    male  35.0      0

   Parch            Ticket     Fare Cabin Embarked
0      0         A/5 21171   7.2500   NaN        S
1      0          PC 17599  71.2833   C85        C
2      0  STON/O2. 3101282   7.9250   NaN        S
3      0            113803  53.1000  C123        S
4      0            373450   8.0500   NaN        S

     PassengerId  Survived  Pclass                                      Name  \
886          887         0       2                     Montvila, Rev. Juozas
887          888         1       1              Graham, Miss. Margaret Edith
888          889         0       3  Johnston, Miss. Catherine Helen "Carrie"
889          890         1       1                     Behr, Mr. Karl Howell
890          891         0       3                       Dooley, Mr. Patrick

        Sex   Age  SibSp  Parch      Ticket   Fare Cabin Embarked
886    male  27.0      0      0      211536  13.00   NaN        S
887  female  19.0      0      0      112053  30.00   B42        S
888  female   NaN      1      2  W./C. 6607  23.45   NaN        S
889    male  26.0      0      0      111369  30.00  C148        C
890    male  32.0      0      0      370376   7.75   NaN        Q

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  891 non-null    int64
 1   Survived     891 non-null    int64
 2   Pclass       891 non-null    int64
 3   Name         891 non-null    object
 4   Sex          891 non-null    object
 5   Age          714 non-null    float64
 6   SibSp        891 non-null    int64
 7   Parch        891 non-null    int64
 8   Ticket       891 non-null    object
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    object
 11  Embarked     889 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
None

       PassengerId    Survived      Pclass         Age       SibSp  \
count   891.000000  891.000000  891.000000  714.000000  891.000000
mean    446.000000    0.383838    2.308642   29.699118    0.523008
std     257.353842    0.486592    0.836071   14.526497    1.102743
```

```
min       1.000000    0.000000    1.000000    0.420000    0.000000
25%     223.500000    0.000000    2.000000   20.125000    0.000000
50%     446.000000    0.000000    3.000000   28.000000    0.000000
75%     668.500000    1.000000    3.000000   38.000000    1.000000
max     891.000000    1.000000    3.000000   80.000000    8.000000

            Parch        Fare
count  891.000000  891.000000
mean     0.381594   32.204208
std      0.806057   49.693429
min      0.000000    0.000000
25%      0.000000    7.910400
50%      0.000000   14.454200
75%      0.000000   31.000000
max      6.000000  512.329200
```

In [46]:
```python
data = {"Name":["John","Alice","Bob"],
        "Age":[25,30,35],
        "City":["New York","London","Paris"]}
df = pd.DataFrame(data)
print("Original df")
print(df)
#rename index
df.rename(index = {0:'A',1:'B',2:'C'},inplace = True)  #inplace = same data fram
print("Modified")
print(df)

#reset index
df.reset_index(inplace = True)
print("Reset")
print(df)
```

```
Original df
    Name  Age      City
0   John   25  New York
1  Alice   30    London
2    Bob   35     Paris
Modified
    Name  Age      City
A   John   25  New York
B  Alice   30    London
C    Bob   35     Paris
Reset
  index   Name  Age      City
0     A   John   25  New York
1     B  Alice   30    London
2     C    Bob   35     Paris
```

In [54]:
```python
data = {
    'Name' : ['John','Anna','John','Anna','John'],
    'Age' : [28,24,28,24,19],
    'City' : ['New York','Los Vegass','New York','Los Angeles','Chicago']
}
df=pd.DataFrame(data)
print(df)
print(df.duplicated()) #check duplicate based on entire data
print(df.duplicated(subset=['Name','Age'])) #check duplicate as per the given su
df.drop_duplicates(inplace=True) #delete duplicates
print(df)
```

```
   Name  Age         City
0  John   28     New York
1  Anna   24   Los Vegass
2  John   28     New York
3  Anna   24  Los Angeles
4  John   19      Chicago
0    False
1    False
2     True
3    False
4    False
dtype: bool
0    False
1    False
2     True
3     True
4    False
dtype: bool
   Name  Age         City
0  John   28     New York
1  Anna   24   Los Vegass
3  Anna   24  Los Angeles
4  John   19      Chicago
```

In [56]:
```
data = {
    'Name' : ['John','Anna','John','Anna','John'],
    'Age' : [28,24,28,24,19],
    'City' : ['New York','Los Vegass','New York','Los Angeles','Chicago']
}
df=pd.DataFrame(data)
print(df)
df.drop_duplicates(keep='last',inplace=True) #delete duplicates but keeps the la
print()
print(df)
```

```
   Name  Age         City
0  John   28     New York
1  Anna   24   Los Vegass
2  John   28     New York
3  Anna   24  Los Angeles
4  John   19      Chicago

   Name  Age         City
1  Anna   24   Los Vegass
2  John   28     New York
3  Anna   24  Los Angeles
4  John   19      Chicago
```

In [57]:
```
data = {
    'Name' : ['Alice','Bob','Charlie'],
    'Age' : [25,30,35],
    'City' : ['New York','London','Paris']
}
df=pd.DataFrame(data)
df.to_csv("Temp_output_1.csv",index=False)
df.to_csv("Temp_output_2.csv",index=True)
```

In [ ]:

Pandas

In [1]:
```
import pandas as pd
```

In [5]:
```
data = [10,20,30,'Jhon',50]
my_series = pd.Series(data)
print(my_series)  #default labelels 0,1,2
```

```
0      10
1      20
2      30
3    Jhon
4      50
dtype: object
```

In [6]:
```
print(my_series[2])
```

```
30
```

In [13]:
```
a = [1,3,5]
my_series = pd.Series(a,index = ["X","Y","Z"]);
print(my_series)
```

```
X    1
Y    3
Z    5
dtype: int64
```

In [11]:
```
print(my_series["X"])
```

```
1
```

In [15]:
```
grades = {"sem1":3.25,"sem2":3.28,"sem3":3.75}
my_grades = pd.Series(grades)  #We can use dictionary to give labels
print(my_grades)
```

```
sem1    3.25
sem2    3.28
sem3    3.75
dtype: float64
```

In [17]:
```
my_series = pd.Series(grades,index = ["sem1","sem2","sem4"])  #values not existi
print(my_series)
```

```
sem1    3.25
sem2    3.28
sem4     NaN
dtype: float64
```

DataFrames

In [20]:
```
data = {"Name":["John","Alice","Bob"],
        "Age":[25,30,35],
        "City":["New York","London","Paris"]}
df = pd.DataFrame(data,index = [1,2,3])
print(df)
```

```
    Name  Age      City
1   John   25  New York
2  Alice   30    London
3    Bob   35     Paris
```

In [25]:
```
data = [["Alice",25,"New York"],
        ["John",30,"London"],
        ["Bob",35,"Paris"]]
df = pd.DataFrame(data,columns = ["Name","Age","City"])
print(df)  #data shall be entered properly else any value is entered anywhere
```

```
    Name  Age      City
0  Alice   25  New York
1   John   30    London
2    Bob   35     Paris
```

In [27]:
```
import numpy as np
data_array=  np.array([["ProductA",15.50,100],
                       ["ProductB",22.00,45],
                       ["ProductC",9.99,250]])
df = pd.DataFrame(data_array,columns = ["item","Price","stcok"])
print(df)
```

```
       item Price stcok
0  ProductA  15.5   100
1  ProductB  22.0    45
2  ProductC  9.99   250
```

In [31]:
```
df = pd.DataFrame()
print(df)
```

```
Empty DataFrame
Columns: []
Index: []
```

In [36]:
```
df = pd.read_csv("titanic.csv")
df
df.head()
```

Out[36]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	I
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0

In [37]:
```
print(df.shape)
```

```
(891, 12)
```

In [43]:
```
data = {"Name":["John","Alice","Bob"],
        "Age":[25,30,35],
        "City":["New York","London","Paris"]}
df = pd.DataFrame(data)
df.set_index("Name",inplace = True)
print(df) #treated as index not as column
```

```
       Age      City
Name
John    25  New York
Alice   30    London
Bob     35     Paris
```

In [46]:
```
data = {"Name":["John","Alice","Bob"],
        "Age":[25,30,35],
        "City":["New York","London","Paris"]}
df = pd.DataFrame(data,index = pd.RangeIndex(5,8,name = "index"))  # index cha n
print(df)
```

```
        Name  Age      City
index
5       John   25  New York
6      Alice   30    London
7        Bob   35     Paris
```

In [69]:
```
print(df.iloc[2])
```

```
index        C
Name       Bob
Age         35
City     Paris
Name: 2, dtype: object
```

In [55]:
```
data = {"Name":["John","Alice","Bob"],
       "Age":[25,30,35],
       "City":["New York","London","Paris"]}
df = pd.DataFrame(data)
print("Original df")
print(df)
#rename index
df.rename(index = {0:'A',1:'B',2:'C'},inplace = True)  #inplace = same data fram
print("Modified")
print(df)

#reset index
df.reset_index(inplace = True)
print("Reset")
print(df)
```

```
Original df
    Name  Age      City
0   John   25  New York
1  Alice   30    London
2    Bob   35     Paris
Modified
    Name  Age      City
A   John   25  New York
B  Alice   30    London
C    Bob   35     Paris
Reset
  index   Name  Age      City
0     A   John   25  New York
1     B  Alice   30    London
2     C    Bob   35     Paris
```

In [71]:
```
print(df.index)
print(df.index.values)
```

```
RangeIndex(start=0, stop=3, step=1)
[0 1 2]
```

In [ ]:

Department of AI_DS
Academic Year: 2026-27 (Odd Sem)
Course: Python for Data Science Lab
Second Year B. Tech Semester: III

PRACTICAL 6

Part A (To be referred by students)

Pandas

SAVE THE FILE AND UPLOAD AS (RollNo_Name_Exp6)

Aim: To understand and apply the basics of working with Pandas Series and Data Frames in Python, focusing on fundamental operations and applications useful in data science and machine learning.

Problem Statement:

Solve the following computational problems by **recognizing** python data structure of pandas and **apply** them to implement programs

1. Create a Pandas Series from a list of 10 random integers and calculate its mean.
   Access the first 5 elements of a Pandas Series using slicing.
2. Create DataFrame from dict.
3. Create a Dataframe using following series.
   1. s1= ['Python','ML','NLP','DL']
   2. s2= ['9','8','7','9']
   3. s3= ['7','8','9','9']
4. Create a DataFrame from a dictionary with columns 'Name', 'Age', and 'City'.
   1. Add a new column 'Score' to a DataFrame with values [85, 90, 78, 92].
   2. Use .iloc to access the first row of a DataFrame.
   3. Use .loc to select the rows where 'Age' is 25 in a DataFrame.
   4. drop the 'City' column using the drop method.
   5. Filter a DataFrame to show rows where 'Age' is greater than 30 using boolean indexing.
   6. Rename the 'Name' column to 'FullName' in a DataFrame and display the DataFrame.
5. Create a Series with a custom index ['a', 'b', 'c'] and access the value at index 'b'.
6. Importing data in (.csv & .excel) pandas.
   1. Read the iris dataset.
   2. Add the meaningful header to the DataFrame of iris dataset.
7. Print the DataFrame (of your choice) in python..
   1. show entire data frame
   2. show first n rows
   3. show bottom n rows
   4. show last 5 rows
8. Analyze the data by performing following operations.
   1. check the data type

2. check the data distribution (only numeric columns)
3. check the data distribution (full summary of data, including objects)
4. provide info of data.

9. Perform following pre-processing on data of your choice
    1. Change the datatype
    2. Delete one row
    3. Delete one column
10. Export the above DataFrame to CSV/JSON/EXCEL.
11. Load the Titanic dataset from Kaggle into a DataFrame and display the first 5 rows.
12. Calculate the correlation matrix of a DataFrame with numerical columns from the Wine Quality dataset.

Topic covered: Pandas

Learning Objective: Learner would be able to

1. Learn to create data frames using pandas package.
2. Learn data analysis using Pandas package.
3. Handle large amount of data to create data visualization.

Theory:

Pandas is a Python library used for working with data sets. It has functions for analyzing, cleaning, exploring, and manipulating data. Pandas use the loc attribute to return one or more specified row(s)

 # Create dataset

```
import pandas as pd
mydataset = {   'cars': ["BMW", "Volvo", "Ford"],   'passings': [3, 7, 2] }
myvar = pd.DataFrame(mydataset)
print(myvar)
```

Pandas series is one-dimensional array holding data of any type. It is like is like a column in a table

 # create series with index value

```
import pandas as pd
a = [1, 7, 2]
myvar = pd.Series(a, index = ["x", "y", "z"])
print(myvar)
print(myvar["y"]) # return value of y
```

 # Create series from Dictionary

```
import pandas as pd
calories = {"day1": 420, "day2": 380, "day3": 390}
```

```
myvar = pd.Series(calories)
print(myvar)
```

O/P - day1 420
day2 380
day3 390
dtype: int64

 # Create DataFrame from two series

```
import pandas as pd
data = {  "calories": [420, 380, 390],  "duration": [50, 40, 45] }
myvar = pd.DataFrame(data)
print(myvar)
print(df.loc[0])   #refer to the 0 row index
myvar 1= pd.DataFrame(data, index = ["day1", "day2", "day3"])
print(myvar1)
print(df.loc[0])   #refer to the 0 row index
```

If your data sets **are stored in a file**, Pandas can load them into a DataFrame.

```
import pandas as pd
df = pd.read_csv('data.csv')  # Give path if not in same directory
print(df)
print(pd.options.display.max_rows)   # Checks maximum number of rows:
print(df.head(10))   # printing the first 10 rows of the DataFrame:
print(df.tail())   # printing the last 5 rows of the DataFrame:
print(df.info())   # Print information about the data:
```

Data Cleaning

Data cleaning means fixing bad data in your data set. Bad data could be: Empty cells, Data in wrong format, Wrong data, Duplicates
In this tutorial you will learn how to deal with all of them.

```
import pandas as pd
df = pd.read_csv('data.csv')
new_df = df.dropna() # Returns a new DataFrame without changing  original DataFrame
df.dropna(inplace = True) # Changes original DataFrame
print(new_df)

df.fillna(130, inplace = True) # Replace NULL values with the number 130:
df["Calories"].fillna(130, inplace = True) # Replace column calories only
x = df["Calories"].mean()
df["Calories"].fillna(x, inplace = True) # Replace by mean value
```

Department of AI_DS
Academic Year: 2026-27 (Odd Sem)
Course: Python for Data Science Lab
Second Year B. Tech Semester: III

Data Plotting

Pandas uses the plot() method to create diagrams. We can use Pyplot, a submodule of the Matplotlib library to visualize the diagram on the screen.

```
import pandas as pd
import matplotlib.pyplot as plt
df = pd.read_csv('data.csv')
df.plot()
df.plot(kind='scatter',x='Duration',y='Calories') # Draws Scatter plot

# To Do -> Plot between Duration and Maxpulse
plt.show()
```

Histogram - A histogram shows us the frequency of each interval, e.g. how many workouts lasted between 50 and 60 minutes?

Use the kind argument to specify that you want a histogram: kind = 'hist'

```
df["Duration"].plot(kind = 'hist')
```

Department of AI_DS
Academic Year: 2026-27 (Odd Sem)
Course: Python for Data Science Lab
Second Year B. Tech Semester: III

PRACTICAL 6

Part B (to be completed by students)

Pandas

1. All the students are required to perform the given tasks in Jupyter Notebook
2. Create a new notebook for each experiment. The filename should be RollNo_Name_Exp6)
3. In the first cell, the student must write his/her Name, roll no and class in the form of comments
4. Every program should be written in separate cells and in the given sequence
5. After completing the experiment, download the notebook in pdf format. The filename should be RollNo_Name_Exp6.pdf).
   Note: - To download as a PDF, go to File and choose Print Preview. This will open a new tab. Now, press Ctrl + P and opt to save as PDF.
6. Upload the pdf on the web portal

In [1]:
```
import pandas as pd
import numpy as np
```

```
C:\Users\djsce.student\AppData\Local\Continuum\anaconda3\lib\site-packages\pandas
\compat\_optional.py:138: UserWarning: Pandas requires version '2.7.0' or newer o
f 'numexpr' (version '2.6.8' currently installed).
  warnings.warn(msg, UserWarning)
```

In [7]:
```
#panda series 10 random integer find mean , access the first 5 elements through
np.random.seed(0)
sr=pd.Series(np.random.randint(1,101,size=10))
print(sr)
print(np.mean(sr))
sr_slice=sr.loc[0:4]
print("\nSliced_Rows :-")
print(sr_slice)
```

```
0    45
1    48
2    65
3    68
4    68
5    10
6    84
7    22
8    37
9    88
dtype: int32
53.5

Sliced_Rows :-
0    45
1    48
2    65
3    68
4    68
dtype: int32
```

In [8]:
```
#create data frame from dictionary
data = {'Name' : ['John','Emma','Michael','Sophia','Aadi'],
        'Age' : [25,30,35,28,69],
        'City' : ['New York','Paris','London','Sydney','Kalyan']}
df = pd.DataFrame(data)
print(df)
```

```
      Name  Age      City
0     John   25  New York
1     Emma   30     Paris
2  Michael   35    London
3   Sophia   28    Sydney
4     Aadi   69    Kalyan
```

In [10]:
```
#create dataframe from the following series
s1 = pd.Series(['Python','ML','NLP','DL'])
s2 = pd.Series(['9','8','7','9'])
s3 = pd.Series(['7','8','9','9'])
df=pd.concat([s1,s2,s3],axis=1)
print(df)
```

```
        0  1  2
0  Python  9  7
1      ML  8  8
2     NLP  7  9
3      DL  9  9
```

In [21]:

```python
#create data frame from dictionary
data = {'Name' : ['John','Emma','Michael','Sophia'],
        'Age' : [25,30,35,28],
        'City' : ['New York','Paris','London','Sydney']}
df = pd.DataFrame(data)
print(df)
print()
df['Scores']=pd.Series([85,90,78,92])
print(df)
print()
print(df.iloc[0])
print()
age25 = df[df["Age"]==25]
print(age25)
print()
df.drop('City',axis=1,inplace=True)
print(df)
print()
agem30 =df[df["Age"]>30]
print(agem30)
print()
df.rename(columns={'Name':'FullName'},inplace=True)
print(df)
```

```
      Name  Age      City
0     John   25  New York
1     Emma   30     Paris
2  Michael   35    London
3   Sophia   28    Sydney
```

```
      Name  Age      City  Scores
0     John   25  New York      85
1     Emma   30     Paris      90
2  Michael   35    London      78
3   Sophia   28    Sydney      92
```

```
Name          John
Age             25
City      New York
Scores          85
Name: 0, dtype: object
```

```
   Name  Age      City  Scores
0  John   25  New York      85
```

```
      Name  Age  Scores
0     John   25      85
1     Emma   30      90
2  Michael   35      78
3   Sophia   28      92
```

```
      Name  Age  Scores
2  Michael   35      78
```

```
  FullName  Age  Scores
0     John   25      85
1     Emma   30      90
2  Michael   35      78
3   Sophia   28      92
```

In [ ]:

Department of Artificial Intelligence (AI) & Data Science

AY: 2026-27
Course: Python for Data Science Laboratory (DJS23SLPC303)
Class: SE/ Sem: III Roll No.: SAPID:

Experiment No. 6: .

Aim: To understand and apply the basics of working with Pandas Series and Data Frames in Python, focusing on fundamental operations and applications useful in data science and machine learning.

Theory

1. What is a Pandas Series? How is it different from a NumPy array?
2. How can you create a DataFrame from a Python dictionary? Give an example.
3. Explain the use of .loc[ ] and .iloc[ ] with suitable examples.
4. How do you check the data type of each column in a DataFrame?
5. Differentiate between head(), tail(), and sample() functions.
6. Give applications of Pandas in data science

Conclusions:

Name:- Aaryan N. Ruparane Roll No:- 5004 Div:- 5
branch:- AI&DS batch:- A1 SAP ID:- 60018250070

Experiment No 6

Aim:-

To understand and apply the basics of working with Pandas Series and DataFrames in Python focusing on fundamental operations useful in data science and Machine learning.

Theory:-

1) What is a Pandas Series? How is it different from a Numpy array?

=> A Pandas Series is a one dimensional labeled array capable of holding data of any type such as integers, floats, or strings. Each element has a corresponding label called an index, which can be integers or custom string labels. It is built on top of Numpy and is therefore fast and memory efficient. A Numpy array uses only integers position to access elements and does not carry an index label. A series adds a labeled index on top, which allows to access by name and enables automatic data alignment when two series are combined. Series also support missing value handling using NaN, which Numpy arrays do not provide by default.
for example:- If two series with different index labels are added, Pandas aligns them by label before performing the operation. This makes series

Sundaram
FOR EDUCATIONAL USE

more suitable than plain. Numpy arrays when working with real world datasets that may have gaps or irregular ordering.

25 How can you create a DataFrame from a Python dictionary? Give an example.

⇒ A DataFrame is a two dimensional table with labelled rows and columns. It can be created by passing a Python dictionary to pd.DataFrame(). The dictionary keys become the column names and the corresponding lists become the column data. If no index is specified, Pandas assigns an integer index starting from 0 automatically. All lists in the dictionary must have the same length, otherwise Pandas raises a ValueError. You can verify the shape and column names of the created DataFrame using df.shape and df.columns.

Eg:-

```
import pandas as pd

data = {
    "Name": ["Alice", "Bob", "Charlie"],
    "Age": [21, 22, 20],
    "Marks": [88, 75, 91]
}
df = pd.DataFrame(data)
print(df)
print("Shape:", df.shape)
print("Columns:", list(df.columns))
```

Sundaram
FOR EDUCATIONAL USE

output :

	Name	Age	Marks
0	Alice	21	88
1	Bob	22	75
2	Charlie	20	91

Shape : (3, 3)
Columns : ['Name', 'Age', 'Marks']

3) Explain the use of .loc[] and .iloc[] with suitable examples.

=> .loc[] is label based indexing. It selects rows and column using their assigned labels (index names and column names). Both the start and end labels are inclusive. If a label does not exist, Pandas raises a KeyError. It also supports boolean conditions for filtering rows.

.iloc[] is integer position based indexing. It selects rows and columns using their 0 based integer positions, following Python slicing rules where the end position is exclusive. It is useful when the index labels are not known or when you need positional access.

Eg :-
```
import pandas as pd
data = {
    "city": ["Mumbai", "Delhi", "Pune", "Chennai"],
    "Population": [7205, 3200, 906, 1160]
}
df = pd.DataFrame(data, index=["r1", "r2", "r3", "r4"])
```

Sundaram
FOR EDUCATIONAL USE

```
print("Using .loc[]:")
print(df.loc["r1":"r3", city])
print("Using .iloc[]:")
print(df.iloc[0:2,0])
```

Output:

```
Using .loc[]:
r1    Mumbai
r2     Delhi
r3      Pune
Using
Name: city, dtype: str
Using .iloc[]:
r1    Mumbai
r2     Delhi
Name: City , dtype: str
```

4) How do you check the datatype of each column in a DataFrame?

⇒ The .dtypes attribute of a DataFrame returns the data type of every column. This is useful to confirm whether columns are stored as the correct type before performing any operations or building machine learning models.
Common dtypes returned are int64 for integer columns, float64 for decimal columns, and Object for string or mixed type columns. If a column has the wrong dtype, it can be converted using the .astype() method.
Eg: df['Age'].astype(float) Converts the Age column from int64 to float64

Sundaram®
FOR EDUCATIONAL USE

Eg:-
```
import pandas as pd

data = {
    "Product": ["Pen", "Book", "Bag"],
    "Price": [10.5, 150.0, 500.0],
    "Quantity": [100, 50, 30]
}
df = pd.DataFrame(data)
print(df.dtypes)
```

output:-

Product	str
Price	float64
Quantity	int64
dtype:	object

5) Differentiate between head(), tail(), and sample() functions.

=> These three functions are used to view a subset of rows from a DataFrame. head() and tail() always return the same rows, while sample() returns a randomly selected set of rows that may differ on each call.

Feature	head()	tail()	sample()
Returns	First n rows (default 5)	Last n rows (default 5)	n random rows (default 1)
Order	Always from the top	Always from the bottom	Random, changes each call

Sundaram®
FOR EDUCATIONAL USE

Use case	Always from the top	Always from the Bottom	Random
Use case	Preview the start of data	Preview the end of data	Get a random sample.
Reproducible	Yes, always the same result	Yes, always the same result	No, varies each execution
Syntax Example	df.head(3)	df.tail(3)	df.sample(3)

6) Give applications of Pandas in data Science.

=> Pandas is widely used across data science tasks. Some key applications are listed below:

i) Data cleaning: Handles missing values using fillna() and dropna(), removes duplicate rows, and corrects inconsistent entries.

ii) Exploratory data Analysis: Provides statistical summaries through describe() and info(), helping understand the distribution and structure of data.

Sundaram
FOR EDUCATIONAL USE

iii) Data Manipulation: Supports filtering rows with conditions, sorting by column values, grouping data using groupby(), and merging multiple DataFrames.

iv) Feature Engineering: Helps create new columns derived from existing ones and apply transformation functions to prepare data for Machine learning models.

v) Time Series Analysis: Supports date indexing, resampling, and rolling window operations, making it useful for analyzing data collected over time.

vi) Data Aggregation: Functions like groupby() combined with agg() allow summarizing large datasets by computing total, averages and counts across groups.

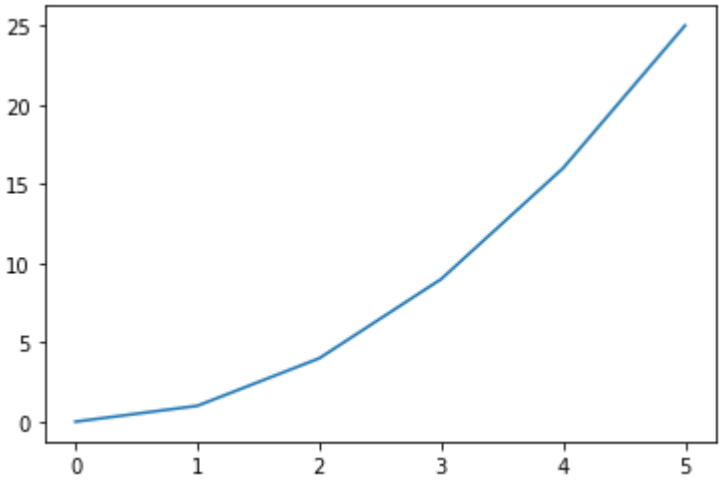
Conclusions:

In this Experiment, Pandas Series and DataFrame structures were studied along with their creation and core operations. Functions such as .loc[], .iloc[], .dtypes, head(), tail(), and sample() were applied on DataFrames to perform label based and position based data access. Pandas proves to be a powerful library for data manipulation and preprocessing, forming a foundational tool in any data science workflow.

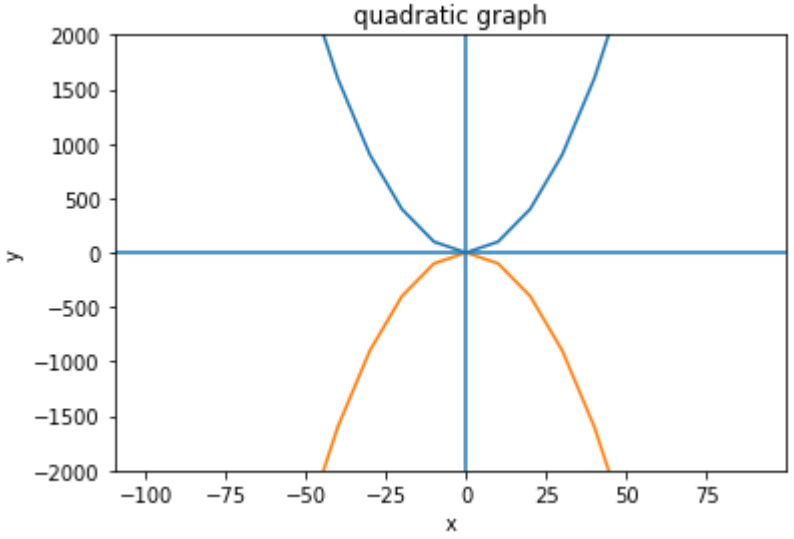
Sundaram
FOR EDUCATIONAL USE

In [17]:
```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
```

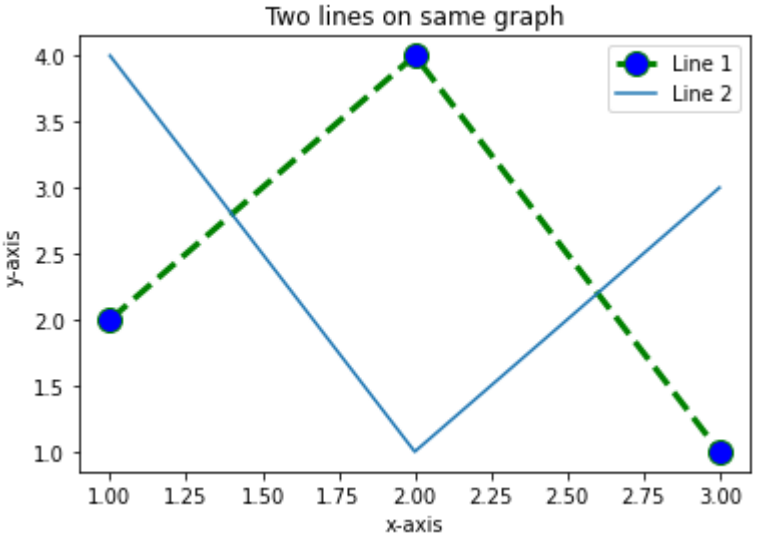
In [2]:
```
x_values = [0,1,2,3,4,5]
y_values = [0,1,4,9,16,25]
plt.plot(x_values,y_values)
plt.show()
```



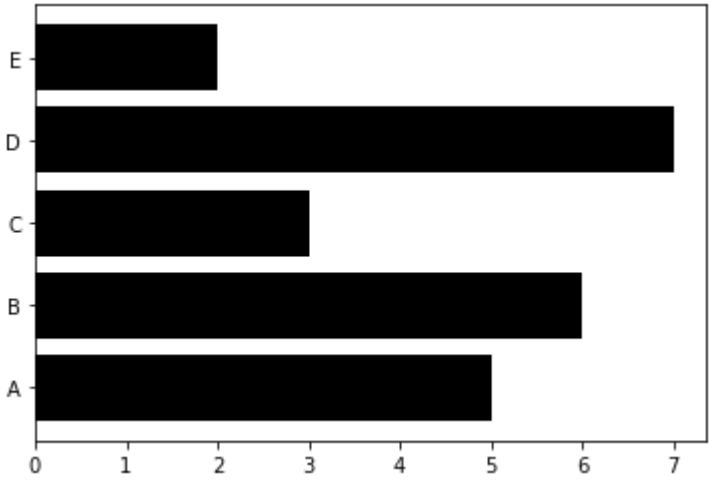
In [3]:
```
y1 = []
y2 = []
x = range(-100,100,10)
for i in x : y1.append(i**2)
for i in x : y2.append(-i**2)
plt.plot(x,y1)
plt.plot(x,y2)
plt.xlabel("x")
plt.ylabel("y")
plt.ylim(-2000,2000)
plt.axhline(0)
plt.axvline(0)
plt.title("quadratic graph")
plt.savefig("quad.png")
plt.show()
```



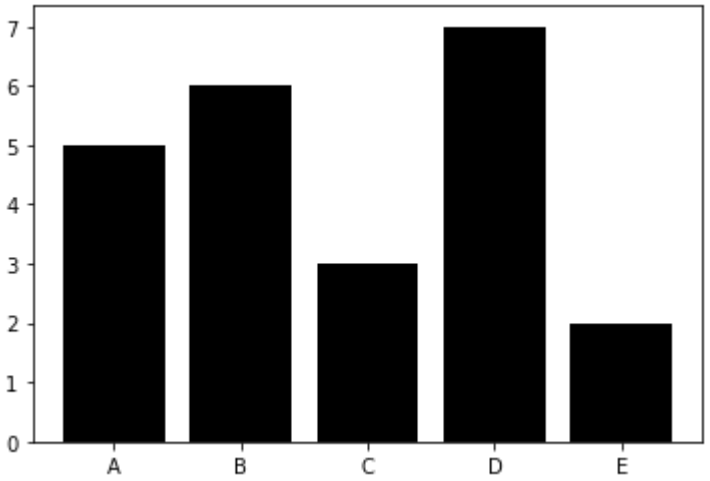
In [4]:
```
x1=[1,2,3]
y1=[2,4,1]
plt.plot(x1,y1,label="Line 1",color='green',linestyle='dashed',linewidth=3,marke
x2=[1,2,3]
y2=[4,1,3]
plt.plot(x2,y2,label="Line 2")
plt.xlabel('x-axis')
plt.ylabel('y-axis')
plt.title("Two lines on same graph")
plt.legend()
plt.show()
```



In [5]:
```
values = [5,6,3,7,2]
names = ["A","B","C","D","E"]
plt.barh(names,values,color="black")
plt.show()
```



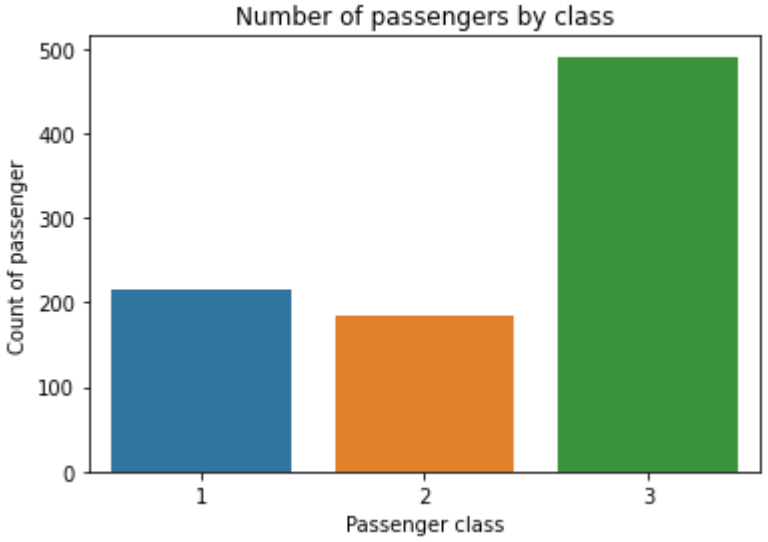
In [6]:
```
values = [5,6,3,7,2]
names = ["A","B","C","D","E"]
plt.bar(names,values,color="black")
plt.show()
```



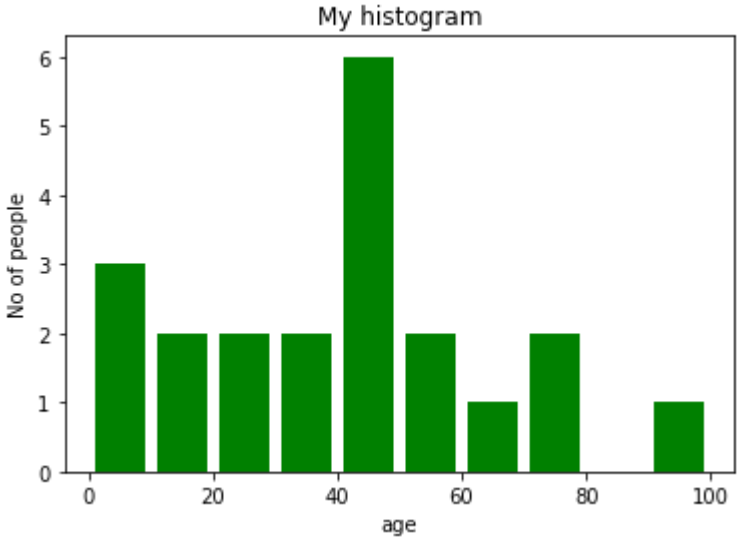
In [7]:
```
import seaborn as sns
```

```
C:\Users\djsce.student\AppData\Local\Continuum\anaconda3\lib\site-packages\pandas
\compat\_optional.py:138: UserWarning: Pandas requires version '2.7.0' or newer o
f 'numexpr' (version '2.6.8' currently installed).
  warnings.warn(msg, UserWarning)
```

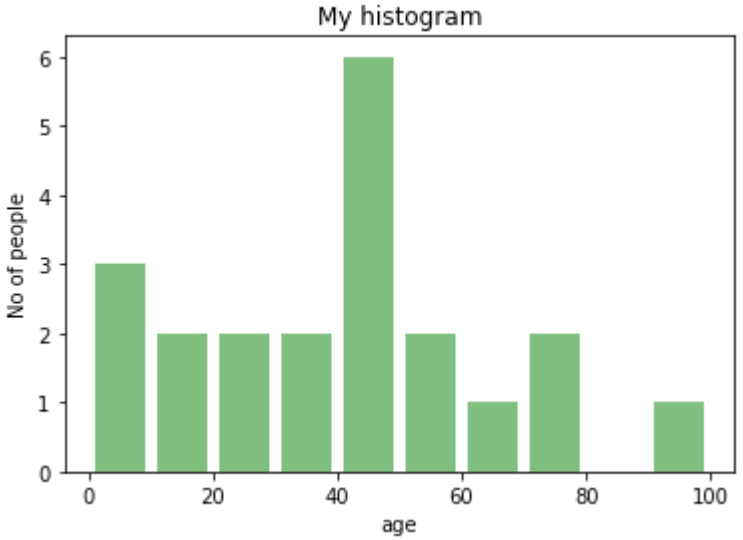
In [8]:
```
titanic = sns.load_dataset('titanic')
sns.countplot(x='pclass',data = titanic)
plt.title("Number of passengers by class")
plt.xlabel("Passenger class")
plt.ylabel("Count of passenger")
plt.show()
```



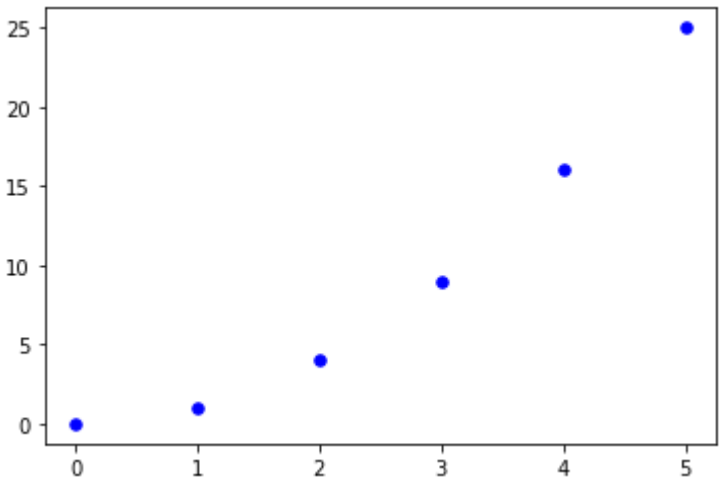
In [9]:
```
ages = [2,5,70,40,30,45,50,45,43,40,444,60,7,13,57,18,90,77,32,21,20,40]
range=(0,100)
bins = 10
plt.hist(ages,bins,range,color='green',histtype='bar',rwidth=0.8)
plt.xlabel('age')
plt.ylabel('No of people')
plt.title('My histogram')
plt.show()
```



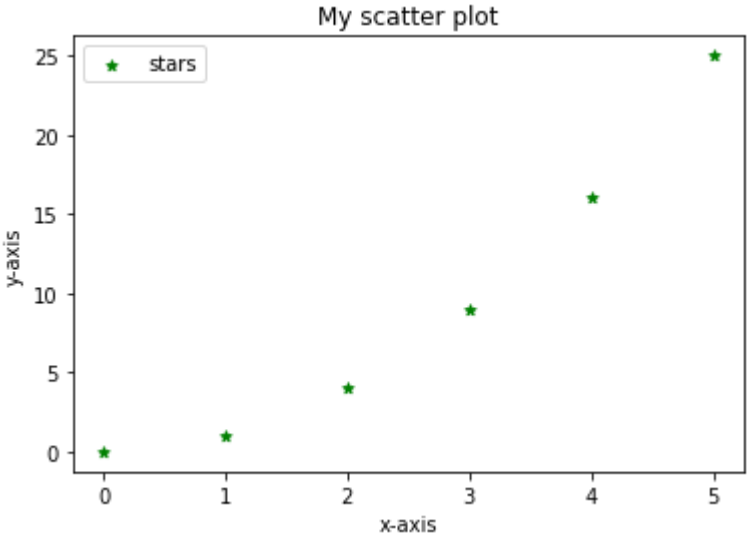
In [10]:
```
ages = [2,5,70,40,30,45,50,45,43,40,444,60,7,13,57,18,90,77,32,21,20,40]
range=(0,100)
bins = 10
plt.hist(ages,bins,range,color='green',histtype='bar',rwidth=0.8,alpha=0.5)
plt.xlabel('age')
plt.ylabel('No of people')
plt.title('My histogram')
plt.show()
```



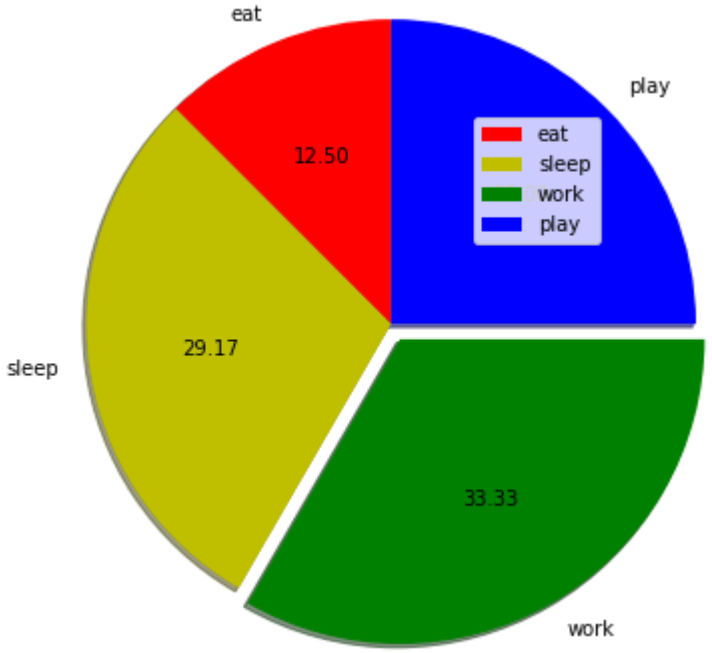
In [11]:
```
x_values = [0,1,2,3,4,5]
y_values = [0,1,4,9,16,25]
plt.scatter(x_values,y_values,s=30,color="blue")
plt.show()
```



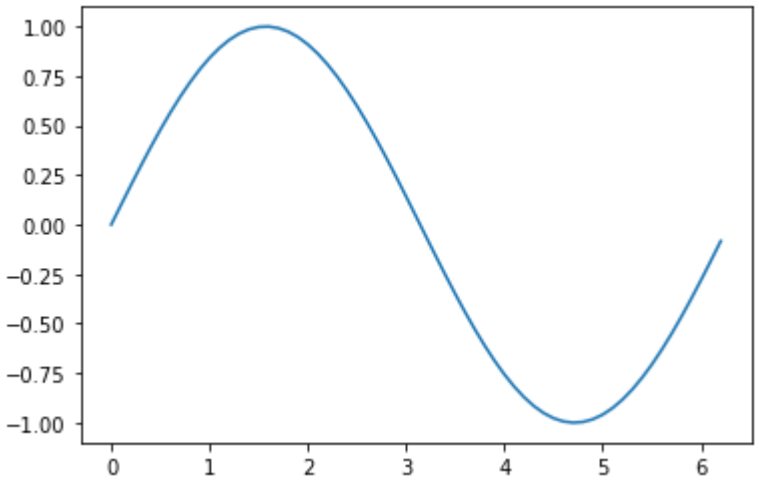
In [12]:
```
x_values = [0,1,2,3,4,5]
y_values = [0,1,4,9,16,25]
plt.scatter(x_values,y_values,label="stars",marker="*",s=30,color="green")
plt.xlabel('x-axis')
plt.ylabel('y-axis')
plt.title('My scatter plot')
plt.legend()
plt.show()
```



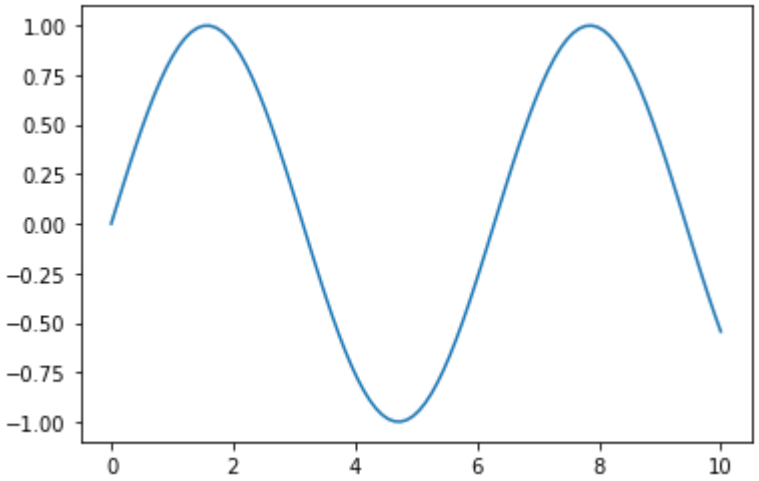
In [13]:
```
activities = ['eat','sleep','work','play']
slices=[3,7,8,6]
colors =['r','y','g','b']
plt.pie(slices,labels=activities,colors=colors,startangle=90,shadow=True,explode
plt.legend(loc = "upper right")
plt.show()
```



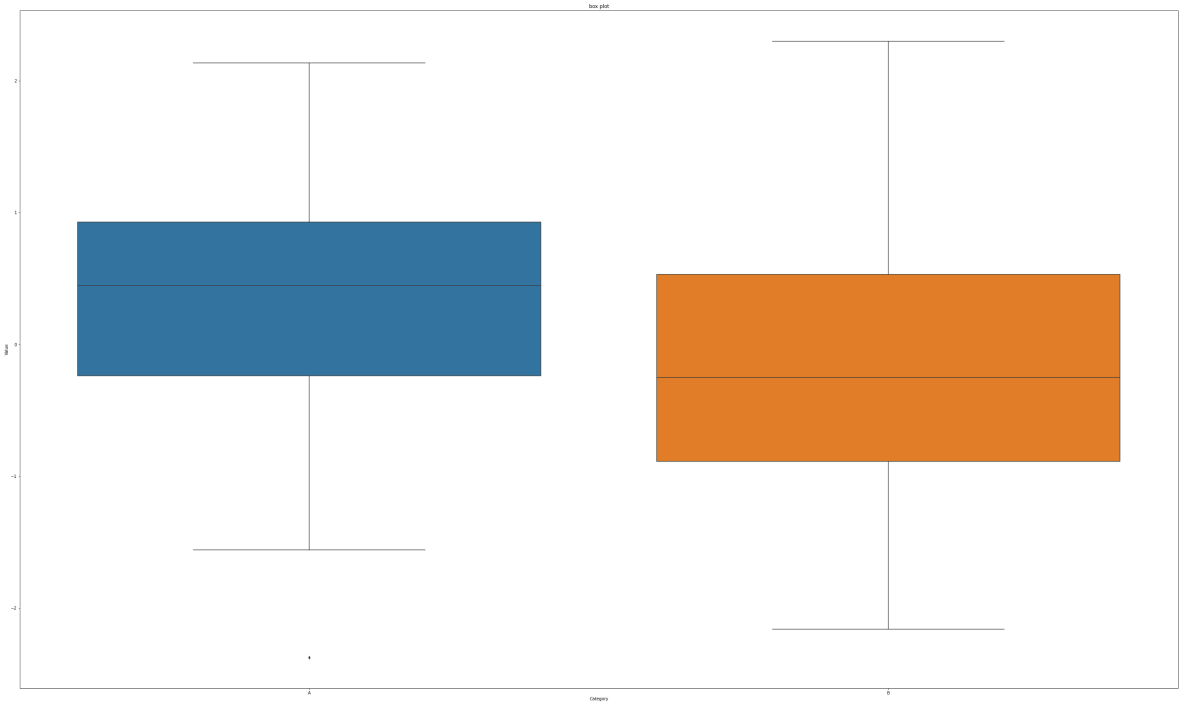
In [14]:
```
x=np.arange(0,2*(np.pi),0.1)
#OR x=np.linspace(0,10,100)
y=np.sin(x)
plt.plot(x,y)
plt.show()
```



In [15]:
```
x=np.linspace(0,10,100)
y=np.sin(x)
plt.plot(x,y)
plt.show()
```

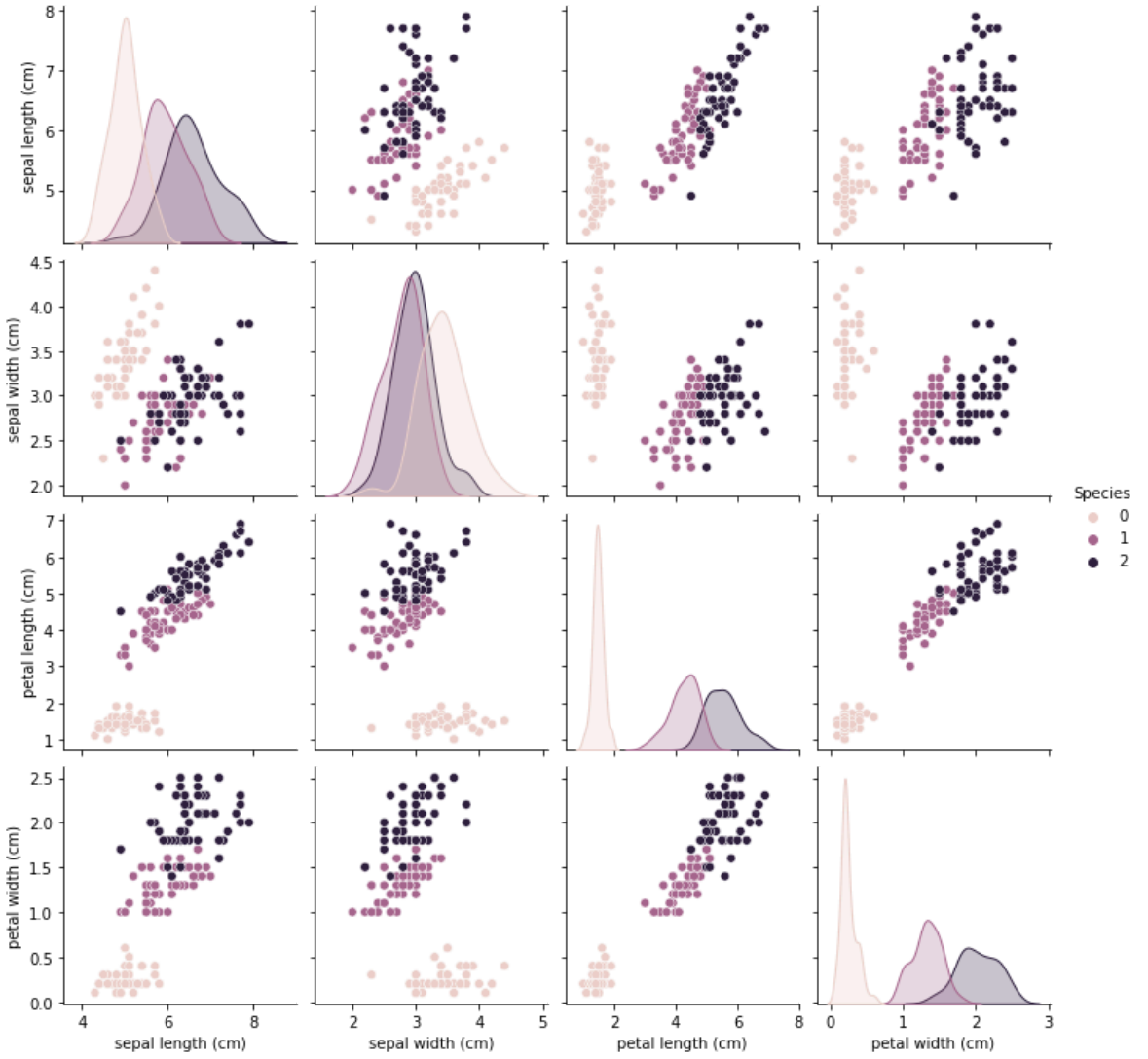


In [24]:
```
data = pd.DataFrame({
    'Category' : ['A']*50 + ['B']*50,
    'Value' : np.random.randn(100)
})
plt.figure(figsize=(50, 30))
sns.boxplot(x='Category',y='Value',data=data)
plt.title("box plot")
plt.show()
```

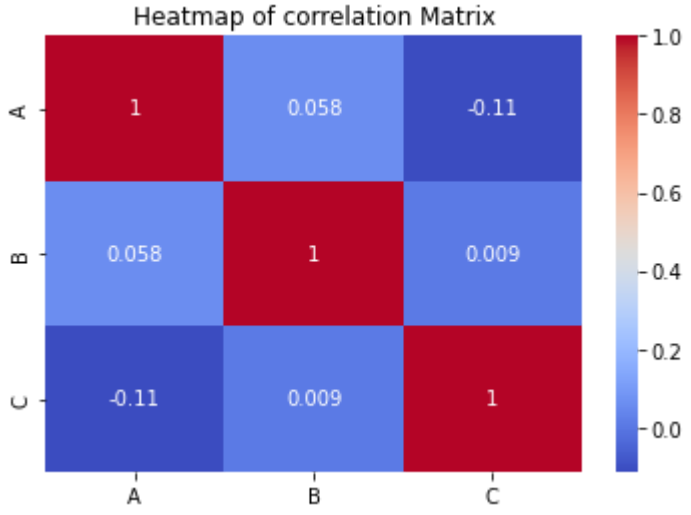


In [28]:
```
import warnings
warnings.filterwarnings("ignore", category=DeprecationWarning)
```

In [30]:
```
from sklearn.datasets import load_iris
iris = load_iris()
data = pd.DataFrame(data=iris.data, columns=iris.feature_names)
data["Species"] = iris.target
sns.pairplot(data, hue="Species")
plt.show()
```



In [31]:
```
data = pd.DataFrame({
    'A':np.random.randn(100),
    'B':np.random.randn(100),
    'C':np.random.randn(100)
})
corr = data.corr()
sns.heatmap(corr,annot=True,cmap='coolwarm')
plt.title('Heatmap of correlation Matrix')
plt.show()
```



In [ ]:

Department of AI_DS
Academic Year: 2025-26 (Odd Sem)
Course: Python for Data Science Lab
Second Year B. Tech Semester: III

PRACTICAL 12

Part A (To be referred by students)

Data Cleaning

SAVE THE FILE AND UPLOAD AS (RollNo_Name_Exp12)

Aim: To understand and apply techniques for handling missing data and finding outliers usingPython with NumPy and pandas, focusing on practical applications in data science and machine learning.

Problem Statement:

Solve the following computational problems to clean the data and identify outliers

1. In the Titanic dataset, replace missing values in the age column with the median age of passengers.
2. In the Titanic dataset, detect outliers in the fare column using Z-scores and display them.
3. Given the Iris dataset, identify and print rows where sepal length (cm) is an outlier using the IQR method.
4. Given the Iris dataset, calculate and print the Z-scores for petal width (cm) and identify outliers.
5. For the Boston Housing dataset, drop rows where the RM (average number of rooms per dwelling) column has missing values.
6. For the Boston Housing dataset, use the IQR method to detect outliers in the AGE column and list them.
7. In the Wine Quality dataset, fill missing values in the total sulfur dioxide column with the mean of that column.
8. In the Wine Quality dataset, replace missing values in the fixed acidity column with 0 and display the cleaned DataFrame.

Link to datasets:

1. **Titanic Dataset**: Titanic: Machine Learning from Disaster
2. **Iris Dataset**: Iris Species
3. **Boston Housing Dataset**: Boston Housing Dataset
4. **Wine Quality Dataset**: Wine Quality Dataset

Department of AI_DS
Academic Year: 2025-26 (Odd Sem)
Course: Python for Data Science Lab
Second Year B. Tech Semester: III

Topic covered: Pandas, Numpy : Data Cleaning

Learning Objective: Learner would be able to

1. Identify and handle missing data
2. Detect and interpret outliers

Theory:

Data preprocessing is a crucial step in data analysis and machine learning. Real-world datasets often contain missing values and outliers, which can negatively impact model performance. This data can be cleaned by handling missing values and detecting outliers using statistical techniques such as **Median/Mean imputation, Z-score, and IQR methods**.

1. Handling Missing Data

Missing data is common in real-world datasets. Proper handling is crucial to ensure accurate analysis and modeling.

1.1 Identifying Missing Data

Example: Identifying missing values in a DataFrame

```
import pandas as pd
import numpy as np

# Sample DataFrame with missing values
df = pd.DataFrame({
'A': [1, 2, np.nan, 4],
'B': [5, np.nan, 7, 8],
'C': [9, 10, 11, 12]
})
```

```
# Identify missing values missing_values = df.isna()

print(missing_values)
# Output:
0       A
False   B
False   C
False
#       1       False   True    False
#       2       True    False   False
#       3       False   False   False
```

1.2 Handling Missing Data

Example: Filling missing values with a specific value (e.g., 0)

```
# Fill missing values with 0
df_filled = df.fillna(0)
print(df_filled)
#
#       Output:
A
B
C
#       0       1.0     5.0     9
#       1       2.0     0.0     10
#       2       0.0     7.0     11
#       3       4.0     8.0     12
```

Example: Filling missing values with the mean of each column

```
# Fill missing values with the mean of each column
df_filled_mean = df.fillna(df.mean())

print(df_filled_mean)
#         Output:
#
#              A  B
0 1.0   5.0       C
9
#         1 2.0   6.0       10
#         2 2.33        7.0 11
#         3 4.0   8.0       12
```

Example: Dropping rows with missing values

```
# Drop rows with missing values df_dropped = df.dropna()

print(df_dropped) # Output:
#         A         B         C # 0 1.0 5.0    9
# 3 4.0 8.0 12
```

2. Finding Outliers

Outliers are values that differ significantly from other observations. Identifying them is crucial for improving model performance.

2.1 Using Z-Scores to Detect Outliers Example: Calculate Z-scores and identify outliers

```
from scipy import stats
# Sample data
data = np.array([10, 12, 14, 15, 100, 18, 20])

# Calculate Z-scores
z_scores = stats.zscore(data)

# Identify outliers (Z-score > 3 or < -3)
 outliers = np.abs(z_scores) > 3

print("Z-scores:", z_scores)
print("Outliers:", data[outliers])

# Output:
# Z-scores: [-0.488889 -0.155555 0.177778 0.322222  5.288889 -
0.044444 0.200000]
# Outliers: [100]
```

2.2 Using IQR (Interquartile Range) to Detect Outliers

Example: Calculate IQR and identify outliers

```
# Sample data
data = np.array([10, 12, 14, 15, 100, 18, 20])

# Calculate Q1 (25th percentile) and Q3 (75th percentile)
Q1 = np.percentile(data, 25)
Q3 = np.percentile(data, 75)
```

```
# Calculate IQR IQR = Q3 - Q1

# Determine outliers
outliers = (data < (Q1 - 1.5 * IQR)) | (data > (Q3 + 1.5 * IQR))
print("IQR:", IQR)
print("Outliers:", data[outliers])

# Output:
# IQR: 9.0
# Outliers: [100]
```

2.3 Identifying Outliers in a DataFrame Using IQR

Example: Find outliers in a DataFrame column

```
# Sample DataFrame df = pd.DataFrame({
'Values': [10, 12, 14, 15, 100, 18, 20]
})

# Calculate Q1 and Q3
Q1 = df['Values'].quantile(0.25)
 Q3 = df['Values'].quantile(0.75)

# Calculate IQR
IQR = Q3 - Q1
# Identify outliers
outliers = (df['Values'] < (Q1 - 1.5 * IQR)) | (df['Values'] > (Q3 +1.5 *IQR))
print("Outliers in DataFrame:", df[outliers]) # Output:
#          Values # 4       100
```

Department of AI_DS
Academic Year: 2025-26 (Odd Sem)
Course: Python for Data Science Lab
Second Year B. Tech Semester: III

PRACTICAL 12

Part B (to be completed by students)

Data Cleaning

1. All the students are required to perform the given tasks in Jupyter Notebook
2. Create a new notebook for each experiment. The filename should be RollNo_Name_Exp12)
3. In the first cell, the student must write his/her Name, roll no and class in the form of comments
4. Every program should be written in separate cells and in the given sequence
5. After completing the experiment, download the notebook in pdf format. The filename should be RollNo_Name_Exp12.pdf).
   Note: - To download as a PDF, go to File and choose Print Preview. This will open a new tab. Now, press Ctrl + P and opt to save as PDF.
6. Upload the pdf on the web portal

In [2]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

In [5]:
```
titanic = pd.read_csv("titanic.csv")
```

In [6]:
```
titanic.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype  
---  ------       --------------  -----  
 0   PassengerId  891 non-null    int64  
 1   Survived     891 non-null    int64  
 2   Pclass       891 non-null    int64  
 3   Name         891 non-null    object 
 4   Sex          891 non-null    object 
 5   Age          714 non-null    float64
 6   SibSp        891 non-null    int64  
 7   Parch        891 non-null    int64  
 8   Ticket       891 non-null    object 
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    object 
 11  Embarked     889 non-null    object 
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
```

In [7]:
```
titanic['Age'] = titanic['Age'].fillna(titanic['Age'].median())
```

In [8]:
```
titanic.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype  
---  ------       --------------  -----  
 0   PassengerId  891 non-null    int64  
 1   Survived     891 non-null    int64  
 2   Pclass       891 non-null    int64  
 3   Name         891 non-null    object 
 4   Sex          891 non-null    object 
 5   Age          891 non-null    float64
 6   SibSp        891 non-null    int64  
 7   Parch        891 non-null    int64  
 8   Ticket       891 non-null    object 
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    object 
 11  Embarked     889 non-null    object 
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
```

In [10]:
```
from scipy import stats

titanic['z_score'] = stats.zscore(titanic['Fare'])
outliers = titanic[titanic['z_score'].abs() > 3]
outliers
```

Out[10]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	
27	28	0	1	Fortune, Mr. Charles Alexander	male	19.0	3	2	19950	2
88	89	1	1	Fortune, Miss. Mabel Helen	female	23.0	3	2	19950	2
118	119	0	1	Baxter, Mr. Quigg Edmond	male	24.0	0	1	PC 17558	2
258	259	1	1	Ward, Miss. Anna	female	35.0	0	0	PC 17755	5
299	300	1	1	Baxter, Mrs. James (Helene DeLaudeniere Chaput)	female	50.0	0	1	PC 17558	2
311	312	1	1	Ryerson, Miss. Emily Borie	female	18.0	2	2	PC 17608	2
341	342	1	1	Fortune, Miss. Alice Elizabeth	female	24.0	3	2	19950	2
377	378	0	1	Widener, Mr. Harry Elkins	male	27.0	0	2	113503	2
380	381	1	1	Bidois, Miss. Rosalie	female	42.0	0	0	PC 17757	2
438	439	0	1	Fortune, Mr. Mark	male	64.0	1	4	19950	2
527	528	0	1	Farthing, Mr. John	male	28.0	0	0	PC 17483	2
557	558	0	1	Robbins, Mr. Victor	male	28.0	0	0	PC 17757	2
679	680	1	1	Cardeza, Mr. Thomas Drake Martinez	male	36.0	0	1	PC 17755	5
689	690	1	1	Madill, Miss. Georgette Alexandra	female	15.0	0	1	24160	2
700	701	1	1	Astor, Mrs. John Jacob (Madeleine Talmadge Force)	female	18.0	1	0	PC 17757	2

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket
716	717	1	1	Endres, Miss. Caroline Louise	female	38.0	0	0	PC 17757
730	731	1	1	Allen, Miss. Elisabeth Walton	female	29.0	0	0	24160
737	738	1	1	Lesurer, Mr. Gustave J	male	35.0	0	0	PC 17755
742	743	1	1	Ryerson, Miss. Susan Parker "Suzette"	female	21.0	2	2	PC 17608
779	780	1	1	Robert, Mrs. Edward Scott (Elisabeth Walton Mc...	female	43.0	0	1	24160

In [13]:
```
iris = pd.read_csv("Iris.csv")
```

In [14]:
```
iris.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 6 columns):
 #   Column         Non-Null Count  Dtype  
---  ------         --------------  -----  
 0   Id             150 non-null    int64  
 1   SepalLengthCm  150 non-null    float64
 2   SepalWidthCm   150 non-null    float64
 3   PetalLengthCm  150 non-null    float64
 4   PetalWidthCm   150 non-null    float64
 5   Species        150 non-null    object 
dtypes: float64(4), int64(1), object(1)
memory usage: 7.2+ KB
```

In [15]:
```
q1 = iris['SepalLengthCm'].quantile(0.25)
q3 = iris['SepalLengthCm'].quantile(0.75)
iqr = q3 - q1
lower_bound = q1 - 1.5 * iqr
upper_bound = q3 + 1.5 * iqr
outliers = iris[(iris['SepalLengthCm'] < lower_bound) | (iris['SepalLengthCm'] >
print(outliers)
```

```
Empty DataFrame
Columns: [Id, SepalLengthCm, SepalWidthCm, PetalLengthCm, PetalWidthCm, Species]
Index: []
```

In [16]:
```
iris['Z_Score'] = stats.zscore(iris['PetalWidthCm'])
outliers = iris[(iris['Z_Score'] < -3) | (iris['Z_Score'] > 3)]
print(outliers)
```

```
Empty DataFrame
Columns: [Id, SepalLengthCm, SepalWidthCm, PetalLengthCm, PetalWidthCm, Species,
Z_Score]
Index: []
```

In [18]:
```
from sklearn.datasets import load_boston
boston = load_boston()
df = pd.DataFrame(boston.data, columns=boston.feature_names)
df['MEDV'] = boston.target
print(df.info())
print(df.head())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 506 entries, 0 to 505
Data columns (total 14 columns):
 #   Column   Non-Null Count  Dtype
---  ------   --------------  -----
 0   CRIM     506 non-null    float64
 1   ZN       506 non-null    float64
 2   INDUS    506 non-null    float64
 3   CHAS     506 non-null    float64
 4   NOX      506 non-null    float64
 5   RM       506 non-null    float64
 6   AGE      506 non-null    float64
 7   DIS      506 non-null    float64
 8   RAD      506 non-null    float64
 9   TAX      506 non-null    float64
 10  PTRATIO  506 non-null    float64
 11  B        506 non-null    float64
 12  LSTAT    506 non-null    float64
 13  MEDV     506 non-null    float64
dtypes: float64(14)
memory usage: 55.5 KB
None
      CRIM    ZN  INDUS  CHAS    NOX     RM   AGE     DIS  RAD    TAX  \
0  0.00632  18.0   2.31   0.0  0.538  6.575  65.2  4.0900  1.0  296.0
1  0.02731   0.0   7.07   0.0  0.469  6.421  78.9  4.9671  2.0  242.0
2  0.02729   0.0   7.07   0.0  0.469  7.185  61.1  4.9671  2.0  242.0
3  0.03237   0.0   2.18   0.0  0.458  6.998  45.8  6.0622  3.0  222.0
4  0.06905   0.0   2.18   0.0  0.458  7.147  54.2  6.0622  3.0  222.0

   PTRATIO       B  LSTAT  MEDV
0     15.3  396.90   4.98  24.0
1     17.8  396.90   9.14  21.6
2     17.8  392.83   4.03  34.7
3     18.7  394.63   2.94  33.4
4     18.7  396.90   5.33  36.2
```

In [21]:
```
df = df.dropna(subset=['RM'])
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 506 entries, 0 to 505
Data columns (total 14 columns):
 #   Column   Non-Null Count  Dtype  
---  ------   --------------  -----  
 0   CRIM     506 non-null    float64
 1   ZN       506 non-null    float64
 2   INDUS    506 non-null    float64
 3   CHAS     506 non-null    float64
 4   NOX      506 non-null    float64
 5   RM       506 non-null    float64
 6   AGE      506 non-null    float64
 7   DIS      506 non-null    float64
 8   RAD      506 non-null    float64
 9   TAX      506 non-null    float64
 10  PTRATIO  506 non-null    float64
 11  B        506 non-null    float64
 12  LSTAT    506 non-null    float64
 13  MEDV     506 non-null    float64
dtypes: float64(14)
memory usage: 59.3 KB
```

In [23]:
```
q1 = df['AGE'].quantile(0.25)
q3 = df['AGE'].quantile(0.75)
iqr = q3 - q1
lower_bound = q1 - 1.5 * iqr
upper_bound = q3 + 1.5 * iqr
outliers = df[(df['AGE'] < lower_bound) | (df['AGE'] > upper_bound)]
print(outliers['AGE'])
```

```
Series([], Name: AGE, dtype: float64)
```

In [24]:
```
wine = pd.read_csv("winequality-red.csv")
wine.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1599 entries, 0 to 1598
Data columns (total 12 columns):
 #   Column                Non-Null Count  Dtype  
---  ------                --------------  -----  
 0   fixed acidity         1599 non-null   float64
 1   volatile acidity      1599 non-null   float64
 2   citric acid           1599 non-null   float64
 3   residual sugar        1599 non-null   float64
 4   chlorides             1599 non-null   float64
 5   free sulfur dioxide   1599 non-null   float64
 6   total sulfur dioxide  1599 non-null   float64
 7   density               1599 non-null   float64
 8   pH                    1599 non-null   float64
 9   sulphates             1599 non-null   float64
 10  alcohol               1599 non-null   float64
 11  quality               1599 non-null   int64  
dtypes: float64(11), int64(1)
memory usage: 150.0 KB
```

In [25]:
```
wine['total sulfur dioxide'] = wine['total sulfur dioxide'].fillna(wine['total s
```

In [26]:
```
wine['fixed acidity'] = wine['fixed acidity'].fillna(0)
print(wine)
```

```
      fixed acidity  volatile acidity  citric acid  residual sugar  chlorides  \
0               7.4             0.700         0.00             1.9      0.076
1               7.8             0.880         0.00             2.6      0.098
2               7.8             0.760         0.04             2.3      0.092
3              11.2             0.280         0.56             1.9      0.075
4               7.4             0.700         0.00             1.9      0.076
...             ...               ...          ...             ...        ...
1594            6.2             0.600         0.08             2.0      0.090
1595            5.9             0.550         0.10             2.2      0.062
1596            6.3             0.510         0.13             2.3      0.076
1597            5.9             0.645         0.12             2.0      0.075
1598            6.0             0.310         0.47             3.6      0.067

      free sulfur dioxide  total sulfur dioxide  density    pH  sulphates  \
0                    11.0                  34.0  0.99780  3.51       0.56
1                    25.0                  67.0  0.99680  3.20       0.68
2                    15.0                  54.0  0.99700  3.26       0.65
3                    17.0                  60.0  0.99800  3.16       0.58
4                    11.0                  34.0  0.99780  3.51       0.56
...                   ...                   ...      ...   ...        ...
1594                 32.0                  44.0  0.99490  3.45       0.58
1595                 39.0                  51.0  0.99512  3.52       0.76
1596                 29.0                  40.0  0.99574  3.42       0.75
1597                 32.0                  44.0  0.99547  3.57       0.71
1598                 18.0                  42.0  0.99549  3.39       0.66

      alcohol  quality
0         9.4        5
1         9.8        5
2         9.8        5
3         9.8        6
4         9.4        5
...       ...      ...
1594     10.5        5
1595     11.2        6
1596     11.0        6
1597     10.2        5
1598     11.0        6

[1599 rows x 12 columns]
```

In [ ]:

Department of Artificial Intelligence (AI) & Data Science

AY: 2025-26
Course: Python for Data Science Laboratory (DJS23SLPC303)
Class: SE/A11/A12/A21/A22 Sem: III Roll No.: SAPID:

Experiment No. 12: Data Cleaning

Aim: To understand and apply techniques for handling missing data and finding outliers usingPython with NumPy and pandas, focusing on practical applications in data science and machine learning.

Theory

1. What is meant by missing data? Why is it important to handle missing values in a dataset?
2. Explain the difference between mean imputation and median imputation. In which case is median preferred?
3. What is an outlier? How do outliers affect the performance of machine learning models?
4. Discuss how improper handling of missing values and outliers can bias model training.
5. Suggest an approach to handle missing categorical data.
6. Can outliers ever be useful? Give an example from real-world data.
7. What are the advantages and limitations of using Z-scores for outlier detection?

Conclusions:

name :- Aaryan W. Raorane Roll No :- 5004 Branch :- AIDS
Div :- 5 batch :- A1 SAPID :- 60018250070

Experiment No 7

06/10

Title :- Data Cleaning

Aim :- To understand and apply techniques for handling missing data and finding outliers using Python with Numpy and Pandas, focusing on practical applications in data science and machine learning.

Theory :-

1) What is meant by missing data? Why is it important to handle missing value in a dataset?

=> Missing data means that no value is stored for a variable in an observation, and pandas shows it as NaN. It arises from skipped survey questions, sensor failures, entry mistakes or failed data merges. Handling it is important because most machine learning algorithm cannot accept NaN, and calculations such as mean or sum before inaccurate. Ignoring missing values also reduces the usable data and can introduce bias in the results.

Code :-

```
import pandas as pd
import numpy as np
df = pd.DataFrame({"Name": ["A", "B", "C", "D"],
                   "Age": [21, np.nan, 23, 22],
                   "Marks": [78, 85, np.nan, np.nan]})
print(df.isnull().sum())
print(df.dropna())
```

Sundaram
FOR EDUCATIONAL USE

output :

```
Name      0
Age       1
Marks     2
dtype: int64
```

	Name	Age	Marks
0	A	21.0	78.0

Q2 Explain the difference between mean imputation and median imputation. In which case is median preffd preferred?

=>

Point	Mean Imputation	Median Imputation
value used	Average of of all available value	Middle value of the sorted available values.
Effect of outliers	Pulled strongly towards extreme values	Not affected by extreme values.
suitable for	Symmetric data without outliers	skewed data or data with outliers.
syntax	fillna(col.mean())	fillna(col.median())

Sundaram
FOR EDUCATIONAL USE

Median is preferred when the column is skewed or has outliers, for example salary because a few very large value raise the mean and the filled values would then be unrealistically high.

code:

```
import pandas as pd
import numpy as np

salary = pd.Series([30,32,35,np.nan,38,400])
print(salary.fillna(salary.mean()))
print(salary.fillna(salary.median()))
```

output:

```
0     30.0
1     32.0
2     35.0
3    107.0
4     38.0
5    400.0
dtype: float64
0     30.0
1     32.0
2     35.0
3     35.0
4     38.0
5    400.0
dtype: float64
```

Sundaram®
FOR EDUCATIONAL USE

Q3 What is an outlier? How do outliers affect the performance of Machine learning models?

=> An outlier is a data point that lies far away from the other observation in a dataset. It may come from a measurement error, a data entry mistake or a genuine rare event. Outliers pull the mean and increase the standard deviation, so the statistics no longer describe the normal data. Models such as linear regression and distance based methods are sensitive to them because the fitted line or the cluster centres shift and the accuracy on normal data falls.

Code:

```
import numpy as np
data = np.array([52, 55, 53, 58, 54, 56, 300])
print("Mean with outlier", round(np.mean(data), 2))
print("Mean without outlier", round(np.mean(data[:6]), 2))
print("Std with outlier", round(np.std(data), 2))
print("Std without outlier", round(np.std(data[:6]), 2))
```

Output:

```
Mean with outlier 89.71
Mean without Outlier 54.69
Std with outlier 85.87
Std without outlier 1.97
```

Sundaram FOR EDUCATIONAL USE

Q4 Discuss how improper handling of missing values and outliers can bias ~~model~~ machine learning models?

⇒ If rows with missing values are simply dropped, a large part of the data may be lost, and if the values are not missing at random the remaining rows no longer represent the population, so the model learns a biased pattern. Filling every gap with the mean reduces the variance of the column ~~bias~~ and weakens its relationship with other variables. Outliers that are left in the data pull the learned parameters towards themselves, while outliers removed without checking may delete genuine extreme cases. The model then performs well on the training data but gives poor predictions on new data.

code:-

```
import pandas as pd
import numpy as np
x = pd.Series([10, 20, 30, 40, 50, np.nan, np.nan, np.nan])
print("std before filling", round(x.std(), 2))
print("std after mean filling", round(x.fillna(x.mean()).std(), 2))
```

output:- std before filling 15.81
std after mean filling 11.95

Sundaram®
FOR EDUCATIONAL USE

Q5 Suggest an approach to handle missing categorical data.

→ Missing categorical values cannot be filled with mean or median. A common approach is mode imputation, where the most frequent category replaces the gaps. Another approach is to create a separate category such as unknown, which also keeps the information that the value was missing. If only a few rows are affected, they can be dropped.

code:

```
import pandas as pd
import numpy as np

df = pd.DataFrame({"City": ["Mumbai", "Pune",
                            np.nan, "Mumbai",
                            np.nan]})
df["Mode"] = df["City"].fillna(df["City"].mode()[0])
df["Unknown"] = df["City"].fillna("Unknown")
```

output:

	City	Mode	Unknown
0	Mumbai	Mumbai	Mumbai
1	Pune	Pune	Pune
2	NaN	Mumbai	Unknown
3	Mumbai	Mumbai	Mumbai
4	NaN	Mumbai	Unknown

Sundaram
FOR EDUCATIONAL USE

Q 6 Can outliers ever be useful? Give an example from real-world data.

=> Yes an outlier is not always an Error, and sometimes it is the most important observation. In banking, a customer who normally spends a few thousand rupees and suddenly makes a transaction of several lakhs is given an outlier, and this is exactly the pattern a fraud detection system looks for. In the same way, an unusually high network traffic reading can point to an intrusion, and a very high body temperature shows a patient who needs attention.

Code :-

```
import pandas as pd
txn = pd.DataFrame({"Amount": [2500, 3200,
                               1800, 2900,
                               950000, 3100]})
print(txn[txn["Amount"] > 100000])
```

output :-

```
        Amount
4       950000
```

Sundaram®
FOR EDUCATIONAL USE

Q7 What are the advantages and limitations of using z-scores for outlier detection?

=> The z score tells how many standard deviations a value lies from the mean. It is calculated as the value minus the mean, divided by the standard deviation, and a value with an absolute z score above 3 is usually treated as an outlier

Advantages	Dis Limitations
simple and quick to calculate using numpy or pandas	Mean and standard deviation are themselves affected by outliers, so extreme values can hide themselves.
Gives a standard scale, so different columns can be compared	Works well only when the data is roughly normally distributed
Threshold of 3 is easy to apply and explain	Not reliable for small datasets.

Sundaram®
FOR EDUCATIONAL USE

code :

```
import numpy as np

data = np.array([50,52,49,51,53,48,50,
                 52,51,49,50,53,47,
                 52,51,50,49,52,51,120])
z = (data - np.mean(data))/np.std(data)
print("z score of last value", round(z[19],2))
print("Outliers", data[np.abs(z)>3])
```

output :-

```
z score of last value 4.34
outliers [120]
```

conclusions :-

Missing values were identified and handled using Numby and pandas by dropping them and by mean, median and mode imputation. outliers were detected using the z score method, and their effect on the mean and standard deviation was observed. choosing a suitable method for each column keeps the data reliable for training machine learning models.

Sundaram
FOR EDUCATIONAL USE

Functions

In [1]:
```
def greet():
    print("I hate to speak Hello World")
```

In [2]:
```
greet()
print("outside function")
```

```
I hate to speak Hello World
outside function
```

In [3]:
```
def greet(name):
    print("Hello",name)
name=input("Enter Name : ")
greet(name)
```

```
Enter Name : Lucifer
Hello Lucifer
```

In [4]:
```
def add_numbers(num1,num2):
    return(num1+num2)
```

In [5]:
```
s=add_numbers(1,1)
print(s)
```

```
2
```

In [6]:
```
def square(num):
    return(num**2)
print(square(2))
```

```
4
```

In [7]:
```
def cube(num):
    return(num**3)
print(cube(2))
```

```
8
```

In [8]:
```
def future_function():
    pass#this will execute without any error or action
future_function()
```

In [9]:
```
def greet():
    message = "Hello"
    print("Local",message)
greet()
message = "Namaste"
print("Global",message)
```

```
Local Hello
Global Namaste
```

In [10]:
```
def outer():
    message = "Local"
    def inner():
        nonlocal message
        message = "Value 1"
```

```
        print("inner :",message)
    inner()
    print("outer :",message)
outer()
```

```
inner : Value 1
outer : Value 1
```

In [11]:
```
# default parameter
def greet(name,message="Hello"):
    print(message,name)
greet("Alice","おはようございます")
greet("Bob")
```

```
おはようございます Alice
Hello Bob
```

variable length arguements

args(passed as tuple) and kwargs (passed as dictionary)

In [12]:
```
def add_all(*nums):
    return(sum(nums))
print(add_all(1,2,3,4,5,6,7,8,9))
```

```
45
```

In [13]:
```
def student(**details):
    print(details)
student(name ="John smith" , age =20 , city ="Mumbai")
```

```
{'name': 'John smith', 'age': 20, 'city': 'Mumbai'}
```

In [14]:
```
def details(name,age=20,*skills,**info):
    print("Name :",name)
    print("Age :",age)
    print("Skills :",skills)
    print("Info :",info)
details("Alex",25,"Python","ML",city="Mumbai",year=2025)
```

```
Name : Alex
Age : 25
Skills : ('Python', 'ML')
Info : {'city': 'Mumbai', 'year': 2025}
```

recursion

In [15]:
```
def factorial(x):
    if x==1:
        return 1
    else:
        return (x*factorial(x-1))
print(factorial(6))
```

```
720
```

lambda function

In [17]:
```
adder = lambda x,y : x+y
print(adder(23135416541654,23435434358719))
```

```
46570850900373
```

lambda in filter

In [19]:
```
sequences = [10,2,8,7,5,4,3,11,0,1]
filtered_result = filter(lambda x:x>4,sequences)
print(list(filtered_result))#here instead of list we can use tuple,set but canno
```

```
[10, 8, 7, 5, 11]
```

lambda in map

In [21]:
```
sequences = [10,2,8,7,5,4,3,11,0,1]
map_result = map(lambda x:x*x,sequences)
print(list(map_result))
```

```
[100, 4, 64, 49, 25, 16, 9, 121, 0, 1]
```

In [26]:
```
from functools import reduce
sequences = [1,2,3,4,5,6,7,8,9,10]
reduce_result = reduce(lambda x,y:x*y,sequences)
print(reduce_result)
```

```
3628800
```

In [30]:
```
sequences = [1,2,3,4,5]
result1 = filter(lambda x:x>3,sequences)
result2 = map(lambda x:x+3,sequences)
result3 = reduce(lambda x,y:x*y ,sequences)
print(list(result1))
print(list(result2))
print(result3)
```

```
[4, 5]
[4, 5, 6, 7, 8]
120
```

In [ ]:
```
```

Department of AI_DS
Academic Year: 2026-27 (Odd Sem)
Course: Python for Data Science Lab
Second Year B. Tech Semester: III

PRACTICAL 7 & 8

Part A (To be referred by students)

Built-in functions and User Defined Functions

SAVE THE FILE AND UPLOAD AS (RollNo_Name_Exp7&8)

Aim: To understand and apply built in and user defined functions in Python programming and explore their usefulness in solving problems.

Problem Statement:

Solve the following computational problems by **recognizing** python data structure and **apply** them to implement following programs

User defined functions

1. Implement program using user defined function to check year is leap or not. A year is a leap if it is divisible by 4 and centennial years (years divisible by 100) are leap years only when they are divisible by 400.
2. Define a function ExtractEven(List) to return all even number from a list. Example l1[1,2,3,4,5,6] method should return [2,4,6].
3. Write a function that returns add, sub, mul and division of two numbers.
4. Define a user defined function to find area of circle or triangle. If one parameter is passed the given function finds the area of circle, if two parameters are passed the same function finds the area of triangle. You have to defined only one parameterized function which accepts either one parameters or two parameters to find area.
5. Write a program to find largest number using variable length argument. (if two parameters are passed it will find largest of two, if three passed it will find largest of 3 and so on …)
6. Implement a program to demonstrate **kwargs
7. Write a program to print Fibonacci series upto n using recursion
8. Define a recursive function find_max(lst) that returns the maximum value in the list lst.
9. Create a recursive function string_length(s) that returns the length of the string s.
10. Define power(base, exp) that calculates base raised to the power of exp using recursion.
11. Write a recursive function print_pattern(n) that prints a pattern of * in n rows.
12. Create a recursive function factorial(n) that returns the factorial of n.
13. Write a function that merges two dictionaries. The merger should handle cases where the dictionaries have overlapping keys by storing values in a list. If a key exists in

both dictionaries, the value in the resulting dictionary should be a list containing the values from both dictionaries.

Built-in Functions

14. Write map function to return a list of numbers which are square of first 10 Fibonacci numbers.
15. Write a lambda function for adding 26 to its argument, also create a lambda function that multiplies argument x with argument y and print the result.
16. Write program using lambda function to display numbers divisible by 9 from the given the list
17. A twitter database is given in which we have username and tweets.
    users = [{"username": 'seema', "tweets": ["i love chocolate", "i am a student"]}, {"username": 'arush', "tweets": []}, {"username": 'kumal', "tweets": ["India", "Python"]}, {"username": 'sam', "tweets": []}, {"username": 'lokesh', "tweets": ["i am good"]},]
    i) Filter out Users which don't have any tweets/Inactive Users
    ii) Filter inactive users with just username in uppercase.
    iii) Return a new list with the string "your name is" + name, but only if length of name is bigger than 4 using map , filter and Lambda

    names=['Rohan','Rhea','bob','to']
    Output:['your name is Rohan', 'your name is Rhea']

Topic covered: User Defined Functions and Built in functions

Learning Objective: Learner would be able to

1. Learn how to define, and invoke user defined functions
2. Apply user defied/ built in functions to solve problems
3. Work on recursion.

Theory:

- Functions are sub program & in Python, interpreter creates an object of function internally.
- Functions are used to perform a specific task
- If there are several tasks, one can write multiple functions
- When we define function inside a class, it is called as method.
- 'def' keyword is used to define functions
- Syntax of function definition:
  def functionname(parameter list):
      #function body

- A functions can not executes its own, it needs to call/invocation when required.
- Function is called using its name and passing values if any.
- Syntax:
  functionname(value/values)

Example:-

```
def main():
    print("Welcome")
print(__name__)

if __name__== "__main__":
    main()
```

- It is also possible to return values back to the calling place from function.
- Return is used to send value back to calling place.

- **Advantages of Functions:-**
  - Reusability
  - Easy to maintain and write
  - Easy to debug
  - Functions provides modularity(part of a program) for programming & work distribution is possible
  -

Retuning Values:-

- In Python's function, we can send one or multiple values back to calling place
- If return is not used in function, it will send None back to the calling place
- Return can send one or multiple values back to the calling place
  - Example.

  - **Returning None**

```
def fun():
        print(1)
a=fun()
print(a)
```

  - **Returning only one Value**

```
def fun():
        return 10
a=fun()
print(a)
```

- **Returning Multiple Values**

```
def fun():
        return 1,2,3,4
a=fun()   #a,b,c,d=1,2,3,4
print(a)
```

- Multiple Values are return in form of a tuple…

Pass by Objects

- There are various function calling mechanisms like
  - Call/pass by Value
  - Call/pass by Address
  - Call/pass by Reference
- But, Python programming supports call/pass by object.
- Because, everything is considered as object in Pyhon.

Example of Number:-	Example of a List

def modify(x):
  x=100
  print("In function",x,id(x))
 	
x=50
print("B4 call x=",x,id(x))
modify(x)
print("After call x=",x,id(x))
	def modify(lst):
	  lst[0]=1000
	  print("In function ",lst,id(lst))
	y=[1,2,3,4]
	print("B4 call",y,id(y))
	modify(y)
	print("After call",y,id(y))

Why this is happening? As Numbers, Strings and Tuples are **immutable**. (If we try to modify, new object will be created)	List and dictionaries are **mutable**, if we modified it, same object will get updated…

Arguments:-

- Formal Arguments/Parameters
- Actual Arguments/Parameters
- Following Types of Arguments are supported by Functions in Python.
  - Positional Arguments
  - Keyword/named Arguments
  - Default Arguments
  - Variable Length Arguments

Recursion:-

- A Function having call to itself.

- Example:

```
def fun():
        fun()
fun()
```

Variable Length Argument(varargs):

- You can specify arbitrary number of arguments(varargs).
- Variable length argument
  - function(farg, *args)
    - Args is implicit tuple
- Keyword variable length argument
  - function(**kwargs)
    - Kwargs is implicit dictionary
    - Example:

```
def fun(**kwargs):
   print(kwargs)
fun(a=2,b='3',c='4')
```

Built-in Functions:-

Lambda ¶

Lambda is a single expression anonymous function.

Often used as an inline function

Syntax: lambda arguments : expression

```
#Write a Python program to create a lambda function
#1.that adds two numbers passed in as an argument
l1 = lambda a,b : a + b
r = l1(10,20)
print(r)
```

Map

The map() function applies a function to every member of an iterable and returns the result.

It takes two parameters: first, the function that is to be applied, and second, the iterables.

Syntax: map(function_namr, iterator)

The returned value from map() is map object, it can be passed to functions like list()/set() to create sequence

```
#WAP to find length of 5 students name stored in list using map()
lst = ["Priyanshu","Shantanu","Parth","Harsh","Krish"]
c = map(len,lst)
l = list(c)
print(l)
```

```
[9, 8, 5, 5, 5]
```

Filter

Syntax: filter(function, Iterable)

```
#Write a program to filter out even and odd numbers from list...
lst = [59,34,35,23,8,46,76,2,43]

odd = filter(lambda x: x % 2, lst)
print("odd numbers are ",list(odd))


evn = filter(lambda x: x % 2 == 0, lst)
print("Even numbers are ",list(evn))
```

```
odd numbers are  [59, 35, 23, 43]
Even numbers are  [34, 8, 46, 76, 2]
```

Reduce

syntax: reduce(function, iterable) ¶

The first argument in reduce() is a function. This function will be applied to all the elements in an iterable in a cumulative manner to compute the result.

The second argument is iterable. Iterables are those python objects that can be iterated/looped over, includes like lists, tuples, sets, dictionaries, generators, iterator, etc

```
# Write a program to find sum of all elements in a list...
from functools import reduce

lst = [1, 2, 3, 4,5,6,]
sum = reduce(lambda x, y: x + y, lst)
print(sum)
```

```
21
```

zip

syntax: zip(*iterables)

zip(iterable1, iterable2, iterable3, ...).

The zip() function in python is an in-built function that takes any number of iterables(list, str, etc.) as an argument and aggregate them together by taking elements from each iterable passed as argument.

It stops when the shortest iterable is exhausted and the elements in longer iterables are left out.

```
# Example...
names = ["Alex", "Stone", "Scott"]

ages = (20, 41, 61,36)

l = [1,2,3,4,5,6]

zp = zip(names, ages, l)

print(list(zp))
```

```
[('Alex', 20, 1), ('Stone', 41, 2), ('Scott', 61, 3)]
```

PRACTICAL 7

Part B (to be completed by students)

Built-in Functions & User Defined Function

1. All the students are required to perform the given tasks in Jupyter Notebook
2. Create a new notebook for each experiment. The filename should be RollNo_Name_Exp7)

3. In the first cell, the student must write his/her Name, roll no and class in the form of comments
4. Every program should be written in separate cells and in the given sequence
5. After completing the experiment, download the notebook in pdf format. The filename should be RollNo_Name_Exp7.pdf).
   Note: - To download as a PDF, go to File and choose Print Preview. This will open a new tab. Now, press Ctrl + P and opt to save as PDF.
   OR
   Download notebook as HTML and open downloaded html(webpage), press CTRL+P and opt to save as PDF.
6. Upload the pdf on the web portal

For Understanding::

```
def sum(a,b)
        if a>10:
           return a+b
        else :
           return a+b+100
print(sum(10,20))

newsum=lambda a,b: a+b
print(newsum(10,20))
```

Department of AI_DS
Academic Year: 2026-27 (Odd Sem)
Course: Python for Data Science Lab
Second Year B. Tech Semester: III

```
# normally we define a function like this
def square(num):
    return num**2

# let's call this function
square(5)  25

# now lambda function implementation for the same function would be
num_square = lambda num: num**2

# let's call num_square
num_square(5)  25
```

---

ids=[20,30,40]
name=['Ram',"lakshman',"Sita']
record=dict(zip(ids,name))

……..

def xyz(n):
return n*n
l=[2,4,7]
c=map(xyz,l)
print(list(c))

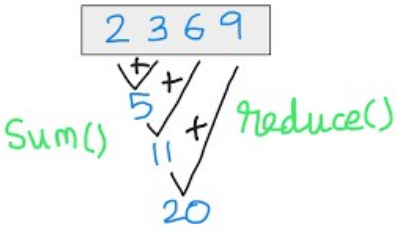
……

d= lambda n:n*n
l=[2,4,7]
c=map(d,l)
print(list(c))

---

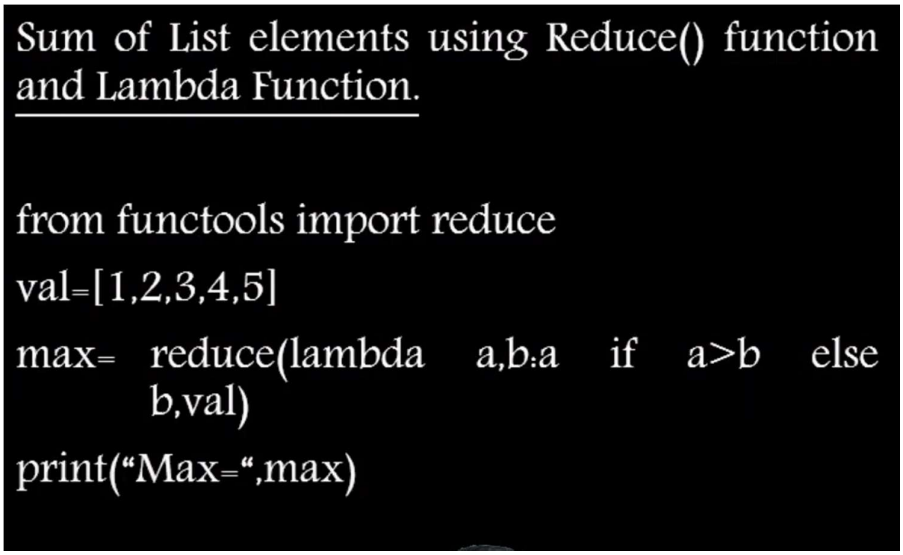
filter(a,b) // a function apply on b sequence

Reduce



Finding Max number in the list using Reduce() function.

```
from functools import reduce
def myFunc(a,b):
        if a>b:
                return a
        else:
                return b
val=[1,2,3,4,5]
max=reduce(myFunc,val)
print("Max=",max)
```




```
# importing reduce function
from functools import reduce

# let's create list of numbers till 10
my_list = [num for num in range(1,11)]
my_list [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

# basically what we need is this
1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9 + 10 55

# using reduce to compute sum of list
total = reduce(lambda x, y: x + y, my_list)
print(total) 55

# using reduce to compute max element from list, expected output 10
max_element = reduce(lambda a, b: a if a > b else b, my_list)
print(max_element) 10
```

Filter , Map, Reduce Functions

Huge Data → Filter ()→ small Data → Map() → change in data → reduce() → reduced data

```python
# function to convert temperature ( C -> F)
def celsius_to_fahrenheit(temp):
    return (temp * 9 / 5) + 32

# function to convert temperature ( F -> C)
def fahrenheit_to_celsius(temp):
    return (temp - 32) * 9 / 5

# list of celsius temperature
c_temp = [17, 20, 35, 0, -2, -18, 68]

# list of fahrenheit temperature
f_temp = [89, 90, 61, 20, 72, 56, 104.4]

#  use regular functions to convert temperature
print(list(map(celsius_to_fahrenheit, c_temp)))
print(list(map(fahrenheit_to_celsius, f_temp)))

# use lambda functions to convert temperature
print(list(map(lambda t: (t * 9 / 5) + 32, c_temp)))
print(list(map(lambda t: (t - 32) * 9 / 5, f_temp)))
```

In [1]:
```python
# Q1: Leap year check
def is_leap(y):
    return (y % 4 == 0 and y % 100 != 0) or (y % 400 == 0)

for y in [2000, 1900, 2024, 2023]:
    print(y, "Leap year" if is_leap(y) else "Not a leap year")
```

```
2000 Leap year
1900 Not a leap year
2024 Leap year
2023 Not a leap year
```

In [2]:
```python
# Q2: Extract even numbers from a list
def ExtractEven(lst):
    return [x for x in lst if x % 2 == 0]

print(ExtractEven([1, 2, 3, 4, 5, 6]))
```

```
[2, 4, 6]
```

In [3]:
```python
# Q3: Add, sub, mul, div of two numbers
def calc(a, b):
    return a + b, a - b, a * b, (a / b if b != 0 else "Division by zero")

print(calc(20, 5))
```

```
(25, 15, 100, 4.0)
```

In [4]:
```python
# Q4: Area of circle (1 argument) or triangle (2 arguments)
def area(a, b=None):
    if b is None:
        return 3.14159 * a * a
    return 0.5 * a * b

print("Circle:", area(5))
print("Triangle:", area(10, 4))
```

```
Circle: 78.53975
Triangle: 20.0
```

In [5]:
```python
# Q5: Largest number using variable length arguments
def largest(*nums):
    return max(nums)

print(largest(4, 9))
print(largest(4, 9, 15))
print(largest(4, 9, 15, 2, 30))
```

```
9
15
30
```

In [6]:
```python
# Q6: Demonstrate **kwargs
def show_info(**kwargs):
    for k, v in kwargs.items():
        print(k, ":", v)

show_info(name="Aaryan", branch="AI and DS", year=2)
```

```
name : Aaryan
branch : AI and DS
year : 2
```

In [7]:
```python
# Q7: Fibonacci series up to n terms using recursion
def fib(n):
    if n <= 1:
        return n
    return fib(n - 1) + fib(n - 2)

n = 10
print([fib(i) for i in range(n)])
```

```
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
```

In [8]:
```python
# Q8: Maximum value in a list using recursion
def find_max(lst):
    if len(lst) == 1:
        return lst[0]
    m = find_max(lst[1:])
    return lst[0] if lst[0] > m else m

print(find_max([3, 8, 1, 25, 7]))
```

```
25
```

In [9]:
```python
# Q9: Length of a string using recursion
def string_length(s):
    if s == "":
        return 0
    return 1 + string_length(s[1:])

print(string_length("Python"))
```

```
6
```

In [10]:
```python
# Q10: Power using recursion
def power(base, exp):
    if exp == 0:
        return 1
    return base * power(base, exp - 1)

print(power(2, 5))
```

```
32
```

In [11]:
```python
# Q11: Print star pattern in n rows using recursion
def print_pattern(n):
    if n == 0:
        return
    print_pattern(n - 1)
    print("*" * n)

print_pattern(5)
```

```
*
**
***
****
*****
```

In [12]:
```python
# Q12: Factorial using recursion
def factorial(n):
    if n <= 1:
        return 1
    return n * factorial(n - 1)

print(factorial(5))
```

```
120
```

In [14]:
```python
# Q13: Merge two dictionaries, overlapping keys stored as a list
def merge_dicts(d1, d2):
    result = dict(d1)
    for k, v in d2.items():
        if k in result:
            result[k] = [result[k], v]
        else:
            result[k] = v
    return result

print(merge_dicts({"a": 1, "b": 2}, {"b": 3, "c": 4}))
```

```
{'a': 1, 'b': [2, 3], 'c': 4}
```

In [15]:
```python
# Q14: Squares of first 10 Fibonacci numbers using map
fibs = [fib(i) for i in range(10)]
print(list(map(lambda x: x * x, fibs)))
```

```
[0, 1, 1, 4, 9, 25, 64, 169, 441, 1156]
```

In [16]:
```python
# Q15: Lambda to add 26, and lambda to multiply x and y
add26 = lambda x: x + 26
mul = lambda x, y: x * y
print(add26(10))
print(mul(6, 7))
```

```
36
42
```

In [17]:
```python
# Q16: Numbers divisible by 9 using lambda
nums = [9, 12, 18, 25, 27, 40, 45, 50]
print(list(filter(lambda x: x % 9 == 0, nums)))
```

```
[9, 18, 27, 45]
```

In [18]:
```python
# Q17 i: Filter out inactive users (no tweets)
users = [
    {"username": 'seema', "tweets": ["i love chocolate", "i am a student"]},
    {"username": 'arush', "tweets": []},
    {"username": 'kumal', "tweets": ["India", "Python"]},
    {"username": 'sam', "tweets": []},
    {"username": 'lokesh', "tweets": ["i am good"]},
]

inactive = list(filter(lambda u: not u["tweets"], users))
print(inactive)
```

```
[{'username': 'arush', 'tweets': []}, {'username': 'sam', 'tweets': []}]
```

In [19]:
```python
# Q17 ii: Inactive users, only username in uppercase
print(list(map(lambda u: u["username"].upper(), filter(lambda u: not u["tweets"]
```

```
['ARUSH', 'SAM']
```

In [20]:
```
# Q17 iii: "your name is" + name, for names with length 4 or more
names = ['Rohan', 'Rhea', 'bob', 'to']
print(list(map(lambda n: "your name is " + n, filter(lambda n: len(n) >= 4, name
```

```
['your name is Rohan', 'your name is Rhea']
```

In [ ]:

Department of Artificial Intelligence (AI) & Data Science

AY: 2026-27
Course: Python for Data Science Laboratory (DJS23SLPC303)
Class: SE/A11/A12/A21/A22 Sem: III Roll No.: SAPID:

Experiment No. 7: Built-in functions and User Defined Functions.

Aim: To understand and apply built in and user defined functions in Python programming and explore their usefulness in solving problems.

Theory

1. What are the advantages of using functions in Python programs?
2. Differentiate between positional arguments, keyword arguments, and default arguments with examples.
3. What is recursion? How does it differ from iteration?
4. Compare lambda functions with normal user-defined functions.
5. Discuss the difference between *local* and *global* scope in relation to Python functions.

Conclusions:

Name:- Aaryan N. Raorane Roll No:- 5004 Branch:- AIDS
Batch:- A1 SAP ID:- 60018250070 Div:- 5

Experiment No. 9 06/10

Title:- Built-in functions and User defined functions.

AIM:- To understand and apply built in and user defined functions in Python programming and explore their usefulness in solving problems.

Theory:-

1- What are the advantages of using functions in Python Programs?

=> Functions allow a block of code to be written once and used many times, which removes repeated code and keeps the program short. A large problem can be divided into smaller functions, so each part is easier to read, test and debug. Any change made only inside a function takes effect wherever it is called, which makes maintenance simple. Function take inputs through parameters and sent results back with return, so the same logic works for different data. Python provides built in functions such as len, max and sum for common tasks, and the programmer writes user defined functions with the def keyword for tasks specific to the program.

Sundaram FOR EDUCATIONAL USE

code :-

```
numbers = [4,9,2,7]

print(len(numbers), max(numbers), sum(numbers))

def area(r):
    return 3.14159 * r * r

print(area(2))
print(area(5))
```

output :-

```
4  9  22
12.56636
78.53975
```

Q2 Differentiate between positional arguements, keyward arguements, and default arguements with examples.

⇒

Point	Positional Arguements	Keyword Arguements	Default Arguements
Meaning	values are matched to parameters by their position	Values are passed along with the parameter name	Parameter has a preset value that is used when no value is passed.

FOR EDUCATIONAL USE

Order	Values are matched to parameters by their order must be followed	Orders does not matter	Must come after the parameters without defaults
Syntax	f(10,20)	f(a=10, b=20)	def f(a,b=20)
Use	short and simple calls	clear calls when there are many parameters	Makes some arguments optional

code:

```
def student(name, age, course='AI and DS')
    print(name, age, course)

student("Riya", 19)                          # positional
student(age=19, name="Riya")                 # Keyword
student("Riya", 19, "cyber security")        # replaces the default.
```

output:

```
Riya  19  AI and DS
Riya  19  AI and DS
Riya  19  Cyber Security.
```

Sundaram
FOR EDUCATIONAL USE

Q 3 What is recursion? How does it differ from iteration?

→ Recursion is a technique in which a function calls itself to solve a smaller version of the same problem until a base case is reached. The base case stops the calls, and the result are then combined as the calls return. Iteration repeats a block of statements using a for or while loop until its condition becomes false.

Point	Recursion	Iteration
working	Function calls itself	Loop repeats the same statements.
Termination	Stops when the base case is reached	stops when the loop condition becomes false
Memory	Each call uses a new stack frame, so more memory is needed.	uses a fixed amount of memory.
speed	Slower because of repeated function calls	faster in general.
Code	shorter and closer to the mathematical definition	usually longer.

Sundaram®
FOR EDUCATIONAL USE

Code :-

```
def factorial(n):
    if n == 0 or n == 1:
        return 1
    return n * factorial(n-1)

def factloop(n):
    result = 1
    for i in range(1, n+1):
        result = result * i
    return result

print(factorial(5))
print(factloop(5))
```

output :-

```
120
120
```

Q4 Compare lambda functions with normal user-defined functions.

⇒

Point	Lambda function	Normal function
Keyword	lambda	def

Sundaram®
FOR EDUCATIONAL USE

Name	Anonymous, but can be assigned to a variable	Has a name given in the definition.
Body	Only a single Expression	Many statements are allowed
Return	Value of the expression is returned automatically	return statement is used
Use	Short operations needed once	Reusable and longer logic

Code:

```
square = lambda x: x * x
add = lambda a, b: a + b

def squaredef(x):
    return x*x

print (square(6), add(3,4))
print ( squaredef(6))
print ((lambda x: x +10 )(5))
```

Output:- 36 7
36
15

Sundaram®
FOR EDUCATIONAL USE

Q5 Discuss the difference between local and global scope in relation to Python functions.

→ A variable created inside a function has local scope, so it exists only while the function runs and cannot be used outside it. A variable created outside all functions has global scope and can be read anywhere in the program. When a function needs to change a global variable it must declare it with the global keyword, otherwise Python creates a new local variable with the same name.

Point	Local scope	Global scope
Defined	Inside a function	Outside all function.
Lifetime	Till the function finishes	Till the program ends.
Access	Only inside that function	Everywhere in the program
Change inside a function	Directly possible	Needs the global keyword.

Sundaram
FOR EDUCATIONAL USE

Code:

```
x = 10
def show():
    x = 5
    print("inside", x)

def change():
    global x
    x = 20

show()
print("outside", x)
change()
print("after change", x)
```

Output:

```
inside 5
outside 10
after change 20
```

Conclusion:

Built in and user defined functions were studied and used to solve small problems in Python. Positional, Keyword and default arguments, recursion and lambda functions were implemented and their differences were observed. Local and global scope decide where a variable can be used, and the global keyword lets a function change a global variable.

Sundaram
FOR EDUCATIONAL USE

```
import MyModule1 as mm1
print(mm1.greet("WisdomKingAR"))
```

```
Hello,WisdomKingAR!
```

```
print(mm1.add(1,2,3,4,5,66,75,9))
```

```
165
```

```
print(mm1.multiply(25,336,45))
```

```
378000
```

```
help('MyModule1')
```

```
Help on module MyModule1:

NAME
    MyModule1 - # coding: utf-8

FUNCTIONS
    add(*args)

    greet(name)

    multiply(*args)

FILE
    c:\users\djsce.student\s004_aaryan_raorane_ai&ds_sy\python\
python_lecture_16_module_exception_and_file_handling\mymodule1.py
```

```
import MyModule2 as mm2

a=mm2.person1["age"]
b=mm2.person1["name"]
c=mm2.person1["country"]
print(a)
print(b)
print(c)
```

```
51
Alice
India
```

```
from MyModule2 import person1
print(person1["age"])
```

```
51
```

```
import math
math.pi
```

```
3.141592653589793
```

```
import statistics
statistics.mean([2,5,6,9])
```

```
5.5
```

```
from datetime import datetime,date,time
today=date.today()
now=datetime.now()
print(today)
print(now.strftime("%d-%m-%y %H:%M:%S")) #formatting for datetime
```

```
2026-10-06
06-10-26 11:03:00
```

```
#exception handling
a=int(input('Enter no 1: '))
b=int(input('Enter no 2: '))
c=0
try:
    c=a/b
except ZeroDivisionError:
    print('not possible')
else:
    print("ohh else worked")
finally:
    print('End')
```

```
Enter no 1: 10
Enter no 2: 0
not possible
End
```

```
#exception handling
a=int(input('Enter no 1: '))
b=int(input('Enter no 2: '))
c=0
try:
    c=a/b
except ZeroDivisionError:
    print('not possible')
else:
    print("ohh else worked")
finally:
    print('End')
```

```
Enter no 1: 10
Enter no 2: 10
ohh else worked
End
```

```python
#exception handling
try:
    a=int(input('Enter no 1: '))
    b=int(input('Enter no 2: '))
    c=0
    c=a/b
except ZeroDivisionError:
    print('Error : Division by zero is not allowed')
except ValueError:
    print('Error : Please Enter numbers only')
else:
    print("Division Successful")
finally:
    print('End of program')
```

```
Enter no 1: ghkwuh
Error : Please Enter numbers only
End of program
```

```python
#exception handling
try:
    a=int(input('Enter no 1: '))
    b=int(input('Enter no 2: '))
    c=0
    c=a/b
except ZeroDivisionError:
    print('Error : Division by zero is not allowed')
except ValueError:
    print('Error : Please Enter numbers only')
else:
    print("Division Successful")
finally:
    print('End of program')
```

```
Enter no 1: 10
Enter no 2: 0
Error : Division by zero is not allowed
End of program
```

```python
#exception handling
try:
    a=int(input('Enter no 1: '))
    b=int(input('Enter no 2: '))
    c=0
    c=a/b
except ZeroDivisionError:
    print('Error : Division by zero is not allowed')
except ValueError:
    print('Error : Please Enter numbers only')
else:
```

```
    print("Division Successful")
finally:
    print('End of program')
```

```
Enter no 1: 10
Enter no 2: 2
Division Successful
End of program
```

```
f=open("testing.txt","r")
```

Dont forget to close the file

```
str1=f.read()
print(str1)
```

```
1. Lorem ipsum dolor sit amet, consectetur adipiscing elit.
2. Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.
3. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris.
4. Nisi ut aliquip ex ea commodo consequat.
5. Duis aute irure dolor in reprehenderit in voluptate velit esse
cillum.
6. Dolore eu fugiat nulla pariatur.
7. Excepteur sint occaecat cupidatat non proident.
8. Sunt in culpa qui officia deserunt mollit anim id est laborum.
9. Curabitur pretium tincidunt lacus.
10. Nulla gravida orci a odio.
11. Nullam varius, turpis et commodo pharetra, est eros bibendum elit.
12. Nec luctus magna felis sollicitudin mauris.
13. Integer in mauris eu nibh euismod gravida.
14. Duis ac tellus et risus vulputate vehicula.
15. Donec lobortis risus a elit
```

```
str2=f.read(5)
print(str2)
```

```
1. Lo
```

```
f.close()
```

```
f1=open("testing-Copy1.txt","r")
```

```
str3=f.read(5)
print(str2)
```

```
---------------------------------------------------------------------------
-----
ValueError                                Traceback (most recent call
last)
<ipython-input-14-83d7a1c90145> in <module>
----> 1 str3=f.read(5)
      2 print(str2)

ValueError: I/O operation on closed file.
```

```
f1.close()
```

```
with open('testing-Copy1.txt', 'r') as f:
    str3 = f.read(100)  # Indented inside the block
    print(str3)       # This works perfectly
```

```
1. Lorem ipsum dolor sit amet, consectetur adipiscing elit.
2. Sed do eiusmod tempor incididunt ut l
```

```
with open("testing2.txt","w") as f:
    f.write("this content is good")
```

```
with open("testing2.txt","r") as f:
    str1 = f.read()
    print(str1)
```

```
this content is good
```

```
with open("testing2.txt","r") as f:
    str1 = f.readline()
    print(str1)
```

```
this content is good
```

```
with open("testing2.txt","r") as f:
    str1 = f.readline(7)
    print(str1)
```

```
this co
```

```
with open("testing.txt","r") as f:
    str1 = f.readlines()
    print(str1)
```

```
['1. Lorem ipsum dolor sit amet, consectetur adipiscing elit.\n', '2.
Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.\n',
'3. Ut enim ad minim veniam, quis nostrud exercitation ullamco
laboris.\n', '4. Nisi ut aliquip ex ea commodo consequat.\n', '5. Duis
aute irure dolor in reprehenderit in voluptate velit esse cillum.\n',
'6. Dolore eu fugiat nulla pariatur.\n', '7. Excepteur sint occaecat
cupidatat non proident.\n', '8. Sunt in culpa qui officia deserunt
mollit anim id est laborum.\n', '9. Curabitur pretium tincidunt
lacus.\n', '10. Nulla gravida orci a odio.\n', '11. Nullam varius,
turpis et commodo pharetra, est eros bibendum elit.\n', '12. Nec
luctus magna felis sollicitudin mauris.\n', '13. Integer in mauris eu
nibh euismod gravida.\n', '14. Duis ac tellus et risus vulputate
vehicula.\n', '15. Donec lobortis risus a elit']
```

```
f=open("testing4.txt","w")
print("Enter text and enter @ at the end : ")
while strn!="@":
    strn=input()
    if(strn!="@"):
```

```
        f.write(str1+'\n')
f.close()
```

```
Enter text and enter @ at the end :
```

```
---------------------------------------------------------------------------
-----
NameError                                 Traceback (most recent call
last)
<ipython-input-2-ff63c1179e7b> in <module>
      1 f=open("testing4.txt","w")
      2 print("Enter text and enter @ at the end : ")
----> 3 while strn!="@":
      4     strn=input()
      5     if(strn!="@"):

NameError: name 'strn' is not defined
```

```
line_number = 2
fo = open("testing.txt","r")
currentline = 1
for line in fo:
    if(currentline == line_number):
        print(line)
        break
    currentline = currentline + 1
```

```
2. Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.
```

```
f=open("testing.txt","r")
print(f.tell())
data = f.read(5)
print(data)
print(f.tell())
```

```
0
1. Lo
5
```

```
f.close()
```

```
infile = open("testing.txt","r")
print("Initially,the position is : ",infile.tell())
infile.seek(10)
print("Position after moving 10 bytes is : ",infile.tell())
infile.seek(0,2)
print("Last position of file : ",infile.tell())
```

```
Initially,the position is :  0
Position after moving 10 bytes is :  10
Last position of file :  800

infile.close()
```

```
f=open("testing.txt","r")
f.seek(101)
print(f.read())
```

```
abore et dolore magna aliqua.
3. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris.
4. Nisi ut aliquip ex ea commodo consequat.
5. Duis aute irure dolor in reprehenderit in voluptate velit esse
cillum.
6. Dolore eu fugiat nulla pariatur.
7. Excepteur sint occaecat cupidatat non proident.
8. Sunt in culpa qui officia deserunt mollit anim id est laborum.
9. Curabitur pretium tincidunt lacus.
10. Nulla gravida orci a odio.
11. Nullam varius, turpis et commodo pharetra, est eros bibendum elit.
12. Nec luctus magna felis sollicitudin mauris.
13. Integer in mauris eu nibh euismod gravida.
14. Duis ac tellus et risus vulputate vehicula.
15. Donec lobortis risus a elit
```

Department of AI_DS
Academic Year: 2026-27 (Odd Sem)
Course: Python for Data Science Lab
Second Year B. Tech Semester: III

PRACTICAL 10

Part A (To be referred by students)

File and Exception Handling

SAVE THE FILE AND UPLOAD AS (RollNo_Name_Exp10)

Aim: To understand and apply file handling and exception handling in Python

Problem Statement:

Understand and apply the various file handling operations to implement following programs

1. Implement a program to handle arithmetic exception
2. Write a program to handle following exceptions
    a. Type Error
    b. Value Error
    c. Make use of else with exception and check its effect
3. Write a program to write 1-20 numbers in a file & display same to the user.
4. Write a program to copy contents (your details) of one file to another file and display contents of both the files.
5. Write a python script to continue writing to a file until user enters choice as y and then on user input n stop writing and read the file and count words in the text file those are ending with alphabet "e".
6. Write a python script to count the number of lines from a text file which is not starting with an alphabet "T".
7. Write a python script to delete the specified line giving the line number from the text file.
8. Program to set file offsets with seek( ) and tell( ) functions in python

Topic covered: File Handling and Exception Handling

Learning Objective: Learner would be able to

1. Understand the various file handling operations
2. Apply the various file handling operations
3. Use exception handling
4. Implement file handling programs using exception handling

Department of AI_DS
Academic Year: 2026-27 (Odd Sem)
Course: Python for Data Science Lab
Second Year B. Tech Semester: III

Theory:

Data stored in variables and lists is temporary — it's lost when the program terminates. Python allows a program to read data from a file or write data to a file. Once the data is saved in a file on computer disk, it will remain there after the program stops running. The data can then be retrieved and used at a later time.

There are two types of files in Python - text files and binary files.

- A **text file** is processed as a sequence of characters. In a text file there is a special end-of-line symbol that divides file into lines. In addition, you can think that there is a special end-of-file symbol that follows the last component in a file. A big advantage of text files is that it may be opened and viewed in a text editor such as Notepad.
- A **binary file** stores data that has not been translated into character form. Binary files typically use the same bit patterns to represent data as those used to represent the data in the computer's main memory. These files are called binary files because the only thing they have in common is that they store the data as sequences of zeros and ones.

Steps to process file Input/output in Python:
Step 1. Open the file : Opening a file creates a connection between the file and the program.
Step 2. Process the file : In this step data is either written to the file (if it is an output file) or read from the file (if it is an input file).
Step 3. Close the file : When the program is finished using the file, the file must be closed. Closing a file disconnects the file from the program.

Python File Handling Operations:

- Open
- Read
- Write
- Close

Other operations include:

- Rename
- Delete

Opening a File

- Open()
- Fileheader =open('file name',"open mode","buffering")
- Various file opening modes
    - w
    - r
    - a

- Example:-
  F=open("myfile.txt","w")

Closing File:-

F.close( )

Example:- Writing Contents to file:

```
f=open('myfile.txt','w')
str=input('Enter text to write to file')
f.write(str)
f.close()
```

Example: - Reading Contents from file:

```
f=open('myfile.txt','r')
str=f.read( )
print(str)
f.close()
```

Example:- Reading first few Contents from file:

```
f=open('myfile.txt','r')
str=f.read(5 )
print(str)
f.close()
```

Program to store group of strings in file?

```
f=open('myfile.txt','w')
print('Enter text and enter @ at end ')
while str!='@':
   str=input()
   if(str!='@'):
      f.write(str+'\n')

f.close()
```

Program to read strings in file

```
f=open('myfile.txt','r')
print('File Contents are… ')
str=f.read()
print(str)
f.close()
```

Program to know whether file exits or not…

```
import os,sys
fname=input('Enter file name')
if os.path.isfile(fname):
   f=open(fname,'r')
```

```
else:
   print('file not exist')
   sys.exit()
print('File contents are')
str=f.read()
print(str)
f.close()
```

The With statement:

- It is used while opening file.
- It take care of closing file
- Need not to close file explicitly
- In case of exception also, it close the file before the exception is handled
- Syntax:-
  With open("filename", "openmode") as fileobject:
- Example:-

```
with open('myfile.txt','w') as f:
        f.write("This contents are written")
```

Python File Methods:
Python has various method available that we can use with the file object. The following table shows file method.

Method	Description
read()	Returns the file content.
readline()	Read single line
readlines()	Read file into a list
truncate(size)	Resizes the file to a specified size.
write()	Writes the specified string to the file.
writelines()	Writes a list of strings to the file.
close()	Closes the opened file.
seek()	Set file pointer position in a file
tell()	Returns the current file location.
fileno()	Returns a number that represents the stream, from the operating system's perspective.
flush()	Flushes the internal buffer.

Reading a File:

In order to read a file in python, we must open the file in read mode.
There are three ways in which we can read the files in python.

- read([n])
- readline([n])
- readlines()

Here, n is the number of bytes to be read.

```
# Reading the entire file at once
filename = "C:/Documents/Python/test.txt"
filehandle = open(filename, 'r')
filedata = filehandle.read()
print(filedata)
```

--OR—

```
with open("large_file.txt", "r") as f:
   print(f.read())
```

```
# Using this function we can read the content of the file on a line by line basis
my_file = open("C:/Documents/Python/test.txt", "r")
print(my_file.readline())
```

--OR—

```
with open("large_file.txt", "r") as f:
   print(f.readline())
```

```
# reading all the lines present inside the text file including the newline characters and
# display as a list
my_file = open("C:/Documents/Python/test.txt", "r")
print(my_file.readlines())
```

```
# Print a specific line from the file, ex: line4
line_number = 4
fo = open("C:/Documents/Python/test.txt", 'r')
currentline = 1
for line in fo:
        if(currentline == line_number):
                print(line)
                break
        currentline = currentline +1
```

Writing into a file:

In order to write data into a file, we must open the file in write mode.

We need to be very careful while writing data into the file as it overwrites the content present inside the file that you are writing, and all the previous data will be erased.

Two methods for writing data into a file:

- write(string)
- writelines(list)

```
my_file = open("C:/Documents/Python/test.txt", "w")
my_file.write("Hello World\n")
my_file.write("Hello Python")
```

```
fruits = ["Apple\n", "Orange\n", "Grapes\n", "Watermelon"]
my_file = open("C:/Documents/Python/test.txt", "w")
my_file.writelines(fruits)
```

Append to File:

To append data into a file we must open the file in 'a+' mode so that we will have access to both the append as well as write modes.

```
my_file = open("C:/Documents/Python/test.txt", "a+")
my_file.write ("Strawberry")
```

```
fruits = ["\nBanana", "\nAvocado", "\nFigs", "\nMango"]
my_file = open("C:/Documents/Python/test.txt", "a+")
my_file.writelines(fruits)
```

The tell() and seek() methods:
Python provides seek() and tell() methods to access data in a random fashion in a file.

The tell() method returns the current file position in a file stream. Syntax:

```
file_object.tell()
```

The seek() method sets the current file position in a file stream and returns the new position. Syntax:

```
file_object.seek(offset [, reference_point])
```

where offset means how many positions you will move; reference_point defines your point of reference:
0: means your reference point is the beginning of the file

1: means your reference point is the current file position
2: means your reference point is the end of the file

```
infile = open("testfile.txt","r")

print("Initially, the position is: ",infile.tell())
infile.seek(10)

print("Position after moving 10 bytes is:",infile.tell())
infile.seek(0,2)

print("Last position of file: ",infile.tell())
```

```
f = open("sample.txt", "r")

# read first line
f.readline()

# get current position of file handle
print(f.tell())
```

```
f = open("sample.txt", "r")

# move to 11 character
f.seek(11)

# read from 11th character
print(f.read())
```

Department of AI_DS
Academic Year: 2026-27 (Odd Sem)
Course: Python for Data Science Lab
Second Year B. Tech Semester: III

PRACTICAL 10

Part B (to be completed by students)

File and Exception Handling

1. All the students are required to perform the given tasks in Jupyter Notebook
2. Create a new notebook for each experiment. The filename should be RollNo_Name_Exp10)
3. In the first cell, the student must write his/her Name, roll no and class in the form of comments
4. Every program should be written in separate cells and in the given sequence
5. After completing the experiment, download the notebook in pdf format. The filename should be RollNo_Name_Exp10.pdf).
   Note: - To download as a PDF, go to File and choose Print Preview. This will open a new tab. Now, press Ctrl + P and opt to save as PDF.
   OR
   Download notebook as HTML and open downloaded html(webpage), press CTRL+P and opt to save as PDF.
6. Upload the pdf on the web portal

In [5]:
```
"""
1. Implement a program to handle arithmetic exception
"""
try:
    num1 = int(input("Enter first number: "))
    num2 = int(input("Enter second number: "))
    result = num1 / num2
    print("Result is", result)
except ZeroDivisionError:
    print("Error: You cannot divide by zero.")
```

```
Enter first number: 0
Enter second number: 0
Error: You cannot divide by zero.
```

In [6]:
```
"""
2. Write a program to handle following exceptions
a. Type Error
b. Value Error
c. Make use of else with exception and check its effect
"""
try:
    data = input("Enter a number: ")
    number = int(data)
    result = data + number
except ValueError:
    print("ValueError: You did not enter a valid integer.")
except TypeError:
    print("TypeError: Cannot add a string to an integer.")
else:
    print("Success: No errors occurred. Result is", result)
```

```
Enter a number: 0
TypeError: Cannot add a string to an integer.
```

In [7]:
```
"""
3. Write a program to write 1-20 numbers in a file & display same to the user.
"""
file = open("numbers.txt", "w")
for i in range(1, 21):
    file.write(str(i) + "\n")
file.close()
print("Numbers written to file. Reading now:")
file = open("numbers.txt", "r")
content = file.read()
print(content)
file.close()
```

```
Numbers written to file. Reading now:
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
```

In [1]:

```
"""
4. Write a program to copy contents (your details) of one file to another file a
"""
file = open("details.txt", "w")
file.write("Name: Aarav Sharma\n")
file.write("Course: B.Tech AI & DS\n")
file.write("College: Mumbai University\n")
file.close()

source = open("details.txt", "r")
content = source.read()
source.close()

destination = open("copy_details.txt", "w")
destination.write(content)
destination.close()

print("Contents of details.txt:")
source = open("details.txt", "r")
print(source.read())
source.close()

print("Contents of copy_details.txt:")
destination = open("copy_details.txt", "r")
print(destination.read())
destination.close()
```

```
Contents of details.txt:
Name: Aarav Sharma
Course: B.Tech AI & DS
College: Mumbai University

Contents of copy_details.txt:
Name: Aarav Sharma
Course: B.Tech AI & DS
College: Mumbai University
```

In [3]:
```
"""
5. Write a python script to continue writing to a file until user enters choice
"""
file = open("user_text.txt", "w")

choice = 'y'
while choice == 'y':
    text = input("Enter text to write: ")
    file.write(text + "\n")
    choice = input("Do you want to continue? (y/n): ")

file.close()

file = open("user_text.txt", "r")
words = file.read().split()
file.close()

count = 0
for word in words:
    if word.endswith('e'):
        count += 1

print("Number of words ending with 'e':", count)
```

```
Number of words ending with 'e': 1
```

In [4]:
```
"""
6. Write a python script to count the number of lines from a text file which is
"""
file = open("sample.txt", "w")
file.write("This is line one\n")
file.write("Another line here\n")
file.write("Testing the script\n")
file.write("Python is fun\n")
file.close()

file = open("sample.txt", "r")
lines = file.readlines()
file.close()

count = 0
for line in lines:
    if not line.startswith('T'):
        count += 1

print("Number of lines not starting with 'T':", count)
```

```
Number of lines not starting with 'T': 2
```

In [5]:
```
"""
7. Write a python script to delete the specified line giving the line number fro
"""
file = open("lines.txt", "w")
file.write("Line 1\n")
file.write("Line 2\n")
file.write("Line 3\n")
file.write("Line 4\n")
file.write("Line 5\n")
file.close()
```

```
line_no = int(input("Enter line number to delete: "))

file = open("lines.txt", "r")
lines = file.readlines()
file.close()

del lines[line_no - 1]

file = open("lines.txt", "w")
file.writelines(lines)
file.close()

print("Line", line_no, "deleted successfully.")

file = open("lines.txt", "r")
print(file.read())
file.close()
```

```
Line 2 deleted successfully.
Line 1
Line 3
Line 4
Line 5
```

In [6]:

```
"""
8. Program to set file offsets with seek( ) and tell( ) functions in python
"""
file = open("offset_demo.txt", "w")
file.write("Hello, this is a file offset demonstration.")
file.close()

file = open("offset_demo.txt", "r")

print("Current position:", file.tell())

file.seek(7)
print("After seek(7), position:", file.tell())
print("Reading from position 7:", file.read(10))

file.seek(0)
print("After seek(0), position:", file.tell())
print("Reading from start:", file.read(5))

file.close()
```

```
Current position: 0
After seek(7), position: 7
Reading from position 7: this is a
After seek(0), position: 0
Reading from start: Hello
```

In [ ]:

Department of Artificial Intelligence (AI) & Data Science

AY: 2025-26
Course: Python for Data Science Laboratory (DJS23SLPC303)
Class: SE/A11/A12/A21/A22 Sem: III Roll No.: SAPID:

Experiment No. 10: File Handling and Exception Handling

Aim: To understand and apply file handling and exception handling in Python

Theory

1. What is an exception? How is it different from an error?
2. List and describe different file opening modes used in Python.
3. What is the use of the close() function in file handling?
4. Differentiate between read(), readline(), and readlines() methods.
5. What is the function of write() and writelines() methods? Give examples.
6. What is the purpose of the seek() and tell() methods?
7. What is the use of the with statement in file handling? How does it simplify file operations?

Conclusions:

Name :- Aaryan N. Rao Rane Roll No :- 5004 Div :- S batch :- A1
branch :- AI&DS SAP ID :- 60018250070

Experiment No 10

22/10

Title :- File Handling and Exception Handling.

Aim :- To understand and apply file handling and Exception Handling in Python.

Theory :-

Q1 What is an Exception? How is it different from an error?

⟹ An Exception is an event that occurs while a program is running and disturbs its normal flow, for Example dividing a number by zero or opening a file that does not exist. When an Exception occurs, Python stops the normal execution and looks for a block of code that can handle it. If no such block is found, the program terminates with a message. An Error is a broader term and a common kind of error is a syntax mistake, which is found before the program starts and cannot be handled during Execution.

Point	Error	Exception.
Meaning	A mistake in the program that stops it from running	An unexpected event that occurs during execution.
When it occurs	Before execution for syntax errors	While the program is running.

Sundaram
FOR EDUCATIONAL USE

Handling	cannot be handled, the code must be corrected	can be handled using try and except.
Example	Missing colon after an if statement	ZeroDivisionError, FileNotFoundError.

code:-

```
try:
    a = 10
    b = 0
    print(a/b)
except ZeroDivisionError:
    print("Exception handled: division by zero is not possible")
print("Program continues after handling")
```

output:-

```
Exception handled: division by zero is not possible
Program continues after handling.
```

Q2 List and describe different file opening modes used in Python

=> The mode is given as the second argument of open() and decides whether the file is read, written or appended, and what happens if the file does not exist.

Sundaram FOR EDUCATIONAL USE

Mode	Description	If file does not exist
r	opens the file for reading only, pointer at the start.	Raises FileNotFoundError
w	Opens the file for writing, old contents erased	creates a new file
a	Opens the file for appending, new data goes at the end.	creates a new file.
r+	Opens the file for both reading and writing.	Raises FileNotFoundError
w+	Opens for writing and reading, old contents erased.	creates a new file.
a+	Opens for appending and reading.	creates a new file.
rb, wb	Binary versions of r and w, used for images and similar files	Same as r and w

Sundaram®
FOR EDUCATIONAL USE

code :-

```
f = open("demo.txt", "w")
f.write("first line\n")
f.close()

f = open("demo.txt", "a")
f.write("second line\n")
f.close()

f = open("demo.txt", "r")
print(f.read())
f.close()
```

output :-

```
First line
second line.
```

Q3 what is the use of the close() function in file handling?

⇒ The close() function closes an open file. closing makes sure that any data still waiting in the Buffer is written to the disk and frees the resources that the operating system had given to the file. A file that is left open may lose data, and it can also stay locked so that other programs cannot use it.

Sundaram®
FOR EDUCATIONAL USE

code :-

```
f = open("notes.txt", "w")
f.write("Data saved to disk")
print("closed before close(): " + str(f.closed))
f.close()
print("closed after close(): " + str(f.closed))
```

output :-

```
closed before close(): False
closed after close(): True
```

Q4 Differentiate between read(), readline(), and readlines() methods.

=>

Method	what it returns	Type of result.
read()	The entire content of the file, or the given number of characters	One string
readline()	Only the next single line from the file	One string.
readlines()	All the lines of the file, one element per line	A list of strings.

Sundaram®
FOR EDUCATIONAL USE

The function repr() is used below only to make the newline characters visible in the output.

Code :-

```
f = open("sample.txt", "w")
f.write("Apple\n Banana\n Cherry\n")
f.close()

f = open("sample.txt", "r")
print(repr(f.read()))
f.close()

f = open("sample.txt", "r")
print(repr(f.readline()))
f.close()

f = open("sample.txt", "r")
print(f.readlines())
f.close()
```

Output :-

```
'Apple\n Banana\n Cherry\n'
'Apple\n'
['Apple\n', 'Banana\n', 'Cherry\n']
```

Sundaram
FOR EDUCATIONAL USE

Q5 What is the function of write() and writelines() methods? Give examples.

=> The write() method writes a single string to the file and returns the number of characters written. The writelines() method writes every string of a list or other sequence to the file in one call. Neither method adds a newline by itself, so the newline character must be included in the text

Code:

```
f = open("out.txt", "w")
n = f.write("Hello World\n")
print("Characters written:" + str(n))

lines = ["Python\n", "File handling\n", "Exceptions\n"]
f.writelines(lines)
f.close()

f = open("out.txt", "r")
print(f.read())
f.close
```

Output:

```
Characters written: 12
Hello World
Python
File handling
Exceptions
```

Sundaram
FOR EDUCATIONAL USE

Q6 What is the purpose of the seek() and tell() methods?

⇒ Every open file has a pointer that marks the position from where the next read or write will happen. The tell() method returns the current position of this pointer as a number of bytes from the start of the file. The seek() method moves the pointer to a chosen position, and seek(0) takes it back to the beginning so that the file can be read again without reopening it.

code:

```
f = open("sample.txt", "r")
print(f.tell())
print(f.read(5))
print(f.tell())
f.seek(0)
print(f.tell())
print(f.readline())
f.close()
```

output:

```
0
apple
5
0
apple
```

Sundaram®
FOR EDUCATIONAL USE

Q7 What is the use of the with statement in file handling? How does it simply simplify file operations?

=> The with statement opens a file and closes it automatically when the block ends, even if an exception occurs inside the block. This removes the need to call close() by hand and avoids the risk of forgetting it. The code also becomes shorter and easier to read, because the work on the file stays together inside one indented block.

code:-

```
with open("sample.txt", "r") as f:
    for line in f:
        print(line.strip())
print("File closed automatically:" + str(f.closed))
```

output:-

```
Apple
Banana
Cherry
File closed automatically: True
```

Sundaram
FOR EDUCATIONAL USE

Conclusions:

File handling in Python lets a program store data permanently by opening, reading, writing and closing files using the correct mode. Exception handling with try, except and finally allows the program to deal with problems such as a missing file without stopping abruptly. The with statement combines both ideas by closing the file safely, which makes the code cleaner and more reliable.

Sundaram®
FOR EDUCATIONAL USE

Object Oriented Programming

In [1]:
```
class stud:
    def __init__(self,a,b):
        self.roll=a
        self.name=b
s1=stud(12,'abc')
print(s1.name)
print(s1.roll)
```

```
abc
12
```

In [4]:
```
class Taxi:
    def __init__(self,driverName,onDuty,cities):
        self.dname=driverName
        self.onduty=onDuty
        self.cities=cities
        self.numPassangers=0
    def changeDriverName(self,newDriverName):
        self.dname=newDriverName
```

In [12]:
```
ourFirstTaxi=Taxi("Aleks",True,["Lund","Malmo"])
print(ourFirstTaxi.cities)
Mytaxi = Taxi("Ali",True,["Lund","Malmo"])
print(Mytaxi.dname)
```

```
['Lund', 'Malmo']
Ali
```

In [13]:
```
Mytaxi.changeDriverName("Akbar")
```

In [14]:
```
print(Mytaxi.dname,Mytaxi.onduty,Mytaxi.cities)
```

```
Akbar True ['Lund', 'Malmo']
```

In [15]:
```
class Student:
    """A class reperesenting a Student"""
    def __init__(self,n,a):
        self.full_name=n
        self.age=a
    def get_age(self):
        return self.age
```

In [16]:
```
f=Student("Python",14)
```

In [17]:
```
f.get_age()
```

Out[17]: 14

In [20]:
```
class Puppy:
    def __init__(self,name,color):
        self.name=name
        self.color=color
    def bark(self):
        print("I am",self.color,self.name)
```

```
p1=Puppy("Bob","Brown")
p1.bark()
p2=Puppy("Olivia","White")
p2.bark()
```

```
I am Brown Bob
I am White Olivia
```

In [22]:
```
class Student(object):
    def __init__(self):
        self.name='Aditya'
        self.age=30
        self.marks=900
    def talk(self):
        print("Name :-",self.name)
        print("Age :-",self.age)
        print("Marks :-",self.marks)

obj=Student()
print(obj.name)
print(obj.age)
print(obj.marks)
obj.talk()
```

```
Aditya
30
900
Name :- Aditya
Age :- 30
Marks :- 900
```

In [23]:
```
class Student:
    def __init__(self,name,age):
        self.fullname=name
        self.age=age
    def display(self):
        print("Student FullName : %s"%self.fullname)
        print("Student Age : %d"%self.age)

s1=Student("Python",14)
s1.display()

s2=Student("Jpython",23)
s2.display()

s3=Student("Leviathan",45)
s3.display()
```

```
Student FullName : Python
Student Age : 14
Student FullName : Jpython
Student Age : 23
Student FullName : Leviathan
Student Age : 45
```

Inheritance

In [24]:
```
class Base:
    def __init__(self):
```

```
        self.name = "base class constructor"
    def display(self):
        print(self.name)
class Derived(Base):
    pass
d = Derived()
d.display()
```

```
base class constructor
```

Method Overriding

In [26]:
```
class Animal:
    def sound(self):
        print("Animal makes a sound")
class dog(Animal):
    def sound(self):
        print("Dog Barks")
a=Animal()
a.sound()
d=dog()
d.sound()
```

```
Animal makes a sound
Dog Barks
```

In [28]:
```
class Rectangle():
    def __init__(self,length,breadth):
        self.length=length
        self.breadth=breadth
    def getArea(self):
        print(self.length*self.breadth,"is area of the rectangle")
class Square(Rectangle):
    def __init__(self,side):
        self.side=side
    def getArea(self):
        print(self.side*self.side,"is area of Square")
```

In [29]:
```
s=Square(4)
r=Rectangle(4,5)
s.getArea()
r.getArea()
```

```
16 is area of Square
20 is area of the rectangle
```

Overloading

In [33]:
```
class Person:
    def Hello(self,name=None):
        if name is not None:
            print("Hello",name)
        else:
            print("Hello")
```

In [34]:
```
obj=Person() #Create instance
obj.Hello() #Call the Method
obj.Hello("Edureka")
```

```
Hello
Hello Edureka
```

In [35]:
```
class Calculator:
    def multiply(self,a,b=1,c=1):
        return a*b*c
calc=Calculator()
print(calc.multiply(5))
print(calc.multiply(5,4))
print(calc.multiply(5,4,3))
```

```
5
20
60
```

Operator Overloading

In [36]:
```
class Test:
    def __init__(self,a):
        self.a=a
    def __add__(self, obj):
        return self.a + obj.a
t1=Test(50)
t2=Test(100)
print(t1+t2)
```

```
150
```

Constructor Overloading

In [37]:
```
class Base:
    def __init__(self):
        self.name = 'base class constructor'
    def display(self):
        print(self.name)
class Derived(Base):
    def __init__(self):
        pass
d = Derived()
d.display()
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[37], line 10
      6 class Derived(Base):
      7     def __init__(self):
      8         pass
      9 d = Derived()
---> 10 d.display()

Cell In[37], line 5, in Base.display(self)
      4     def display(self):
----> 5         print(self.name)

AttributeError: 'Derived' object has no attribute 'name'
```

In [38]:
```
class Base:
    def __init__(self):
        self.name = 'base class constructor'
```

```
    def display(self):
        print(self.name)
class Derived(Base):
    def __init__(self):
        self.name="Derived class Constructor"
d = Derived()
d.display()
```

```
Derived class Constructor
```

Super Method

In [41]:
```
class Base:
    def __init__(self,nm):
        self.name = nm
    def display(self):
        print(self.name)
class Derived(Base):
    def __init__(self):
        super().__init__("Super is Accessible")
d = Derived()
d.display()
```

```
Super is Accessible
```

In [ ]:
```

```

PRACTICAL 11

Part A (To be referred by students)

Object Oriented Programming, Class Object, self, __init__, instance variables and methods

SAVE THE FILE AND UPLOAD AS (RollNo_Name_Exp11)

Aim: To implement Object Oriented Programming Concept in Python

Problem Statement:

Apply core and object oriented Python concepts to build following applications

1. Create a Vehicle class without any variables and methods.
2. Create a class with instance attributes

   Write a Python program to to create a Vehicle class with max_speed and mileage instance attributes

3.Create a child class Bus that will inherit all of the variables and methods of the Vehicle class

```
class Vehicle:
   def __init__(self, name, max_speed, mileage):
      self.name = name
      self.max_speed = max_speed
      self.mileage = mileage
```

4.Create a Bus object that will inherit all of the variables and methods of the parent Vehicle class and display it.

Expected Output:

Vehicle Name: School Volvo Speed: 180 Mileage: 12

5.Create a Bus class that inherits from the Vehicle class. Give the capacity argument of Bus.seating_capacity() a default value of 50.

Use the following code for your parent Vehicle class.

```
class Vehicle:
    def __init__(self, name, max_speed, mileage):
        self.name = name
        self.max_speed = max_speed
        self. mileage = mileage

    def seating_capacity(self, capacity):
        return f"The seating capacity of a {self.name} is {capacity} passengers"
```

Expected Output:

```
The seating capacity of a bus is 50 passengers
```

6. Class Inheritance

Given:

Create a **Bus** child class that inherits from the Vehicle class. The default fare charge of any vehicle is **seating capacity * 100**. If Vehicle is **Bus** instance, we need to add an extra 10% on full fare as a maintenance charge. So total fare for bus instance will become the **final amount = total fare + 10% of the total fare.**

```
        self.name = name
        self.mileage = mileage
        self.capacity = capacity

    def fare(self):
        return self.capacity * 100

class Bus(Vehicle):
    pass

School_bus = Bus("School Volvo", 12, 50)
print("Total Bus fare is:", School_bus.fare())
```

Expected Output:

```
Total Bus fare is: 5500.0
```

7.Check type of an object

Write a program to determine which class a given Bus object belongs to.

8.Determine if School_bus is also an instance of the Vehicle class

9.Determine if School_bus is Sub class of Vehicle Class

10.Create a child class for Bus Class named Mini bus inheriting Busclass and price attribute and Print Price Method (Multilevel Inheritance)

Use the following code for your parent Vehicle class. We need to access the parent class from inside a method of a child class.

```
class Vehicle:
    def __init__(self, name, mileage, capacity):
        self.name = name
        self.mileage = mileage
        self.capacity = capacity

    def fare(self):
        return self.capacity * 100
```

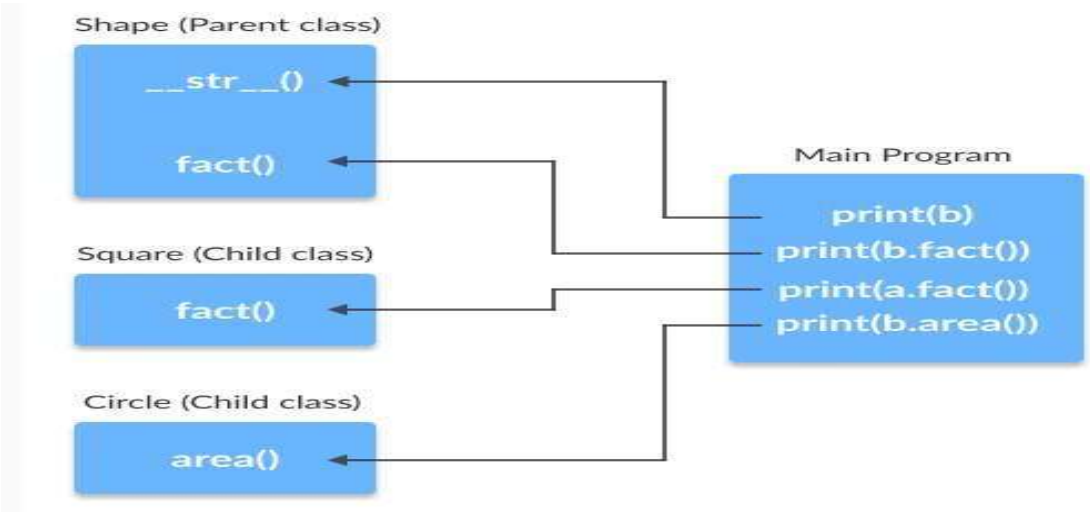
```
class Bus(Vehicle):
    pass

School_bus = Bus("School Volvo", 12, 50)
print("Total Bus fare is:", School_bus.fare())
```

11. Create a car class inheriting Vehicle class and add type and Print attribute and display Type method method for the same

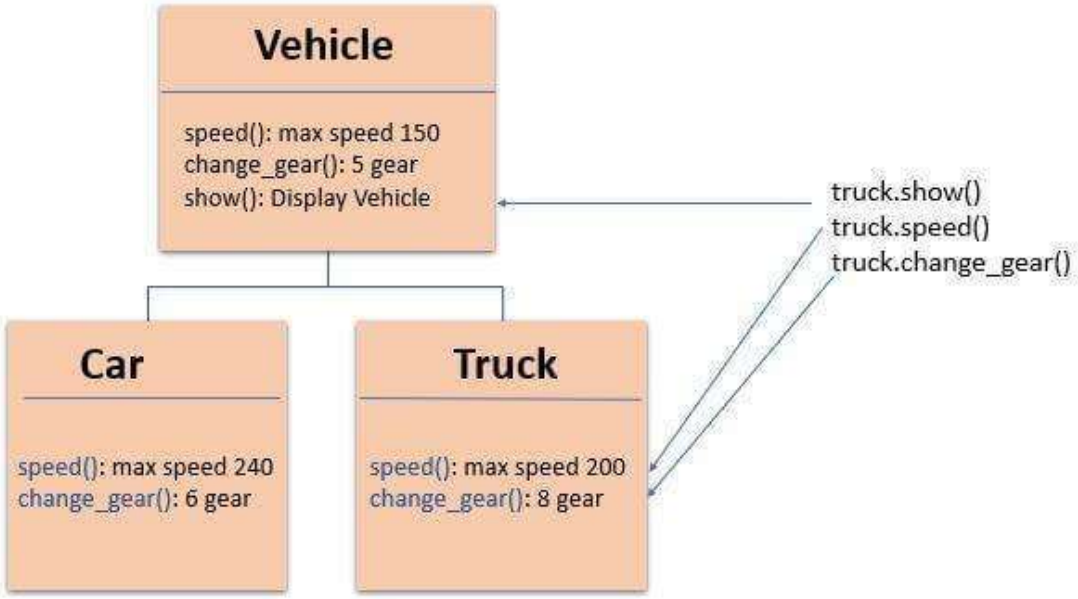
Use the following code for your parent Vehicle class. We need to access the parent class from inside a method of a child class.

```
class Vehicle:
    def __init__(self, name, mileage, capacity):
        self.name = name
        self.mileage = mileage
        self.capacity = capacity

    def fare(self):
        return self.capacity * 100

class Bus(Vehicle):
    pass

School_bus = Bus("School Volvo", 12, 50)
print("Total Bus fare is:", School_bus.fare())
```

12.Write a program in python to Implement Method Overriding considering the following



13.Write a program in python to Implement Method Overriding considering the following classes, methods and Variables



Method overridden in Car and Truck class

14.Write a program in python to demonstrate how method overloading is achieved. Create a function to perform addition operation. If 2 no's are given it should perform addition of 2 no's otherwise it should add 3 nos.

15. Write a program in python to perform method overloading. Create a function to calculate area of given shape. If 1 side is given calculate area of square, if Length and breadth is given calculate the area of Rectangle.

Department of AI_DS
Academic Year: 2026-27 (Odd Sem)
Course: Python for Data Science Lab
Second Year B. Tech Semester: III

16.Write a program to Implement Abstract class using Following problem statement
The Employee class represents an employee, either full-time or hourly. the Employee class should be an abstract class because there're only full-time employees and hourly employees, no general employees exist.
The Employee class should have a property that returns the full name of an employee. In addition, it should have a method that calculates salary. The method for calculating salary should be an abstract method.
The Full_time_Employee class inherits from the Employee class. It'll provide the implementation for the get_salary () method.
Since full-time employees get fixed salaries, you can initialize the salary in the constructor of the class.
The HourlyEmployee also inherits from the Employee class.
However, hourly employees get paid by working hours and their rates. Therefore, you can initialize this information in the constructor of the class.
To calculate the salary for the hourly employees, you multiply the working hours and rates.

17. Write a Python Program to overload + operator to add Point (X1, Y1) and Point (X2, Y2)

18.Write a Python Program to perform division of Two Numbers, take both the nos from users from user, use TRY, EXCEPT and FINALLY block to raise an Exception when Diving Number by Zero.

Topic covered: Object Oriented Programming, Class, Object, Instance Method, Instance Variables etc.

Learning Objective: Learner would be able to

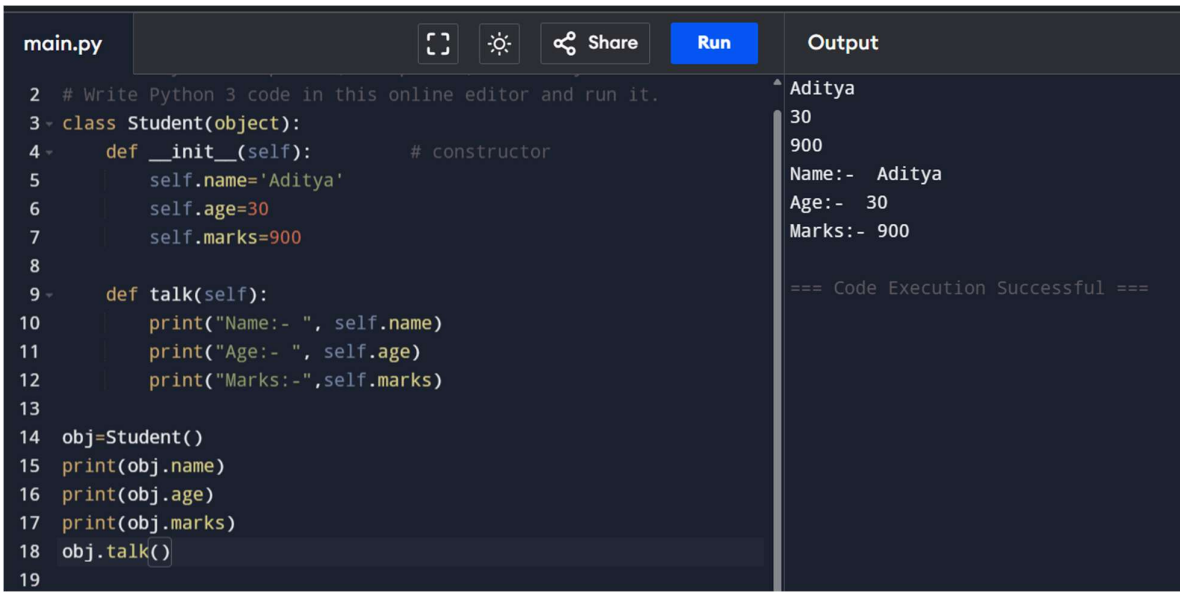
1. Understand the OOP concepts in Python
2. Apply OOP to Solve real world applications

Department of AI_DS
Academic Year: 2026-27 (Odd Sem)
Course: Python for Data Science Lab
Second Year B. Tech Semester: III

Theory:

Program to show use of class and Objects…

```
class Student(object):
    def __init__(self):        # constructor
        self.name='Aditya'
        self.age=30
        self.marks=900

    def talk(self):
        print("Name:- ", self.name)
        print("Age:- ", self.age)
        print("Marks:-",self.marks)

obj=Student()
print(obj.name)
print(obj.age)
print(obj.marks)
obj.talk()
```



self Variable:-

- self is a default variable that contains the memory address of the instance of current class.
- so, we can use 'self' to refer to all instance variable and methods.
- Self is used as first parameter in constructor as

- def __init__(self): --> in this case self is used to indicate instance variable of class

- self can be used as first parameter in the instance methods as:
  - def talk(self):
  - here, talk is instance method as it act on instance variable.

Constructor:-

- Used to initialize instance variables.
- In constructor we can create instance variable and initialize them with some value.
- The first parameter of constructor is self-variable that contains the memory address of the instance.
- Example:-

```
def __init__(self):        # constructor
    self.name='Aditya'
```

- Here, constructor have only one parameter and name is instance variable.
- A constructor is called when we create an instance as:
- S1=Student()
- In, Student ( ), empty parenthesis represents we are not passing any values to constructor.
- Self is passed implicitly…

Constructor with Parameters:-

```
class Student(object):

    def __init__(self,n='',m=0,a=0):    # constructor with default arg.
        self.name=n
        self.marks=m
        self.age=a

    def talk(self):
        print("Name:- ", self.name)
        print("Age:- ", self.age)
        print("Marks:-",self.marks)

obj1=Student()
obj1.talk()
obj2=Student("Yash",600,21)
obj2.talk()
```

main.py | Share | Run | Output

```
# Online Python compiler (interpreter) to run Python online.
# Write Python 3 code in this online editor and run it.
class Student():

    def __init__(self,n='',m=0,a=0):    # constructor with
        default arg.
        self.name=n
        self.marks=m
        self.age=a

    def talk(self):
        print("Name:- ", self.name)
        print("Age:- ", self.age)
        print("Marks:-",self.marks)

obj1=Student()
obj1.talk()
obj2=Student("Yash",600,21)
obj2.talk()
```

```
Name:-
Age:-  0
Marks:- 0
Name:-  Yash
Age:-  21
Marks:- 600

=== Code Execution Successful ===
```

- In above case we passed, three arguments to student instance.
- Main duty of constructor is to initialize or store the beginning values into the instance variable.
- Constructor is called only once at the time of creating object.
- If 4-objects are created then constructor is called 4 –times.

Department of AI_DS
Academic Year: 2026-27 (Odd Sem)
Course: Python for Data Science Lab
Second Year B. Tech Semester: III

Types of Variables:-

- Instance Variables
  - Its separate copy is created in every instance.
- Class/Static Variables
  - Single copy is available to all instances.

```
class Student(object):
    cnt=0   #class variable
    def __init__(self,n='',m=0,a=0):
        self.name=n
        self.marks=m
        self.age=a

    def talk(self):
        print("Name:- ", self.name)
        print("Age:- ", self.age)
        print("Marks:-",self.marks)

    @classmethod     # built in decorator statement
    def count(cls):
        cls.cnt+=1

obj1=Student()
obj1.talk()
obj2=Student("Yash",600,21)
obj2.talk()
obj1.count()
obj2.count()
print("No of Objects", obj1.cnt)
```

```
class Student():
    cnt=0   #class variable
    def __init__(self,n='',m=0,a=0):
        self.name=n
        self.marks=m
        self.age=a
    def talk(self):
        print("Name:- ", self.name)
        print("Age:- ", self.age)
        print("Marks:-",self.marks)
    @classmethod     # built in decorator statement
    def count(cls):
        cls.cnt+=1
obj1=Student()
obj1.talk()
obj2=Student("Yash",600,21)
obj2.talk()
obj1.count()
obj2.count()
```

```
Name:-
Age:-  0
Marks:- 0
Name:-  Yash
Age:-  21
Marks:- 600
No of Objects 2

 No of Objects 2

=== Code Execution Successful ===
```

Class Method:-

- It use first parameter by default 'cls' with which we can access the class variable.
- Example:-

```
@classmethod     # built in decorator statement
    def count(cls):
    cls.cnt+=1
```

- We can also have other parameters in the class method in addition to the 'cls' parameter.
- We can access the class variables from outside the class using class name. classname.variable…

```
class Student(object):
   cnt=0   #class varaible
   def __init__(self,n='',m=0,a=0):
      Student.cnt+=1
      print("Student:-",Student.cnt)
      self.name=n
      self.marks=m
      self.age=a

      def talk(self):
      print("Name:- ", self.name)
      print("Age:- ", self.age)
      print("Marks:-",self.marks)
```

```
    @classmethod     # built in decorator statement
    def count(cls):
        print("No of Objects", cls.cnt)

obj1=Student()
obj1.talk()
obj2=Student("Yash",600,21)
obj2.talk()
obj2.count()
print("No of Objects", obj1.cnt)
```

Types of Methods:-

- Instance Methods
  - Accessor Methods
    - Access or read the data of variable
    - They do not modify the values/data of variable
    - Are also called getter method
    - Written in the form getXXX( )
  - Mutator Methods
    - Not only read but also modify them.
    - Also called setter method
    - Written in the form setXXX( )
- Class Methods
  - These methods act on class level.
  - It can act on class or static variable
  - These methods are written using @classmethod decorator
  - By default first parameter is cls.
  - These methods are generally called using classname.methodname()

- Static Methods
  - We required static methods when some processing is related to the class but does not need the class or its instances to perform any work.
  - Example, counting number of instances of class.
  - Static methods are written with @staticmethod decorator
  - These methods are generally called using classname.methodname()
  - Example:-

```
class Test:
    c=0 #this is class/static variable

    def __init__(self):
        Test.c+=1
    @staticmethod
    def objCount():
        print('No of objects are ',Test.c)

t1=Test()
t2=Test()
t3=Test()
t4=Test()
Test.objCount()
```

Department of AI_DS
Academic Year: 2026-27 (Odd Sem)
Course: Python for Data Science Lab
Second Year B. Tech Semester: III

```
class Test:
    c=0 #this is class/static variable

    def __init__(self):
        Test.c+=1
    @staticmethod
    def objCount():
        print('No of objects are ',Test.c)

t1=Test()
t2=Test()
t3=Test()
t4=Test()
Test.objCount()
```

```
No of objects are  4

=== Code Execution Successful ===
```

PRACTICAL 11

Part B (to be completed by students)

Object Oriented Programming, Class Object, self, __init__, instance variables and methods

1. All the students are required to perform the given tasks in Jupyter Notebook
2. Create a new notebook for each experiment. The filename should be RollNo_Name_Exp11)
3. In the first cell, the student must write his/her Name, roll no and class in the form of comments
4. Every program should be written in separate cells and in the given sequence
5. After completing the experiment, download the notebook in pdf format. The filename should be RollNo_Name_Exp11.pdf).
   Note: - To download as a PDF, go to File and choose Print Preview. This will open a new tab. Now, press Ctrl + P and opt to save as PDF.
   OR
   Download notebook as HTML and open downloaded html(webpage), press CTRL+P and opt to save as PDF.
6. Upload the pdf on the web portal

Department of AI_DS
Academic Year: 2026-27 (Odd Sem)
Course: Python for Data Science Lab
Second Year B. Tech Semester: III

PRACTICAL 12

Part A (To be referred by students)

Class variables & methods, accessing class attribute, inheritance, overriding

SAVE THE FILE AND UPLOAD AS (RollNo_Name_Exp12)

Aim: To apply core and object-oriented programming concepts in Python such as class variables, methods, inheritance, method overriding, to build modular and reusable programs.

Problem Statement:

Apply core and object oriented Python concepts to build following applications

1. Write a Python class named Student with two attributes student_id, student_name as a class variables. Add a new attribute student_class as class variable. Create a class function/method to display the entire attributes and their values in Student class with exception handling.
2. Create Account class for a bank with variables number and balance. Create constructor and two user defined function deposit and withdraw with appropriate arguments. Create class customer inherited from Account and with variables name, address, dob. Define constructor. Display details of customer along with balance amount after deposit and withdraw operation.
3. Create a class Shape with a method area ( ), which accept one argument to find area of square. Two class Circle and Rectangle are derived from Shape in which area method is also implemented (method overriding). The main program should create objects of Circle & Rectangle, accept suitable input and display area of Circle, Rectangle and Square.
4. Create a Python class Employee with attributes: name, age and salary. Create a display() method that displays the name, age and salary of an object created via the Employee class. Create a child class Manager which inherits from the Employee class and which also has a Department attribute. Create a method displayManager() that displays the name, age, salary and department of an object created via the Manager class. Create a Manager object via an instantiation on the Manager class and then test the displayManager() method.

Topic covered: Class variables & methods, accessing class attribute, inheritance, overriding

Department of AI_DS
Academic Year: 2026-27 (Odd Sem)
Course: Python for Data Science Lab
Second Year B. Tech Semester: III

Learning Objective: Learner would be able to

1. Work with class variables, methods & accessing class attribute
2. Implement Inheritance
3. Overriding

Theory:

Inheritance:

- Deriving a new class from existing class, such that derived class inherits members from super class.
- Syntax is
  Class SubClass(BaseClass):
- In inheritance we always creates object as sub class as copy of super class object is available to sub class object.
- If we created object of super class only super class members are accessible not sub class member.

Constructor in Inheritance:-

- Constructor in super class is accessible to sub class..
- Example:_

```
class Base:
   def __init__(self):
      self.name="Base class Constructor "
   def display(self):
      print(self.name)

class Derived(Base):
   pass
d=Derived()
d.display()
```

Overriding Super class Constructors and Methods:-

- When we write a constructor in sub class, the super class constructor is not available to sub class.
- In this case only sub class constructor is accessible from the sub class object.
- This means sub class constructor is replacing super class constructor
- Example:-

```
class Base:
   def __init__(self):
      self.name="Base class Constructor "
   def display(self):
      print(self.name)

class Derived(Base):
```

```
    def __init__(self):
        pass

d=Derived()
d.display()
```

- Error occurred :-
AttributeError: 'Derived' object has no attribute 'name'

- The above concept is called constructor overriding.
- Similarly if we write method in super class and sub class, then sub class method overrides methods in super class… this is called method overriding…
- Above program can be executed again with some change…
- Program to show used of constructor overriding :

Example:-

```
class Base:
    def __init__(self):
        self.name="Base class Constrctor "
    def display(self):
        print(self.name)

class Derived(Base):
    def __init__(self):
        self.name="Derived class Constrctor "

d=Derived()
d.display()
```

Output:-
Derived class Constrctor

The super( ) method…

- It is built in method which is useful to call the super class constructor or methods from the sub class.
- Super class constructor can be accessed from sub class constructor using super()
- Super can be used as:
    - Super().__init__() # to invoke constructor
    - Super().__init__(args_list) # to invoke constructor
    - Super().method() # to invoke method of super class
- When there is constructor with parameter in super class, we have to create another constructor with parameters in sub class and call the super class constructor using

super() from sub class constructor. [this is not the rule… its good practice of programming]

- Accessing Super class constructor from sub class:-

```
class Base:
    def __init__(self,nm):
        self.name=nm
    def display(self):
        print(self.name)

class Derived(Base):
    def __init__(self):
        super().__init__('Super is accessible')

d=Derived()
d.display()
```

- We can also access method of super class using super()
- Python program to access base class constructor and method in sub class using super()

```
class Square:
    def __init__(self,side):
        self.side=side
    def area(self):
        print('Area of sq is ',self.side*self.side)
class Rectangle(Square):
    def __init__(self,x,y):
        self.x=x
        self.y=y
        super().__init__(x)
    def area(self):
        super().area()
        print('Area of Rectangle  is ',self.x*self.y)
r=Rectangle(10,20)
r.area()
```

Method Overloading:-

- It is not available in Python.

- It can be achieved using method with multiple parameters with default value of arguments(default arguments)

Method Overriding:-

- Same method in super and sub class, sub class overrides super class metho

PRACTICAL 12

Part B (to be completed by students)

Class variables & methods, accessing class attribute, inheritance, overriding

1. All the students are required to perform the given tasks in Jupyter Notebook
2. Create a new notebook for each experiment. The filename should be RollNo_Name_Exp10)
3. In the first cell, the student must write his/her Name, roll no and class in the form of comments
4. Every program should be written in separate cells and in the given sequence
5. After completing the experiment, download the notebook in pdf format. The filename should be RollNo_Name_Exp10.pdf).
   Note: - To download as a PDF, go to File and choose Print Preview. This will open a new tab. Now, press Ctrl + P and opt to save as PDF.
   OR
   Download notebook as HTML and open downloaded html(webpage), press CTRL+P and opt to save as PDF.
6. Upload the pdf on the web portal

Practical 11: Object Oriented Programming

Aim: To implement Object Oriented Programming concepts in Python (class, object, self, `__init__`, instance variables, methods, inheritance, overriding and overloading).

Q1. Create a Vehicle class without any variables and methods

In [2]:
```
class Vehicle:
    pass          # empty class body

v = Vehicle()     # create an object of the class
print("Vehicle object created successfully.")
print("Type of object:", type(v))
```

```
Vehicle object created successfully.
Type of object: <class '__main__.Vehicle'>
```

Q2. Create a Vehicle class with max_speed and mileage instance attributes

In [3]:
```
class Vehicle:
    def __init__(self, max_speed, mileage):
        self.max_speed = max_speed      # instance variable
        self.mileage = mileage          # instance variable

    def display(self):
        print(f"Max speed : {self.max_speed} km/h")
        print(f"Mileage   : {self.mileage} km/l")

car = Vehicle(180, 18)
bike = Vehicle(120, 45)

print("Car details")
car.display()
print()
print("Bike details")
bike.display()
```

```
Car details
Max speed : 180 km/h
Mileage   : 18 km/l

Bike details
Max speed : 120 km/h
Mileage   : 45 km/l
```

Q3. Create a child class Bus that will inherit all of the variables and methods of the Vehicle class

In [4]:
```
class Vehicle:
    def __init__(self, name, max_speed, mileage):
        self.name = name
        self.max_speed = max_speed
        self.mileage = mileage

class Bus(Vehicle):
    pass          # inherits everything from Vehicle

print("Class Bus created. Parent class of Bus:", Bus.__bases__[0].__name__)
```

```
Class Bus created. Parent class of Bus: Vehicle
```

Q4. Create a Bus object that will inherit all of the variables and methods of the parent Vehicle class and display it

Expected Output: `Vehicle Name: School Volvo Speed: 180 Mileage: 12`

In [5]:
```
class Vehicle:
    def __init__(self, name, max_speed, mileage):
        self.name = name
        self.max_speed = max_speed
        self.mileage = mileage

class Bus(Vehicle):
    pass

School_bus = Bus("School Volvo", 180, 12)
print("Vehicle Name:", School_bus.name, "Speed:", School_bus.max_speed, "Mileage
```

```
Vehicle Name: School Volvo Speed: 180 Mileage: 12
```

Q5. Create a Bus class that inherits from the Vehicle class. Give the capacity argument of Bus.seating_capacity() a default value of 50

Expected Output: `The seating capacity of a bus is 50 passengers`

In [6]:
```
class Vehicle:
    def __init__(self, name, max_speed, mileage):
        self.name = name
        self.max_speed = max_speed
        self.mileage = mileage

    def seating_capacity(self, capacity):
        return f"The seating capacity of a {self.name} is {capacity} passengers"

class Bus(Vehicle):
    # override the parent method and give capacity a default value of 50
    def seating_capacity(self, capacity=50):
        return super().seating_capacity(capacity)

School_bus = Bus("bus", 180, 12)
print(School_bus.seating_capacity())
```

```
The seating capacity of a bus is 50 passengers
```

Q6. Class Inheritance

The default fare charge of any vehicle is **seating capacity * 100**. If the Vehicle is a **Bus** instance, add an extra 10% of the total fare as a maintenance charge, so the final amount = total fare + 10% of the total fare.

Expected Output: `Total Bus fare is: 5500.0`

In [7]:
```
class Vehicle:
    def __init__(self, name, mileage, capacity):
        self.name = name
        self.mileage = mileage
        self.capacity = capacity

    def fare(self):
        return self.capacity * 100

class Bus(Vehicle):
    def fare(self):
        amount = super().fare()
        amount += amount * 10 / 100      # extra 10% maintenance charge
        return amount

School_bus = Bus("School Volvo", 12, 50)
print("Total Bus fare is:", School_bus.fare())
```

```
Total Bus fare is: 5500.0
```

Q7. Check type of an object

Write a program to determine which class a given Bus object belongs to.

In [8]:
```
print(type(School_bus))
print("Class name:", type(School_bus).__name__)
```

```
<class '__main__.Bus'>
Class name: Bus
```

Q8. Determine if School_bus is also an instance of the Vehicle class

In [9]:
```
print(isinstance(School_bus, Vehicle))
```

```
True
```

Q9. Determine if School_bus is a sub class of the Vehicle class

In [10]:
```
print(issubclass(type(School_bus), Vehicle))
print(issubclass(Bus, Vehicle))
```

```
True
True
```

Q10. Create a child class for Bus class named Minibus inheriting the Bus class with a price attribute and a Print Price method (Multilevel Inheritance)

In [11]:
```python
class Vehicle:
    def __init__(self, name, mileage, capacity):
        self.name = name
        self.mileage = mileage
        self.capacity = capacity

    def fare(self):
        return self.capacity * 100

class Bus(Vehicle):
    pass

class Minibus(Bus):
    def __init__(self, name, mileage, capacity, price):
        super().__init__(name, mileage, capacity)   # access the parent class fr
        self.price = price

    def print_price(self):
        print("Price of", self.name, "is:", self.price)

mini = Minibus("Mini Volvo", 15, 20, 2500000)
mini.print_price()
print("Total Bus fare is:", mini.fare())
print("MRO:", [c.__name__ for c in Minibus.__mro__])
```

```
Price of Mini Volvo is: 2500000
Total Bus fare is: 2000
MRO: ['Minibus', 'Bus', 'Vehicle', 'object']
```

Q11. Create a Car class inheriting the Vehicle class, add a type attribute and print it using a display type method

In [12]:
```python
class Vehicle:
    def __init__(self, name, mileage, capacity):
        self.name = name
        self.mileage = mileage
        self.capacity = capacity

    def fare(self):
        return self.capacity * 100

class Car(Vehicle):
    def __init__(self, name, mileage, capacity, type):
        super().__init__(name, mileage, capacity)   # access the parent class fr
        self.type = type
```

```
    def display_type(self):
        print("Type of", self.name, "is:", self.type)

c1 = Car("Honda City", 18, 5, "Sedan")
print("Type attribute:", c1.type)
c1.display_type()
print("Total fare is:", c1.fare())
```

```
Type attribute: Sedan
Type of Honda City is: Sedan
Total fare is: 500
```

Q12. Method Overriding (Shape, Square and Circle)

`print(b)` calls `Shape.__str__()`, `b.fact()` calls `Shape.fact()`, `a.fact()` calls `Square.fact()` (overridden) and `b.area()` calls `Circle.area()`.

In [13]:
```
class Shape:
    def __str__(self):
        return "I am a two-dimensional shape."

    def fact(self):
        return "Shapes are two-dimensional."

class Square(Shape):
    def __init__(self, length):
        self.length = length

    def fact(self):                        # overrides Shape.fact
        return "Squares have each angle equal to 90 degrees."

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return 3.14159 * self.radius ** 2

a = Square(4)
b = Circle(7)

print(b)
print(b.fact())
print(a.fact())
print(b.area())
```

```
I am a two-dimensional shape.
Shapes are two-dimensional.
Squares have each angle equal to 90 degrees.
153.93791
```

Q13. Method Overriding (Vehicle, Car and Truck)

Vehicle: speed() max speed 150, change_gear() 5 gear, show() displays Vehicle. Car overrides speed (240) and change_gear (6). Truck overrides speed (200) and change_gear

(8). `truck.show()` uses the parent method.

In [14]:
```
class Vehicle:
    def speed(self):
        print("Max speed 150")

    def change_gear(self):
        print("5 gear")

    def show(self):
        print("Display Vehicle")

class Car(Vehicle):
    def speed(self):
        print("Max speed 240")

    def change_gear(self):
        print("6 gear")

class Truck(Vehicle):
    def speed(self):
        print("Max speed 200")

    def change_gear(self):
        print("8 gear")

print("Truck")
truck = Truck()
truck.show()
truck.speed()
truck.change_gear()

print("\nCar")
car = Car()
car.show()
car.speed()
car.change_gear()

print("\nVehicle")
v = Vehicle()
v.show()
v.speed()
v.change_gear()
```

```
Truck
Display Vehicle
Max speed 200
8 gear

Car
Display Vehicle
Max speed 240
6 gear

Vehicle
Display Vehicle
Max speed 150
5 gear
```

Student class with class variables and a class method

Create class with student_id and student_name as class variables

In [1]:
```
class Student:
    student_id = 101                # class variable
    student_name = "Aarav Sharma"   # class variable

    @classmethod
    def display(cls):
        try:
            print(f"{'student_id':<15}{cls.student_id}")
            print(f"{'student_name':<15}{cls.student_name}")
            print(f"{'student_class':<15}{cls.student_class}")
        except AttributeError as e:
            print(f"Attribute error caught: {e}")

# student_class is not added yet, so the exception handler runs
Student.display()
```

```
student_id     101
student_name   Aarav Sharma
Attribute error caught: type object 'Student' has no attribute 'student_class'
```

Add a new class variable student_class and display all attributes

In [2]:
```
Student.student_class = "SE A1"    # new class variable added from outside the c

Student.display()
```

```
student_id     101
student_name   Aarav Sharma
student_class  SE A1
```

Account and Customer classes (inheritance)

In [3]:
```
class Account:
    def __init__(self, number, balance):
        self.number = number
        self.balance = balance

    def deposit(self, amount):
        if amount <= 0:
            print("Deposit amount must be positive.")
        else:
            self.balance += amount
            print(f"Deposited {amount}. Balance is now {self.balance}.")

    def withdraw(self, amount):
        if amount > self.balance:
            print("Insufficient balance, withdrawal failed.")
```

```
        else:
            self.balance -= amount
            print(f"Withdrew {amount}. Balance is now {self.balance}.")


class Customer(Account):
    def __init__(self, number, balance, name, address, dob):
        super().__init__(number, balance)    # call the Account constructor
        self.name = name
        self.address = address
        self.dob = dob

    def display(self):
        print("Customer Details")
        print(f"{'Name':<17}{self.name}")
        print(f"{'Address':<17}{self.address}")
        print(f"{'DOB':<17}{self.dob}")
        print(f"{'Account Number':<17}{self.number}")
        print(f"{'Balance':<17}{self.balance}")

print("Classes Account and Customer defined successfully.")
```

```
Classes Account and Customer defined successfully.
```

Perform deposit and withdraw, then display customer details

In [4]:
```
c1 = Customer(1001, 5000, "Priya Patel", "Andheri, Mumbai", "12/05/2005")

c1.deposit(2000)
c1.withdraw(1500)
print()
c1.display()
```

```
Deposited 2000. Balance is now 7000.
Withdrew 1500. Balance is now 5500.

Customer Details
Name              Priya Patel
Address           Andheri, Mumbai
DOB               12/05/2005
Account Number    1001
Balance           5500
```

Shape, Circle and Rectangle classes (method overriding)

In [5]:
```
class Shape:
    def area(self, side):                # area of a square
        return side * side


class Circle(Shape):
    def area(self, radius):              # overrides Shape.area
        return 3.14 * radius * radius


class Rectangle(Shape):
    def area(self, length, breadth):     # overrides Shape.area
```

```
        return length * breadth

print("Classes Shape, Circle and Rectangle defined successfully.")
```

```
Classes Shape, Circle and Rectangle defined successfully.
```

Create objects and display the areas

In [6]:
```
side = 4
radius = 7
length = 10
breadth = 5

s = Shape()
c = Circle()
r = Rectangle()

print(f"Area of Square    : {s.area(side):.2f}")
print(f"Area of Circle    : {c.area(radius):.2f}")
print(f"Area of Rectangle : {r.area(length, breadth):.2f}")
```

```
Area of Square    : 16.00
Area of Circle    : 153.86
Area of Rectangle : 50.00
```

Employee and Manager classes (inheritance)

In [7]:
```
class Employee:
    def __init__(self, name, age, salary):
        self.name = name
        self.age = age
        self.salary = salary

    def display(self):
        print(f"Name   : {self.name}")
        print(f"Age    : {self.age}")
        print(f"Salary : {self.salary}")


class Manager(Employee):
    def __init__(self, name, age, salary, department):
        super().__init__(name, age, salary)   # call the Employee constructor
        self.department = department

    def displayManager(self):
        self.display()                        # reuse the parent method
        print(f"Department : {self.department}")

print("Classes Employee and Manager defined successfully.")
```

```
Classes Employee and Manager defined successfully.
```

Create a Manager object and test displayManager()

In [8]:
```
m1 = Manager("Rohan Mehta", 35, 85000, "Data Science")
m1.displayManager()
```

```
Name   : Rohan Mehta
Age    : 35
Salary : 85000
Department : Data Science
```

Department of Artificial Intelligence (AI) & Data Science

AY: 2026-27
Course: Python for Data Science Laboratory (DJS23SLPC303)
Class: SE/A11/A12/A21/A22 Sem: III Roll No.: SAPID:

Experiment No. 11 & 12: Object-Oriented Programming

Aim: To implement the Object Oriented Programming Concept in Python.

Write Python programs to implement Inheritance & Polymorphism.

Theory:

1. What is Object-Oriented Programming (OOP)? How is it different from procedural programming?

2. Define the following terms with examples:
   a) Class
   b) Object
   c) Constructor
   d) Destructor

3. What is the purpose of the __init__() method in Python classes?

4. What do you mean by inheritance? List and explain any two types of inheritance supported in Python.

5. What is method overriding? How does it differ from method overloading?

6. Explain the use of the super() function with an example.

7. What is polymorphism? Provide an example demonstrating polymorphism in Python.

8. What is the role of the self parameter in Python class methods?

9. Write a short note on the advantages of using OOP principles in real-world applications.

Conclusions:

Name :- Aaryan V. Rodrane Roll No :- 5004 Div :- 5
Batch :- A1 Branch :- AI&DS SAP ID :- 60018250070

Experiment No 11 & 12

Title :- Object - oriented Programming and inheritance, overriding and polymorphism

Aim :- To Implement the Object Oriented Programming concept in Python, write Python programs to implement Inheritance & Polymorphism.

Theory :-

Q1 What is Object - Oriented Programming (OOP)? How is it different from procedural programming?

=> OOP is a style of programming where a program is built from objects that hold data and the methods that work on the data. Its main ideas are encapsulation, inheritance and polymorphism.

Basis	Procedural Programming	Object oriented Programming
Approach	Program is a sequence of functions	Program is a set of interacting objects
Data	Shared openly between functions	Kept inside objects with their methods.
Reuse	Limited, by calling functions	High, through inheritance.

Sundaram
FOR EDUCATIONAL USE

Q2 Define the following terms with examples
a) class
b) object
c) constructor
d) destructor

=>a) class:-

A class is a blueprint that defines the attributes and methods of a group of similar objects.

b) Object:-

An object is an instance of a class. Each object holds its own copy of the data.

c) constructor:-

A constructor is the special method that runs automatically when an object is created and sets its initial values.

d) Destructor:-

A Destructor is the special method that runs when an object is destroyed, and it is used to release resources.

Sundaram
FOR EDUCATIONAL USE

code :-

```
class student:
    college = "DJSCE"

    def __init__(self, name, roll):
        self.name = name
        self.roll = roll
        print("Object created for", self.name)

    def show(self):
        print("Name:", self.name, "| Roll:",
              self.roll)
    def __del__(self):
        print("Object destroyed for", self.name)

s1 = student("Rahul", 12)
s1.show()
del s1
```

Output :-

```
Object created for Rahul
Name: Rahul | Roll: 12
Object destroyed for Rahul
```

Sundaram
FOR EDUCATIONAL USE

Q3 what is the purpose of the __init__() method in Python classes?

⇒ The init method is the constructor of a class. It runs automatically at object creation and assigns starting values to the attributes.

code:

```
class Book:
    def __init__(self, title, price):
        self.title = title   # attribute set at creation
        self.price = price

b = Book("Python Basics", 350)
print(b.title, b.price)
```

output:

Python Basics 350

Q4 what do you mean by inheritance? List and Explain any two types of inheritance supported in Python.

⇒ Inheritance lets a child class acquire the attributes and methods of a parent class, which avoids rewriting code. Single inheritance means one child derives from one parent. Multilevel inheritance means a class derives from a child class, forming a chain.

Sundaram
FOR EDUCATIONAL USE

code:

```
# Single inheritance
class Vehicle:
    def Start(self):
        print("Vehicle started")

class Car(Vehicle):
    def drive(self):
        print("Car is moving")

c = Car()
c.start()
c.drive()

# Multilevel inheritance
class Animal:
    def eat(self):
        print("Animal eats")

class Dog(Animal):
    def bark(self):
        print("Dog barks")

class Puppy(Dog):
    def weep(self):
        print("Puppy weeps")
p = Puppy()
p.eat()
p.bark()
p.weep()
```

Sundaram FOR EDUCATIONAL USE

output :-

```
Vehicle started
car is moving
Animal eats
Dog barks
Puppy weeps
```

Q5 what is method overriding? How does it differ from method overloading?

⇒ Method overriding means a child class gives its own version of a method already defined in the parent. Overloading means several methods share one name with different parameters.

Basis	Overriding	Overloading
where	Between parent and child class	within the same class.
Signature	Same name and parameters	Same name, different parameters.
Python	Supported directly	Not supported directly, default arguments are used.

Sundaram
FOR EDUCATIONAL USE

code:

```
class Shape:
    def area(self):
        print("area of a generic shape")

class Square(Shape):

    def __init__(self, side):
        self.side = side
    def area(self):
        print("area of square:", self.side *
                                 self.side)

Square(4).area()
Shape().area()

# overloading style: One method, default arguments
class calc:
    def add(self, a, b=0, c=0):
        return (a+b+c)

k = calc()
print(k.add(2))
print(k.add(2,3))
print(k.add(2,3,4))
```

Output:
```
area of square: 16
area of a generic shape
2
5
9
```

Sundaram FOR EDUCATIONAL USE

Q6 Explain the use of the super() function with an example.

⇒ The super() function give acess to the parent class, so the child can call the parent constructor or methods without naming the parent

Code:-

```
class Person:
    def __init__(self, name):
        self.name = name

class Employee(Person):
    def __init__(self, name, salary):
        super().__init__(name)
        self.salary = salary

    def display(self):
        print("Name:", self.name)
        print("Salary:", self.salary)

e = Employee("Riya", 45000)
e.display()
```

Output:-

```
Name: Riya
Salary: 45000
```

Sundaram
FOR EDUCATIONAL USE

Q7 What is polymorphism? Provide an example demonstrating polymorphism in Python.

⇒ Polymorphism means one method name behaving differently for different objects. In the example, each class defines sound() in its own way and the same call works for all

Code:

```
class Cat:
    def sound(self):
        return "Meow"
class Cow:
    def sound(self):
        return "Moo"
class Duck:
    def sound(self):
        return "Quack"

for animal in (Cat(), Cow(), Duck()):
    print(animal.sound())
print(len("Python"), len([1,2,3]), len({"a":1}))
```

Output:

```
Meow
Cow
Quack
6 3 1
```

Sundaram FOR EDUCATIONAL USE

Q8 what is the role of the self parameter in Python class methods?

⇒ The self parameter refers to the current object. It is the first parameter of every instance method and is used to reach the attributes and methods of that object. Python passes it automatically.

code:

```
class Counter:
    def __init__(self, start):
        self.value = start
    def increase(self):
        self.value += 1
        return self
c = Counter(5)
c.increase().increase()
print("Value:", c.value)
```

Output:

Value: 7

Sundaram®
FOR EDUCATIONAL USE

@9 write a short note on the advantage of using OOP principles in real-world applications.

⇒ OOP makes real world applications easier to build and maintain. The main advantages are given below.

Advantage	Explanation	Real world use
Reusability	Inheritance lets existing code be reused	Savings and current accounts reuse a common account class
Easy maintenance	Code is divided into classes, so changes stay local	Fixing the student class does not disturb other classes.
Data Protection	Encapsulation keeps data inside the object	Account balance is changed only through its methods
Real entity modelling	Objects map naturally to real things	Students, vehicles and Employees.
Flexibility	Polymorphism lets one call work on many objects.	Different shapes use one area() call
Scalability	New classes can be added without rewriting old ones	Adding a new product prototype in a shop system.

Sundaram
FOR EDUCATIONAL USE

Programs:

Program 1: Inheritance (single and multilevel)

```
# Base class
class Employee:
    def __init__(self, name, empId):
        self.name = name
        self.empId = empId
    def details(self):
        print("Name:", self.name)
        print("ID:", self.empID)
# child of Employee (single inheritance)
class Manager(Employee):
    def __init__(self, name, empId, department):
        super().__init__(name, empId)
        self.department = department
    def details(self):
        super().details()
        print("Department:", self.department)
# child of Manager (multilevel inheritance)
class Manager(Employee):
    def __init__(self, name, empId, department, company):
        super().__init__(name, empId, department)
        self.company = company
    def details(self):
        super().details()
        print("company:", self.company)
```

Sundaram
FOR EDUCATIONAL USE

```
m = Manager("Neha", 101, "Sales")
m.details()
print()
d = Director("Arjun", 1, "Finance", "TechCorp")
d.details()
```

Output:

```
Name: Neha
ID: 101
Department: Sales

Name: Arjun
ID: 1
Department: Finance
Company: TechCorp
```

Program 2: Polymorphism using method overriding

```
import math
class Shape:
    def area(self):
        return 0
class Rectangle(Shape):
    def __init__(self, length, breadth):
        self.length = length
        self.breadth = breadth
```

FOR EDUCATIONAL USE

Sundaram®

```
def area(self)
    return self.length * self.breadth

class circle(shape):
    def __init__(self, radius)
        self.radius = radius
    def area(self):
        return round(math.pi * self.radius **2 , 2)

class Triangle(shape):
    def __init__(self, base, height):
        self.base = base
        self.height = height
    def area(self):
        return 0.5 * self.base * self.height

shapes = [Rectangle(5,3), circle(7) Triangle(6, 4)]
for s in shapes:
    print(type(s).__name__, "area =", s.area())
```

Output:- Rectangle area=15
circle area=153.94
Triangle area=12.0

Conclusion:-

Object oriented concepts were implemented in Python using classes and objects. Inheritance allowed child classes to reuse parent code, with super() linking the two. Polymorphism let one method name act differently across classes, keeping the code short and easy to extend.

Sundaram
FOR EDUCATIONAL USE

Data cleaning

In [8]:
```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    'A':[1,2,np.nan,4],
    'B':[5,np.nan,7,8],
    'C':[9,10,11,12]
})
missing_values = df.isna()
print(missing_values)
```

```
       A      B      C
0  False  False  False
1  False   True  False
2   True  False  False
3  False  False  False
```

In [9]:
```
df_filled=df.fillna(0)
print(df_filled)
```

```
     A    B   C
0  1.0  5.0   9
1  2.0  0.0  10
2  0.0  7.0  11
3  4.0  8.0  12
```

In [10]:
```
df_filled_mean = df.fillna(df.mean())
print(df_filled_mean)
```

```
          A         B   C
0  1.000000  5.000000   9
1  2.000000  6.666667  10
2  2.333333  7.000000  11
3  4.000000  8.000000  12
```

In [11]:
```
df_dropped=df.dropna()
print(df_dropped)
```

```
     A    B   C
0  1.0  5.0   9
3  4.0  8.0  12
```

Forward filling

In [16]:
```
data = {
    'A':[1,None,None,4,None,6]
}
df = pd.DataFrame(data)
print("Before Forward Filling")
print(df)
df['A']=df['A'].fillna(method="ffill")
print("\nAfter Forward Filling")
print(df)
```

```
Before Forward Filling
     A
0  1.0
1  NaN
2  NaN
3  4.0
4  NaN
5  6.0

After Forward Filling
     A
0  1.0
1  1.0
2  1.0
3  4.0
4  4.0
5  6.0
```

Backward filling

In [17]:
```
data = {
    'A':[1,None,None,4,None,6]
}
df = pd.DataFrame(data)
print("Before Backward Filling")
print(df)
df['A']=df['A'].fillna(method="bfill")
print("\nAfter Backward Filling")
print(df)
```

```
Before Backward Filling
     A
0  1.0
1  NaN
2  NaN
3  4.0
4  NaN
5  6.0

After Backward Filling
     A
0  1.0
1  4.0
2  4.0
3  4.0
4  6.0
5  6.0
```

Z-score = (x-u)/d

x = data point

u = mean

d = deviation

In [20]:
```
from scipy import stats
data = np.array([10,12,14,15,100,18,20])
z_scores = stats.zscore(data)
outliers = np.abs(z_scores)>2
```

```
print("Z-scores : ",z_scores)
print("Outliers : ",data[outliers])
```

```
Z-scores :  [-0.56729735 -0.50055648 -0.43381562 -0.40044519  2.43604155 -0.30033
389
 -0.23359303]
Outliers :  [False False False False  True False False]
```

In [25]:
```
data = np.array([10,12,14,15,100,18,20])
Q1 = np.percentile(data,25)
Q3 = np.percentile(data,75)
IQR = Q3 - Q1
outliers = ((data < (Q1-1.5*IQR))|(data > (Q3+1.5*IQR)))
print("IQR : ",IQR)
print("Outliers : ",data[outliers])
print(Q1)
print(Q3)
```

```
IQR :  6.0
Outliers :  [100]
13.0
19.0
```

In [26]:
```
df = pd.DataFrame({
    'values' : [10,12,14,15,100,18,20]
})
Q1 = df['values'].quantile(0.25)
Q3 = df['values'].quantile(0.75)
IQR = Q3 - Q1
outliers = ((df['values'] < (Q1 - 1.5*IQR))|(df['values'] > (Q3+1.5*IQR)))
print("Outliers in DataFrame : ",df[outliers])
```

```
Outliers in DataFrame :     values
4     100
```

In [ ]:
```
```

Department of AI_DS
Academic Year: 2026-27 (Odd Sem)
Course: Python for Data Science Lab
Second Year B. Tech Semester: III

PRACTICAL 13

Part A (To be referred by students)

Data Visualisation

SAVE THE FILE AND UPLOAD AS (RollNo_Name_Exp13)

Aim: To understand and apply data visualisation techniques using Matplotlib and Seaborn in Python, focusing on data science, machine learning applications, and statistical analysis.

Problem Statement:

Solve the following computational problems to visualise the data

1. In the Titanic dataset, create a bar chart to show the count of passengers in each 'Pclass'.
2. For the Titanic dataset, plot a histogram of 'Age' to see how many passengers fall into different age ranges.
3. In the Titanic dataset, generate a pie chart to display the proportion of survivors vs. non-survivors.
4. From the Iris dataset, create a scatter plot of 'SepalLength' vs. 'SepalWidth' using different colors for each 'Species'.
5. For the Iris dataset, plot a histogram of 'PetalLength' to show the distribution of petal lengths.
6. In the Iris dataset, create a bar chart to display the average 'PetalWidth' for each 'Species'.
7. In the Boston dataset, plot a scatter plot of 'RM' (average number of rooms) vs. 'MEDV' (median value of homes).
8. For the Boston dataset, create a histogram of 'AGE' (proportion of units built before 1940).
9. In the Boston dataset, generate a bar chart to show the count of records for different 'CHAS' (Charles River dummy variable) values.
10. From the Wine dataset, plot a scatter plot of 'Alcohol' vs. 'ColorIntensity'.
11. In the Wine dataset, create a histogram of 'Hue' to visualize the distribution of hue values.

12. For the Wine dataset, generate a bar chart to display the average 'Flavanoids' for each wine 'Class'.

Link to datasets:

1. **Titanic Dataset**: Titanic: Machine Learning from Disaster
2. **Iris Dataset**: Iris Species
3. **Boston Housing Dataset**: Boston Housing Dataset
4. **Wine Quality Dataset**: Wine Quality Dataset

Topic covered: Matplotlib and Seaborn

Learning Objective: Learner would be able to

1. Explain the purpose and importance of data visualization in data analysis.
2. Identify different types of plots (bar, histogram, pie, scatter, box, etc.) and their appropriate use cases.
3. Describe the key components of Matplotlib and Seaborn libraries used for visualization in Python.

Theory:
Introduction to Matplotlib and Seaborn

Matplotlib is a plotting library for creating static, animated, and interactive visualizations in Python. Seaborn builds on Matplotlib and provides a high-level interface for drawing attractive statistical graphics.

Basic Visualizations with Matplotlib

Line Plot

Task: Plot a simple line chart.

Example:

```
import matplotlib.pyplot as plt
```

```
import numpy as np

# Sample data
x = np.linspace(0, 10, 100) y = np.sin(x)
# Create a line plot
plt.plot(x, y)
plt.title('Sine Wave')
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.show()
```

Explanation: plt.plot(x, y) creates a line plot with x as the x-axis and y as the y-axis

Scatter Plot

Task: Plot a scatter plot to show the relationship between two variables.

Example:

```
import matplotlib.pyplot as plt

# Sample data
x = np.random.rand(100)
y = np.random.rand(100)

# Create a scatter plot
plt.scatter(x, y)
plt.title('Scatter Plot')
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.show()
```

Explanation: plt.scatter(x, y) creates a scatter plot with x and y as the coordinates of the points.

Histogram

Task: Create a histogram to visualize the distribution of a dataset.

Example:

```
import matplotlib.pyplot as plt

# Sample data
data = np.random.randn(1000)

# Create a histogram
plt.hist(data, bins=30, edgecolor='black')
plt.title('Histogram')
plt.xlabel('Value')
plt.ylabel('Frequency')
plt.show()
```

Explanation: plt.hist(data, bins=30) creates a histogram of data with 30 bins.

Advanced Visualizations with Seaborn

Box Plot

Task: Create a box plot to visualize the distribution of a dataset.

Example:

```
import seaborn as sns import pandas as pd

# Sample data
data = pd.DataFrame({ 'Category': ['A']*50 + ['B']*50, 'Value': np.random.randn(100)
})

# Create a box plot
sns.boxplot(x='Category', y='Value', data=data) plt.title('Box Plot')
plt.show()
```

Explanation: sns.boxplot(x='Category', y='Value', data=data) creates a box plot of Value by Category.

Pair Plot

Task: Visualize pairwise relationships in a dataset.

Example:

```
import seaborn as sns import pandas as pd
from sklearn.datasets import load_iris


# Load Iris dataset iris = load_iris()
data = pd.DataFrame(data=iris.data, columns=iris.feature_names)
data['Species'] = iris.target

# Create a pair plot sns.pairplot(data, hue='Species') plt.title('Pair Plot')
plt.show()
```

Explanation: sns.pairplot(data, hue='Species') creates a pair plot for all features colored by species.

Heatmap

Task: Create a heatmap to visualize the correlation matrix of a dataset.

Example:

```
import seaborn as sns import pandas as pd

# Sample data
data = pd.DataFrame({
'A': np.random.randn(100),
'B': np.random.randn(100), 'C': np.random.randn(100)
})

# Compute correlation matrix corr = data.corr()

# Create a heatmap
```

sns.heatmap(corr, annot=True, cmap='coolwarm') plt.title('Heatmap of Correlation Matrix') plt.show()
Explanation: sns.heatmap(corr, annot=True) creates a heatmap of the correlation matrix with annotations.

Data Science and ML Applications

Visualizing Model Performance

Task: Plot ROC curve to evaluate classification model performance.

Example:

```
import matplotlib.pyplot as plt
from sklearn.metrics import roc_curve, auc
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import label_binarize

# Generate sample data
X, y = make_classification(n_samples=1000, n_classes=2, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Train a logistic regression model model = LogisticRegression() model.fit(X_train, y_train)

# Compute ROC curve
y_score = model.decision_function(X_test) fpr, tpr, _ = roc_curve(y_test, y_score) roc_auc =
auc(fpr, tpr)

# Plot ROC curve
plt.plot(fpr, tpr, color='blue', lw=2, label=f'ROC curve (area = {roc_auc:.2f})') plt.plot([0, 1], [0, 1],
color='grey', linestyle='--')
plt.title('Receiver Operating Characteristic (ROC)')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend(loc='lower right') plt.show()
```

Explanation: This code plots the ROC curve of a logistic regression model to evaluate its performance.

Visualizing Feature Importance Task: Plot feature importance for a trained model. **Example:**

```
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestClassifier from sklearn.datasets import load_iris
import pandas as pd

# Load Iris dataset iris = load_iris()
X, y = iris.data, iris.target

# Train a RandomForest model model = RandomForestClassifier() model.fit(X, y)

# Get feature importances
importances = model.feature_importances_ features = iris.feature_names

# Create a bar plot plt.barh(features, importances) plt.xlabel('Importance') plt.title('Feature Importance') plt.show()
```

Department of AI_DS
Academic Year: 2025-26 (Odd Sem)
Course: Python for Data Science Lab
Second Year B. Tech Semester: III

PRACTICAL 13

Part B (to be completed by students)

Data Cleaning

1. All the students are required to perform the given tasks in Jupyter Notebook
2. Create a new notebook for each experiment. The filename should be RollNo_Name_Exp13)
3. In the first cell, the student must write his/her Name, roll no and class in the form of comments
4. Every program should be written in separate cells and in the given sequence
5. After completing the experiment, download the notebook in pdf format. The filename should be RollNo_Name_Exp12.pdf.
   Note: - To download as a PDF, go to File and choose Print Preview. This will open a new tab. Now, press Ctrl + P and opt to save as PDF.
6. Upload the pdf on teams

In [2]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
C:\Users\djsce.student\AppData\Local\Continuum\anaconda3\lib\site-packages\pandas
\compat\_optional.py:138: UserWarning: Pandas requires version '2.7.0' or newer o
f 'numexpr' (version '2.6.8' currently installed).
  warnings.warn(msg, UserWarning)
```

In [6]:
```
titanic = pd.read_csv("titanic (1).csv")
titanic.head()
```

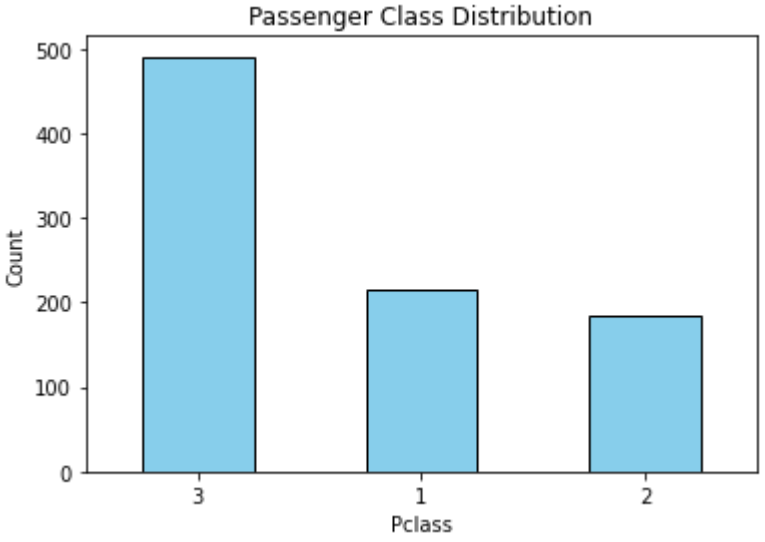
Out[6]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	I
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0

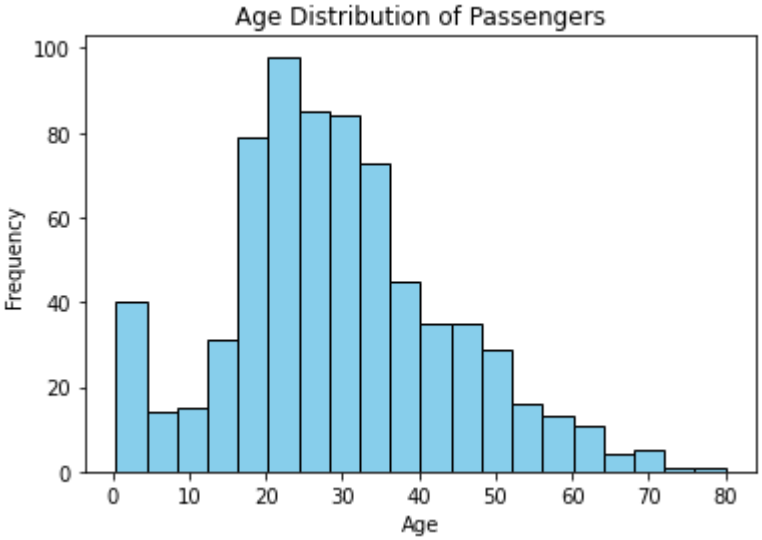
In [3]:
```
titanic.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype  
---  ------       --------------  -----  
 0   PassengerId  891 non-null    int64  
 1   Survived     891 non-null    int64  
 2   Pclass       891 non-null    int64  
 3   Name         891 non-null    object 
 4   Sex          891 non-null    object 
 5   Age          714 non-null    float64
 6   SibSp        891 non-null    int64  
 7   Parch        891 non-null    int64  
 8   Ticket       891 non-null    object 
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    object 
 11  Embarked     889 non-null    object 
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
```

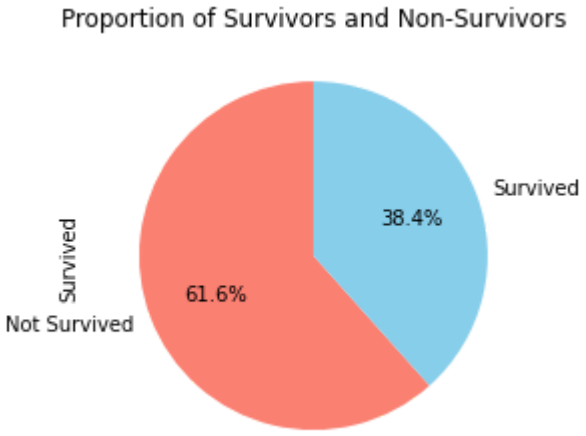
In [8]:
```
titanic['Pclass'].value_counts().plot(kind='bar', color='skyblue', edgecolor='bl
plt.title('Passenger Class Distribution')
plt.xlabel('Pclass')
plt.ylabel('Count')
plt.xticks(rotation=0)
plt.show()
```



In [10]:
```
titanic['Age'].plot(kind='hist', bins=20, color='skyblue', edgecolor='black')
plt.title('Age Distribution of Passengers')
plt.xlabel('Age')
plt.ylabel('Frequency')
plt.show()
```



In [11]:
```
titanic['Survived'].value_counts().plot(
    kind='pie',
    autopct='%1.1f%%',
    colors=['salmon', 'skyblue'],
    labels=['Not Survived', 'Survived'],
    startangle=90
)
plt.title('Proportion of Survivors and Non-Survivors')
plt.show()
```



In [15]:
```
iris = pd.read_csv("Iris.csv")
iris.head()
```

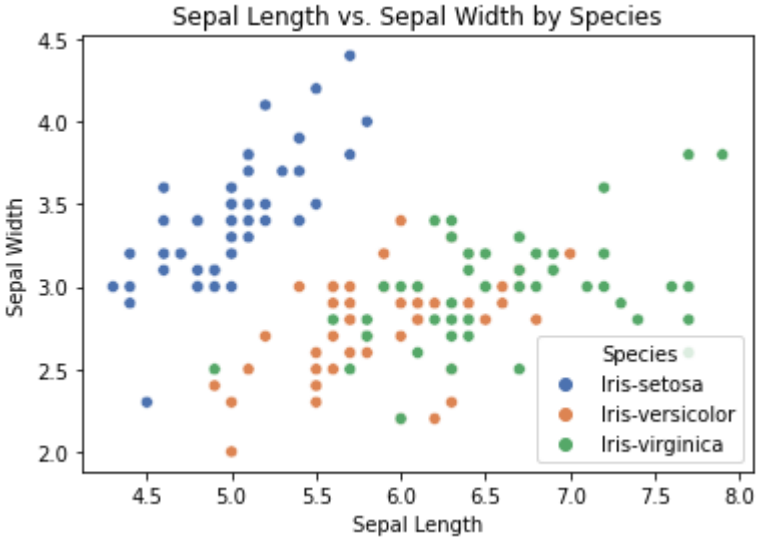
Out[15]:

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa

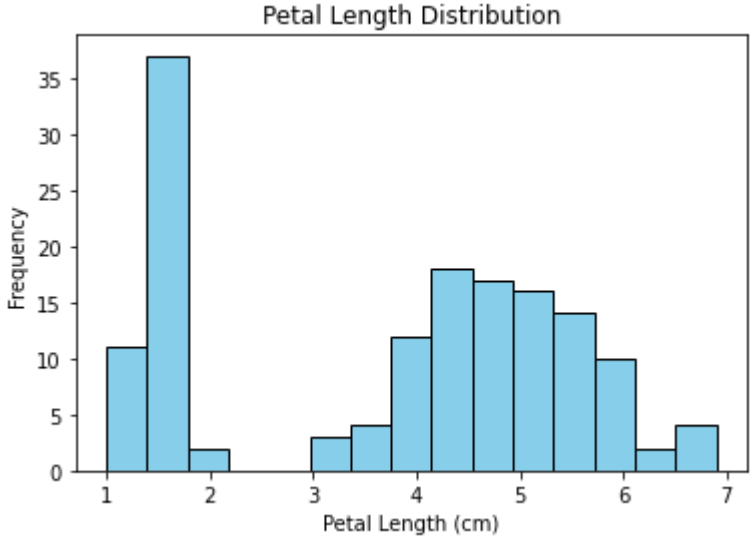
In [14]:
```
iris.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 6 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Id             150 non-null    int64
 1   SepalLengthCm  150 non-null    float64
 2   SepalWidthCm   150 non-null    float64
 3   PetalLengthCm  150 non-null    float64
 4   PetalWidthCm   150 non-null    float64
 5   Species        150 non-null    object
dtypes: float64(4), int64(1), object(1)
memory usage: 7.2+ KB
```

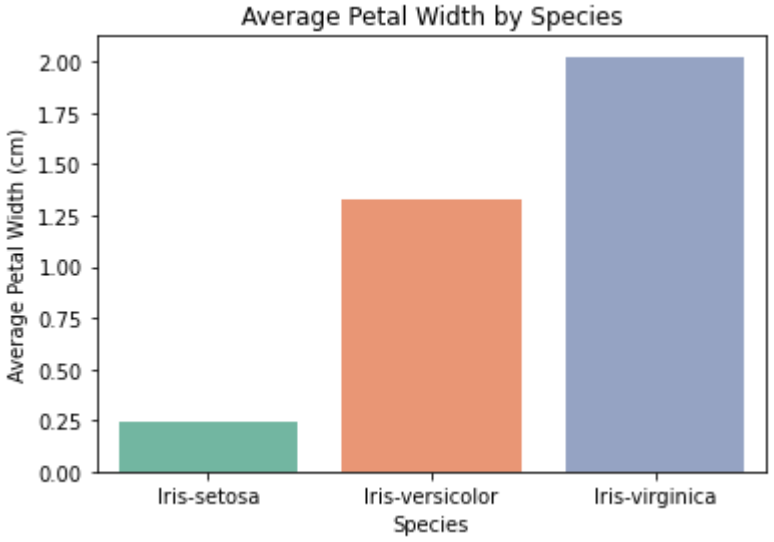
In [19]:
```
sns.scatterplot(data=iris, x='SepalLengthCm', y='SepalWidthCm', hue='Species', p
plt.title('Sepal Length vs. Sepal Width by Species')
plt.xlabel('Sepal Length')
plt.ylabel('Sepal Width')
plt.show()
```



In [20]:
```
iris['PetalLengthCm'].plot(kind='hist', bins=15, color='skyblue', edgecolor='bla
plt.title('Petal Length Distribution')
plt.xlabel('Petal Length (cm)')
plt.ylabel('Frequency')
plt.show()
```



In [21]:
```
sns.barplot(data=iris, x='Species', y='PetalWidthCm', errorbar=None, palette='Se
plt.title('Average Petal Width by Species')
plt.xlabel('Species')
plt.ylabel('Average Petal Width (cm)')
plt.show()
```



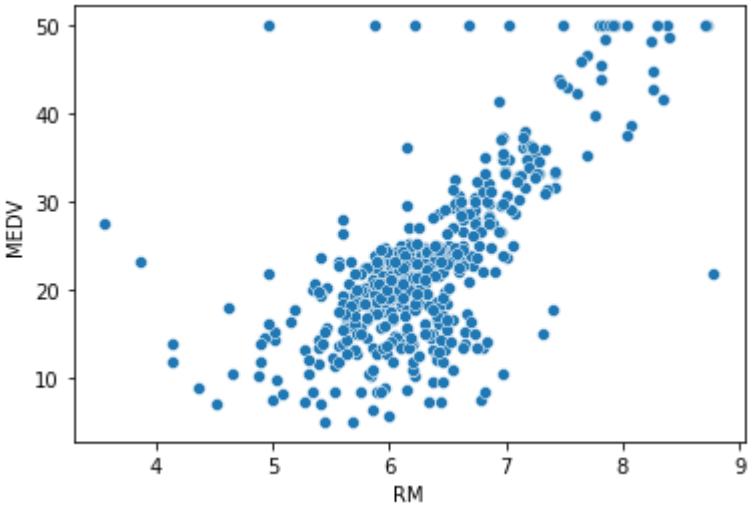
In [22]:
```
data_url = "http://lib.stat.cmu.edu/datasets/boston"
raw_df = pd.read_csv(data_url, sep=r"\s+", skiprows=22, header=None)
data = np.hstack([raw_df.values[::2, :], raw_df.values[1::2, :2]])
target = raw_df.values[1::2, 2]
columns = ['CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX', 'RM', 'AGE', 'DIS', 'RAD', 'TAX
boston = pd.DataFrame(data, columns=columns)
boston['MEDV'] = target
```

In [23]:
```
boston.head()
```

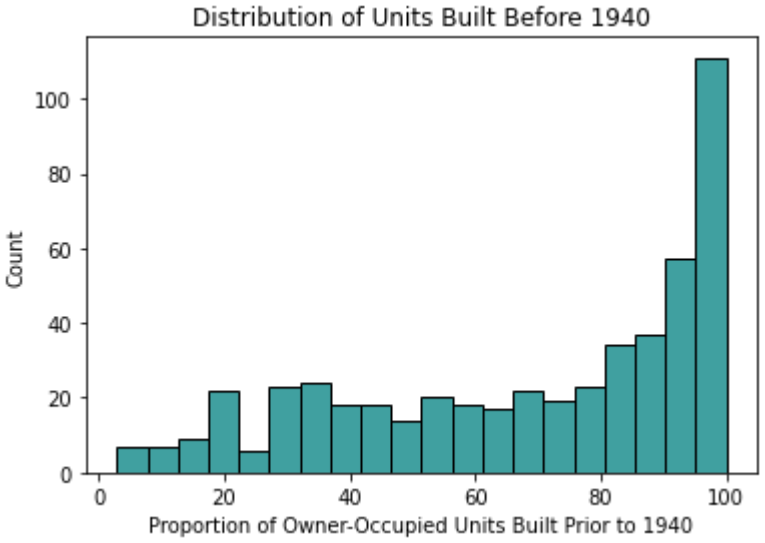
Out[23]:

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	
0	0.00632	18.0	2.31	0.0	0.538	6.575	65.2	4.0900	1.0	296.0	15.3	396.9
1	0.02731	0.0	7.07	0.0	0.469	6.421	78.9	4.9671	2.0	242.0	17.8	396.9
2	0.02729	0.0	7.07	0.0	0.469	7.185	61.1	4.9671	2.0	242.0	17.8	392.8
3	0.03237	0.0	2.18	0.0	0.458	6.998	45.8	6.0622	3.0	222.0	18.7	394.6
4	0.06905	0.0	2.18	0.0	0.458	7.147	54.2	6.0622	3.0	222.0	18.7	396.9

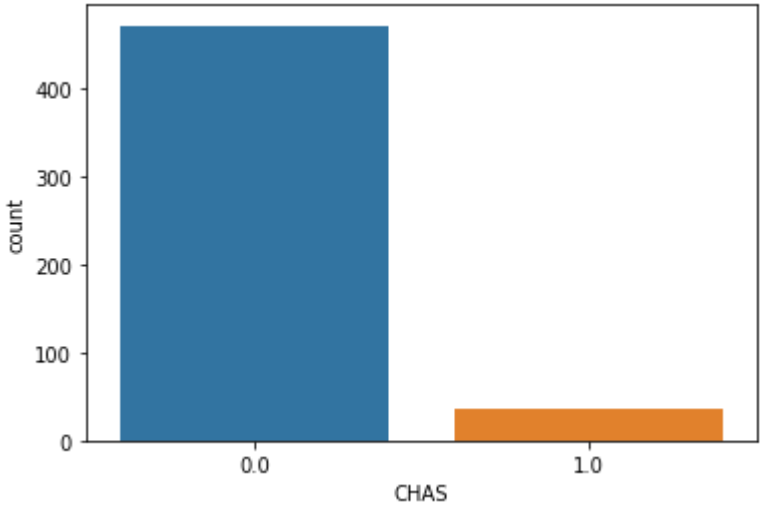
In [24]:
```
sns.scatterplot(data=boston, x='RM', y='MEDV')
plt.show()
```



In [27]:
```
sns.histplot(data=boston, x='AGE', bins=20, color='teal')
plt.title('Distribution of Units Built Before 1940')
plt.xlabel('Proportion of Owner-Occupied Units Built Prior to 1940')
plt.ylabel('Count')
plt.show()
```



In [28]:
```
sns.countplot(data=boston, x='CHAS')
plt.show()
```



In [4]:
```
wine = pd.read_csv("WineQT (1).csv")
wine.head()
```

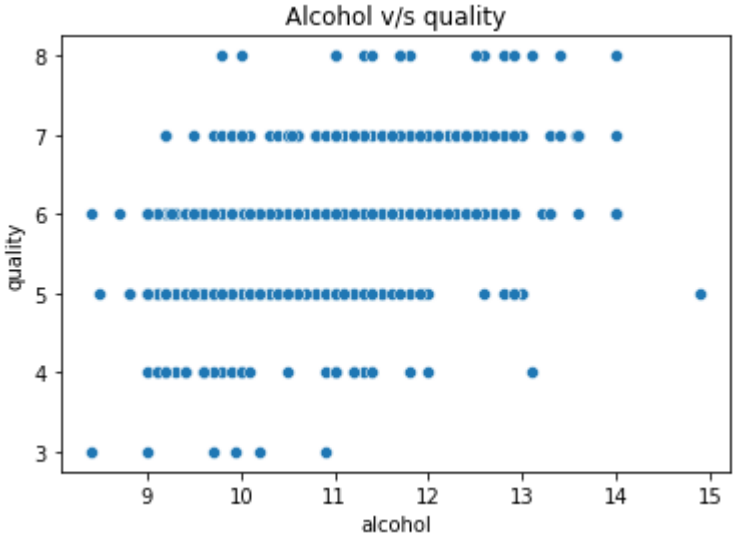
Out[4]:

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates
0	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.56
1	7.8	0.88	0.00	2.6	0.098	25.0	67.0	0.9968	3.20	0.68
2	7.8	0.76	0.04	2.3	0.092	15.0	54.0	0.9970	3.26	0.65
3	11.2	0.28	0.56	1.9	0.075	17.0	60.0	0.9980	3.16	0.58
4	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.56

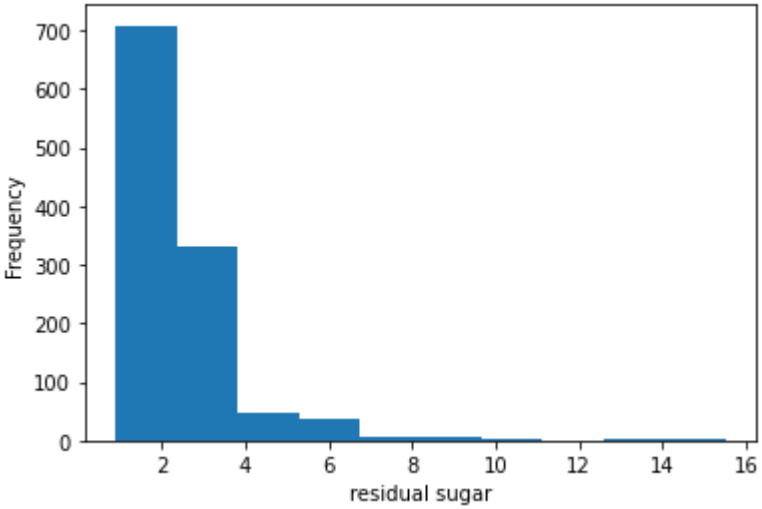
In [5]:
```
wine.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1143 entries, 0 to 1142
Data columns (total 13 columns):
 #   Column                Non-Null Count  Dtype  
---  ------                --------------  -----  
 0   fixed acidity         1143 non-null   float64
 1   volatile acidity      1143 non-null   float64
 2   citric acid           1143 non-null   float64
 3   residual sugar        1143 non-null   float64
 4   chlorides             1143 non-null   float64
 5   free sulfur dioxide   1143 non-null   float64
 6   total sulfur dioxide  1143 non-null   float64
 7   density               1143 non-null   float64
 8   pH                    1143 non-null   float64
 9   sulphates             1143 non-null   float64
 10  alcohol               1143 non-null   float64
 11  quality               1143 non-null   int64  
 12  Id                    1143 non-null   int64  
dtypes: float64(11), int64(2)
memory usage: 116.2 KB
```

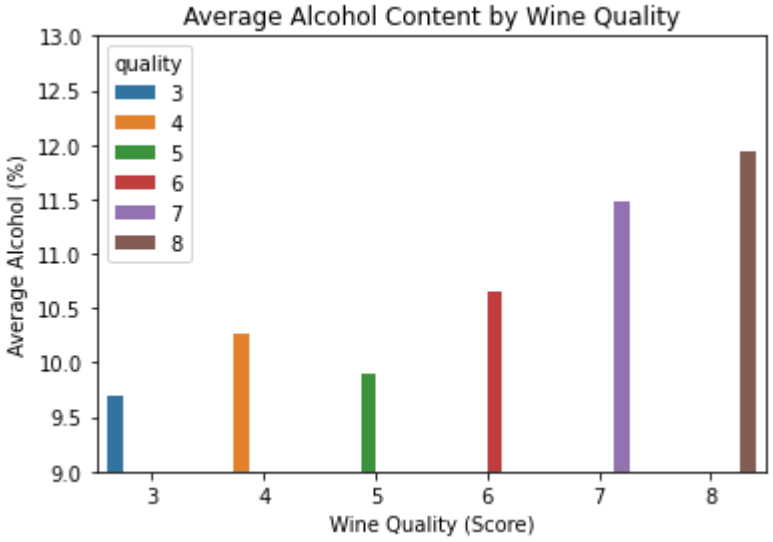
In [7]:
```
sns.scatterplot(data=wine, x='alcohol', y='quality')
plt.title("Alcohol v/s quality")
plt.show()
```



In [8]:
```
plt.hist(wine['residual sugar'])
plt.xlabel('residual sugar')
plt.ylabel('Frequency')
plt.show()
```



In [10]:
```
avg_alcohol = wine.groupby('quality')['alcohol'].mean().reset_index()
sns.barplot(x='quality', y='alcohol', data=avg_alcohol, hue='quality')
plt.title('Average Alcohol Content by Wine Quality')
plt.xlabel('Wine Quality (Score)')
plt.ylabel('Average Alcohol (%)')
plt.ylim(9, 13)
plt.show()
```


Average Alcohol Content by Wine Quality
quality
3
4
5
6
7
8
Average Alcohol (%)
13.0
12.5
12.0
11.5
11.0
10.5
10.0
9.5
9.0
3
4
5
6
7
8
Wine Quality (Score)

In [ ]:

Department of Artificial Intelligence (AI) & Data Science

AY: 2026-27

Course: Python for Data Science Laboratory (DJS23SLPC303)

Class: SE/A11/A12/A21/A22 Sem: III Roll No.: SAPID:

Experiment No. 13: Data Visualization

Aim:

Theory

1. What is the importance of data visualization in data science and machine learning?
2. Differentiate between Matplotlib and Seaborn libraries in Python.
3. Explain the role of the following types of plots and mention one use case for each:
   a) Bar Chart b) Histogram c) Pie Chart d) Scatter Plot
4. What is the difference between a box plot and a histogram in terms of data interpretation?

Conclusions:

Name :- Aaryan V. Raorane Roll No :- 5004 Branch :- AIEDS
Batch :- A1 Div :- 5 SAP ID :- 60018250070

Experiment No 13

06/10

Title :- Data Visualization

Aim :- To apply and understand data visualization techniques using Matplotlib and Seaborn in Python.

Theory :-

Q1 What is the importance of data visualization in data science and machine learning?

=> Data visualization means presenting data in graphical form, such as charts and plots, so that patterns can be understood quickly. In data science it is used during exploratory analysis to find trends, relationships, skewness and outliers that are hard to notice in raw tables. It helps in checking data quality before cleaning and modelling, and in choosing suitable features and algorithms. After a model is trained, plots of accuracy, loss and predictions show how well it performs. Graphs also communicate the findings clearly to people who are not familiar with the technical details.

code :-

```
import matplotlib.pyplot as plt
months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun"]
sales = [20, 25, 31, 38, 44, 52]
plt.plot(months, sales, marker="o")
plt.xlabel("Month")
plt.ylabel("sales in thousands")
plt.title("Monthly Sales Trend")
plt.show()
```

Sundaram
FOR EDUCATIONAL USE

Q2 Differentiate between Matplotlib and seaborn libraries in Python.

⇒

Point	Matplotlib	seaborn
Nature	Basic, low level plotting library	High level library built on top of Matplotlib
code length	More lines are needed for a good looking plot	Fewer lines are needed.
Default look	Plain	Attractive themes and colour palettes.
Data input	lists and arrays	Works directly with pandas DataFrames using column names.
statistical plots	Have to be built Manually.	Ready functions such as boxplot, heatmap and pairplot.
control	Complete control over every Element	Less control, Matplotlib is used to fine tune.

Sundaram
FOR EDUCATIONAL USE

code :

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
y = [2, 4, 6, 8, 10]
plt.bar(x, y, color = "steelblue")
plt.xlabel("x")
plt.ylabel("y")
plt.title("Matplotlib Bar Plot")
plt.show()

import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt.

df = pd.DataFrame({"Branch": ["AI", "IT", "CS", "EXTC"],
                   "Students": [60, 55, 50, 40]})
sns.barplot(x = "Branch", y = "students", data = df)
plt.title("Seaborn Bar plot")
plt.show()
```

Sundaram®
FOR EDUCATIONAL USE

Q3 Explain the role of the following types of plots and mention one use case for each:

a) Bar chart
b) Histogram
c) Pie chart
d) Scatter plot

⇒ a) Bar chart:

A Bar chart uses rectangular bars whose heights show the value of each category, so categories can be compared easily. It is used, for example, to compare the number of students in each branch.

b) Histogram:

A Histogram divides numeric data into bins and shows how many values fall in each bin, so it shows the shape of the distribution. It is used for example, to see how the marks of a class are spread.

c) Pie Chart:

A pie chart divides a circle into slices whose sizes show the share of each category in the whole. It is used, for example, to show the percentage of a monthly budget spent on each head.

Sundaram
FOR EDUCATIONAL USE

d) Scatter Plot :

A Scatter Plot places each observation as a point using two numeric variables, so it shows the relationship between them. It is used, for example, to study how study hours are related to marks.

code :

```
import matplotlib.pyplot as plt

fig, ax = plt.subplot(2,2, figsize=(9,7), layout="constrained")
ax[0,0].bar(["AJ", "IT", "CS", "EXTC"], [60,55,50,40])
ax[0,0].set(title="Bar chart")

marks = [35,42,48,50,52,55,58,60,61,63,65,69,70,72,78,85]
ax[0,1].hist(marks, bin=5, edgecolor="black")
ax[0,1].set(title="Histogram")

ax[1,0].pie([40,25,20,15], autopct="%1.0f%%",
            labels=["Rent", "Food", "Travel", "Other"])
ax[1,0].set(title="Pie chart")

hours = [1,2,3,4,5,6,7,8]
score = [35,42,50,55,63,70,78,85]
ax[1,1].scatter(hours, score)
ax[1,1].set(title="Scatter Plot")

plt.show()
```

Sundaram
FOR EDUCATIONAL USE

Q4 what is the difference between a box plot and a histogram in terms of data interpretation?

⇒

Point	Box Plot	Histogram
shows	Median, quartiles, spread and outliers	frequency of values in each bin
Outliers	shown directly as separate points beyond the whiskers	seen only as small isolated bars far from the rest.
shape	gives a limited idea of shape and cannot show two peaks	shows skewness and the number of peaks clearly
comparing groups	Many boxes can be placed side by side easily	Becomes crowded for many groups.
dependence on bins	Doesn't depend on bins	appearance changes with the number of bins.

Sundaram®
FOR EDUCATIONAL USE

code:

```
import matplotlib.pyplot as plt

data = [12, 15, 14, 16, 18, 17, 15, 14, 16, 19, 13, 15, 45]
fig, ax = plt.subplots(1, 2, figsize=(4, 4), layout="constrained")
ax[0].boxplot(data)
ax[0].set(title="Box plot")
ax[1].hist(data, bins=6, edgecolor="black")
ax[1].set(title="Histogram")
plt.show()
```

Conclusion:

Data visualization was performed using Matplotlib and seaborn to present data in graphical form. Bar charts, histograms, pie charts, scatter plots and box plots were created and the information given by each was understood. The choice of plot depends on the type of data and on the question that has to be answered.

Sundaram®
FOR EDUCATIONAL USE

In [1]:
```
import sqlite3
conn = sqlite3.connect('college.db')
cur = conn.cursor()
cur.execute('''CREATE TABLE IF NOT EXISTS student(
roll_no INTEGER PRIMARY KEY,
name TEXT NOT NULL,
dept TEXT,
marks INTEGER)''')
print("Table Created Successfully")
```

```
Table Created Successfully
```

In [2]:
```
cur.execute("ALTER TABLE student ADD COLUMN email TEXT")
print("Column added successfully")
conn.commit()
conn.close()
```

```
Column added successfully
```

In [4]:
```
import sqlite3

conn = sqlite3.connect('college.db')
cur = conn.cursor()

cur.execute("INSERT INTO student (roll_no,name, dept, marks, email) VALUES (1,'a
cur.execute("INSERT INTO student (roll_no,name, dept, marks, email) VALUES (2,'x

conn.commit()
conn.close()
```

In [7]:
```
import sqlite3
conn = sqlite3.connect('college.db')
cur = conn.cursor()
cur.execute("UPDATE student SET marks = 95 WHERE roll_no = 1")
conn.commit()
print("record updated successfully")
```

```
record updated successfully
```

In [ ]:
```
```

Department of AI_DS
Academic Year: 2025-26 (Odd Sem)
Course: Python for Data Science Lab
Second Year B. Tech Semester: III

PRACTICAL 11

Part A (To be referred by students)

Database Connectivity and DDL/DML Operations Using SQLite in Python

SAVE THE FILE AND UPLOAD AS (RollNo_Name_Exp11)

Aim: To demonstrate database connectivity using SQLite and perform DDL and DML operations through Python.

Problem Statement:

Develop a Python program to perform the following operations using SQLite:

a. Create a database and establish connection.
b. Create a table named STUDENT with fields: Roll_No, Name, Age, and Branch.
c. Insert at least three records into the STUDENT table.
d. Update the age of a student based on the Roll_No.
e. Delete a student record.
f. Fetch and display all records from the table.
g. Implement proper exception handling for database operations.

Topic Covered:

1. SQLite Database Connection
2. DDL/DML Queries (CREATE, INSERT, UPDATE, DELETE, SELECT)
3. Exception Handling in Database Operations

Learning Objective:

The learner will be able to:

1. Establish database connectivity with Python.
2. Perform SQL operations such as insert, update, delete, and fetch using Python.
3. Apply exception handling in database programming.

Theory:

SQLite is a lightweight, disk-based database that doesn't require a separate server process. Python provides a built-in module `sqlite3` which allows for seamless database handling.

SQLite Architecture

SQLite consists of components like Tokenizer, Parser, Code Generator, Virtual Database Engine (VDBE), B-Tree Storage Engine, and Pager.

Python–SQLite Connectivity

Python's sqlite3 module provides an interface for interacting with SQLite databases. Steps include importing module, connecting, creating cursor, executing SQL, committing, and closing connection.

DDL (Data Definition Language)

DDL commands define database structure. Examples: CREATE TABLE, ALTER TABLE, DROP TABLE.

DML (Data Manipulation Language)

DML commands manage data in tables. Examples: INSERT, UPDATE, DELETE, SELECT.

Transactions in SQLite

Transactions ensure atomicity and reliability. COMMIT saves changes, ROLLBACK reverts to last commit.

Cursor Methods in sqlite3

Cursor methods include execute(), executemany(), fetchone(), fetchall(), and fetchmany().

Sample Program 1: DDL Operations

```
import sqlite3
conn = sqlite3.connect('college.db')
cur = conn.cursor()

cur.execute('''CREATE TABLE IF NOT EXISTS student(
            roll_no INTEGER PRIMARY KEY,
            name TEXT NOT NULL,
            dept TEXT,
            marks INTEGER)''')
print("Table created successfully.")

cur.execute("ALTER TABLE student ADD COLUMN email TEXT")
print("Column added successfully.")

conn.commit()
conn.close()
```

Sample Program 2: DML Operations

```
import sqlite3

conn = sqlite3.connect('college.db')
cur = conn.cursor()

cur.execute("INSERT INTO student (roll_no, name, dept, marks, email) VALUES (1,
'Arjun', 'AI&DS', 85, 'arjun@gmail.com')")
cur.execute("INSERT INTO student (roll_no, name, dept, marks, email) VALUES (2,
'Meera', 'CSE', 90, 'meera@gmail.com')")
conn.commit()
print("Records inserted successfully.")
```

```
cur.execute("UPDATE student SET marks = 95 WHERE roll_no = 1")
conn.commit()
print("Record updated successfully.")

cur.execute("DELETE FROM student WHERE roll_no = 2")
conn.commit()
print("Record deleted successfully.")

cur.execute("SELECT * FROM student")
rows = cur.fetchall()

print("\nStudent Details:")
for row in rows:
    print(row)

conn.close()
```

Sample Output

```
Table created successfully.
Column added successfully.
Records inserted successfully.
Record updated successfully.
Record deleted successfully.

Student Details:
(1, 'Arjun', 'AI&DS', 95, 'arjun@gmail.com')
```

Department of AI_DS
Academic Year: 2025-26 (Odd Sem)
Course: Python for Data Science Lab
Second Year B. Tech Semester: III

PRACTICAL 11

Part B (to be completed by students)

1. All the students are required to perform the given tasks in Jupyter Notebook
2. Create a new notebook for each experiment. The filename should be RollNo_Name_Exp11)
3. In the first cell, the student must write his/her Name, roll no and class in the form of comments
4. Every program should be written in separate cells and in the given sequence
5. After completing the experiment, download the notebook in pdf format. The filename should be RollNo_Name_Exp11.pdf).
   Note: - To download as a PDF, go to File and choose Print Preview. This will open a new tab. Now, press Ctrl + P and opt to save as PDF.
   OR
   Download notebook as HTML and open downloaded html(webpage), press CTRL+P and opt to save as PDF.
6. Upload the pdf on the web portal

Create database and connection

In [1]:
```
import sqlite3
from sqlite3 import Error

DB_NAME = "college.db"

try:
    conn = sqlite3.connect(DB_NAME)   # creates the file if it doesn't exist
    print(f"Connected to '{DB_NAME}' successfully.")
except Error as e:
    print(f"Connection error: {e}")
```

```
Connected to 'college.db' successfully.
```

Create STUDENT table

In [2]:
```
try:
    conn.execute("""
        CREATE TABLE IF NOT EXISTS STUDENT (
            Roll_No INTEGER PRIMARY KEY,
            Name    TEXT NOT NULL,
            Age     INTEGER CHECK (Age > 0),
            Branch  TEXT NOT NULL
        )
    """)
    conn.commit()
    print("Table STUDENT created successfully.")
except Error as e:
    print(f"Table creation error: {e}")
```

```
Table STUDENT created successfully.
```

Insert three records

In [3]:
```
students = [
    (101, "Aarav Sharma", 19, "AI & DS"),
    (102, "Priya Patel",  20, "Computer"),
    (103, "Rohan Mehta",  21, "IT"),
]

try:
    conn.executemany(
        "INSERT INTO STUDENT (Roll_No, Name, Age, Branch) VALUES (?, ?, ?, ?)",
        students
    )
    conn.commit()
    print("3 records inserted successfully.")
except sqlite3.IntegrityError as e:
    print(f"Insert failed (duplicate Roll_No): {e}")
except Error as e:
    print(f"Insert error: {e}")
```

```
3 records inserted successfully.
```

Update age based on Roll_No

In [4]:
```
roll_no = 102
new_age = 22

try:
    cur = conn.execute(
        "UPDATE STUDENT SET Age = ? WHERE Roll_No = ?",
        (new_age, roll_no)
    )
    conn.commit()
    if cur.rowcount:
        print(f"Age of Roll_No {roll_no} updated to {new_age}.")
    else:
        print(f"No student found with Roll_No {roll_no}.")
except Error as e:
    print(f"Update error: {e}")
```

```
Age of Roll_No 102 updated to 22.
```

Delete a student record

In [5]:
```
roll_no = 103

try:
    cur = conn.execute("DELETE FROM STUDENT WHERE Roll_No = ?", (roll_no,))
    conn.commit()
    if cur.rowcount:
        print(f"Student with Roll_No {roll_no} deleted.")
    else:
        print(f"No student found with Roll_No {roll_no}.")
except Error as e:
    print(f"Delete error: {e}")
```

```
Student with Roll_No 103 deleted.
```

Fetch and display all records

In [6]:
```
try:
    cur = conn.execute("SELECT Roll_No, Name, Age, Branch FROM STUDENT ORDER BY
    rows = cur.fetchall()

    print(f"{'Roll_No':<10}{'Name':<15}{'Age':<6}{'Branch'}")
    print("-" * 45)
    if not rows:
        print("(no records)")
    for r in rows:
        print(f"{r[0]:<10}{r[1]:<15}{r[2]:<6}{r[3]}")
except Error as e:
    print(f"Fetch error: {e}")
```

```
Roll_No   Name           Age   Branch
-------------------------------------------
101       Aarav Sharma   19    AI & DS
102       Priya Patel    22    Computer
```

Exception handling demo (duplicate Roll_No)

In [7]:
```
try:
    conn.execute(
        "INSERT INTO STUDENT (Roll_No, Name, Age, Branch) VALUES (?, ?, ?, ?)",
        (101, "Duplicate Entry", 20, "IT")
    )
    conn.commit()
except sqlite3.IntegrityError as e:
    print(f"IntegrityError caught: {e}")
except Error as e:
    print(f"Database error caught: {e}")
finally:
    conn.close()
    print("Connection closed.")
```

```
IntegrityError caught: UNIQUE constraint failed: STUDENT.Roll_No
Connection closed.
```

Department of Artificial Intelligence (AI) & Data Science

AY: 2026-27

Course: Python for Data Science Laboratory (DJS23SLPC303)

Class: SE/A11/A12/A21/A22 Sem: III Roll No.: SAPID:

Experiment No. 14: SQLite, DDL and DML

Aim: To establish a connection between Python and an SQLite database and perform Data Definition Language (DDL) and Data Manipulation Language (DML) operations programmatically.

Theory

1. What is SQLite? How does it differ from other database management systems such as MySQL or PostgreSQL?

2. Why is SQLite considered a serverless and lightweight database?

Conclusions:

Name:- Aaryan N. Raorane Div:- 5 Roll No.:- 5004
Batch:- A1 Branch:- AI & DS SAP ID:- 60018250070

Experiment No 14

Title:- SQLite, DDL and DML

Aim:- To establish a connection between Python and an SQLite database and perform Data Definition Language (DDL) and Data Manipulation Language (DML) operations programmatically.

Theory:-

Q1 What is SQLite? How does it differ from other database management systems such as MySQL or PostgreSQL?

⇒ SQLite is a small relational database engine that is built into the application as a library. It stores the complete database, with its tables and data, in a single file on the disk, and it is accessed using SQL commands. Python includes the sqlite3 module in its standard library, so a database can be created and used without installing anything extra.

Point	SQLite	MySQL	PostgreSQL
Architecture	Embedded library inside the application	client and server	client and server.

Sundaram
FOR EDUCATIONAL USE

Server Process	Not required	A server must be running	A server must be running.
Storage	one single file	Files managed by the server	Files managed by the server
Setup	No installation or configuration needed.	Needs installation and configuration	Needs installation and configuration.
Users and access	No user accounts or passwords	User accounts with row privileges	User accounts with roles and privileges.
Concurrent use	Best for one or a few writers at time	Many users at the same time	Many users and heavy workloads.
Typical use	Small apps, mobile apps, learning testing	web applications	Large Business system.

Sundaram
FOR EDUCATIONAL USE

The first program connects Python to an sqlite database and performs the data Definition language operation CREATE TABLE, followed by the Data Manipulation language operations INSERT and SELECT

code:

```
import sqlite3

conn = sqlite3.connect("college.db")
cur = conn.cursor()

cur.execute("CREATE TABLE IF NOT EXISTS
            students(id INTEGER PRIMARY KEY,
                     name TEXT,
                     marks INTEGER)")

cur.execute("INSERT INTO students(name, marks)
            VALUES('Aarav', 82)")
cur.execute("INSERT INTO students(name, marks)
            VALUES('Riya', 91)")
conn.commit()

cur.execute("SELECT * FROM students")
for row in cur.fetchall():
    print(row)

conn.close()
```

Sundaram
FOR EDUCATIONAL USE

output:-

(1, 'Aarav', 82)
(2, 'Riya', 91)

The second program uses the DML operations UPDATE and DELETE, and the DDL operations ALTER TABLE and DROP TABLE. The commit() method saves the changes to the database file.

code:-

```
import sqlite3

conn = sqlite3.connect("college.db")
cur = conn.cursor()

cur.execute("UPDATE students SET marks=88
            WHERE name='Aarav'")
cur.execute("DELETE FROM students
            WHERE name='Riya'")
cur.execute("ALTER TABLE students
            ADD COLUMN city TEXT")
conn.commit()
cur.execute("SELECT * FROM students")
print(cur.fetchall())
cur.execute("DROP TABLE students")
conn.commit()
conn.close()
```

Sundaram
FOR EDUCATIONAL USE

output :-

[(1, 'Arav', 88, None)]

Q> why is SQLite considered a serverless and lightweight database?

=> SQLite is serverless because it does not need a seperate database server process. The SQLite library runs inside the same process as the Python program and reads and writes the database file directly, so there is no network connection, no port and no login to set up. It is lightweight because the library is small, it needs no installation or configuration, it uses little memory, and the whole database is kept in one ordinary file that can be copied, moved or deleted like any other file.

code :-

```
import sqlite3
import os

conn= sqlite3.connect("light.db")
conn.execute("CREATE TABLE demo(id INTEGER)")
conn.commit()
conn.close()
```

Sundaram
FOR EDUCATIONAL USE

print("Database file exists: " + str(os.path.exists("light.db")))

print("size in bytes: " + str(os.path.getsize("light.db")))

output :-

Database file exists : True
Size in Bytes : 8192

The output shows that creating a database only produced one small file on the disk, with no server started at any point.

conclusion :-

A connection between Python and SQLite was made using the sqlite3 module, and tables were created, changed and removed using DDL commands. Records were inserted, updated, deleted and displayed using DML commands, with commit() saving the changes. SQLite needs no server and keeps the database in one file, which makes it a simple choice for small applications.

Sundaram
FOR EDUCATIONAL USE